

分子动力学模拟及其 LAMMPS 实现

分子模拟全能助手

(添加微信: [lmp_zhushou](#), 进入微信群, 帮助他人, 共建社区)

(讲义尚未完成, 持续更新中)

声明: 本讲义及其附带的讲解视频和脚本都是免费的!

获取 68 个 in 和 data 文件加微信 [lmp_zhushou](#)。

如果你觉得本讲义有用, 请免费分享给他人!

目录

前言	4
1. 分子动力学基础	6
1.1 原理	7
1.2 势能	10
1.3 温度、压强	22
1.4 牛顿方程的数值解法和周期性边界条件	23
1.5 LAMMPS 软件的并行通信算法	25
2. Linux 系统常见命令和操作	31
3. LAMMPS 介绍和并行版安装 (Linux 版)	32
3.1 LAMMPS 介绍	32
3.2 LAMMPS 能做的	33
3.3 LAMMPS 不能做的	38
3.4 LAMMPS 编译过程	39
4. LAMMPS 模拟基本流程	41
4.1 LAMMPS 的 in 文件简介	41
4.2 in 文件的命令解析规则	43
4.3 LAMMPS 的 in 文件基本内容 (很重要必须认真读)	45
4.4 LAMMPS 的运行方法	57
4.5 屏幕日志文件输出	59
5. 建模和后处理软件概述	61
6. LAMMPS 模拟与后处理技巧和教程	66
6.1 LAMMPS 输出轨迹用 vmd 后处理氢键	66
6.2 LAMMPS 建模神器-Enhanced Monte Carlo (EMC)的安装	69
6.3 怎么强迫高分子伸展和蜷缩	71
6.4 乙醇水溶液液滴-moltemplate 结合 packmol 建模	72
6.5 ReaxFF 防止弛豫发生分解反应	76
6.6 铁表面镀碳膜-a toy case	77
6.7 初始密度很小使用 npt 弛豫如何防止出现 PPPM 错误 (NaOH 溶液例子)	78
6.9 两种局部应力计算方法	86
6.10 水泥体系的钙矾石拉伸的 LAMMPS 模拟	87
6.11 ReaxFF 力场之键结参数验证-对比 DFT 和 ReaxFF	94
6.12 ReaxFF 力场之 Qeq 电荷计算参数验证	102
6.13 有机分子热分解的 ReaxFF 模拟及活化能计算	106
6.14 离子液体建模教程	109
6.15 SPC/E 模型下水的表面张力计算	113
6.16 纳米气泡电润湿形态变化	114
6.17 耗散粒子动力学	118
6.18 氯化钠 NaCl 熔盐粘度和扩散系数模拟	120
6.19 铝粉锻造 LAMMPS 模拟	127
7. 出错解决思路	129
8. 专题一: 纳米流动	131
8.1 引言	131

8.2 in 文件讲解	136
8.3 后处理	139
9. 专题二: 纳米流体	141
9.1 基本传热性质模拟	142
9.2 Green-Kubo 法计算热导率和粘度	144
9.3 RNEMDS 方法计算热导率和粘度	148
10. 专题三: 润湿	150
10.1 moltemplate 建模	151
10.2 in 文件讲解	157
10.3 MS+packmol 建模	158
10.3.1 所需软件	158
10.3.2 使用 Packmol 建立液滴模型	160
10.3.3 使用 packmol 和 MS 建模	162
10.3.4 使用 MS 建立基底+基底液滴合并	163
10.3.5 LAMMPS 的 data 文件导出与 in 文件编写	169
11. 专题四: 拉伸、压缩、剪切、弯曲、扭转、疲劳	176
11.1 引言	176
11.2 拉伸	176
11.3 压缩	177
11.4 剪切	178
11.5 弯曲	179
11.6 扭转	180
11.7 疲劳	181
12. 专题五: 传热	182
13. 专题六: 回转半径和扩散	182
13.1 建模	182
13.2 模拟	185
13.3 结果	188
14. 专题八: 摩擦和磨损	188
14.1 引言	188
14.2 一个简单的例子	188
14.3 二维粗糙度铜块对磨模拟	190
15. 专题九: 反应分子动力学 ReaxFF	197
15.1 ReaxFF 力场公式介绍	197
15.2 ReaxFF 力场拟合软件介绍	203
14.2.1 安装	203
14.2.2 基本使用	207
15.3 ReaxFF 力场验证	213
15.4 ReaxFF 力场开发	229
16. 专题十: 耗散粒子动力学	229
16.1 简单流动	229
16.2 滑移边界条件控制方法	231
17. 专题十一: 自由能	234
18. 专题十二: LAMMPS 实现小分子有机半导体的沉积	234

18.1 研究背景.....	234
18.2 建模细节.....	235
18.3 模拟过程.....	235
18.4 模拟结果.....	251
18.5 结果分析.....	251
19. LAMMPS 源代码解析	255
19.1 LAMMPS 的一个时间步是如何工作的.....	255

前言

分子动力学模拟在科研中的使用越来越广泛。LAMMPS 作为分子动力学中一款优秀的开源软件，使用率非常高。大量高水平的论文都是采用 LAMMPS 完成的，其中不乏 Nature, Science, PRL, JACS, Nature Communication, PNAS 等顶刊。但是对于零基础的同学，特别是本科生和低年级硕士生，LAMMPS 的学习曲线还是比较陡峭的。很多同学都是课题组内第一个使用分子动力学模拟和 LAMMPS，其困难可想而知。如果能有一个针对 LAMMPS 的系列教程对学生掌握这个工具会大有裨益。本教程就是希望能够在学生使用 LAMMPS 初期降低学习难度，少走弯路，快速上手，将自己的精力集中在具体的科学问题上，而不是软件的学习上。本教程假设你之前对 Linux, 分子动力学和 LAMMPS 一无所知，从头开始讲，因此比较浅显，并未就某些问题进行深入介绍（其实是深入的我也不会，哈哈）。

我将分子动力学研究科学问题分为三个部分：软件使用，分子动力学理论，科学问题发现。如果把分子动力学比作武林，那么软件的使用仅仅是招式，分子动力学理论则是内功心法，而科学问题则是对手。要想成为一名武林高手，在江湖上闯出一番事业，既要熟练掌握各种招式，也要有很深的内功修为，同时也要

有一些值得尊敬的对手。只有用深厚的内功催动精妙的招式, 才能打败可敬的对手, 受到江湖传颂。本教程只是 LAMMPS 的使用教程, 它会教给你使用 LAMMPS 的一招一式, 但要将招式练得熟练, 还需要自己在科研中积累大量模拟经验。而内功心法的修炼则需要长时间的学习经典教材和文献。好的对手必须要广泛阅读文献, 与导师深入沟通确定具有很好学术价值的科学问题。那么有没有一种方法像武侠小说中的主角, 掉下悬崖然后打通任督二脉, 迅速成为高手。答案是当然是没有的。但是作为物理学分支, 我推荐阅读《费曼物理讲义》第一卷, 可以起到类似的作用。总之, 科研没有坦途, 静心学习, 努力思考必有所成。

“LAMMPS 即 Large-scale Atomic/Molecular Massively Parallel Simulator, 可以翻译为大规模原子分子并行模拟器, 主要用于分子动力学相关的一些计算和模拟工作, 一般来讲, 分子动力学所涉及到的领域, LAMMPS 代码也都涉及到了。LAMMPS 由美国 Sandia 国家实验室开发, 以 GPL license 发布, 即开放源代码且可以免费获取使用, 这意味着使用者可以根据自己的需要自行修改源代码。LAMMPS 可以支持包括气态, 液态或者固态相形态下、各种系综下、百万级的原子分子体系, 并提供支持多种势函数。且 LAMMPS 有良好的并行扩展性。”——摘自百度百科。LAMMPS 的官网是 <https://www.lammps.org/>。在 LAMMPS 的官网中有大量关于软件的信息, 可以好好探索一番。有几个地方可以经常看看。第一是 Publication 部分 <https://www.lammps.org/papers.html>, 记录引用 LAMMPS 的所有文章, 多看看对自己的研究有所启发。Maillist 部分 <https://www.lammps.org/mail.html> 是 LAMMPS 的支持社区。当你有问题的时候可以检索 maillist, 你遇到的问题别人也遇到过。所以查看开发者对这些问题的解答。

如何学习 LAMMPS? 你可以把本教程当做一个开始。LAMMPS 的手册中详细介绍了软件的各个方面。一定要好好阅读手册。手册的前四章和第八章要认真阅读。经常使用的命令也要仔细阅读。当你要实现某种功能的时候就把手册打开看看命令。仔细阅读学习 LAMMPS 官方手册是成为 LAMMPS 高手的必经之路, 阅读学习一本分子动力学模拟经典教材是增加修为的关键。希望本教程能教会你 LAMMPS 的基本招式。祝你好运!

LAMMPS 官网推荐了几本分子动力学模拟教材:

Books about Molecular Dynamics generally or LAMMPS specifically

Note that these are not endorsements of particular books. We simply want to make the LAMMPS user community aware of them as potentially useful resources.

General MD books:

- Allen & Tildesley - [Computer Simulation of Liquids](#)
- Frenkel & Smit - [Understanding Molecular Simulation: From Algorithms to Application](#)
- Griebel, Knapek, Zumbusch - [Numerical Simulation in Molecular Dynamics: Numerics, Algorithms, Parallelization, Applications](#)
- Tuckerman - [Statistical Mechanics: Theory and Molecular Simulation](#)

Books about LAMMPS specifically:

- Mubin & Li - [Extending and Modifying LAMMPS](#) (see below)



1. 分子动力学基础

1.1 原理

本节只是简要介绍分子动力学的原理，如果要深入学习相关内容，可阅读前言中推荐的教材。

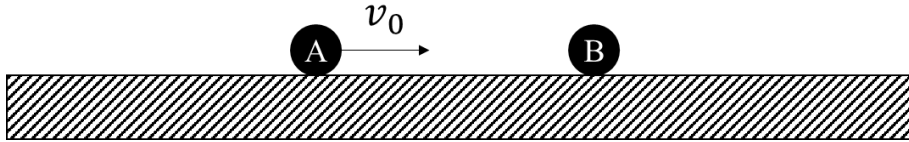
分子动力学的基础是牛顿力学，也即经典力学。经典力学中有三个主要内容：质点、力和运动。牛顿第二定律是经典力学的核心，下面两个质点的方程大家一定不陌生。

$$\begin{aligned}\frac{dx}{dt} &= v \\ m \frac{dv}{dt} &= F(x)\end{aligned}$$

我们把他写成离散的形式，也称为差分形式：

$$\begin{aligned}\Delta x &= v \Delta t \\ \Delta v &= \frac{F(x)}{m} \Delta t\end{aligned}$$

如果 Δt 足够小那么离散形式可以以足够的精度近似连续形式。首先我们假设质点的受力之依赖于它的位置，也就是说只是位置的函数。事实上这是一个很普遍成立的假设，比如重力或者万有引力，在质量确定的情况下只依赖于两个物体之间的相对距离。在比如静电力，在电荷确定的情况下只依赖于两个带电质点的相对距离。有了这个假设，我们考虑一个一维的简单情形。有两个质点放置在光滑的地面上，如下图。



在 $t = 0$ 时刻，质量为 m_A 位于 x_0^A 的质点 A 以速度 v_0^A 朝质量为 m_B 位于 x_0^B 且静止（即 $v_0^B = 0$ ）的质点 B 运动。两个质点之间的受力 F_{AB} 只是质点 A 与质点 B 之间距离 x_{AB} 的函数，也即只要知道 x_{AB} ，我们就能计算出来 F_{AB} 。然后，我们要问的问题是在后续的时间中两个质点是怎么运动的？我们这时设定一个很小的 Δt 。我们知道初始速度那么有

$$\begin{aligned}x_{t_0+\Delta t}^A &= x_0^A + v_0^A \Delta t \\ x_{t_0+\Delta t}^B &= x_0^B + v_0^B \Delta t \\ v_{t_0+\Delta t}^A &= v_0^A + \frac{F_0^{AB}}{m_A} \Delta t\end{aligned}$$

$$v_{t_0+\Delta t}^B = v_0^B + \frac{F_0^{AB}}{m_B} \Delta t$$

这样我们就知道了 $t_0 + \Delta t$ 时刻, 质点 A 和 B 的位置和速度。由新的位置又可以得到新的受力。重复上述公式就可以知道后续所有时刻两个质点的位置和速度, 也就是知道两个质点的运动细节。总结一下我们结算所有时刻质点的运动细节要提前知道哪些信息。质点的质量、初始速度、初始位置和依赖于位置的受力函数。如果我们再选一个足够小的 Δt , 那么我们就能够准确的得到质点 A 和 B 的所有运动信息。

我们把情形扩展到二维。在一个光滑的台球桌面上, 白球在初始时刻以确定的速度撞向其他摆好的台球。如果我们把台球都看成质点, 又知道所有台球的质量、初始速度、初始位置和依赖于位置的受力函数, 我们就能预测击球后所有时刻台球的运动轨迹, 从而判断台球是否能够进袋。我们再把情形扩展到三维。考虑宇宙中只存在太阳和太阳系的八大行星。我们是否能够通过万有引力去预测所有行星的轨迹。答案当然是可以的。只是此时我们知道某个行星除了来自太阳的引力, 还有来自其他行星的应力。我们在计算某个行星的受力时, 要考虑该行星与其余所有天体之间的受力。这就是经典力学能够告诉我们的东西。

在这个概念上, 拉普拉斯提出了一个想法。如果存在一只神奇的动物, 它能基于经典力学解算出世界上所有东西的位置和速度并能加以分析, 那么世界上发生的一切及其背后的原因都一览无余的呈现在它眼前。这就是经典力学的世界观和认识论。分子动力学的理论就是基于这样的认识方法。像上面的例子一样, 分子动力学的目的就是为了求解任意时刻所有组成物体原子的位置和速度, 从而认识物质世界。

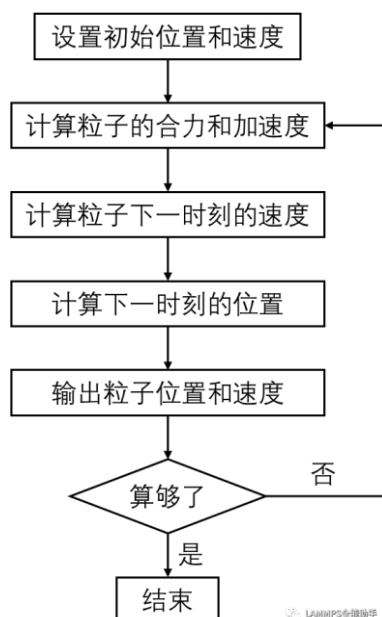


为什么这个方法具有很大的威力呢? 这是多年科学实践告诉我们的。事实上

著名物理学家理查德·费曼对这个问题有着精彩的描述。理查德·费曼在其传世物理教材《费曼物理讲义》中提出这样一个有意思的问题：“假如由于某种大灾难，所有的科学知识都丢失了，只有一句话传给下一代，那么怎样才能用最少的词汇来表达最多的信息呢？”费曼认为这句话应该是原子假设：“**所有物体都是用原子构成的——这些原子是一些小小的粒子，它们一直不停地运动着，当彼此略微离开地时互相吸引，当彼此过于靠近时又相互排斥。**”这句话之所以重要是因为基于这个原子假设我们能解释巨量自然界中的现象。比如，气体压强来自于大量气体分子对壁面撞击；水蒸发吸热来自于水分子从体相水中逃逸至外界环境而带走能量所致。。。。更多详细例子可阅读《费曼物理讲义》第一卷的相关内容。

费曼把原子假设放在如此高度的原因总结起来就是——**掌握组成物体内部原子的行为就可解释和理解物体的性质和行为**。只是这个时候我们关注的是大量原子在一起表现出来的整体特性，而不是关注具体某个原子的运动细节。事实上，实践告诉我们大量原子在一起就会表现出某些特定的确定的性质和行为，也就我们实际生活生产中能直接测量和感受的性质和行为。到这里我们可以看到分子动力学实际上就是一只拉普拉斯兽。分子动力学的基本任务就是获取物体在任意时刻组成原子的所有位置和动量然后利用统计力学知识理解物体的性质和行为。如果这样还不能说服你分子动力学的实力。就讲一个事实——2013 年诺贝尔化学奖颁给了创立跨尺度分子模拟的三位科学家。通过分子动力学模拟我们可以深刻理解我们所研究的对象。目前各种硬件超强的计算能力为分子动力学创建了一个大有可为的时代！！

分子动力学的基本任务就是获得研究对象不同时刻的位置和动量，然后基于统计力学知识获得想要的物理量，解释对象的性质和行为。因此，分子动力学的模拟流程超级简单。第一步，设置研究对象组成原子或者粒子（下面统一称为粒子）初始位置和速度；第二步，基于粒子的位置计算每个粒子受到的合力，并基于牛二定律计算粒子的加速度；第三步，基于加速度计算粒子下一时刻的速度；第四步，基于下一时刻的速度计算下一时刻的位置；第五步，循环第二步到第四步的过程，得到一系列时刻粒子的位置和速度；第六步，基于位置和速度信息得到描述对象性质和行为的物理量。



分子动力学的基本原理就是这么简单，正因为简单所以有效。需要指出的是在实际编程求解的过程考虑到计算精度等原因上述过程在某些细节上会略有改动。但是，基本上你可以认为这就是分子动力学的原理和流程。上述过程由LAMMPS 帮你实现，你需要做的就是设置研究对象的基本信息，告诉给LAMMPS，然后开始模拟。

1.2 势能

前面我们知道要获得所有原子的轨迹，需要知道原子的质量，初始位置，初始速度和依赖于位置的受力函数。原子的质量，初始速度和初始位置都是我们人为设定的值。不同研究对象的原子质量，初始速度和位置很容易获得。不同研究对象更本质的差别在于依赖于位置的受力函数，这个函数又被称为相互作用。相互作用的形式和参数决定了研究对象的性质和行为。在分子动力学中，这个相互作用通过原子之间的势能进行描述，因此又称为势能函数或势函数。而受力是势能对位置的负梯度，因为实践表明原子之间的受力都是保守力。举个例子，重力和重力势能。取竖直向上为正方向，则重力表达式为

$$-mg$$

重力势能的表达式是

$$mgh$$

$$-(mgh)' = -mg$$

即重力势能对位置的负梯度就是重力。为什么要用势能，而不是受力来表达相互

作用呢？这是因为在绝大多数模拟中采用的是笛卡尔坐标。势能是标量，而力是矢量，数学上处理起来更容易。在计算受力的时候，直接用势能对两个原子之间的相对位置进行求导，然后将这个力乘以方向余弦就能得到xyz三个方向上的受力。举例说明，在三维空中有两个原子

$$i(x_i, y_i, z_i) \quad j(x_j, y_j, z_j), \quad \Delta x = x_i - x_j, \quad \Delta y = y_i - y_j, \quad \Delta z = z_i - z_j$$

两个原子之间的距离

$$r = \sqrt{(\Delta x)^2 + (\Delta y)^2 + (\Delta z)^2}$$

那么j原子对i原子的受力就为

$$F_x = -U'(r) \frac{\Delta x}{r}, \quad F_y = -U'(r) \frac{\Delta y}{r}, \quad F_z = -U'(r) \frac{\Delta z}{r}$$

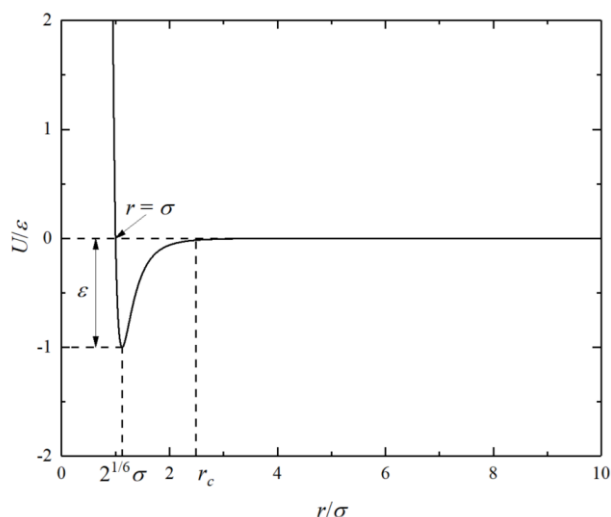
这样不管是程序的简洁性还是计算效率都会很好。

所以要开展分子动力学计算，首先必须确定所有原子之间的势能函数，即势能关于两个原子之间相对位置的函数。确定函数分为两个部分：函数表达式和表达式中的常数值。对于一个对象，描述其相互作用的势能函数表达式和表达式中常数值的集合称为该对象的力场。通常一种力场可以用来描述一类对象。分子动力学模拟一般研究的对象可以分为：惰性气体，分子物质，金属晶体和非金属晶体。分子动力学模拟绝大多数情况用来研究固体和液体，即凝聚态。对于气体分子动力学模拟效率太低，有更好的办法进行研究。

简单液体

原子之间相互作用只具有范德瓦尔斯作用的物质。描述范德瓦尔斯作用最常用的表达式是 Lennard-Jones 12/6 势能。通常用 LJ12/6 描述的物质也称为简单液体。

$$U(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]$$

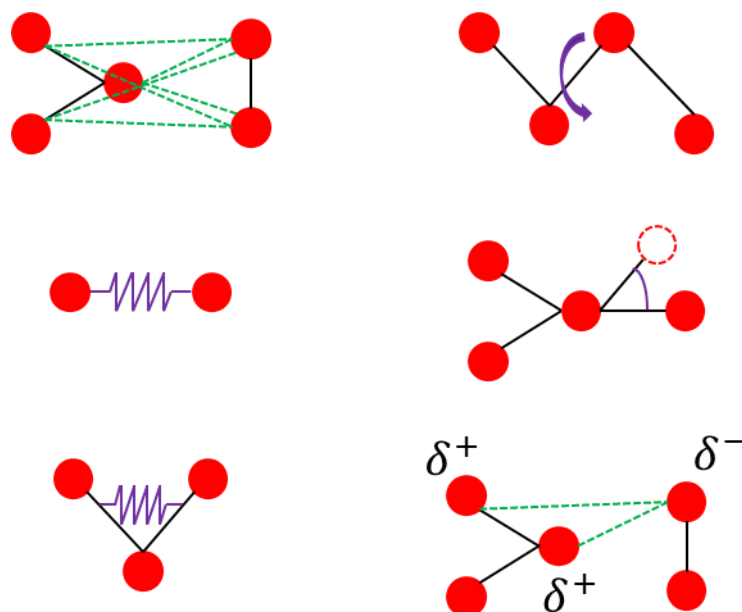


上图是 LJ/126 势函数的图像。对于该势函数有几个重要的点要说一下。势能的最低点的到零点之间的绝对值为 ϵ , 称为势能函数的特征能量, 势能的零点位于 $r = \sigma$ 处, σ 称为势能函数的特征长度。势能函数的最低点位于 $r = 2^{1/6}\sigma$ 处, 在该点处势能对位置的导数是零, 所以该点处受力也为零。该点也称为平衡位置, 我们可以看到在平衡位置出势能是负的。所以当体系处于平衡状态时, 原子之间的相对距离大概也在平衡位置附近, 所以势能为负的。这也就是为什么大多数情况分子动力学算出来的势能都是负的。在 $r < 2^{1/6}\sigma$, 两个原子之间表现为相互排斥, 在 $r > 2^{1/6}\sigma$, 两个原子之间表现为相互吸引。从势函数的图像上可以看出在 r 较大的时候, 势能趋于零, 其导数也趋于零。也就是当两个原子距离很远的时候, 原子之间的相互作用可以忽略不计。因此在实际计算的时候都会使用截断的 LJ/126。截断距离成为截断半径, 其值大概在 2.5σ 。就像重力势能一样, 势能的计算需要一个参考位置。LJ/126 的参考位置选择在了无限远处。类似 LJ/126 势能以及其他拓展的 LJ 势能, 都称为对势, 其含义是两个原子之间的相互作用只由这一对原子决定, 与其它相邻原子没有关系。

分子物质

原子之间有化学键连接的物质。比如常见的水, 甲烷, 乙醇, 醋酸, 氯化钠, 聚乙烯, 酯类, 氨基酸, RNA, DNA 等。分子物质是分子动力学模拟应用最广的对象之一。为了描述分子物质研究者开发出了一系列力场。值得说明的是力场是一个经验性的东西。研究者通过自己的物理直觉和反复试验, 最终建立起适合某一类物质的力场。对于分子物质, 相互作用包括两类: 非键结势能和键结势能。非键结用来描述没有化学键连接原子之间的相互作用, 键结势能用来描述有化学

键连接原子之间的相互作用, 如下图。



在一个体系中任意两个非键结的原子, 如果其距离小于截断半径那么都要考虑它们之间的非键结相互作用。对于大多数力场非键结势能用 LJ12/6 描述。对于键结原子之间的相互作用则通过键结势函数进行描述。键结相互作用分为键能 (bond), 键角能 (angle), 二面角能 (dihedral), 离平面能 (improper)。两个以化学键相连的两个原子都具有键能相互作用, 当化学键偏离平衡长度的时候两个原子就会出现相互作用。两个化学键连接的相邻原子之间均具有键角能, 当三个原子之间的角度偏离平衡角度的时候就会产生键角能。四个化学键连接的相邻原子之间会存在二面角能相互作用, 当四个原子构成的二面角偏离平衡二面角的时候四个原子之间会出现二面角相互作用。四个相邻化学键连接的相邻原子之间有时还会存在离平面势能作用, 当第四个原子偏离前三个原子所构成平面的平均夹角时, 四个原子之间会存在离平面能相互作用。不同的分子会具有不同的键结势能。如氧气分子就只具有键能相互作用, 水分子具有键能和键角相互作用, 十二烷分子具有键能, 键角能和二面角相互作用分子。元素构成较为复杂的分子, 如 RNA, 就会同时具有四种键结相互作用。具体一个分子具有哪些键结能相互作用, 由描述该分子的力场所定义。你只需要使用合适的软件在建模的时候会自动定义好各种键结信息和势能函数的参数。值得指出的是, 在分子体系中原子一般都带有电荷, 所以非键结原子之间会存在静电相互作用。在大多数力场中, 原子的电荷都是部分电荷, 也就是电荷一般都是零点几几几。这是由于在共价键中任何一

个原子都并没有完全得到或失去一个电子, 只是部分的得到或失去电子, 因此在描述其电荷的时候都是零点几几几。少数强电解质会有整数电荷, 比如氯化钠中的氯离子和钠离子, 分别是-1 和+1 电荷。这里的电荷单位是一个电子或质子所带的电荷量, 也就是单位电荷量。

为什么会采用多种键结势能函数来描述分子体系呢? 这是因为键结的原子之间电子云的相互作用会对分子的构型进行限制, 导致分子会具有特定的几何形状。分子动力学为描述这种情形就采用这样键能组合的形式。这是一种经验的方法。实践表明这是一种很好的办法。只要有合适的势函数形式和参数, 采用的力场就能够准确描述分子的构型和之间的相互作用。目前研究者针对分子体系已经开发出多套力场。李继存老师对分子体系的力场有一篇文章做了很好的总结, 现引述如下 (具体网址见[科学网—力场与拓扑之二: 如何选择力场 - 李继存的博文 \(sciencenet.cn\)](http://www.sciencenet.cn)):

性能

模拟有机小分子热力学性质用 Charmm generalized >= OPLS-AA >= GAFF, 但实际上 GAFF 已经很好了。它们计算各种有机分子的密度、蒸发焓都很准确, 但是介电常数、等温压缩系数计算得都一般。GAFF 对于带有硝基的分子不好。OPLS 和 GAFF 对于苯甲醛、甲酸, 以及有两个及以上 Br 或 Cl 相距较近的情况都不好。

对蛋白质构象的模拟: ff99SB+ildn+nmr > CHARMM27 >OPLS>>ff99。

Berger: 专门用于磷脂的力场

PFF=Polarizable Force Field

VAMM=Virtual atom molecular mechanics

材料力场

cvff(consistent valence forcefield): 参数用于有机分子、蛋白质模拟, 函数形式略复杂。cvff_aug 是对其扩展, 可以用于研究硅酸盐、铝硅酸盐、磷酸盐、泥土

CFF(consistent family of forcefield): 包括 CFF91 和 CFF95。适用面很广, 涵盖有机无机小分子、聚合物、多糖和生物大分子, 还支持金属。函数形式挺复杂。参数由从头算获得, 非键参数从 CVFF 弄来, 不适合凝聚相模拟。

pcff: 基于 CFF91, 适用范围做了扩展, 主要用于聚合物和有机材料, 也能用于

无机材料, 还有糖、核酸、脂的参数。

COMPASS=Condensed-phase Optimized Molecular Potentials for Atomistic Simulation Studies: 在 pcff 基础上改进的新版本, 同样由从头算获得参数, 在凝聚相模拟方面大有改善。适用于有机和一些无机分子、高分子, 常用于材料领域的各种性质计算, 不支持生物分子。模拟超临界水不错。能够适应很宽范围的压强和温度。MS 中 COMPASS(即 2.8)>COMPASS2.7>COMPASS2.6。在 MaterialStudio 中御用, 参数是加密不公开的, 虽然 lammps 也能用, 但是参数不全。

普适力场

Dreiding: 普适型力场, 但支持的元素有限, 并非涵盖整个周期表。可以用于有机、生物、主族无机分子。结构、结合能的计算结果精度一般。没有指定计算电荷的方法, 建议用拟合静电势电荷, 原文用 Gasteiger 勉强应付。

UFF=Universal Force Field: 涵盖整个周期表的普适型力场。比 Dreiding 更好。函数不复杂。UFF 计算结构、结合能的计算结果精度一般, 主要适合找不到适合的力场时凑合用。默认用 QEq 电荷。缺少纯金属单质的参数, 都是离子的。

反应力场

REBO: 反应力场, 主要用于固体, 无非键参数。后来添加了 C、H 的 LJ 参数的叫做 AI-REBO 力场。

ReaxFF=Reactive force field: ADF、MS、lammps 支持, 作者本人也有相应的程序但不公开。这是反应力场, 引入键级概念, 可以研究模拟过程中的设计拓扑结构改变的化学反应, 挺流行。对于结构新鲜的体系可能差一些。用于爆炸、燃烧过程的反应貌似不错。支持所有主族和部分过渡金属元素。电荷由 EEM 获得。参数很多很复杂难找全。形式复杂, 步长需要比较小, 比如 0.1~0.5fs。计算速度慢, 貌似介于半经验与一般分子力场耗时之间, 只适合最多用于几千个分子, 再多的话就得和普通力场联用(类似 QM/MM)。

专业小分子力场

MMX: 早期的有机小分子力场。

MM2/MM3/4=Molecular Mechanics versions 2/3/4: 用于小分子。函数复杂。计算小分子结构能量好, 适合构象搜索、计算频率、获得最稳定结构。由于 VDW 参

数不好, 凝聚相问题差。MM3 是这一类中最流行的。MM3 也有蛋白质参数, 叫 MM3PRO。

MMFF=Merck Molecular Force Field: 各方面类似于 MM2/3, 但比之更好。在 CFF 之后才发展的, 也是从头算计算参数, 但提升至 MP2 级别。参数比较细而且广泛。凝聚相问题仍不好。

大分子力场

CHARMM=Chemistry at HARvard Macromolecular Mechanics, 函数形式简单, 包含以下版本:

CHARMM19: 联合原子力场, 也适用于蛋白质。

CHARMM22: 全原子力场, 适用于蛋白质。

CHARMM22/CMAP: 适用于隐势 GBSW。名字编号是最初使用这种力场的 CHARMM 版本得名的。

CHARMM27: 适用于核酸和脂(膜)。与 22 结合可以混合使用模拟蛋白+核酸体系。应当用 TIP3P, 是在这种水模型下拟合的。

CHARMM General Force Field(CGenFF): 用于药物类小分子, 也可视为通用有机小分子力场。可结合其它 CHARMM 全原子力场使用。

CHARmm: CHARMM 的商业版本, 归 Accelrys。

AMBER=Assisted Model Building with Energy Refinement: 适合蛋白和核酸的凝聚相模拟, 有机小分子支持得少。函数形式简单。包含以下版本:

ff12 力场(parm10.dat+frcmod.ff12SB): 与 ff10 的区别是对蛋白的骨架和侧链扭转项参数做了进一步修正以更吻合实验。核酸的参数没变。这是目前最好的 amber 力场。

ff10 力场 (parm10.dat): 对 ff99 的各种参数补丁的集合, 相当于 parm99.dat+frcmod.ff03+bsc0+chi.OL3+新的离子参数+原子和残基名的修改以顺应 PDB format version 3。蛋白的参数和 ff99SB 相同。

ff99SBildn(frcmod.ff99SBildn): 对 ff99SB 的异亮氨酸、亮氨酸、天冬氨酸、天冬酰胺的侧链参数改进的补丁。

ff99SBnmr(frcmod.ff99SBnmr): 基于 NMR 数据对 ff99SB 的骨架参数进行修正的补丁。

ff03.r1 力场(parm99.dat+frcmod.ff03): ff99 力场的修改版。获取电荷时通过连续介电模型表现溶剂可极化效应, 修改了蛋白 phi、psi 骨架参数, 减少了对螺旋构象的偏爱。核酸参数相对于 ff99 没变。ff03.r1 与 amber9 中的 ff03 略有不同, 那时仍用的是 ff94 的方法得来的碳、氮端基原子电荷, 如果仍想用那时代的 ff03 就调用 oldff/leaprc.ff03。

ff03ua 力场(parm99.dat+frcmod.ff03+frcmod.ff03ua): ff03 力场的 united-atom 版本, 侧链的氢原子被 united 了, 骨架上的氢原子和芳香环上的氢原子仍被保留。由于骨架还是全原子故骨架势参数没变, 侧链上的参数因用了 united 故重新拟合。核酸参数完全没变, 且还是全原子。

ff02 力场(parm99.dat+frcmod.ff02pol.r1): ff99 力场的可极化版, 给原子上增加了可极化的偶极子。frcmod.ff02pol.r1 是对原 ff02 的扭转参数的修正。

ff02EP 力场(parm99EP.dat+frcmod.ff02pol.r1): ff02 力场基础上给诸如氧、氮、硫原子增加了偏离原子中心的点电荷以表现孤对电子效应。据称比 ff02 稍好点。

ff99 力场(parm99.dat): 大部分参数来自 ff94 力场, 修改了许多扭转角的参数。甘氨酸的骨架参数有问题, 螺旋和延展构象的平衡性不对。而对于 DNA, ff99 长时间模拟中亚稳态占统治地位, 即 alpha 和 gamma 二面角倾向于分别为 gauche+和 trans 状态。虽然在 RNA 中也有这问题, 但不严重。ff99 的这些毛病在 ff94 里也有。

ff99SB 力场(parm99.dat+frcmod.ff99SB): 对 ff99 的蛋白二面角参数进行修正, 二级结构间分布的比例得到了改善, 也解决了甘氨酸骨架参数问题。

bsc0(frcmod.parmbsc0): 解决上述 ff99 在核酸模拟问题上的补丁, 同时还改进了 RNA 的糖苷的 gamma 二面角扭转势。可参考 <http://mmb.pcb.ub.es/PARMBSC0>。

ff99SB+bsc0 力场: 把 bsc0 补丁用到 ff99SB 上, 相对于 ff99 同时增进对蛋白和核酸的效果。这个组合使 gamma 二面角过分偏离了 trans 型。如果初始结构有很多 gamma 角为 trans 的情况, 还是用 ff99 比较好。

ff99SBildn(frcmod.ff99SBildn): 在 ff99SB 基础上修改氨基酸侧链参数的补丁。

ff99SBnmr(frcmod.ff99SBnmr): 在 ff99SB 基础上修改骨架扭转项参数以更符合 NMR 数据的补丁。

ff98 力场(parm98.dat): 对 ff94 改进了糖苷的扭转角参数。

ff96 力场(parm96.dat): 与 ff94 扭转角不同, 算出来的能量更接近量化结果。来自 Beachy et al, 由于构象有明显偏向 beta 等问题, 使用不广泛。

ff94 力场(parm94.dat): 来自 Cornell, Kollman et al。适合溶剂环境。电荷由 RESP HF/6-31G* 获得。

ff86 力场(parm91X.dat): 将 ff84 扩展为全原子力场。和 ff84 一样对氢键也是用 Lennard-Jones 10-12 势, 故如果想在 sander 里用 ff84/86, 得重新带着 -DHAS_10_12 选项编译。之所以相应的文件叫 parm91X 是因为对原始 ff86 做了一些修正。(parm91X.dat 是 parm91.dat 的补完版, 加入了一些非键项, 但非键项比如 Mg、I 等的参数都没调好, 只是近似。)

ff84(parm91X.ua.dat): 最早的 AMBER 力场, 用于模拟核酸和蛋白质的联合原子力场。不推荐使用, 但在真空或者距离依赖的介电常数下模拟还有用。

parmAM1 和 parmPM3 力场(parmAM1.dat/parmPM3.dat): 用这个参数对蛋白质优化可以得出与 AM1/PM3 相同的优化结果。如今已没什么价值。

GAFF 力场(gaff.dat)=Generation Amber Force Field: 普适型有机小分子力场, 函数形式和 AMBER 力场相同, 与 AMBER 力场完全兼容。

GLYCAM-06 力场(GLYCAM_06g.dat): 对以前 GLYCAM 力场做了改进, 并且纳入了一小部分脂类的参数。

GLYCAM-04EP 力场(GLYCAM04EP.dat): 将 GLYCAM04 扩展到可用于 TIP5P 模型下的模拟。给氧加上非原子中心点电荷表现孤对电子效应。

GLYCAM-04 力场(GLYCAM04.dat)=glycans and glycoconjugates in AMBER: 专用于糖的模拟, 和 AMBER 完全兼容, 可一起用于糖蛋白的模拟。官网:

<http://glycam.ccruc.uga.edu/ccrc/index.jsp>

lipid11: 与 Amber 力场完全兼容的模拟磷脂和胆固醇的力场。

AMOEBA: 可极化力场, 能模拟水、蛋白、单价离子、有机分子。速度比起一般的固定点电荷力场慢 8 倍。

GROMOS=Groningen Molecular Simulation: 适合烷烃、蛋白、核酸凝聚相的模拟, 函数形式简单。A 前边的数字代表此力场所含原子类型数目, 后面的是版本号。

G43B1: 适用于真空下的模拟, 现在基本不用。

G43A1: 是最早的 GROMOS96 力场, 是联合原子力场 (极化的氢才表达出来),

用于凝聚相, 后面那些都是对这个的小幅改进

G43A2: 是改进了烷烃中原子类型的二面角参数

G45A3: 是改了烷烃的 VDW 参数以更好地符合实验数据 (气化焓、可压缩率、压强、水合热等)

G45A4: 是为了改进核酸的模拟, 增加了二面角参数、原子类型、改动了原子电荷

G53A5 和 G53A6: 调整了参数, 分别使生物分子 (蛋白、DNA、糖、脂) 在环己烷和水中的溶解自由焓贴近实验值。但是蛋白模拟时的结构比起 G45A4 却有了缺陷。

G54A7/B7: 修改了 psi/phi 的扭转角参数以改正模拟蛋白质时对螺旋的稳定性问题, 加入了一个-CH₃ 的原子类型, 修改了 Na⁺和 Cl⁻以符合其水合能, 加入了一个和手性改变相关的 improper 项。模拟蛋白质的稳定性比 G53A6 更好了。G54B7 是用于真空模拟, 是在 G53B6 基础上做了类似 G54A7 对 G53A6 的改进得到的。

OPLS=Optimized potentials for liquid simulations: Jorgensen 搞的。适合有机小分子、蛋白凝聚相的模拟。函数形式简单。最初 OPLS-UA 是联合原子力场, 现在一般都用 OPLS-AA 全原子力场。

MARTINI: 流行的粗粒化力场, 四个原子 (氢也算) 凑一个原子, 可以做磷脂、蛋白质、聚合物、糖、胆固醇。在 gromacs 里可以用。一般 25~40fs 一步, 不能考虑二级结构的变化, 必须事先指定好。

<http://md.chem.rug.nl/~marrink/coarsegrain.html>

ENCAD=Energy Calculation and Dynamics: 不流行。专门模拟溶液下蛋白和核酸, 侧重于模拟过程中的能量保守性。

Shinoda2007=粗理化力场, 专用于模拟水与表面活性剂体系, 平均三个重原子折合一个 bead。

水模型

不一定水模型是力场御用的就一定比用其它的水好。对于非极化的水, TIP4P-EW 是最好的。对于可极化的水, AOMEBA 是最好的。

TIP3P: amber、gaff 御用的。

TIP4P: OPLS 御用的。

TIP4P-EW: 在 TIP4P 基础上专为 ewald 计算优化, 对水的凝聚相各种热力学、结构性性质表现得比其它水模型都好。

SPC: GROMOS 力场御用的。

SPCE: 对纯水体系比 SPC、TIP3P 都好。但是有人说这个在纯水下表现好, 在混合体系疏水性略强。

shell 水模型: 在 gmx 中直接支持。就是给普通的水中间加了个振子表现可极化效应。虽然形式是所有可极化水模型里几乎最简单的, 耗时少, 但是步长必须设得很短, 0.2fs。

AMOEBA 水模型 =Atomic Multipole Optimized Energetics for Biomolecular Application: 主要在 Tinker 中使用, amber 也支持。包括比较复杂的可极化水模型, 是三点模型, 在原子中心上除了单极矩(原子电荷)外还用偶极矩、四极矩来精确描述电荷分布, 并且给予原子由拟合实验值得到的原子极化率, 在外场下产生诱导偶极矩以使电荷分布能响应实际环境, 氢的范德华作用也考虑进去。不是一般的 LJ 势。这个水是柔韧的, 因此不要用约束算法固定住它的结构。这个水模型管在纯水液相状况下比 TIP5P 没什么改进还多花时间, 但普适性比起其它模型都好得多, 气相、溶液相都适用, 而不是像固定点电荷水模型只在特定环境下才好。对结构、热力学数据、结合能重现得都好。

金属

磷脂膜模拟的力场

Gromos96: rtp 本身自带了 DPPC 参数, 结果不好。

CHARMM27 及改进版 CHARMM36c: 专门且常用的膜力场。

Glycam06: 支持了少数磷脂分子, 非主流。

GAFF: GAFF 力场没有膜的参数, 直接用在膜模拟效果不好。

Lipid11(2012): Skjevik 提出的膜力场, 作为 amber 系列力场的扩展, 参数来自 GAFF, 几种头部(PC,PE,PS,PH,P2,PGR,PGS,PI)和几种尾部可以自由搭配(模块化)组成磷脂, 还支持胆固醇, 完全兼容 amber 力场, leap 已支持。非主流。
Dickson(2012)的 GAFFlipid 力场只是一个阶段性的膜力场, 将会被融合进 Lipid11。
Berger(1997): 联合原子膜力场。成键参数基于 GROMOS87, LJ 参数基于 OPLS-UA, 适合搭配 Gromos87, 很常用也很好, 几乎是唯一致命的问题在于不直接兼

容 Gromos96, 若搭配 OPLSAA 需要很留神。虽然也有一些人结合 Gromos96 来模拟膜蛋白, 但终究比较古怪, 需谨慎。原文只给出了 DPPC 的参数, 后来又有人基于此弄了其它磷脂的。Berger 本身没直接提供参数和拓扑文件, Peter Tieleman 基于 Berger 的参数制作了 DPC、POPC、DPPC、DMPC、DLPC、DOPC、PLPC、POPE 的 itp 文件, 都需要 lipid.itp 中的参数, 可以在这里下载:

<http://wcm.ucalgary.ca/tieleman/downloads>

G43A1-S3 (2006): Chiu 弄的兼容 Gromos43A1 的膜力场。支持 PC/PE/sphingomyelin 和 cholesterol。此力场的 POPC 不建议使用。

Kukol(2009): 完全兼容 Gromos96 G53A6 的膜力场, 烷烃链是联合原子, 结果很好, 和 Berger 相仿佛, 弥补了它不支持 Gromos96 的遗憾。拓扑文件从原文的补充材料里得到。包含 DPPC、DMPC、POPG、POPC、DMPC 的参数。此力场的 POPC 不建议使用。

DAVID POGER(2010): 完全兼容 gromos96 G53A6 的膜力场。JCC 的文章中只提出了 DPPC 的参数, JCTC 的文章中还提出了 DLPC、DMPC、DOPC、POPC 的参数。网址和 gmx 的拓扑文件:

<http://compbio.chemistry.uq.edu.au/~david/research/lipids.htm>

Stockholm lipids (Slipids) (2012): Jambeck 弄的全原子膜力场。兼容 amber。支持 DPPC、DLPC、DMPC、POPC、DOPC、SOPC、POPE、DOPE、sphingomylin、PG 和 PS 头部集团、胆固醇。gromacs 的拓扑文件和预平衡的结构从这里下:

http://people.su.se/~jjm/Stockholm_Lipids/Downloads.html

MARTINI : 粗粒化。网址和 gmx 的拓扑文件:

<http://md.chem.rug.nl/cgmartini/index.php/downloads>

Lipidbook 汇总了各种膜力场的参数: <http://lipidbook.bioch.ox.ac.uk>

ATB 带了几十种兼容 gromos96 的膜参数(可能对应的 gromos96 版本不同), ATB 也能自动生成新的磷脂的参数。ATB 网址和 gmx 的拓扑文件:

<http://compbio.biosci.uq.edu.au/atb/>

测试总结(<http://md.chem.rug.nl/cgmartini/index.php/blog/269-jungle2>): slipid 最好, CHARMM36 其次, berger 很好, Kukol 和 poger 说得过去, 后者貌似相对略好。磷脂平均表面积、平均体积、头部集团距离都很容易算准, 所有膜力场都没问题,

但扩散能力、等热表面可压缩率大多膜力场难以算准。值得一提的是，一种膜力场当中，某种磷脂算得好绝不代表另外磷脂就算得好。

金属晶体和非金属晶体

这两类物质具有特殊的结构，研究者为它们开发了额外的势函数，称为多体势函数，来描述原子之间的作用。所谓多体就是说两个原子之间的相互作用不仅取决于这两个原子还与其周围的原子有关系。针对某种物质研究者已经开发出相应的力场参数，只需要搜索一般就能找到对应的势函数参数或文件。我对这些势函数不太了解，就不在这里赘述了。

以上就是关于势能和势函数的内容。需要指出的是为了提高模拟精度，研究者还针对开发出了针对某一种物质的势函数。当你要模拟这些物质时，直接查找原始文件中的参数就行。关于力场的选择，当然是选针对要研究体系的力场。但是对于 LAMMPS 来说建模是个很麻烦的事。如果你能找到一种力场描述的体系，并且成功建模，那你就偷着乐吧。不同力场之间的差别并不是那么明显。事实上，分子动力学模拟最大的价值就在于给出研究对象的一般性规律和背后的机理，而不是计算出某些精确的数值。而大多数力场能够捕捉研究对象的本质特征，这就基本上能够保证这个力场能够再现研究对象的本质规律，这对于我们来说就已经足够了。当然，力场的开发一直在发展中以越来越精确的描述研究对象。比如几年来发展出来的机器学习力场就能够以量子的精度开展分子动力学模拟。这是分子动力学模拟方法中的一大进展。当你要验证一个力场的时候，计算研究对象你所关心的物理量的数值和实验或者文献对比，如果接近，力场就能用。

1.3 温度、压强

温度对于分子动力学模拟是一个很重要的物理量，因为分子动力学一般用来模拟凝聚态，如果温度过高就气化了。初中物理告诉我们温度是原子热运动剧烈程度的度量。原子热运动越剧烈，温度越高。也就是说温度和原子的热运动速度是直接相关的。根据统计力学三维系统中温度和原子速度具有以下对应关系

$$\frac{3}{2}Nk_B T = \sum_{i=1}^N \frac{1}{2}m_i v^2$$

其中， N 是原子总个数， k_B 是玻尔兹曼常数。通过调整原子的速度就可以实现控制系统的温度。

压强是另一个重要的物理量。压强在分子动力学中这样定义的

$$P = \frac{Nk_B T}{V} + \frac{\sum_i^{N'} r_i \cdot f_i}{3V}$$

其中, V 是系统体积。第一项来自原子热运动的贡献。第二项来自原子之间相互作用的贡献, 称为维里项。通过调整系统的体积和原子的位置就可以实现控制系统的压强。对于液体来说压强就等于我们平时所接触到的压强。而对于固体来说这里的压强实际上就是内应力。事实上, 压强其实就是系统内部应力张量的对角线分量。压强和温度都是强度量, 就是说其值与系统规模无关。与之相对应的是广延量, 比如能量, 当系统规模增大时, 系统能量增加。

更多分子动力学的细节将结合 LAMMPS 使用过程进行介绍。

1.4 牛顿方程的数值解法和周期性边界条件

设一个对象或系统由 n 个原子构成, 这 n 个原子的位置为 $\mathbf{r}=(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_n)$, 速度为 $\mathbf{v}=(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n)$ 以及质量为 $\mathbf{m}=(m_1, m_2, \dots, m_n)$ 。原子受到的力为其总势能的梯度, 由此根据牛顿第二定律建立任一原子的运动方程为

$$m_i \ddot{\mathbf{r}}_i = -\nabla U_i$$

对于有 n 个原子的系统就有 n 个运动方程。在每一时刻对这些方程进行积分并结合初始时刻所有原子的位置和速度就可获得后续任意时刻的原子位置和速度。系统所有原子任意时刻的位置和速度称为系统原子的轨迹。基于对系统原子轨迹的分析即可获得系统的性质和属性。给出了最原始的原子运动控制方程, 该方程对应统计力学中的微正则系综。

对控制方程采用速度-Verlet 法, 该方法在不同的时间步计算原子的位置和速度, 即在 $t+h/2$ 时刻更新原子的速度, 然后在 $t+h$ 更新原子位置, 最后再更新在 $t+h$ 的速度。首先将 $\dot{\mathbf{r}}(t)$ 在 $t+h/2$ 和 $t-h/2$ 时刻进行泰勒展开, 有

$$\dot{\mathbf{r}}(t+h/2) = \dot{\mathbf{r}}(t) + h/2 \ddot{\mathbf{r}}(t) + (h^2/4) \dddot{\mathbf{r}}(t) + O(h^3/9)$$

$$\dot{\mathbf{r}}(t-h/2) = \dot{\mathbf{r}}(t) - h/2 \ddot{\mathbf{r}}(t) + (h^2/4) \dddot{\mathbf{r}}(t) - O(h^3/9)$$

用两式相减可得 $t+h/2$ 的速度,

$$\dot{\mathbf{r}}(t+h/2) = \dot{\mathbf{r}}(t-h/2) + h \ddot{\mathbf{r}}(t)$$

由将上式重新改写为

$$\mathbf{r}(t+h) = \mathbf{r}(t) + h[\dot{\mathbf{r}}(t) + h/2 \ddot{\mathbf{r}}(t)] + O(h^3)$$

上式中 $\dot{\mathbf{r}}(t)+h/2\ddot{\mathbf{r}}(t)$ 为 $\dot{\mathbf{r}}(t+h/2)$ ，有

$$\mathbf{r}(t+h)=\mathbf{r}(t)+h\dot{\mathbf{r}}(t+h/2)$$

如果不需要 t 时刻的速度，则模拟过程中仅需存储 $t-h/2$ 时刻的速度和 t 时刻的位置，如此可减少内存使用空间。而但通常都会进一步存储 t 时刻的速度。 t 时刻的速度可由 $t-h/2$ 的速度获得

$$\dot{\mathbf{r}}(t) = \dot{\mathbf{r}}(t-h/2) + h/2\ddot{\mathbf{r}}(t)$$

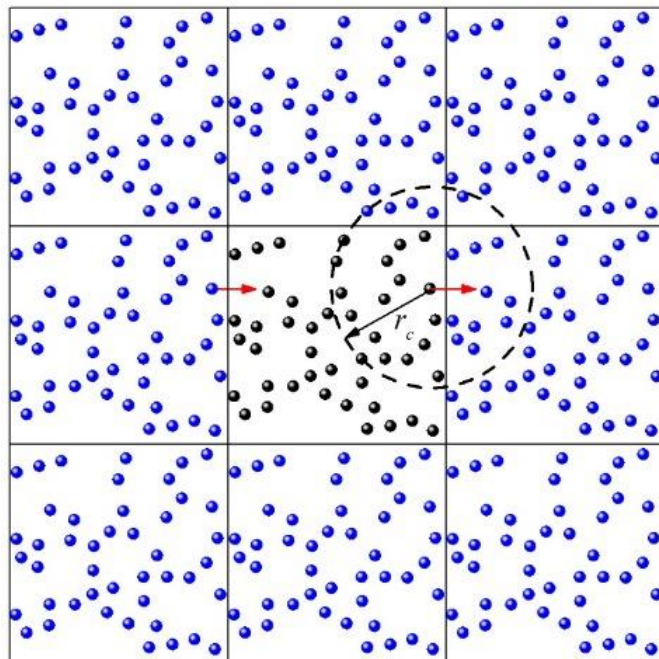
由于在实际的分子动力学模拟中，会先给定初始时刻系统内所有原子的位置和速度以及为避免在不同时间步计算位置和速度，通常采用如下更为简便的方法实施跳蛙法

$$\dot{\mathbf{r}}(t+h/2)=\dot{\mathbf{r}}(t)+h/2\ddot{\mathbf{r}}(t)$$

$$\mathbf{r}(t+h)=\mathbf{r}(t)+h\dot{\mathbf{r}}(t+h/2)$$

$$\dot{\mathbf{r}}(t+h)=\dot{\mathbf{r}}(t+h/2)+h/2\ddot{\mathbf{r}}(t+h)$$

周期性边界条件等价于在模拟区域周围放置与模拟区域完全相同的镜像。下图为二维情形下模拟区域及其周围的周期性镜像。周期性边界条件具有两个特点。第一个特点是当某个原子跨越一个边界时会立刻从对面的边界重新进入模拟区域。第二个特点与计算原子所受非键结力有关。我们知道原子间的非键结力在两个原子距离较远时可以忽略不计，因此在计算某个原子所受非键结力时，会设置一个截断半径 r_c ，即一个原子只有处于以该原子为中心， r_c 为半径的圆形区域（三维条件下为球形区域）内时才考虑这个原子对该原子的受力。周期性边界条件要求在计算边界附近原子的受力时，不仅要考虑本身模拟区域中截断半径以内的原子，还要将最近的周期性镜像区域中处于截断半径所表征区域内的原子也考虑在内，这就等价于考虑了在对面边界附近的原子。



二维周期性边界条件示意图。

在计算原子位置以及原子受力时,周期性边界条件的这两个特点在每一步计算时都要被考虑。在每一步更新原子位置后,都要对原子位置与边界位置进行比较,如果超出边界位置,则需要替换掉直接计算得到的原子位置,使其重新回到模拟区域中。以 x 方向为例,设第 i 个原子的位置为 x_i ,模拟盒子在 x 方向上的左右边界为 0 和 L_x 。每一时间步对 i 原子 x 方向上的位置检测为:如果 $x_i < 0$,那么 x_i 的值用 $x_i + L_x$ 替换;如果 $x_i \geq L_x$,那么 x_i 的值用 $x_i - L_x$ 替换。在计算受力时,周期性边界条件具有类似的影响。考虑计算 j 原子对 i 原子在 x 方向上的受力。设 d_{ijx} 为两个原子在 x 方向上的距离,如果 $d_{ijx} < -L_x/2$,那么 d_{ijx} 就用 $d_{ijx} + L_x$ 替换;如果 $d_{ijx} \geq L_x/2$,那么 d_{ijx} 就用 $d_{ijx} - L_x$ 替换。由周期性边界条件的这两个特点可知,非键结力的截断半径的大小不能超过模拟区域最小尺寸一半。此外,在实际分子动力学模拟中,当某个原子一个时间步内的在某个方向上移动距离超过了该方向上模拟区域的尺寸,此时意味着原子之间受到了非常大的力,应当终止模拟,因为模拟系统出现了偏离物理实际的动力学行为。

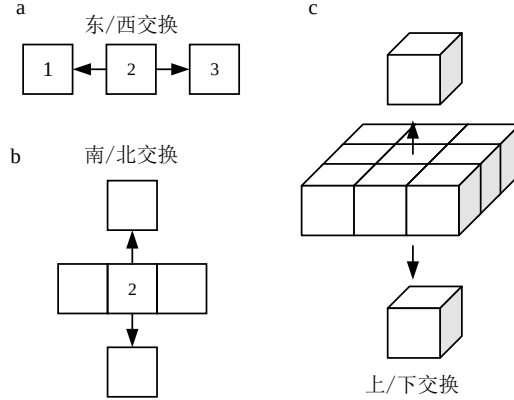
1.5 LAMMPS 软件的并行通信算法

在大多数情况下, LAMMPS 数值积分原子,分子或者宏观颗粒等粒子的牛顿运动方程。这些粒子通过短程或者长程(一般指库仑力)相互作用。为了计算效

率, LAMMPS 采用邻居列表的方法来追踪某个粒子周围对其产生相互作用的粒子。这些列表根据粒子靠近时的相互排斥的特点进行优化, 从而保证系统的某处的局部密度不会过大。在多核并行计算时, LAMMPS 采用基于空间分配技术将模拟区域分割为小的三维子区域, 然后将每一个子区域分配给一个处理器进行计算。处理器之间通过 MPI 进行消息通信并且将跨越子区域边界的原子记录为 ghost 原子以备调用。当原子跨越子区域边界后就会被分配给新的处理器。为了计算某个处理器负责的原子所受的力, 该处理仅需要掌握其周围附近原子的位置即可。因此空间分配算法处理器之间的通信是局部的, 这与基于原子分配和受力分配算法不同 (这两种算法的通信是全局的)。

此处仅介绍三维长方体模拟区域的情形。此时, 分配给每个处理的子区域的大小和形状取决于原子数 N 和处理器个数 P 以及模拟区域的长宽比。模拟区域三个方向上设定的处理器个数应当使得每个处理器负责子区域的形状越接近正方体越好。这样可以使不同处理器之间的通信量最小。在空间配算法中, 通信消耗的时间与模拟子区域的表面积成正比。在空间分配算法中, 每个处理器负责两个数据结构, 一个存储该处理负责子区域内的 N/P 个原子, 另一个存储周围子区域的原子。在第一个数据结构中, 每个处理器存储原子的完整信息, 如位置, 速度, 邻居列表等。这个数据结构以链表的形式存储, 这样就允许原子在相邻子区域交换时, 可以插入或删除链表中的原子信息数据。第二个数据结构只存储原子的位置。处理器之间在每一时间步的通信可以确保其负责的原子信息是最新的。下图为处理器与其周围处理器之间的通信示意图。第一步每个处理器与其东西相邻的处理进行通信。处理器 2 将其区域内落入处理器 1 中原子受力截断半径的原子位置存入缓存区。如果在东西方向上子区域的长度 d 小于截断半径 r_s , 则将处理器 2 的所有原子存入缓存区。现在每个处理器向西将信息传入相邻处理器(2 传到 1)并且结构来自向东方向的信息。每个处理器把接受到的信息存入第二个数据结构中。然后, 这个过程反过来, 每个处理器向东传出信息, 接受来自西边的信息。如果 $d > r_s$, 那么每个处理器在东西方向上就完成了所需交换的信息 (即原子位置)。如果 $d < r_s$, 那么东西方向上的信息传递会被重复直到所有落入相关截断半径的原子的信息传递给对应的处理器。然后类似的过程在南北方向上重复, 如图 2.4.1 (b) 所示。差别就在于发送给相邻的处理器信息不仅包含此处理器负责

子区域内的原子（在它的第一个数据结构中的原子），而且任何原子被相邻处理所需要的原子位置均需要发送。对于 $d = r_s$ ，这就相当于把三个盒子的所有原子位置放在一条信息中传递出去了，如图 2.4.1（b）所示。最后在信息传递过程在上下方向重复进行。现在原子位置一个平面 9 个盒子的信息都会被传递给相邻的处理器。



上述空间分配和消息传递算法具有以下几个特点来降低计算过程中所需要的通信。首先当 $d \geq r_s$ 时，本来一个处理器要与周围 26 个处理器进行通信，现在仅需要六次数据交换。而且，如果并行计算机是一个超立方配置的，那么模拟区域的分配可以使得所有的六个这些处理器可以直接与中心的处理器联系。这样通信就会快速而且无争用的。其次，当 $d < r_s$ 时，所需的原子信息需要来自更远处的子区域。这仅会增加少量的数据交换且这些交换也是与相邻的六个处理器进行的。这个特性使得在较小的体系下采用较多的处理器进行计算也会有很好的效率。

第三个特点是数据交换量是最少的。每个处理器所需要的只是落入这个处理器负责子区域的截断半径内的原子。第四处理器所有接受到的原子信息都可以连续的存储在该处理器的第二个数据结构中。除非需要创建拟发送的缓存信息，否则并不需要花时间重排收到的信息。最后，拟发送的缓存信息可以被很快的创建。对两个数据结构的扫描只需要隔几个时间步才进行（因为此时邻居列表会被重建）以确定哪些原子需要被发送。扫描程序会创建需要被通信的原子列表。在后续的几个时间步中，这个列表仍可以采用，从而直接索引需要的原子，快速缓存通信信息。

下图为空间分配算法 S1 的大纲。设子区域 z 分配给了处理器 P_z ，其中 z 从 0 到 $P - 1$ 。处理器 P_z 在 x_z 中存储了该处理器负责的 N/P 个原子的位置，在 f_z 中存储了这些原子的受力。步骤（1a）到（1c）是邻居列表的建立。邻居列表建立每个

几个时间步才会建立。在其他算法中用于通信的原子列表在每一时间步都需要建立。在步骤 1 (a) 中有原子从子区域 z 中移动出去, 此时需要删除这些原子在子区域 z 对应的 x_z (第一个数据结构) 中的信息, 并把这些信息存储为缓存信息。这些原子与 6 个相邻的处理器通过 Fig.2.4.1 所示的通信方式进行交换。同时, 处理器 P_z 接受到新的原子并把它们添加至 x_z 中。接着在步骤 (1b) 中, 所有子区域 z 截断半径内的原子与相邻处理器进行通信, 并将通信原子建立列表存储。

- (1a) 将需要的原子移动到新的子区域
- (1b) 建立需要被交换的原子列表
- (1c) 在子区域 z 中建立邻居列表
 - $d < 2r_s$ 全对法
 - $d \geq 2r_s$ binning 法
- (2) 在子区域 z 内计算原子受力, 重复存储受力结果
- (4) 更新子区域 z 中的原子位置和受力
- (5) 在子区域边界与相邻处理器交换原子位置
 - $d < r_s$ 发送 N/P 个位置至多个相邻处理器
 - $d \approx r_s$ 发送 N/P 个位置至最邻近处理器
 - $d > r_s$ 发送子区域表面附近的位置至最邻近处理器

当步骤 (1a) 和 (1b) 完成后, 处理器的两个数据结构都只最新的, 此时就可以建立邻居列表了。如果原子 i 和原子 j 都在子区域 z 内, 则 (ij) 原子对只需在列表中存储一次。如果原子 i 和原子 j 在两个相邻的子区域中, 则两个处理器分别存储原子 i 和原子 j 的列表。如果不这样做, 那么处理器计算的力必须互相通信。下面会阐述一个修正的算法, 避免这样两次计算原子受力。当 $d < 2r_s$ 时, 直接对子区域 z 内的每个原子针对两个数据结构扫描找到其邻居相互作用。当 $d > 2r_s$ 时, 对于围绕着子区域 z 的薄壳, 在每个方向上大概有四个或者更多的薄层。此时, 装入薄层法构建邻居列表会有更好的效率。在两个数据结构中的原子均被装入尺寸为 r_s 的区域内。在子区域 z 内的每个原子周围的区域都会被检测, 以查找可能的邻居。

处理器 P_z 现在就可以基于建立的邻居列表在步骤 (2) 中计算其负责原子的受力。当发生相互作用的两个原子都在子区域 z 内时, 受力会被存储两次, 一次是

原子 i , 一次是原子 j 。当发生作用的两个原子处在不同的子区域内时, 各个处理器会存储其各自负责原子的受力。在受力 f_z 计算完成后就可以在步骤 (4) 中更新原子的位置。最后这些更新后的原子位置必须和相邻的处理器进行通信为下一个时间步的操作准备。这发生在步骤 (5) 采用图 2.4.1 所示的通信方式并利用之间建立的列表。在这个操作中的数据交换量是受力计算的截断半径和子区域长度相对值的函数。同时, 需要指出的是在那些邻居列表已经建立的时间步中, 步骤 (5) 不是一定要执行的。

S1 算法中的通信操作发生在步骤 1 (a), 1 (b) 和 (5) 中。其中步骤 1 (b) 和 (5) 的通信是相同的。这些步骤的通信消耗的时间与数据交换量有关。对于步骤 (5), 如果我们假设原子的密度是均匀的, 那么数据量正比于数据所占的体积即, $(d + 2r_s)^3 - d^3$ 。在体积 d^3 的区域内大于有 N/P 个原子。有三种情况需要考虑。第一种, 当 $d < r_s$ 时, 来自多个周围的子区域的数据均要被考虑且这一操作量级为 $8r_s^3$; 第二种, 当 $d \approx r_s$ 时, 来自周围 26 个子区域的数据都要被考虑, 此时操作的数据量量级为 $27N/P$; 最后当 $d \gg r_s$ 时, 只有在子区域 z 六个面附近的原子需要被交换。此时交换的数据量量级比例于子区域的表面积, 也就是 $6r_s N/P^{2/3}$ 。这三种情况都列在了步骤 (5) 中。在图 2.4.2 中, 使用了 Δ 来代表在给定 N , P 和 r_s 时, 三个中的哪一个会被应用。在步骤 (1a) 中涉及到的数据交换比较少因为并非所有都会跨越子区域的边界。但是这个操作的数据量依旧比例于子区域的表面积, 因此也标示其比例于 Δ 。

在步骤 1 (c)、(2) 和 (4) 中是计算部分。所有这些操作都比例于 N/P , 除了在 1 (c) 和 (2) 中需要额外处理存储在第二个数据结构中的来自周围子区域的原子。这些原子的个数比例于 Δ , 因此也包含在了这些步骤的比例量级中。在步骤 1 (c) 和 (2) 中的主要对计算的消耗贡献的量级为 $N/2P$, 因为在盒子内部每一对原子的受力只计算和存储一次。当系统的规模也即整个模拟区域比增大时, 也即 d 相对于 r_s 增大, Δ 对计算的贡献减小, 整体计算的量级趋向于 $N/2P$ 。此时, 每个处理器主要工作于计算其内部原子的信息, 并且与周围的处理器只交换少量的信息用以更新自己边界处的原子信息。

算法 S1 的一个很重要的特点就是两个数据结构只有隔几个时间步才在重新建立邻居列表时更新。特别的, 即使原子移动超出了子区域的边界, 它也不会被

分配给新的处理器直到步骤(1a)被执行。处理器 P_z 依旧会正确的计算原子受力, 只要满足以下两个条件。第一, 在两次邻居列表被构建时, 任一个原子都不会移动超过子区域边界 d 。第二, 所有在 r_s 范围内的临近原子在每一时间步都被更新。另一种方法是讲更新后跨越边界的原子立刻交换给相邻的处理器。这样做的好处是当邻居列表没有构建时, 仅需要交换 r_c 范围的原子。这可以减少通信量, 因为 $r_s > r_c$ 。但是, 这样的话被交换原子的邻居列表的信息也需要发送。邻居列表中的信息是原子对存储其相邻原子当地内存位置的索引。如果原子被频繁的发送到新的处理器, 那么这些索引就没有意义了。为了克服这一点, 我们的操作是给每个原子分配一个全局指针, 该指针随着原子在处理器之间交换。全局指针和局部内存之间的对应必须被每个处理器存储在长度为 N 的向量中。或者, 当全局指针在邻居列表中被引用是, 这个全局指针必须被排序和检查以找到它正确对应的原子。前一个方案限制模拟体系的规模; 后一个方案需要额外的排序和排查。在算法 S1 中应用 Tamayo 和 Giles 的方法可以是代码不那么复杂, 并且降低超量的计算和通信。这不会影响大规模的体系并且会提高算法在中等规模下的表现。

利用牛顿第三定律而对 S1 算法进行改进, 改进算法称为 S2。如果处理器 P_z 只从西面, 南面和上面方向接受原子而向东面, 北面和下面送出原子, 这样相互作用只需要被计算一次, 即使两个原子在不同的子区域。这要求将计算的力从相反方向送还给负责该原子的处理器, 也即在算法中的第三步。这样操作并不会减小通信的数据量, 因为原来一半的信息会被通信两次, 但是这确实避免了对跨越两个盒子的相互作用的重复计算。关于这个需要指出两点。第一, 从整体来看算法 S2 比 S1 节省的时间并不多, 特别是对较大的体系。因此只有在(1c)和(2)步节省了 Δ 项的时间。第二, 对于大规模的体系, 空间分配算法可以通过优化单个处理器计算力的效率来提升算法的效率。在矢量机器上, 对于计算力和构建邻居列表的过程中需要特别注意数据结构和循环的顺序来获得高效的浮点率。实施 S2 算法需要额外注意子区域边界和角落处的原子来保证这些原子的受力只被计算一次, 而这会妨碍单处理器的优化。

最后一个问题是在空间分配算法中负载平衡很重要。当模拟区域内的原子数密度大致均匀分布时, 算法 S1 的负载才会平衡。如果模拟区域的形状不是长方体, 那么很难讲模拟区域均匀的分成多份。此时, 更复杂的算法被开发出来, 来

应对非长方体的模拟区域和原子密度非均匀分布的体系，但是这会使得子区域不是长方体或者以非长方体的方式连接相邻的子区域。这些情况都会使得向子区域分配原子和相邻子区域的通信变得低效和复杂。如果在分子动力学模拟过程中，体系的物理密度会一直发生改变，那么负载平衡问题是复杂的。任何动态的负载平衡都要求额外的计算资源和数据迁移。

空间分配算法是最高效的，但是其效率依赖于模拟的几何形状并且在具体实施时程序也是较复杂的。

2. Linux 系统常见命令和操作

LAMMPS 是在针对 Linux 系统开发的。虽然 Windows 系统也能用但是会出现很多问题，所以强烈推荐你使用 Linux 系统运行 LAMMPS。要搞模拟你一定要熟悉 Linux 的命令行操作逻辑，要习惯使用脚本来写 LAMMPS 命令。很多软件包括开源的和商业的都是采用脚本控制的方式来执行的。

在 Linux 系统中使用最多的不是鼠标点点点，而是在终端中输入命令行来执行操作。执行一条命令是指在终端中输入该命令，然后回车执行。在 Ubuntu 系统中可以使用 `Ctrl+Alt+T` 来打开终端，其他 Linux 操作系统可以在软件列表中点击打开终端。这里只列举最常用的命令。

`cd` 命令用来进入某个目录，比如 `cd /home` 就是进入 `home` 文件夹。在 Linux 中，`.` 表示当前目录，`..` 表示上一级目录。`ls` 命令列出当前目录下的所有文件和文件夹。`pwd` 命令返回当前目录的路径。`mkdir` 创建文件夹，如 `mkdir software` 表示在当前目录下创建 `software` 文件夹，`mkdir /home/softcare`，表示在 `/home` 文件夹下创建 `software` 文件夹。`rm` 命令删除文件或文件夹。`tar` 解压缩命令当压缩文件的扩展名是 `.tar.gz` 时可以使用 `tar -xvzf` 压缩文件名，将压缩文件解压。`which` 命令，查找可执行文件的路径，如 `which lmp_g++_mpich` 就是查找可执行文件 `lmp_g++_mpich` 的所在路径，如果返回路径那么这个命令就可以被系统找到，如果什么也没返回就是找不到。`echo` 命令返回变量的值，如 `echo $PATH` 就是返回 `PATH` 的值。环境变量是指系统或软件定义的变量，一般用于文件查找。最常见的环境变量就是 `PATH`，该变量记录了众多可执行文件的位置。如果一个可执行文件的路径被添加到 `PATH` 中了，那么在运行该文件时可以直接在终端中写该文件的名字，如果没有被添加那么就要写出该文件的绝对路径才能执行该文件。将

一个路径添加到 PATH 中需要修改 .bashrc 文件。输入执行 `vim ~/.bashrc`, 就可以打开 .bashrc 文件, 按 `i` 键进入编辑模式, 将光标定位在文件末尾并添加以下一行 `PATH=$PATH:<路径名>`, 按 `esc` 键退出编辑模式, 输入 `:wq` 后回车, 这样就保存好了刚才的编辑, 并退出了 .bashrc 文件回到了终端。在终端输入 `source ~/.bashrc` 就可以是刚才的操作发生作用。sudo 是临时取得 root 权限命令, 当你要在系统目录中添加或修改文件的时候就需要把 sudo 加到执行命令前面, 如 `sudo cp ./lmp /usr/bin` 就是把当前目录下的 lmp 文件拷贝到系统目录 /usr/bin 中。带有 sudo 的命令回车执行后需要输入用户密码, 当你在输入密码的时候终端上不会显示密码, 但是确实是输进去了, 输完回车如果密码正确就开始执行命令了。其他命令可以百度能够找到大量资料。当你在使用 Linux 系统时需要问一个问题, 你就百度, 一定能找到答案。

3. LAMMPS 介绍和并行版安装 (Linux 版)

3.1 LAMMPS 介绍

LAMMPS 是一种基于经典力学的分子动力学 (MD) 代码, 用于模拟液态、固态或气态的粒子系统。它可以使用各种原子间势函数 (力场) 和边界条件, 对原子、聚合物、生物、固态 (金属、陶瓷、氧化物)、粒状、粗粒化或宏观系统进行模拟。它可以对几千到几百万甚至数十亿粒子的 2d 或 3d 系统进行模拟。

LAMMPS 可以在笔记本电脑或台式机上编译和运行, 但其是专为并行计算机而设计的。它可以在支持 MPI 消息传递库的任何并行机器上运行。这包括共享内存、分布式内存集群和超级计算机。部分 LAMMPS 功能还支持 OpenMP 多线程、矢量化和 GPU 加速。

LAMMPS 是用 C++ 编写的, 需要一个至少与 C++-11 标准兼容的编译器。早期版本是用 F77、F90 和 C++-98 编写的。所有版本都可以以源代码形式在 LAMMPS 网站下载。

LAMMPS 旨在易于修改或扩展新功能, 例如新的力场、原子类型、边界条件或操作。在最一般的意义上, LAMMPS 将一组相互作用的粒子的牛顿运动方程进行积分。单个粒子可以是原子或分子或电子, 粗粒原子簇, 或介观或宏观材料团块。LAMMPS 包含的相互作用大多是短程的, 但还包括一些长程作用。

LAMMPS 使用邻居列表来跟踪附近的粒子。这些列表针对具有短距离排斥

粒子的系统进行了优化，因此粒子的局部密度永远不会变得太大。这与用于模拟等离子体或引力体（例如星系形成）的方法形成对比。

在并行机器上，LAMMPS 使用空间分解技术将模拟域划分为计算成本相等的子区域，其中每个子区域分配给每个处理器。处理器与其子域边界的原子通信并存储“ghost”原子信息。此外，还可以使用多线程并行化和带有粒子分解的 GPU 加速。

3.2 LAMMPS 能做的

一般功能

- 以单核或多核并行方式运行
- 分布式内容并行通信（MPI）
- 共享式内容多线程并行计算（OpenMP）
- 模拟区域空间分解 MPI 并行计算
- 粒子分解的 OpenMP 和 GPU 并行计算
- GPLv2 开放获取
- 高度可迁移的 C++-11 代码
- 功能模块化设计
- 只依赖于 MPI 库
- 其他库只有在部分可选的功能模块中需要
- GPU (CUDA, OpenCL, HIP, SYCL), Intel Xeon Phi, and OpenMP 支持的多核特性
- 非常容易添加或扩展新功能
- 通过一个 in 文件（文本文件）来运行
- in 文件语法支持使用变量和公式
- in 文件语法支持循环和退出循环
- 在一个 in 文件中以并行的方式同时运行一个或多个模拟
- 可以编译为一个库，通过库接口调用 LAMMPS 和提供一个 Python 包装器
- 与其他代码耦合：LAMMPS 调用其他代码，其他代码调用 LAMMPS，或者互相调用

粒子类型

- 原子
- 粗粒化粒子 (如珠子-弹簧高分子)
- 联合原子高分子或有机分子
- 全原子高分子, 有机分子, 蛋白质, DNA
- 金属
- 颗粒材料
- 粗粒化介观模型
- 有限尺寸的球或椭球粒子
- 有限尺寸的线段 (二维) 或三角形颗粒 (三维)
- 有限尺寸的圆角多边形 (二维) 和多面体 (三维) 粒子
- 点偶极矩粒子
- 磁自旋粒子
- 多个粒子构成的刚体
- 上述模型的组合

相互作用势能 (力场)

- 对势: Lennard-Jones, Buckingham, Morse, Born-Mayer-Huggins, Yukawa, soft, class 2 (COMPASS), hydrogen bond, tabulated
- 带电对势: Coulombic, point-dipole
- 多体势: EAM, Finnis/Sinclair EAM, modified EAM (MEAM), embedded ion method (EIM), EDIP, ADP, Stillinger-Weber, Tersoff, REBO, AIREBO, ReaxFF, COMB, Streitz-Mintmire, 3-body polymorphic, BOP, Vashishta
- 机器学习势: SNAP, GAP, ACE, N2P2, RANN, AGNI
- charge, point-dipoles 和 LJ dispersion 长程相互作用: Ewald, Wolf, PPPM (similar to particle-mesh Ewald), MSM
- 极化模型: QEq, core/shell model, Drude dipole model
- 电荷平衡: QEq via dynamic, point, shielded, Slater methods
- 粗粒化势: DPD, GayBerne, REsquared, colloidal, DLVO
- 介观势: granular, Peridynamics, SPH, mesoscopic tubular potential (MESONT)

- 半经验势: multi-ion generalized pseudopotential theory (MGPT), second moment tight binding + QEq (SMTB-Q), density functional tight-binding (LATTE)
- 电子力场 (eFF, AWPMD)
- 键势: harmonic, FENE, Morse, nonlinear, class 2, quartic (breakable), tabulated
- 键角势: harmonic, CHARMM, cosine, cosine/squared, cosine/periodic, class 2 (COMPASS), tabulated
- 二面角势: harmonic, CHARMM, multi-harmonic, helix, class 2 (COMPASS), OPLS, tabulated
- 离平面势: harmonic, cvff, umbrella, class 2 (COMPASS), tabulated
- 高分子势: all-atom, united-atom, bead-spring, breakable
- 水分子势: TIP3P, TIP4P, SPC, SPC/E and variants
- 石墨烯和同类物的层间势
- 金属有机框架势 (QuickFF, MO-FF)
- 隐式溶剂势: hydrodynamic lubrication, Debye
- 与 CHARMM, AMBER, DREIDING, OPLS, GROMACS, COMPASS 兼容的势
- 通过 kim 命令访问 OpenKIM 势库
- 组合势: 多种势在一个模拟中组合
- 叠加势: 多种势可以在一个模拟中叠加并可以分配权重

创建原子

- 从一个文件中读取原子坐标
- 从一个或多个晶格参数中创建原子
- 根据分组或几何区域删除原子
- 多次复制已有模型
- 移动或旋转原子

系综, 约束和边界条件

- 二维或三维系统 2d or 3d systems
- 正交或非正交模拟盒子
- NVE, NVT, NPT, NPH, Parrinello/Rahman 积分器
- 对特定组或几何区域中原子进行温度控制
- 通过 Nose/Hoover 或 Berendsen 在 1 到 3 维进行压强控制
- 变形盒子 (拉伸或剪切)
- 弹簧 (伞状) 受力约束
- 刚性体约束
- 键或键角 SHAKE 约束
- 流形表面的运动约束
- 蒙特卡洛键断裂、形成、交换、基于模板的反应建模
- 原子/分子插入和删除
- 各种各样的, 静态的和移动的虚拟墙
- 非平衡分子动力学 (NEMD)
- 各种附加边界条件和约束

积分器

- velocity-Verlet 积分器
- 布朗动力学
- 刚体积分
- 通过共轭梯度或最速下降松弛的能量最小化
- rRESPA 分级时间步长
- 重新运行输出文件的后处理命令

输出

- 用以热力学信息的日志输出
- 原子坐标速度
- 二进制重启文件
- 并行输出轨迹文件或重启文件
- 单原子量
- 用户定义的系统量或单原子量

- 自定义的分区统计
- 空间时间平均的单原子量
- 时间平均或直方图的系统量
- 内置的, XYZ, XTC, DCD, CFG 格式的原子快照

多副本模型

- nudged elastic band
- hyperdynamics
- parallel replica dynamics
- temperature accelerated dynamics
- parallel tempering
- path-integral MD: first variant <fix_pimd>, second variant <fix_ipi>
- multi-walker collective variables with Colvars and Plumed

特殊功能

LAMMPS 可以使用可选包编译, 这些包实现了各种附加功能。这些是你可能认为不是典型经典 MD 内容的 LAMMPS 功能:

- 静态和动态负载平衡, 可选递归二等分分解
- 一般化的非球形颗粒
- 随机旋转动力学 stochastic rotation dynamics (SRD)
- 实时可视化和交互式 MD, 内置图像和视频渲染器
- 计算虚拟衍射图案
- 计算有限温度声子色散和最小化结构的动力学矩阵
- 与有限元联用的原子到连续介质耦合
- 通过 POEMS 库进行耦合刚体积分
- QM/MM 耦合
- 通过 GCMC 和 tfMC 以及原子交换的蒙特卡罗
- 路径积分分子动力学
- 低密度流体的直接模拟蒙特卡罗
- 近场动力学模拟
- 格子玻尔兹曼流体

- 靶向和导向分子动力学
- 双温电子模型

3.3 LAMMPS 不能做的

LAMMPS 旨在成为用于分子动力学 (MD) 模拟的快速并行引擎。它仅提供少量功能来设置模拟和分析其输出。具体来说, LAMMPS 的构思和设计目的不是:

- 通过 GUI 运行
- 构建分子系统或构建分子拓扑结构
- 自动分配力场参数
- 对 MD 模拟执行复杂的分析
- 以交互方式可视化 MD 模拟
- 绘制你的输出数据

多年来, 通过添加到 LAMMPS 的功能或与 LAMMPS 紧密斜街或扩展外部工具, 上述限制中的一些已经减少或消除。

如何执行上述任务可以考虑以下建议:

GUI: LAMMPS 可以构建为一个库, 并且提供了一个包装库接口的 Python 包装器。因此, GUI 界面可以用运行 LAMMPS 并可视化或绘制其输出的 Python (或 C 或 C++, 如果需要) 编写的功能。这方面的例子在 `python` 目录中提供, 并在 Python 文档页面上进行了描述。此外, 还有几个 GUI 前端。

建模: 几个预处理工具与 LAMMPS 一起分发。其中一些将由其他 MD 代码 (如 CHARMM、AMBER 或 Insight) 生成的输入文件转换为 LAMMPS 输入格式。其中一些是简单的程序, 可以构建简单的分子系统, 例如线性珠状弹簧聚合物链。

moltemplate 程序是一个真正的分子构建器, 可以生成复杂的分子模型。

LAMMPS 网站的前/后处理页面描述了用于此任务的各种第三方工具。此外, 一些 LAMMPS 内部命令允许重建或选择性地添加拓扑信息, 并提供插入分子模板而不是原子以构建大分子系统的选项。

力场分配: 假设提供 CHARMM、AMBER 或 BIOVIA (前身为 Accelrys) 力场文件, 上面描述的 CHARMM、AMBER 和 Insight 转换工具也可以 LAMMPS 的格式分配力场参数。ParmEd 和 InterMol 工具在各种 MD 模拟程序之间转换力

场和拓扑数据方面特别强大和灵活。

模拟分析: 对于后处理, 可以将快照等 LAMMPS 输出转换为其他 MD 或后处理代码使用的格式。在某种程度上, 这种转换可以通过连接到 VMD molfile 插件直接在 LAMMPS 内部完成。rerun 命令还允许对现有轨迹进行一些后处理, 并且通过能够读取各种文件格式, 这也可以用于分析来自其他 MD 代码的轨迹。一些与 LAMMPS 一起打包的后处理工具会进行这些转换。tools/pytho 目录中提供的脚本可以提取和处理转储文件中的数据, 以便更轻松地导入其他程序。

可视化: LAMMPS 可以通过其输出图像命令即时生成 NETPBM、JPG 或 PNG 快照图像, 并将它们传递给外部程序 FFmpeg 以从中生成视频。对于高质量的交互式可视化, 有许多优秀的免费工具可供使用。

3.4 LAMMPS 编译过程

这里介绍在 ubuntu 在编译并行版 LAMMPS 并执行。首先假设你已经有了一台安装了 ubuntu 系统的电脑。你的用户名叫 me。电脑联网。分别在 LAMMPS 官网, MPICH2 官网和 FFTW 官网下载对应的 lammps, mpich 和 fftw 的最新版源代码, 然后

第一步按 Ctrl+Alt+T 打开一个终端, 输入执行以下命令

```
sudo apt-get update
```

```
sudo apt-get install build-essential
```

```
sudo apt-get install vim
```

第二步输入执行

```
mkdir /home/me/software
```

```
mkdir /home/me/software/mpich
```

```
mkdir /home/me/software/fftw
```

第三步将下载好的三个源代码的压缩文件拷贝进/home/me/software 目录下

第四步输入执行

```
tar -xvf <lammps 压缩文件的名称>
```

```
tar -xvf <mpich 压缩文件的名称>
```

```
tar -xvf <fftw 压缩文件的名称>
```

第五步进入解压出来的 mpich 的目录执行

```
./configure --prefix=/home/me/software/mpich --disable-fortran
```

make

make install

第六步进入解压出来的 fftw 的目录执行

```
./configure -prefix=/home/me/software/fftw
```

make

make install

第七步输入执行

```
vim ~/.bashrc
```

打开.bashrc 文件后按 i 键进入边界模式，将光标定位在文件末尾并添加以下一行

```
PATH=$PATH:/home/me/software/mpich/bin
```

按 esc 键退出编辑模式，输入:wq 后回车，这样就保存好了刚才的编辑，并退出了.bashrc 文件回到了终端。在终端输入

```
source ~/.bashrc
```

```
which mpirun
```

若输出了 mpirun 的目录则 mpich 安装配置成功

第八步终端输入执行以下命令

```
cd /home/me/software/解压出来 lammmps 文件夹名称/src
```

第九步打开目录 /home/me/software/ 解压出来 lammmps 文件夹名称 /src/MAKE/OPTIONS 下的 Makefile.g++_mpich 文件。在文件以下位置修改为

```
MPI_INC = -DMPICH_SKIP_MPICXX -DOMPI_SKIP_MPICXX=1 -I/home/me/software/mpich/include
```

```
MPI_PATH = -L/home/me/software/mpich/lib
```

```
MPI_LIB =
```

```
FFT_INC = -I/home/me/software/fftw/include
```

```
FFT_PATH = -L/home/me/software/fftw/lib
```

```
FFT_LIB =
```

保存退出。

第十步在终端依次执行

make yes-molecule

make yes-kspace

make yes-rigid

make yes-manybody

make g++_mpich

等待编译完成 src 目录下会生成 `lmp_g++_mpich` 的可执行文件, 将此可执行文件拷贝至你的 in 文件所在的位置输入

`mpirun -np <并行核数> ./lmp_g++_mpich -in <in 文件名字>`

运行正常结束则编译成功。

```
LAMMPS (1 Feb 2014)
Reading data file ...
  orthogonal box = (-0.0319996 0.00499916 -1.5) to (149.733 149.77 90)
  1 by 1 by 1 MPI processor grid
  reading atoms ...
  49032 atoms
  reading velocities ...
  49032 velocities
  scanning bonds ...
  2 = max bonds/atom
  scanning angles ...
  1 = max angles/atom
  reading bonds ...
  11520 bonds
  reading angles ...
  5760 angles
Finding 1-2 1-3 1-4 neighbors ...
  2 = max # of 1-2 neighbors
  1 = max # of 1-3 neighbors
  1 = max # of 1-4 neighbors
  2 = max # of special neighbors
17280 atoms in group spce
31752 atoms in group SiO2
```



4. LAMMPS 模拟基本流程

4.1 LAMMPS 的 in 文件简介

LAMMPS 通过从输入脚本（文本文件）读取命令来执行计算，一次一行。当输入脚本结束时，LAMMPS 退出。这与在开始计算之前读取和处理整个输入的程序不同。

每个命令都会导致 LAMMPS 立即采取一些行动，而不考虑以后可能处理的任何命令。命令可以设置内部变量、读入文件或运行模拟。这些动作可以分为三类：

- 更改全局设置的命令（例如：`timestep`、`newton`、`echo`、`log`、`thermo`、`restart`），
- 添加、修改、删除或替换“run”期间执行的“style”的命令（例如：`pair_style`、

`fix`、`compute`、`dump`、`thermo_style`、`pair_modify`), 以及

- c) 执行“`run`”或执行某些其他计算或操作的命令(例如: `print`、`run`、`minimize`、`temper`、`write_dump`、`rerun`、`read_data`、`read_restart`)

a) 类中的命令具有默认设置, 这意味着只需要在希望更改默认设置时使用该命令。

在许多情况下, 输入脚本中命令的顺序并不重要, 但当全局状态在 c)类别中的命令之间发生更改时, 可能会产生影响。以下规则适用:

LAMMPS 不会读取您的整个输入脚本, 然后使用所有设置执行模拟。相反, 输入脚本一次读取一行, 每个命令在读取时生效。因此这个命令序列:

```
timestep 0.5
```

```
run 100
```

```
run 100
```

与下面这个命令是不同的:

```
run 100
```

```
timestep 0.5
```

```
run 100
```

在第一种情况下, 指定的时间步长 (0.5 fs) 用于两次模拟, 每次 100 个时间步长。

在第二种情况下, 第一个 100 步模拟使用默认时间步长 (1.0 fs), 第二个使用 0.5 fs 时间步长。

有些命令只有在跟随其他命令时才有效。例如, 在定义原子并使用 `group` 命令定义哪些原子属于该组之前, 您不能设置一组原子的温度。

有时, 命令 B 将使用可以由命令 A 设置的值。这意味着如果要产生所需的效果, 命令 A 必须在输入脚本中的命令 B 之前。例如, `read_data` 命令通过设置模拟盒子并将原子分配给处理器来初始化系统。如果不需要默认值, 则需要在 `read_data` 之前使用处理器和边界命令来告诉 LAMMPS 如何将处理器映射到模拟盒子。

LAMMPS 检测到许多输入脚本错误并打印错误或警告消息。每个命令的文档都列出了如何使用该命令的限制。

您可以使用 `-skiprun` 命令行标志让 LAMMPS 跳过任何“`run`”、“`minimize`”或类

似命令的执行, 以检查整个输入的语法是否正确, 以避免在长时间运行时因拼写错误或语法错误而崩溃。

4.2 in 文件的命令解析规则

输入脚本中的每个非空行都被视为一个命令。LAMMPS 命令区分大小写。命令名称是小写的, 指定的命令参数也是如此。大写字母可用于文件名或用户选择的 ID 字符串。

以下是 LAMMPS 如何解析输入脚本中的每一行的 6 条规则:

1. 如果该行的最后一个可打印字符是“&”字符, 则假定该命令在下一行继续。通过删除“&”字符和换行符将下一行连接到上一行。这允许长命令跨两行或多行继续。
2. 从第一个“#”字符开始的所有字符都被视为注释并被丢弃。此规则的例外情况在 6 中进行了描述。请注意, 尾部“&”字符后的注释将阻止命令在下一行继续。另请注意, 对于多行命令, 单个前导“#”将注释掉整个命令。
3. 重复搜索该行以查找\$字符, 这些字符表示用文本字符串替换的变量。该规则的例外情况在 6 中描述。如果\$后跟大括号“{}”中的文本, 则变量名是大括号内的文本。如果\$后面没有大括号, 则变量名是紧跟\$的单个字符。因此, `${myTemp}` 和 `$x` 指的是名为“myTemp”和“x”的变量, 而“\$xx”将被解释为一个名为“x”的变量后跟一个“x”字符。

变量如何转换为文本字符串取决于它是什么类型的变量; 它可以是存储多个文本字符串的变量, 并返回其中一个。返回的文本字符串可以是多个“单词”(空格分隔), 然后将其解释为输入命令中的多个参数。该变量还可以存储一个数字公式, 该公式将被计算并将其数字结果作为字符串返回。

作为一种特殊情况, 如果 \$ 后跟括号“()”, 则括号内的文本将被视为“立即”变量并作为等式变量进行评估。这是一种在输入脚本中使用数字公式而无需将它们分配给变量名称的方法。例如, 这 3 个输入脚本行:

```
variable X equal (xlo+xhi)/2+sqrt(v_area)
region 1 block $X 2 INF INF EDGE EDGE
variable X delete
```

可以替换为:

```
region 1 block $((xlo+xhi)/2+sqrt(v_area)) 2 INF INF EDGE EDGE
```

这样您就不必定义（或丢弃）临时变量，在这种情况下为“X”。

此外，“立即”变量表达式后面可以跟一个冒号，后面跟一个 C 风格的格式字符串，例如“:%f”或“:%.10g”。格式字符串必须适合双精度浮点值。格式字符串用于输出变量表达式求值的结果。如果未指定格式字符串，则默认使用高精度“%.20g”。这对于将打印输出格式化为所需的精度非常有用：

```
print "Final energy per atom: $(pe/atoms:%10.3f) eV/atom"
```

请注意，大括号或立即形式的变量都不能包含嵌套的\$字符以供其他变量替换。

因此你不能这样做：

```
variable      a equal 2
variable      b2 equal 4
print         "B2 = ${b$a}"
```

也不能为立即变量指定像“\$(x-1.0)”这样的表达式，但可以使用 \$(v_x-1.0)，因为后者是等式变量的有效语法。

4.一行命令被分成由空格（制表符、空格）分隔的“单词”。请注意，单词因此可以包含字母、数字、下划线或标点字符。

5.第一个词是命令名称。该行中所有连续的单词都是参数。

6.如果希望将带有空格的文本视为单个参数，则可以将其括在单引号或双引号或三引号中。如果使用“&”字符，则用单引号或双引号括起来的长单个参数可以跨越多行，如上所述。当这些行连接在一起时（并且“&”字符和换行符被删除），文本将变成单行。如果希望参数的多行保留其换行符，则可以将文本括在三引号中，在这种情况下不需要“&”字符。例如：

```
print "Volume = $v"
print 'Volume = $v'
if "${steps} > 1000" then quit
variable a string "red green blue &
                    purple orange cyan"
print ""
System volume = $v
System temperature = $t
""
```


在每种情况下，当单引号、双引号或三引号括起来的单个参数存储在内部时，它们都会被删除。引号之间的“#”或“\$”字符在 2 中不会被视为注释指示符，也不会被替换为 3 中的变量。

注意

如果参数本身是一个需要带引号的参数的命令（例如，使用 `print` 命令作为 `if` 的或 `run every` 命令的一部分），则可以以通常的方式嵌套单引号、双引号或三引号。只允许嵌套一级，但这对于大多数用例来说应该足够了。

ASCII 与 UTF-8

LAMMPS 在内部需要并处理 7 位 ASCII 格式的文本。许多现代环境使用 UTF-8 编码，它是 7 位 ASCII 字符表的超集，因此大多兼容。但是，有几个非 ASCII 字符可能看起来与它们的 ASCII 等效字符非常相似或不可见（因此它们看起来像一个空白），但编码不同。众所周知，Web 浏览器、PDF 查看器、文档编辑器有时会将其中一个替换为另一个以获得更好看的输出。但是，这可能会导致问题，例如，当使用来自网页的输入文件示例的剪切-粘贴时，或者当使用文档编辑器（不是专用的纯文本编辑器）来编写 LAMMPS 输入时。LAMMPS 将尝试检测这一点，并将非 ASCII 字符替换为已知的 ASCII 等效字符。如果发生这种情况，还会打印警告。建议在 LAMMPS 输入、数据和潜在文件中完全避免使用此类字符。

4.3 LAMMPS 的 in 文件基本内容（很重要必须认真读）

LAMMPS 的模拟需要写一个脚本，称之为 in 文件。我将一个典型的 in 文件书写分为六个部分：1、定义模拟的基本规则，2、定义对象的几何模型，3、定义对象的力场参数，4、定义分组信息，5、弛豫部分，6、正式模拟过程。

第一部分 定义模拟的基本规则

根据前面说的分子动力学原理，那么第一步就是设置粒子的初始位置和速度？等等。在这之前我们还需要做一些前期准备工作。首先是设置单位制，即涉及到的物理量的单位都是什么。为什么要这么做呢？就是为了能够定量描述我们的对象。回想在高中的著名实验——小球平抛。我们怎么描述小球的运动呢？我们会说小球经过多少秒，向前运动了多少米，下落了多少米。这里就采用了国际制单位描述小球的运动。只有先确定了单位制，我们才好定量描述研究对象。不同的

研究对象, 为了描述方便会采用不同的单位制。比如描述高铁的速度我们会用 km/h, 而不是 m/s。同样, 在 LAMMPS 中面对不同的研究对象, LAMMPS 内置了不同的单位制。所以 in 文件中的

第一条命令要写的就是设置单位制, 即写 units 命令。语法就是、

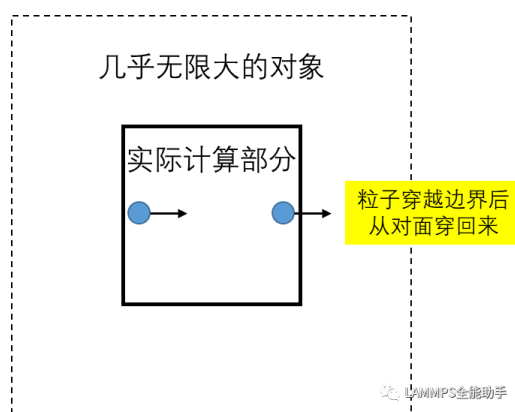
units <style> #举例: units real

LAMMPS 所有的单位制可在手册中查到。常见的单位系统有, 描述分子体系的 real, 描述金属体系的 metal, 描述简单液体的 lj。

第二条命令是设置边模拟维度, 即你模拟的是三维问题还是二维问题。通常我们的模拟都是三维的。语法是

dimension <N> #举例: dimension 3

第三条命令是设置边界条件, 即 boundary 命令。通常我们希望描述对象的体相性质。但是, 看看阿佛加德罗常数的值我们就可以知道实际的对象都会包含超级多的粒子。这么多的粒子的爱计算机无法承受。那怎么办呢? 分子动力学中采用了有限代替无限的做法。就是取对象的一部分然后加上周期性边界条件来代替几乎无限大的对象。如图中所示所谓周期性边界条件就是粒子如果从边界穿出去, 就会从对面穿回来。这样就实现了以有限的粒子数代替了无限大的对象。由边界所围起来的区域通常称为模拟盒子。模拟盒子的形状最常用的就是长方体。之所以采用长方体是因为编程容易处理。



设置边界条件在分子动力学模拟中优先级很高, 所以第二条命令要写边界条件, 语法是

boundary <x> <y> <z> #举例: boundary p p p

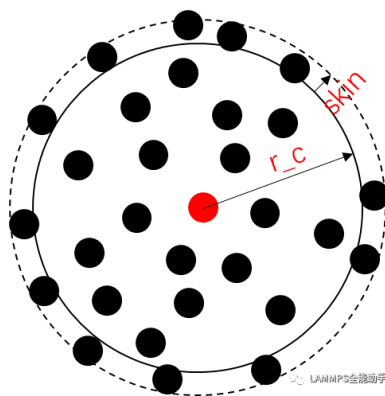
boundary p p p 就是指三个方向上都是周期性边界条件。这个设置在 LAMMPS 中最常用。LAMMPS 中还提供了其他类型的边界条件, 常用的有固定边界条件

(f) 和缩放边界条件 (s)。当我们需要在某个方向上不想采用周期性时就可用 f 或 s。f 作用就是粒子从边界跑出去就回不来了，计算就把它丢掉了。s 的作用是边界的位置会随着粒子的位置而缩放，当最靠近边界的粒子往外移动时边界位置也跟着移动，反之亦然。需要指出的是每个方向上都有两个边界，当写一个参数是如 p 就表示该方向上两个边界都是周期性，f 和 s 也如此。但是，还可以这么写 boundary p fs p，这就表示在 x 和 z 方向上是周期性边界，在 y 方向上下面是 f，上面是 s。因此写该条命令时根据需要进行设置。

第四条命令是设置建立邻居列表的，即 neighbor 命令。根据分子动力学原理我们需要计算每个原子的受力。而任何一个粒子的受力都来自于其周围粒子给它的合力。但是，我们知道只有距离较近的粒子才会对该粒子产生受力，距离较远的粒子给与的力可以忽略不计。这就是邻居的含义。所以要计算一个粒子所受的合力，首先要找到该粒子周围有哪些邻居，即建立邻居列表。每一个粒子都有自己的邻居列表。neighbor 的语法是

neighbor <skin> <style> #举例：neighbor 3 bin

所谓邻居就是距离中心粒子一定范围内的粒子，这个范围称之为作用力的截断半径 (r_c)。也就是以中心粒子为球心， r_c 为半径内的所有粒子都是邻居。skin 是比 r_c 多出来的一部分，所以建立邻居列表的时候是以 $r_c + \text{skin}$ 为半径进行寻找的。为什么要这么做呢？这是因为建立邻居列表是挺费时间的一件事，那么好不容易建立起来那就尽可能多用几次。而事实上由于分子动力学模拟的时间步长较短，粒子在一个时间步长内走不了多少距离。那么只要邻居列表中的原子移动距离都小于 skin，那么这个列表就可以在下一步中继续使用。这就是 skin 的含义。后面 style 指的是建立邻居列表的方法，最常用的就是 bin 方法。这个方法的具体含义可以查看分子动力学教材。入门只需要知道 bin 在大多数情况下都是最快的，用它就没错了。而 r_c 在后面的命令中设置。



第五条命令是设置邻居列表的更新频率, 即 **neigh_modify** 命令。LAMMPS 提供了很多方式设置邻居列表的更新频率。如果邻居中的粒子已经不在 $r_c + \text{skin}$ 范围内了, 那么邻居列表就必须更新了。当需要更新而未更新时就会出现 Dangerous builds。当算完之后 LAMMPS 输出的 Dangerous builds 不为零, 那么你要修改 neigh_modify 命令了, 或者加大 skin 的值。这条命令使用比较复杂, 需要根据模拟经验和具体的模拟测试确定最优值。对于新手, 可以直接使用以下设置:

```
neigh_modify every 1 delay 0 check yes
```

以上都是计算设置, 每个模拟这些设置都大同小异。下面开始就是针对模拟对象的设置。

第六条命令设置原子类型, 即 **atom_style** 命令。我们模拟的基本构成是原子。不同的对象粒子所具有的属性是不同的。对于某个对象, 粒子不需要的属性就不需要设置, 以节省内存加速计算。比如研究对象是分子 (如水分子), 那么构成分子的粒子 (style 为 full) 就需要具有以下属性: 粒子编号, 粒子所属分子的编号, 粒子类别 (atom type), 带电量, x 坐标, y 坐标, z 坐标。如果研究对象是惰性气体 (如氩) 或金属单质 (如铁), 那么粒子 (style 为 atomic) 的属性就是: 粒子编号, 粒子类别, x 坐标, y 坐标, z 坐标。LAMMPS 中内置了相当多的原子类型可以查看手册寻找自己需要的类型。这里要说的是 atom style 和 atom type 中文翻译都一样, 具体是啥意思。简单来说 atom style 定义了大类, atom type 定义了大类中的小类。比如一个水分子中, 所有粒子的 atom_style 都是 full, 但是在这个 style 下, 又区分为氧原子 type 和氢原子 type。一个模拟中可以有多个 type, 但是只能有一个 style。atom_style 定义好后就约束了 data 文件中具体内容。atom_style 的语法是

`atom_style <style> #举例: atom_style full`

粒子属性定义完后, 就可以定义粒子之间相互作用的方式了。所谓相互作用其实就是粒子之间作用力的计算公式。计算公式是通过势能来表述的。因为势能在空间上的梯度就是力, 即 $\mathbf{F} = -\text{div}\mathbf{U}$ 。比如重力势能对高度求导就是重力。为什么要用势能表述呢? 因为势能是标量而力是矢量, 因此使用标量在构建或者拟合相互作用表达式时就容易的多。而在实际计算中只需要对势能进行求导就能获得受力。任何对象的组成粒子之间相互作用都分为非键结和键结。非键结就是不同通过化学键的相互作用, 键结就是通过化学键的相互作用。有些对象只有非键结相互作用, 如液态氩或单质铁。有些对象两种都有, 比如水或蛋白质。非键结相互作用包括范德华和静电。在 LAMMPS 非键结相互作用即范德华和静电通过 `pair_style` 和 `pair_coeff` 命令设置。`pair_style` 和 `pair_coeff` 定义了短程相互作用, 即相互作用具有截断半径。范德华仅有短程相互作用, 而静电是既有短程又有长程作用, 如果体系中有静电作用, 那么定义完 `pair_style` 和 `pair_coeff`, 还要定义长程静电求解类型 `kspace_style`。用在分子体系的键结相互作用分为键 (`bond`), 键角 (`angle`), 二面角 (`dihedral`), 离平面 (`improper`)。不同的对象会包含不同的键结作用, 比如水分子就只包括键和键角, 而分子量稍大的分子会需要全部的键结相互作用来描述。这些相互作用构成描述体系的力场。这些力场是从量子力学出发而得到的经典近似。以下为定义相互作用的命令:

第七条命令设置 `pair_style`。首先定义非键结相互作用。不同的力场有不同的 `pair_style`, 模拟中要根据对象选择合适的 `pair_style`。LAMMPS 集成了目前几乎所有的 `pair_style`。模拟时只需要根据对象进行选择即可。`pair_style` 对于绝大多数模拟都需要定义。不同的 `pair_style` 具有不同的语法, 具体可查阅手册。`pair_style` 中一般会定义截断半径的大小。以最简单的包含范德华的 `pair_style` 为例, 其语法是

`pair_style lj/cut <cutoff> #举例: pair_style lj/cut 9.0`

第八条命令设置 `bond_style`。这条命令是可选的, 根据不同的对象可定。其设置简单, 没啥好说的, 就是计算两个粒子通过化学键的相互作用, 其语法为 `bond_style <style> #举例: bond_style harmonic`

第九条命令设置 `angle_style`。这条命令是可选的, 根据不同的对象可定。其

设置简单, 没啥好说的, 就是计算相邻三个粒子的相互作用, 其语法为

`angle_style <style> #举例: angle_style harmonic`

第十条命令设置 `dihedral_style`。这条命令是可选的, 根据不同的对象可定。其设置简单, 没啥好说的, 就是计算四个相邻粒子之间的相互作用, 其语法为 `dihedral_style <style> #举例: dihedral_style opls`

第十一条命令设置 `improper_style`。这条命令是可选的, 根据不同的对象可定。其设置简单, 没啥好说的, 就是计算四个相邻粒子之间的相互作用, 其语法为

`improper_style <style> #举例: improper_style opls`

第十二条命令设置 `special_bonds`。这条命令是可选的, 当有键结作用出现时, 设定有键结相互作用的粒子之间是否还具有 `pair` 相互作用。一般来说, 是没有的, 所以这条命令一般写作:

`special_bonds lj/coul 0.0 0.0 0.5`

第十三条命令设置 `kspace_style`。这条命令是可选的, 当有静电作用出现时, 需要加上。不同的 `pair_style` 对应不同的 `kspace_style`, 其语法:

`kspace_style <style> <value> #举例: kspace_style pppm 0.0001`

以上就是第一部分的内容。对于不同的研究对象力场设置部分会有所不同。如简单液体只需要 `pair_style` 就行了, 分子物质一般都会设置, 晶体也一般只需要 `pair_style`。如果你的对象有多种物质还可以使用 `hybrid` 类的力场命令进行组合设置。

第二部分 定义对象的几何模型

第十四条命令 `read_data`。这条命令用来读取粒子的初始位置, 键结信息等。这些信息写在一个 `data` 文件中。建立 `data` 文件称为 LAMMPS 的建模过程。LAMMPS 的建模是个非常重要的步骤。我们可以在 `in` 文件中利用 `creat_box` 和 `create_atoms` 命令建立。但是这两条命令只能用来创建晶体结构。对于分子结构需要自己使用第三方工具或者编程建立 `data` 文件实现。我习惯于自己写 `data` 文件建立模拟对象的构型。LAMMPS 的 `data` 文件建立比较麻烦, 一般要使用多个工具联合建模。这里不再赘述, 后面会专门讲 LAMMPS 建立 `data` 文件的教程。`data` 文件中的信息要根据 `atom_style` 和分子的拓扑结构进行建立。以一个由分子

构成的模拟对象, 也即 atom_style 是 full 为例 data 文件内容为:

LAMMPS Description #第一行写描述信息

```
116803 atoms #共多少个 atoms
70386 bonds #共多少个 bonds
41643 angles #共多少个 angles
13700 dihedrals #共多少个 dihedrals
2550 impropers #共多少个 impropers
```

```
191 atom types #共多少个 atoms types
195 bond types #共多少个 bonds types
356 angle types #共多少个 angles types
548 dihedral types #共多少个 dihedrals types
102 improper types #共多少个 angles types
```

```
0 97.92 xlo xhi #x 方向的上下边界
0 97.92 ylo yhi #y 方向的上下边界
-15.0 160 zlo zhi #z 方向的上下边界
```

Masses

```
1 15.9994 # 1 号 atom type 的质量
2 1.008 # 2 号 atom type 的质量
.....
```

Atoms

```
1 1 1 0.0 0.0 0.0 145.0 #粒子编号, 粒子所属分子的编号, 粒子类别 (atom type),
带电量, x 坐标, y 坐标, z 坐标
2 1 1 0.0 0.0 2.04 147.04
```

.....

Bonds

1 2 13825 13826 #bond 的编号, bond type, 组成 bond 的第一个, 第二个的原子编号

2 3 13826 13827

.....

Angles

1 2 13825 13826 13827 #angle 的编号, angle type, 组成 angle 的第一个, 第二个, 第三个的原子编号

2 3 13825 13826 13828

.....

Dihedrals

1 1 13825 13826 13828 13829 #dihedral 的编号, dihedral type, 组成 dihedral 的第一个, 第二个, 第三个, 第四个的原子编号

2 2 13826 13828 13829 13830

.....

Impropers

1 1 13826 13825 13827 13828 #improper 的编号, improper type, 组成 improper 的第一个, 第二个, 第三个, 第四个的原子编号

2 2 13835 13834 13836 13837

.....

第三部分 定义对象的物理模型

这部分命令用来指定势函数中的具体参数是多少。

第十五条命令 pair_coeff。根据定义的 pair_style 和粒子 type 的个数来决定 pair_coeff 的内容和条数。具体书写时可查看手册中对应的 pair_style 的例子。如 pair_style 是 lj/cut，粒子 type 个数为 2，则该命令写为

```
pair_coeff 1 1 0.1553 3.166 9.0
```

```
pair_coeff 1 2 0.1423 3.125 9.0
```

```
pair_coeff 2 2 0.2753 3.196 9.0
```

第十六条命令 bond_coeff。根据定义的 bond_style 和 bond type 的个数来决定 bond_coeff 的内容和条数。具体书写时可查看手册中对应的 bond_style 的例子。

第十七条命令 angle_coeff。根据定义的 angle_style 和 angle type 的个数来决定 angle_coeff 的内容和条数。具体书写时可查看手册中对应的 angle_style 的例子。

第十八条命令 dihedral_coeff。根据定义的 dihedral_style 和 dihedral type 的个数来决定 dihedral_coeff 的内容和条数。具体书写时可查看手册中对应的 dihedral_style 的例子。

第十九条命令 improper_coeff。根据定义的 improper_style 和 improper type 的个数来决定 improper_coeff 的内容和条数。具体书写时可查看手册中对应的 improper_style 的例子。

第四部分 定义分组信息

很多情况下我们需要对研究对象的不同部分施加不同的操作。LAMMPS 中通过对对象中的原子进行分组实现。

第二十条命令 group。group 命令可以采用多种方式进行分组定义

```
region fix block INF INF INF INF INF 10.0 units box
```

```
group fix region fix
```

```
group thermos type 1
```

```
group a1 id >= 76
```

```
group sub id 100:10000:10
```

```
group big molecule 123
```

```
group polyA molecule <> 50 250
group boundary subtract all a2 a3
group boundary union lower upper
group boundary intersect upper flow
group boundary delete
group mine dynamic all region myRegion every 100
```

第五部分 弛豫

在弛豫部分我们首先需要定义基本输出日志信息, 这是为了监控我们的模拟是否正确。一般输出的都是系统量, 如温度、压强、总动能、总势能等。所以

第二十一条命令 thermo。 thermo 命令控制每多少步输出一次。如果不设置 LAMMPS 默认只在第一步和最后一步输出日志信息。语法是

```
thermo <N> #举例: thermo 1000
```

第二十二条命令 thermo_style。 thermo_style 控制究竟要输出哪些全局量。具体能够输出哪里量可查阅手册 thermo_style 命令描述。如果不设置该条命令, 那么 LAMMPS 就会输出默认信息。其语法是

```
thermo_style <style> <args>
```

第二十三条命令 dump。 该条命令用来每隔一段时间输出系统所有原子的信息, 这些信息可以是原子的位置, 受力, 速度, 质量, 编号等。当你要输出所有原子的信息时就是用这条命令。将这条命令的输出文件进行可视化可以初步判断你的模拟是否正确。以下是最常用的设置

```
dump 1 all custom 10000 dump.lammpstrj id mol type mass x y z
```

第二十四条命令 minimize。 这条命令是可选的。如果你的几何模型和物理模型都设置正确, 那么可以不用这条命令。这条命令执行的是能量最小化。为什么要执行能量最小化呢? 这是因为我们在建模的时候, 有时几何模型和物理模型不是那么完美, 经常出现的就是某些原子之间的位置靠得太近, 这就会导致系统的能量很大, 容易出现运行错误。执行能量最小化就是将系统的能量调节至尽可能小。LAMMPS 实现这个命令的方式是通过计算原子之间的受力和能量, 通过一些优化算法, 移动原子的位置从而是体系总能量达到尽可能小。所以如果你发现你的体系中能量超级大, 压强超级大而运行出错, 那么你可以试试能量最小化命

令。一般使用下面这些参数设置就能实现你的目的, 如果能量最小化后还出错, 那么你就要排查你的几何模型和物理模型是不是太差了, 能量最小化也无能为力了。至于能量最小化采用什么算法, 用默认的就行了, 也就是说你不用有额外的设置, 直接 `minimize` 就行了, 花里胡哨的没啥用。

```
minimize 0.0 1.0e-8 1000 100000
```

第二十五条命令 `compute`。该命令是 LAMMPS 中极其重要的一类命令。当你需要计算某个量时你就可以考虑使用 `compute` 命令。LAMMPS 提供了丰富的 `compute` 命令, 具体使用时, 根据你的需求, 查阅手册看 LAMMPS 中有没有实现你需要的功能的 `compute`。`compute` 和 `dump` 一样可以针对某个 `group` 的粒子进行计算。一个 `in` 文件中可以有多个 `compute`, 但每个 `compute` 的 `id` 不能重复。计算出来的值可以被很多命令使用, 如 `dump`, `fix` 等。最常用的就是计算温度, 如

```
compute myT all temp
```

第二十六条命令 `velocity`。`velocity` 命令有不少功能, 最常用的功能就是根据目标温度创建粒子的初始速度。最常用的形式为

```
velocity all create 300.0 4928459 rot yes dist gaussian
```

第二十七条命令 `timestep`。该命令设定求解牛顿差分方程中 Δt 的大小。不同体系会要求不同大小的 Δt 。一般来说分子体系会要求 $\Delta t = 1\text{fs}$ 。简单液体体系可放宽至 $\Delta t = 5\text{fs}$ 。晶体体系可以是 $\Delta t = 2\text{fs}$ 。反应力场 `reaxff` 体系会要求更小的时间步, $\Delta t = 0.1 - 0.5\text{fs}$ 。时间步长的选取原子是, 要足够小以准确捕捉体系中最快的动力学过程, 又要足够大以节省计算时间。通常你的时间步长大小参考文献设置就行了。或按照上面举的例子进行设置, 就没啥问题了。

第二十八条命令 `fix`。`fix` 是 LAMMPS 中另一类极其重要的命令。其实现功能就是对某个组的粒子施加某种操作。最重要的功能就是用来更新某个组的粒子的速度和位置, 因此每个 `in` 文件中都必然会有 `fix` 命令。我们通常所说的系综就是通过 `fix` 命令实现。如 `fix nve` 实现 `nve` 系综, `fix nvt` 实现 `nvt` 系综, `fix npt` 实现 `npt` 系综。关于系综这里要多说一点。系综是统计力学的基本概念。所谓系综, 简单来说就是系统无限个副本的集合。但是分子动力学中的系综和统计力学上的系综有联系又有区别。比如在 LAMMPS 中 `nve` 系综就是只直接求解牛顿方程而

不对速度和位置做任何干涉的系统。把 `fix nve` 与温度控制 `fix` 联用就实现 `nvt` 系统, 把 `fix nve` 与温度控制的 `fix` 和压强控制的 `fix` 同时联用就实现了 `npt` 系统。也可以只使用 `fix nvt` 命令直接实现 `nvt`, 只使用 `fix npt` 命令实现 `npt` 系统。系统当你的边界条件是 `p` 或 `f` 时系统体积是不变的。当你使用 `s` 边界时系统体积是可变得, 此时也就没有什么 `nvt` 或 `nve` 了。但是需要强调的是 `npt` 是通过改变盒子体积来实现控压的所以在控压的方向上只能使用 `p` 边界条件。所以基本上你可以忘记系统的概念。把 `fix nve` 或 `fix nvt` 或 `fix npt` 当做更新原子位置和速度的命令, `fix nve` 直接根据牛二进行更新, `fix nvt` 在更新的时候会对速度进行干涉以达到设定的温度, `fix npt` 会对速度和位置同时进行干涉以达到设定的温度和压强。至于 `nvt` 和 `nve` 下体积变不变取决于你的边界条件。同一个系统中可能同时存在 `fix nve` 和 `fix nvt`, 这是因为有些模拟要求一部分原子的位置和速度直接根据牛二进行更新, 而另一部分却要求在更新原子速度时保持温度不变。但是不会同时出现 `fix nve` 和 `fix npt`, 因为 `fix npt` 在根据设定的压强在更新原子位置和速度时, 计算的是所有原子对压强的贡献。所以 `fix npt` 不适合局部控压。

某个组的粒子只有使用了 `fix+` 系统的命令才会更新位置和速度。因此如果你想保持某组的粒子在模拟过程中固定不动, 最简单的办法就是不对它施加任何 `fix` 命令。通常我们在使用 `fix` 时都会使用先来一条 `fix+` 系统, 然后调用其他 `fix` 实现某些目的, 如要让模拟对象发生一定变形, 就用 `fix deform`, 要让对对象施加一个电场就用 `fix efield`。当你想用实现某种操作时, 打开手册查看 `fix` 的一些命令看有没有实现你功能的 `fix` 命令。`fix` 的某些命令也可以用于输出信息, 如 `fix ave/chunk` 输出模拟盒子不同位置处的原子属性的时间平均值; `fix ave/time` 输出某些全局量的平均值。

第二十九条命令 `run`。终于要开始跑了。这个命令最常用的就是你要 `run` 多少步。在弛豫阶段一般会 `run 1ns` 的长度, 通常是 1000000 步。这里可以控制采用什么算法对牛二进行求解。采用默认的就行, 也就是说你不用额外设置, 直接 `run` 就行了。花里胡哨的没啥用, 默认的往往是最好的。其设置是

`run 1000000`

以上就是弛豫部分。为什么要进行弛豫呢? 这是因为系统中原子的初始位置和初始速度都是认为设定的, 这是系统的状态并不在热力学平衡状态。通常我们

希望在正式模拟时系统的起点在热力学平衡状态。只有经过弛豫后系统才会达到热力学平衡状态。在弛豫开始的时候你会看到系统的温度和压强会有非常剧烈的变化，这是正常的。经过一段时间后温度和压强这些量就会在一个值附近小幅振荡。那么怎么判断系统是否达到平衡了呢？这是个挺麻烦的事，不同的体系有不同的标准。通常会看系统的温度，压强，能量这些值不会在持续上升或下降就差不多平衡了。但是这个事很难说。所以为了保险起见，通常模拟中会设置冗余的弛豫步数来确保系统达到平衡。这个值一般是 1000000 步。需要说明的是，系统温度是根据系统动能算出来的，一般很快就不咋变了。但是压强是根据受力算出来的振荡会很大。一般要看温度或压强的时间平均值，也就是不同时间步的平均值来判断系统的温度或压强是否达到了设定值。

第三十条命令 `write_restart`。这个命令可以写出 `restart` 文件，以便后续从这个点接着算。`restart` 文件是一个二进制文件。使用这条命令是个好的习惯，可以让你省很多事，又不会额外增加计算量。其语法是

```
write_restart <filename> #举例 write_restart restart.equ
```

最后一部分 正式模拟

这部分其实才是模拟的核心。不同的研究对象和目的设置和使用的命令都会不同。无法在基本流程中写，将在后面的专题中介绍。

4.4 LAMMPS 的运行方法

LAMMPS 从命令行运行，通过 `-in` 命令行标志从文件读取命令，或者从标准输入读取命令。建议使用“`-in in.file`”形式（参见下面的注释）。LAMMPS 可执行文件的名称是 `lmp` 或 `lmp_<machine>`，其中 `<machine>` 是编译 LAMMPS 时使用的机器字符串。这在使用传统方法（例如使用 `make mpi`）编译 LAMMPS 时是必需的，但在使用 CMake 配置和编译 LAMMPS 时是可选的：

```
$ lmp_serial -in in.file
$ lmp_serial < in.file
$ lmp -in in.file
$ lmp < in.file
$ /path/to/lammps/src/lmp_serial -i in.file
$ mpirun -np 4 lmp_mpi -in in.file
$ mpiexec -np 4 lmp -in in.file
```

```
$ mpirun -np 8 /path/to/lammps/src/lmp_mpi -in in.file
```

```
$ mpiexec -n 6 /usr/local/bin/lmp -in in.file
```

通常在输入脚本所在的目录中运行 LAMMPS 命令。这也是默认生成输出文件的地方，除非在输入脚本或命令行中提供特定的其他路径。与上面的一些示例一样，LAMMPS 可执行文件本身可以放在其他地方。

注意：重定向运算符“<”在与 mpirun 或 mpiexec 并行运行时并不总是有效；对于那些系统，-in 形式是必需的。

当 LAMMPS 运行时，它会在屏幕上打印信息和一个名为 log.lammps 的日志文件。有关输出的更多信息在屏幕和日志文件输出页面上给出。

如果 LAMMPS 在输入脚本中或在运行模拟时遇到错误，它将打印错误消息并停止或警告消息并继续。有关 LAMMPS 可以或无法检测到的各种错误的讨论、所有 ERROR 和 WARNING 消息的列表以及如何处理它们。

LAMMPS 可以在任意数量的处理器上运行相同的问题，包括单个处理器。理论上，应该在任何数量的处理器和任何机器上得到相同的结果。在实践中，由于使用浮点数学而导致的数值四舍五入会导致分子动力学轨迹的细微差异和最终发散。

LAMMPS 可以在一个或多个处理器的物理内存中运行尽可能大的问题。如果内存不足，则必须在更多处理器上运行或定义更小的问题。所需的内存量以及它在处理器之间的分布情况可能会因使用的模型、设置和命令而异。

如果通过 mpirun 并行运行 LAMMPS，应该知道 processors 命令，它控制 MPI 任务如何映射到模拟盒子，以及控制 MPI 任务如何分配到节点的物理核心的。这些设置可以提高性能，尽管默认值通常就足够了。

例如，将 MPI 任务（进程）绑定到物理内核（处理器亲和性）通常很重要，这样操作系统就不会在模拟期间迁移它们。如果这不是您机器上的默认行为，则可以使用 mpirun 选项“-bind-to core”（OpenMPI）或“-bind-to core”（MPICH）。

如果使用的 LAMMPS 命令支持多线程，可以在启动 LAMMPS 之前通过环境变量 OMP_NUM_THREADS 设置每个 MPI 任务的线程数：

```
$ export OMP_NUM_THREADS=2      # bash
```

```
$ setenv OMP_NUM_THREADS 2      # csh or tcsh
```

这也可以通过 `package` 命令或通过调用 `package` 命令的 `-pk` 命令行开关来完成。

您可以使用 `examples` 或 `bench` 目录中提供的任何输入脚本来试验运行 LAMMPS。输入脚本命名为 `in.*`, 示例输出命名为 `og.*.P`, 其中 `P` 是运行它的处理器数量。一些 `examples` 或 `bench` 测试需要使用可选包编译 LAMMPS。

4.5 屏幕日志文件输出

当 LAMMPS 读取输入脚本时, 它会在屏幕和日志文件中打印有关设置模拟所采取的重要操作的信息。当模拟准备开始时, LAMMPS 会执行各种初始化, 并打印有关它将要执行的运行的信息, 包括模拟所需的内存量 (以 MBytes/处理器为单位)。它还打印系统初始热力学状态的详细信息。在运行过程中, 每隔几个时间步就会定期打印热力学信息。当运行结束时, LAMMPS 会打印最终的热力学状态和模拟的总运行时间。它还附加了有关模拟的 CPU 时间和存储要求的统计信息。示例统计信息:

```
Loop time of 2.81192 on 4 procs for 300 steps with 2004 atoms

Performance: 18.436 ns/day 1.302 hours/ns 106.689 timesteps/s
97.0% CPU use with 4 MPI tasks x no OpenMP threads

MPI task timings breakdown:
Section | min time | avg time | max time | %varavg | %total
-----|-----|-----|-----|-----|-----
Pair    | 1.9808   | 2.0134   | 2.0318   | 1.4    | 71.60
Bond    | 0.0021894 | 0.0060319 | 0.010058 | 4.7    | 0.21
Kspace  | 0.3207   | 0.3366   | 0.36616  | 3.1    | 11.97
Neigh   | 0.28411  | 0.28464  | 0.28516  | 0.1    | 10.12
Comm    | 0.075732 | 0.077018 | 0.07883  | 0.4    | 2.74
Output  | 0.00030518 | 0.00042665 | 0.00078821 | 1.0    | 0.02
Modify  | 0.086606 | 0.086631 | 0.086668 | 0.0    | 3.08
Other   |          | 0.007178 |          |        | 0.26

Nlocal: 501 ave 508 max 490 min
Histogram: 1 0 0 0 0 1 1 0 1
Nghost: 6586.25 ave 6628 max 6548 min
Histogram: 1 0 1 0 0 0 1 0 0 1
Neighs: 177007 ave 180562 max 170212 min
Histogram: 1 0 0 0 0 0 0 1 1 1

Total # of neighbors = 708028
Ave neighs/atom = 353.307
Ave special neighs/atom = 2.34032
Neighbor list builds = 26
Dangerous builds = 0
```

第一部分提供了全局循环时序摘要。循环时间是模拟运行的总挂钟时间。提供性

能信息是为了方便帮助预测运行所需的物理模拟需要多长时间。CPU 使用行提供每个 MPI 任务的 CPU 使用率；它应该接近 OpenMP 线程数(或不使用 OpenMP 的 1) 的 100% 倍。较低的数字对应于由于文件 I/O 或线程利用率不足而导致的延迟。

MPI 任务部分将 CPU 运行时间（以秒为单位）细分为主要类别：

pair = 非键合力计算

bond = 键相互作用：键、角、二面角、不正确

Kspace = 远程交互：Ewald、PPPM、MSM

Neigh = 邻居列表构造

Comm = 原子及其属性的处理器间通信

output = 热力学信息和转储文件的输出

modify = fix 调用的 fix 和 compute

other = 所有剩余时间

对于每个类别，都会细分任何处理器在此类计算上花费的最少、平均和最多的挂墙时间。“%varavg”是最大值或最小值与平均值变化的百分比。这表明负载不平衡。接近 0 的百分比是完美的负载平衡。百分比较大则是是不平衡。最后的“%total”列是在这个类别中花费的总循环时间的百分比。

使用 timer full 设置时，会添加一个附加列，该列还以百分比形式打印 CPU 利用率。此外，当使用 timer full 并且 package omp 命令处于活动状态时，会提供类似的时间摘要，用于在线程区域中监控线程利用率和负载平衡所花费的时间。还添加了一个新的线程计时部分，其中列出了将每个线程数据元素减少到存储以进行非线程计算所花费的时间。这些线程计时仅针对第一个 MPI 等级测量，因此，由于 MPI 任务的细分可以从 MPI 等级更改为 MPI 等级，因此对于各个等级，这种分解可能非常不同。这是示例输出：

```
Thread timings breakdown (MPI rank 0):
Total threaded time 0.6846 / 90.6%
Section | min time | avg time | max time | %varavg | %total
-----|-----|-----|-----|-----|-----
Pair    | 0.5127   | 0.5147   | 0.5167   | 0.3     | 75.18
Bond    | 0.0043139 | 0.0046779 | 0.0050418 | 0.5     | 0.68
Kspace  | 0.070572  | 0.074541  | 0.07851  | 1.5     | 10.89
Neigh   | 0.084778  | 0.086969  | 0.089161  | 0.7     | 12.70
Reduce  | 0.0036485 | 0.003737  | 0.0038254 | 0.1     | 0.55
```


上面的第三部分列出了每个处理器存储的拥有原子 (Nlocal)、幽灵原子 (Nghost) 和成对邻居的数量。最大值和最小值给出了这些值在处理器之间的分布, 其中 10 位直方图显示了分布。直方图计数的总数等于处理器的数量。

5. 建模和后处理软件概述

本节介绍了包装 LAMMPS 的商业和免费软件, 为开发模型、运行仿真和分析结果提供了一个用户友好的环境。

[Materials Design, Inc.](#)

Materials Design, Inc. 开发了 MedeA®, 这是一种原子模拟和建模环境, 可提供与 LAMMPS 一起使用的生产力、模型构建和分析工具。MedeA® 使用流程图简化了 LAMMPS 模拟。生成的 LAMMPS 模拟很容易在同事之间共享、编辑以供将来重复使用, 并且可以由 LAMMPS 专家进行定制。

使用 MedeA®, 可以使用各种晶体系统方法构建原子模型, 并使用 MedeA® 非晶和热固性构建器, 这是为 LAMMPS 创建输入的工具。它们还为预测机械性能提供了经过验证的方法, 包括基于 LAMMPS 模拟的弹性常数、扩散率、传输性能和内聚能。MedeA® 还有助于管理用于模拟有机、无机和金属系统的力场; 材料设计团队在为有机和金属系统开发精确力场方面拥有良好的记录。

[Scienomics](#)

Scienomics 公司开发了一个 LAMMPS 接口, 作为其材料和工艺模拟 (MAPS) 平台的一部分, 该平台允许 LAMMPS 新手和专家快速有效地创建 LAMMPS 输入文件-用于原子模拟和 DPD 模拟。Scienomics 为 LAMMPS 用户提供完整的电话和电子邮件支持。MAPS 中的 LAMMPS 插件还允许用户为 LAMMPS 创建输入文件, 对 LAMMPS 模拟的输出进行可视化和分析。MAPS 也有许多力场可供选择(用于原子建模和粗粒度建模)与 LAMMPS。他们还有一个名为 Amorphous Builder 和 Crosslink Builder 的工具, 可用于为 LAMMPS 创建输入几何图形, 以及用于分析 ReaxFF 模拟结果的工具和其他强大的功能, 用于计算弹性属性、结构属性、传输属性等关键属性。

MAPS 是一个综合性的软件平台, 结合了构建-模拟-分析的工作流程。这使用户能够以高效和无缝的方式对复杂材料系统进行建模, 从包括 LAMMPS 在内的一系列仿真引擎中进行选择以对过程进行建模, 并分析输出以提取许多属性,

例如机械、热、传输、流变等. 有关所有这些工具和服务的更多详细信息, 请参见 Scienomics 网页。

[XenoView](#)

由 Sergei Shenogin (伦斯勒纳米技术中心) 开发。该软件对非商业用户免费。XenoView 是基于 Windows 的分子动力学模拟软件。它的界面为从头开始构建结构提供了广泛的工具。此外, 您可以从多种格式 (例如 PDB 格式) 导入结构。键可以自动定义, 力场可以自动分配。XenoView 可以导出可在 LAMMPS 中用于大规模模拟的数据文件和输入脚本。有关详细信息, 请参阅 XenoView 主页。

[Atomman](#)

atomman 工具是由 NIST 的 Lucas Hale 开发的开源 (免费) 软件。它是一个用于与大规模原子系统交互的 Python 包。它允许用户完全从 Python 中准备、运行和分析 MD 模拟。

- 允许对数百万个原子进行高效和快速的计算, 每个原子都有许多自由定义每个原子属性。
- 生成包含缺陷的原子系统, 例如点缺陷和位错单极子。
- 用于缺陷分析的内置函数, 例如滑移向量和 Nye 张量。
- 直接从 Python 调用 LAMMPS 并立即检索结果数据或 LAMMPS 错误语句。
- 轻松将系统与 ASE 和 PyMatGen 的其他 Python 原子环境相互转换。
- 可以加载基于 CIF 晶体结构文件、POSCAR 和 LAMMPS 原子和转储文件的系统。
- 内置单位转换工具。
- 独立于平台: 适用于 Linux、Windows 和 Mac。

[LOOS](#) = Lightweight Object-Oriented Structure analysis library

LOOS 是一个用于分析分子动力学模拟的软件包。它与包无关、开源, 可在所有主要的 Linux 发行版以及 OSX 上运行。它分发了大约 150 个预打包的分析工具, 范围从标准任务 (轨迹操作、主成分分析等) 到新工具 (评估模拟收敛、测量膜和膜蛋白的特性)。此外, 它专为快速开发新的分析工具而设计, 尤其是在使用 Python 包装器时。LOOS 可从 <https://github.com/GrossfieldLab/loos> 下载

[Freud](#) = analysis library for post-processing MD and MC data

Freud 是一个 Python 库, 由 Sharon Glotzer 的小组 (密歇根大学) 开发和支持, 它提供了一套简单、灵活、强大的工具, 用于分析从分子动力学或蒙特卡罗模拟中获得的轨迹。高性能、并行化 C++ 用于计算标准工具, 例如径向分布函数、相关函数、阶参数和聚类, 以及包括平均力和扭矩 (PMFT) 势和局部环境匹配在内的原始分析方法。freud 库支持多种输入格式和输出 NumPy 数组, 支持与许多典型材料科学工作流程的科学 Python 生态系统集成。

教程: [g\(r\) 分析](#)

文档: <https://freud.readthedocs.io>

源代码: <https://github.com/glotzerlab/freud>

Molecular Builders

要模拟分子系统, LAMMPS 要求您输入分子拓扑结构 (键、角、二面体等列表) 以及适合您的模型的力场系数。因此, 构建分子系统的任务是一个预处理步骤, 并且本身可能是一项复杂的任务。

本节介绍有助于自动化此过程的工具。

一般来说, 封装可以从键拓扑推断角度、二面角和不正确的相互作用。他们有生成分子几何的命令。他们可以从 PACKMOL 和其他 PDB 文件生成器生成的文件中读取坐标。

[Enhanced Monte Carlo \(EMC\)](#)

由位于 verizon.net 的't Veld (BASF) 的 Pieter J. 开发和维护。

增强型 Monte Carlo 或简称 EMC 为使用 COMPASS、CHARMM、OPLS、Martini、DPD 或胶体力场创建和操作粒子模拟的输入结构提供了环境。为此, 脚本语言管理对其功能的访问。当前版本通过 SMILES 字符串提供对分子或粗粒度结构的操作, 在需要时为选定的力场键入这些结构, 并应用蒙特卡罗原理构建构象以不重叠原子。EMC 提供 LAMMPS、PDB 和 XYZ 格式的输出端口。可以在 montecarlo.sourceforge.net 上找到适用于 Linux、MacOS 或 Windows 的编译版本。

[Moltemplate](#)

由 Andrew Jewett (UCSB) 开发和维护。

该工具与 LAMMPS 一起在 tools/moltemplate 目录中分发。有关更多详细信

息, 请参见该目录和 [moltemplate](#) 主页。

Moltemplate 专为构建粗粒度生物分子模型而设计。**Moltemplate** 可以同时创建: **lammps DATA** 文件(包含几何和拓扑)和 **lammps INPUT** 脚本(包含力场、修复和组)。与其他转换工具生成的文件不同, **moltemplate** 允许用户访问 **LAMMPS** 中可用的所有力场。用户可以将分子保存为 **moltemplate** 紧凑、可读的模板文件格式(“.LT”), 并与他人共享。分子可以用作更大分子的构建块。“罐装”力场(例如 **Dreiding**、**GAFF**、**TraPPE** 和用户创建)也可以(原则上)以这种格式保存并稍后应用于分子。

分子可以被复制、组合和链接在一起以定义新分子。(这些可用于定义更大的分子。支持无限级别的对象组合、嵌套和继承。)一旦构建, 可以自定义单个分子和子单元(原子和键, 以及子单元可以移动、删除和替换)。

[VMD TopoTools](#)

由 Axel Kohlmeyer (Temple U 开发和维护。

有关详细信息, 请参阅 [TopoTools](#) 主页。

Topotools 是一个分子生成器, 它利用 **VMD** 和 **TCL** 的强大功能创建 **lammps DATA** 文件并将它们转换为其他格式或从其他格式转换。**Topotools** 有两个组件: 一个可以提取和操作拓扑信息的中间件脚本, 以及在它之上构建的几个高级应用程序, 例如可以使其能够读取/写入数据文件、复制和合并系统。与 **VMD** 一起, **topotools** 可以从 **PDB** 文件、**PSF** 文件和原子对距离推断拓扑结构, 使蛋白质溶剂化。

[InterMol a conversion tool for molecular dynamics simulations](#)

IntelMol 是用 **Python** 编写的, 可以在 **InterMol** 中本地进行 **Desmond** <=> **Gromacs** <=> **Lammps** 转换; **AMBER** -> **X** 是通过将 **AMBER** 转换为 **GROMACS**, 然后使用 **ParmEd** 转换为其他程序来执行的。**AMBER** -> **CHARMM** 由 **ParmEd** 直接执行。

[Avogadro](#)

作为开源项目开发和维护。有关详细信息, 请参阅 [Avogadro](#) 主页。

Avogadro 是一款先进的分子编辑器和可视化工具, 专为计算化学、分子建模、生物信息学、材料科学和相关领域的跨平台使用而设计。它提供灵活的高质量

渲染和强大的插件架构。

[Packmol](#)

作为开源项目开发和维护。有关详细信息, 请参阅 [Packmol](#) 主页。

[Packmol](#) 通过在定义的空间区域中包装分子来创建分子动力学模拟的初始点。包装保证短程排斥相互作用不会破坏模拟。

[Atomsk](#)

作为开源项目开发和维护。有关详细信息, 请参阅 [Atomsk](#) 主页。

[Atomsk](#) 旨在创建、操作和转换原子系统。它支持多种文件格式, 其中包括 LAMMPS 文件格式, 还支持 VASP、Quantum Espresso、IMD、DL_POLY、Atomeye CFG 格式或 xCrySDen XSF 格式, 可以轻松转换文件以进行从头计算、经典势模拟或可视化。此外, [atomsk](#) 还可以对原子位置进行一些简单的变换, 如旋转、变形、插入位错。

[OCTA](#) 和 [J-OCTA](#)

[OCTA](#) 是一个开源软件包, 由模拟引擎 (分子动力学、流变模拟、自洽场理论、有限元方法等) 和用于建模软物质系统的 GUI (可视化、简单分子构建器和分析工具) 组成。

[OCTA](#) 还为多种模拟器的协作使用提供了环境, 即多物理场和多尺度模拟。

LAMMPS 和 COGNAC (MD 引擎) 文件之间的转换程序也包括在内。此功能使 LAMMPS 用户能够使用 [OCTA](#) GUI 并结合其他理论 (如 SCFT) 运行经典的 MD 模拟。

通过使用商业版本的 [J-OCTA](#), 还可以进行全原子和粗粒度 MD 的复杂分子构建。

[Molecular Simulation Design Framework \(MoSDeF\)](#)

[MoSDeF](#) 提供了一组可扩展的 Python 工具, 旨在促进使用分子模拟对软物质系统进行初始化、原子分型和筛选。

[Automated Topology Builder \(ATB\) and Repository](#) - molecular topology building blocks for organic molecules

ATB 项目由 Alan E. Mark 教授 (澳大利亚昆士兰大学) 领导; ATB 网站上有一份其他贡献者的列表。

ATB 以与 LAMMPS 和其他分子动力学软件包兼容的格式提供有机分子的拓扑文件。LAMMPS 拓扑文件支持使用 `moltemplate` 工具 (随 LAMMPS 分发) 构建复杂系统。`Moltemplate` 允许结合力场参数和分子模板文件来构建复杂的系统, 从而实现类似于在生物分子 MD 模拟包 (例如 GROMOS、AMBER、CHARMM 等) 中构建系统拓扑的工作流程。

ATB 站点为各种分子 (> 20,000 并且还在增长) 提供拓扑结构。可以使用基于名称或化学式的内部搜索工具并从结果列表中选择分子来找到分子。一旦进入感兴趣分子的页面, 您可以选择“Molecular Dynamics (MD) Files”并选择“LAMMPS”作为输出“格式”如果数据库中不存在分子, 可以通过提交新的要处理的分子。

Data Sites

本节列出了提供可与 LAMMPS 一起使用的数据的 WWW 站点。例如, 原子配置, 可用于初始化 LAMMPS 模拟的模型。或者使用 LAMMPS 完成的计算存档, 可以浏览以获取物理或模型。

[JARVIS database for MD potential calculations on DFT geometries](#)

JARVIS for force-fields (JARVIS-FF) 是一个高通量计算数据库, 用于对具有各种力场/原子间势的密度泛函理论 (DFT) 优化的几何结构进行 LAMMPS 计算。该项目的目标是通过网络界面为评估力场提供一个简单的查找表, 并提高数据的可重复性。JARVIS-FF 是美国国家标准与技术研究院 (NIST) 材料基因组计划 (MGI) 的一部分。

[Orsi group at Queen Mary University of London](#)

Mario Orsi 的小组为各种水系统 (不同的力场) 收集了很好的 LAMMPS 输入脚本和数据文件。他们也在为蛋白质和脂质膜系统添加文件。

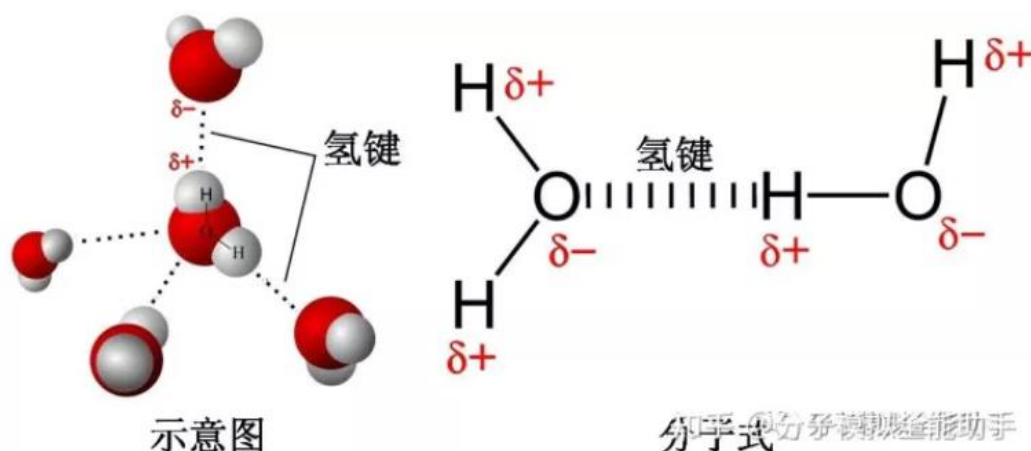
6. LAMMPS 模拟与后处理技巧和教程

6.1 LAMMPS 输出轨迹用 vmd 后处理氢键

内容来自公众号: 分子模拟全能助手

LAMMPS 的计算功能强大, 作为计算引擎首屈一指, 但是 LAMMPS 后处理功能不充分。这就需要借助一些后处理工具。`vmd` 是一款优秀的开源可视化及后处理工具之一。使用 `vmd` 可以方便地分析 LAMMPS 的轨迹。其中, 最常用的就

是进行氢键的分析。以下是相关教程, 教程如有错误欢迎加微信 lmp_zhushou 进行反馈。也可加微信 lmp_zhushou 进入微信群进一步交流。



使用 vmd 对 LAMMPS 的轨迹进行氢键分析需要 LAMMPS 输出特定格式的轨迹。需要使用的命令当然是 dump, 但是需要输出 xyz 格式的轨迹, 同时使用 dump_modify 进行元素映射。以下是一个例子。假设你的体系中有 5 个 atom type, 分别是 1, 2, 3, 4, 5, 这五个分别对应 C、H、O、N、P。那么 dump 和 dump_modify 命令需要这样写

```
dump myDump all xyz 100 dump.xyz
```

```
dump_modify myDump element C H O N P
```

这样输出的轨迹就是包含元素名的 xyz 格式的轨迹。通常情况对于经典力场同一个元素会有多个 type, 比如你有 9 个 atom type: 1, 2, 3, 4, 5, 6, 7, 8, 9, 分别对应 C, H, O, C, N, C, P, N, P。那么你的 dump 和 dump_modify 就应该这样写

```
dump myDump all xyz 100 dump.xyz
```

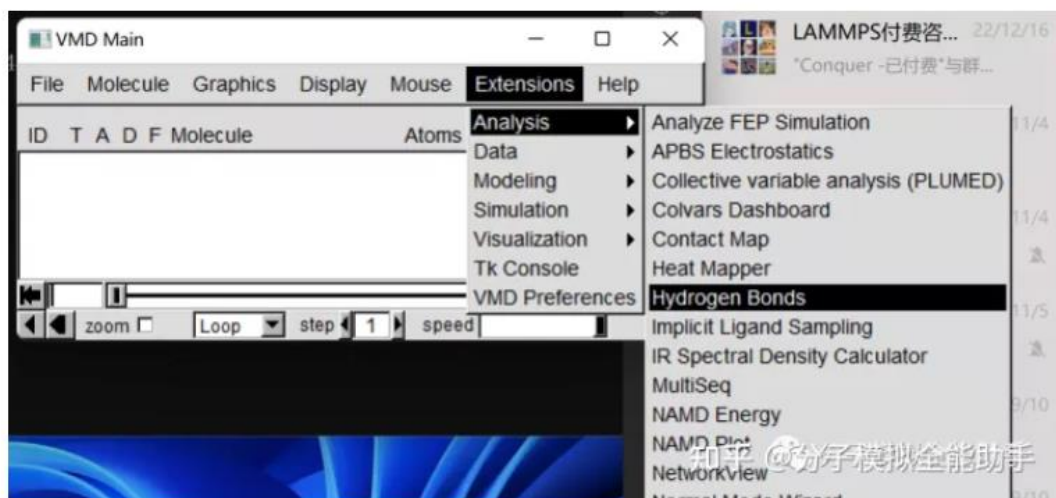
```
dump_modify myDump element C H O C N C P N P
```

这里有一个小技巧, 如果你的体系中有多个 O 或者其他极性原子, 比如水分子中的 O 和乙醇中的 O, 而你想看乙醇中的 O 与水分子中的 H 之间的氢键那么你可以自己重新定义水分子中的 O, 把这个 O 故意分配成 N 或者。这样在 vmd 中做选择的时候就可以把不同分子中的 O 区分开来。

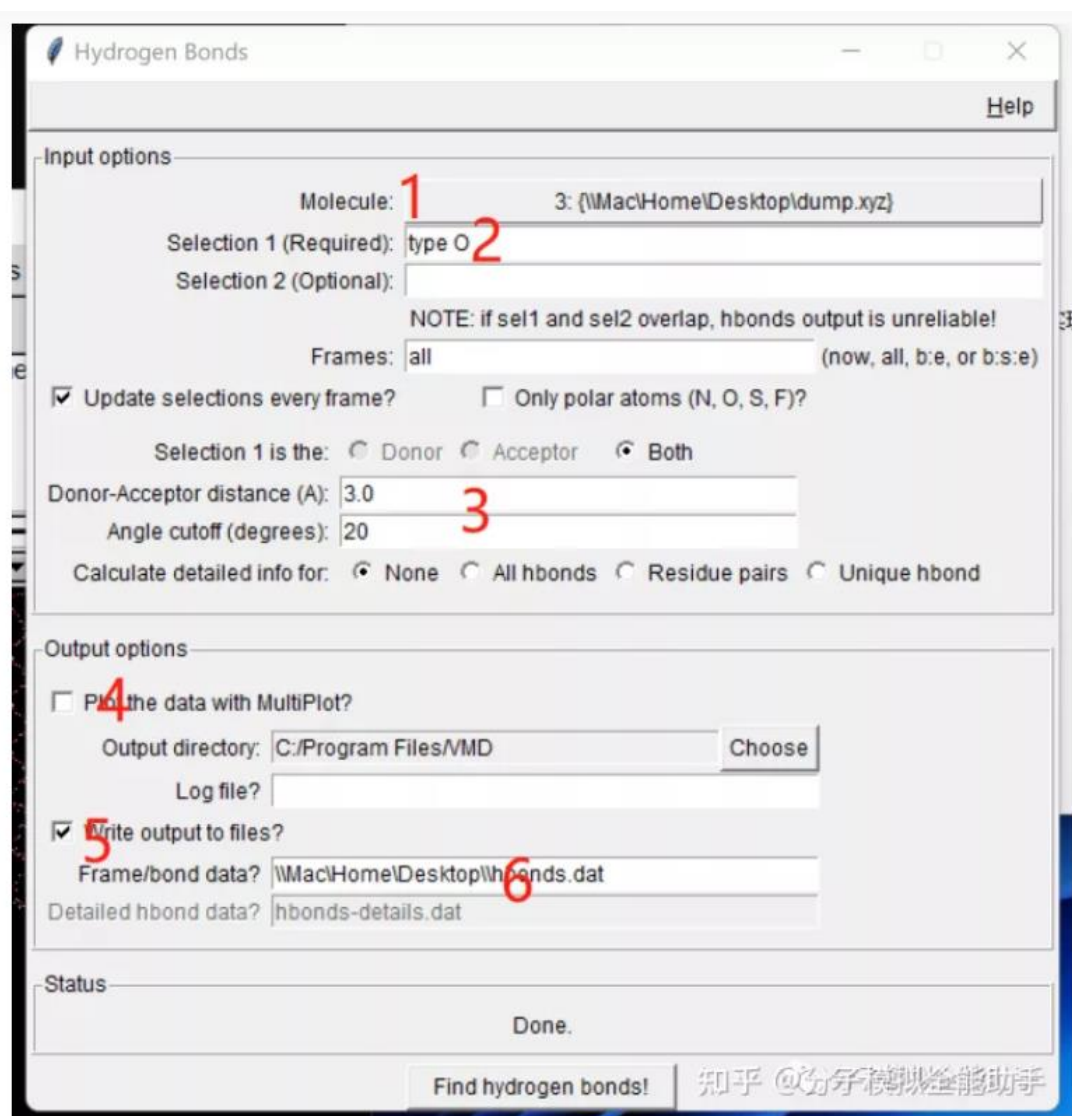
有了轨迹后就可以导入 vmd 中分析氢键了。

由于后缀是.xyz 所以 vmd 可以直接识别, 把轨迹拖入 vmd 的主窗口就可以自动

读取了。读取结束后, 按如下菜单选择打开氢键分析窗口



氢键分析窗口如下



第一步: 选择你刚才导入的 xyz 格式的轨迹

第二步: 选择要分析的元素。如果 Selection 1 既是 donor 又是 acceptor 那么只需要设置 Selection 1 就行了, 然后把后面的 Selection 1 is the 选择为 Both。如果 Selection 1 是 donor 那么就要设置 Selection 2 了。这个时候一般会把 Selection 1 设置为 donor, Selection 2 为 acceptor。也就是说 Selection 1 is the 那里选择为 Donor。这里的选择设置需要根据自己的情形。设置选择是输入 type O 就是选择为 O, type N 就是选择 N。如果设置为 all 就是选择所有。这个时候可以勾选 Only polar atoms(N, O, S, F)?

第三步: 设置氢键的几何标准, 这个一般取默认值就行, 如果需要修改则需要参考文献中的设置。

第四步: 取消绘图勾选。

第五步: 勾选输出文件信息。

第六步: 设置氢键信息的输出路径和文件名。这里最好修改为非 C 盘路径。

最后点击 Find hydrogen bonds! 按钮开始分析氢键。

结束后, 氢键信息就在输出的文件中了。

6.2 LAMMPS 建模神器-Enhanced Monte Carlo (EMC)的安装

Enhanced Monte Carlo (EMC)是除 moltemplate 之外另一款与 LAMMPS 完美对接的建模软件, 功能也很强大。特别的它的设置十分简单。

最重要的是这个软件支持自动原子分配, 这就省老鼻子事了。

EMC 的主页是: <https://montecarlo.sourceforge.net/emc/Welcome.html>

EMC 同时支持 Windows, Linux 和 MacOS, 在主页下载对应的版本就行。

EMC 提供的是可执行文件, 因此无需编译。只需要添加三个环境变量就行了。

最新版的 EMC 是 9.4.4 版本, 解压缩安装包后会出现 v9.4.4 这个文件夹, 假设你把 v9.4.4 这个文件夹放在了路径下/home/me/software。这个路径中 me 是你的 Linux 用户名, 你需要改成自己的用户名。那么需要添加的三个环境变量分别是:

首先执行

在 Linux 下执行

```
vim ~/.bashrc
```

或在 MacOS 下执行

```
vim ~/.zshrc
```

在打开的文件末尾添加以下三行（注意路径是自己的实际路径）

```
export EMC_ROOT=/home/me/software/v9.4.4
```

```
export PATH=/home/me/software/v9.4.4/bin:$PATH
```

```
export PATH=/home/me/software/v9.4.4/scripts:$PATH
```

保存退出后执行

```
source ~/.bashrc
```

或

```
source ~/.zshrc
```

这样 EMC 就安装好了。

下面是一个示例。首先你需要建立一个输入文件 setup.esh

EMC 建模基于 SMILES 字符串，而这个字符串可以很长很长（但要确保正确）

比如

```
#!/usr/bin/env emc_setup.pl
```

```
ITEM      OPTIONS
```

```
replace    true
```

```
mass       true
```

```
ntotal     1000
```

```
density    0.01
```

```
field      pcff
```

```
build_dir  .
```

```
ITEM      END
```

```
ITEM      SHORTHAND
```

```
mol
```

```
O=C(CN(CC(OCCOCCOC(CN(CCC1=CC=C(O)C(O)=C1)CC(OC[C@])(COC(C=C)
)=O)(COC(CNCCC2=CC=C(O)C(O)=C2)=O)COCC(COC(C=C)=O)(COC(CN(CC3
=CC=CC=C3)CC(OC[C@])(COC(C=C)=O)(COC(CN(CC(OCCOCCOC(CN(CCC4=
CC=C(O)C(O)=C4)CC(OC[C@])(COC(CN(CC(OCCCCOC(CN(CCC5=CC=C(O)C(
O)=C5)CC(OCC(COC(C=C)=O)(COC(C=C)=O)COCC(COC(CN(CC(OCCOCCOC
(CN(CC(OCCOCCOC(CN(CCC6=CC=C(O)C(O)=C6)CC(OCC(COC(C=C)=O)(CO
C(C=C)=O)COC[C@])(COC(C=C)=O)(COC(CN(CC(OCCOCCOC(C=C)=O)=O)CC
C7=CC=C(O)C(O)=C7)=O)COC(CNCCC8=CC=C(O)C(O)=C8)=O)=O)=O)CC
C9=CC=C(O)C(O)=C9)=O)=O)CCC%10=CC=C(O)C(O)=C%10)=O)(COC(C=C)=
O)COC(C=C)=O)=O)=O)CCC%11=CC=C(O)C(O)=C%11)=O)(COC(C=C)=O)
COC[C@])(COC(CN(CC(OCCOCCOC(C=C)=O)=O)CCC%12=CC=C(O)C(O)=C%
12)=O)(COC(CNCC%13=CC=CC=C%13)=O)COC(C=C)=O)=O)=O)CC%14=C
```

```
C=CC=C%14)=O)COCC(COC(C=C)=O)(COC(C=C)=O)COC(C=C)=O)=O)CO
C(C=C)=O)=O)=O)CCC%15=CC=C(O)C(O)=C%15)OC[C@](COC(C=C)=O)(
COC(CN(CC(OCCCCOC(C=C)=O)=O)CC%16=CC=CC=C%16)=O)COCC(COC(C
=C)=O)(COC(C=C)=O)COC(C=C)=O,1
```

```
ITEM    END
```

```
ITEM    COMMENTS
```

```
ITEM    GROUPS
```

```
mol
```

```
O=C(CN(CC(OCCOCCOC(CN(CCC1=CC=C(O)C(O)=C1)CC(OC[C@](COC(C=C
)=O)(COC(CNCCC2=CC=C(O)C(O)=C2)=O)COCC(COC(C=C)=O)(COC(CN(CC3
=CC=CC=C3)CC(OC[C@](COC(C=C)=O)(COC(CN(CC(OCCOCCOC(CN(CCC4=
CC=C(O)C(O)=C4)CC(OC[C@](COC(CN(CC(OCCOCCOC(CN(CCC5=CC=C(O)C(
O)=C5)CC(OCC(COC(C=C)=O)(COC(C=C)=O)COCC(COC(CN(CC(OCCOCCOC
(CN(CC(OCCOCCOC(CN(CCC6=CC=C(O)C(O)=C6)CC(OCC(COC(C=C)=O)(CO
C(C=C)=O)COC[C@](COC(C=C)=O)(COC(CN(CC(OCCOCCOC(C=C)=O)=O)CC
C7=CC=C(O)C(O)=C7)=O)COC(CNCCC8=CC=C(O)C(O)=C8)=O)=O)=O)CC
C9=CC=C(O)C(O)=C9)=O)=O)CCC%10=CC=C(O)C(O)=C%10)=O)(COC(C=C)=
O)COC(C=C)=O)=O)=O)CCC%11=CC=C(O)C(O)=C%11)=O)(COC(C=C)=O)
COC[C@](COC(CN(CC(OCCOCCOC(C=C)=O)=O)CCC%12=CC=C(O)C(O)=C%
12)=O)(COC(CNCCC%13=CC=CC=C%13)=O)COC(C=C)=O)=O)=O)CC%14=C
C=CC=C%14)=O)COCC(COC(C=C)=O)(COC(C=C)=O)COC(C=C)=O)=O)CO
C(C=C)=O)=O)=O)CCC%15=CC=C(O)C(O)=C%15)OC[C@](COC(C=C)=O)(
COC(CN(CC(OCCCCOC(C=C)=O)=O)CC%16=CC=CC=C%16)=O)COCC(COC(C
=C)=O)(COC(C=C)=O)COC(C=C)=O
```

```
ITEM    END
```

```
ITEM    CLUSTERS
```

```
mol      mol,1
```

```
ITEM    END
```

```
# Comments end
```

```
ITEM    END
```

然后执行

```
emc_setup.pl setup.esh
```

这个会生成一个 build.emc 的文件, 执行

Linux

```
emc_linux64 build.emc
```

macos

```
emc_macos build.emc
```

这样就可以生成 data 文件了。

6.3 怎么强迫高分子伸展和蜷缩

有时候平衡状态下高分子的状态是自发伸展的活蜷缩的, 怎么通过命令强迫高分子进行伸展或蜷缩。答案是使用

`fix spring/rg command`

该命令对 `group` 中的每个原子施加一个弹簧力是的高分子的回转半径趋向设定值。比如平衡状态下高分子是蜷缩的, 那么可以设定一个较大的回转半径值, 使的高分子的回转半径从小变到达实现伸展。如果平衡状态下你的高分子是伸展的, 那么可以设定一个较小的回转半径值, 使得高分子的回转半径从大变到小实现蜷缩。假设初始回转半径是 100 埃, 那么你可以设定一个 1000 埃大小的回转半径进行高分子强迫伸展。命令示例如下:

#real 单位

`fix sp all spring/rg 5.0 1000.0`

6.4 乙醇水溶液液滴-moltemplate 结合 packmol 建模

moltemplate 可以结合 packmol 进行建模。很方便, 但是有几个细节需要注意。在 `lt` 文件的 `atom` 定义部分和 `xyz` 或 `pdb` 文件中原子顺序要一致。`system.lt` 中 `import` 的顺序要和 `mix.inp` 文件中 `xyz` 或 `pdb` 文件的顺序一样。后面 `system.lt` 阵列的时候也要按照 `import` 的顺序。参考后面的各个文件就可以照猫画虎生成自己的 `data` 文件。

写好各个文件后首先执行

`packmol < mix.inp`

将生成的 `system.xyz` 拷贝到 `lt` 文件所在的目录下然后执行

`moltemplate.sh -atomstyle "full" -xyz system.xyz system.lt`

由于乙醇是调用 `oplsaa` 力场生成的, `moltemplate` 会把所有的 `oplsaa` 中的 `type` 都写到 `data` 文件中, 因此还需要执行

`cleanup_moltemplate.sh`

由此就完成了 `data` 文件

下面是各个文件

`wat.lt:`

`wat {`

```
write("Data Atoms") {
  $atom:O $mol:. @atom:O -0.8476 0.000000000 0.00000 0.000000
```

```

$atom:H1 $mol:. @atom:H 0.4238 -0.816490431 0.00000 0.57735867
$atom:H2 $mol:. @atom:H 0.4238 0.816490431 0.00000 0.57735867
}

write_once("Data Masses") {
  @atom:O 15.9994
  @atom:H 1.008
}

write("Data Bonds") {
  $bond:OH1 @bond:OH $atom:O $atom:H1
  $bond:OH2 @bond:OH $atom:O $atom:H2
}

write("Data Angles") {
  $angle:HOH @angle:HOH $atom:H1 $atom:O $atom:H2
}

write_once("In Settings") {
  bond_coeff @bond:OH 600.0 1.0
  angle_coeff @angle:HOH 75.0 109.47
  pair_coeff @atom:O @atom:O 0.1553 3.166
  pair_coeff @atom:H @atom:H 0.0 0.0
  group spce type @atom:O @atom:H
  fix fShakeSPCE spce shake 0.0001 10 100 b @bond:OH a @angle:HOH
}

write_once("In Init") {
  units real
  atom_style full
  pair_style lj/cut/coul/long 9.0
  bond_style harmonic
  angle_style harmonic
  kspace_style pppm 0.0001
}

}

yichun.lt:
import oplsaa.lt

yichun inherits OPLSAA {

  # atom-id mol-id atom-type charge x y
  z

```

```

write("Data Atoms") {

    $atom:c1    $mol:... @atom:80    0.0    1.000    1.00000    0.00000
    $atom:c2    $mol:... @atom:99    0.0    -0.515    1.00000    0.00000
    $atom:o     $mol:... @atom:96    0.0    -0.999    1.00000    1.33488
    $atom:h1    $mol:... @atom:85    0.0    1.390    1.00129    -1.02181
    $atom:h2    $mol:... @atom:85    0.0    1.387    0.11898    0.52261
    $atom:h3    $mol:... @atom:85    0.0    1.385    1.87982    0.52630
    $atom:h4    $mol:... @atom:85    0.0    -0.907    0.11841    -0.51618
    $atom:h5    $mol:... @atom:85    0.0    -0.897    1.89393    -0.50150
    $atom:h6    $mol:... @atom:97    0.0    -0.661    0.19796    1.76839
}

write('Data Bond List') {
    $bond:cc $atom:c1 $atom:c2
    $bond:co $atom:c2 $atom:o
    $bond:ch1 $atom:c1 $atom:h1
    $bond:ch2 $atom:c1 $atom:h2
    $bond:ch3 $atom:c1 $atom:h3
    $bond:ch4 $atom:c2 $atom:h4
    $bond:ch5 $atom:c2 $atom:h5
    $bond:oh $atom:o $atom:h6
}

}

system.lt:
import "wat.lt"
import "yichun.lt"

# Periodic boundary conditions:
write_once("Data Boundary") {
    0.0  100  xlo xhi
    0.0  100  ylo yhi
    0.0  100  zlo zhi
}

# Generate an array of 864 = 12 x 12 x 6 Butane molecules
# which (more or less) uniformly fills the simulation box:

WAT = new wat [10].move(0, 0, 5.2)
      [10].move(0, 5.2, 0)

```

[10].move(10.4, 0, 0)

YICHUN = new yichun [10].move(0, 0, 5.2)
[10].move(0, 5.2, 0)
[10].move(10.4, 0, 0)

wat.xyz:

3

water

O 0.000000000 0.00000 0.000000
H -0.816490431 0.00000 0.57735867
H 0.816490431 0.00000 0.57735867

yichun.xyz:

9

yichun

c1 1.000 1.00000 0.00000
c2 -0.515 1.00000 0.00000
o -0.999 1.00000 1.33488
h1 1.390 1.00129 -1.02181
h2 1.387 0.11898 0.52261
h3 1.385 1.87982 0.52630
h4 -0.907 0.11841 -0.51618
h5 -0.897 1.89393 -0.50150
h6 -0.661 0.19796 1.76839

mix.inp:

tolerance 4.0

seed 12345

filetype xyz

output system.xyz

structure wat.xyz

number 1000

inside sphere 50 50 50 40

end structure

```
structure yichun.xyz
  number 1000
  inside sphere 50 50 50 40
end structure
```

6.5 ReaxFF 防止弛豫发生分解反应

当使用建模软件建好有机模型后在弛豫的时候分子通常都会发生分解。这个有多种原因。主要有两种：第一力场参数不符合你模拟的体系；第二建立的模型在 reaxff 力场下偏离平衡状态太远。当你确认力场合适的时候，可以采用给分子加一个额外的 bond 约束。当你使用建模软件，比如 ms 建好模型后将 data 文件中 angle, dihedral 和 improper 部分都删除，只保留 bond 部分，并把所有的 bond type 都改为 1。然后添加 bond_style harmonic 进行额外的约束。然后开始弛豫，在弛豫的过程中不断减小 bond_coeff 的能量项逐渐消除约束的强度直到完全消除约束，然后写出 data 文件。这就要求你要重复执行多次计算才行，可以使用 loop 实现循环计算。此时的 data 文件就是弛豫好的模型，可以直接开始跑反应。添加 bond 约束的命令如下：

```
bond_style harmonic
bond_coeff 1 500 1.2
```

#这条命令一定要加否则会使用默认的设置，而消除 reaxff 对键结原子的应用

```
special_bonds lj/coul 1.0 1.0 1.0
```

完整的 in 文件如下：

```
units      real
boundary    s s s
atom_style  full
neighbor    3.0 bin
neigh_modify every 1 delay 0 check no

read_data  50.data

pair_style  reaxff NULL safezone 4.8 mincap 400 minhbonds 200
pair_coeff  * * SiONH.ff C H O N
bond_style harmonic
bond_coeff 1 500.00 1.2
special_bonds lj/coul 1.0 1.0 1.0

timestep    0.25
```



```
velocity all create 300.0 4928459 rot yes dist gaussian
thermo 100
thermo_style custom step temp pe ke
thermo_modify flush yes

fix          1 all nvt temp 300.0 300.0 25.0
fix          3 all qeq/reaxff 1 0.0 10.0 1e-6 reaxff

dump 1 all custom 500 dump.lammpstrj id mol type q x y z
fix sp all spring/rg 5.0 100.0

run 1000000
```

6.6 铁表面镀碳膜-a toy case

铁表面镀碳膜, 一个简单的示例。

in 文件如下

```
# ----- Init Section -----

boundary p p p
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes
atom_style atomic
units metal
pair_style tersoff/zbl

# ----- Atom Definition Section -----
lattice bcc 2.855324
region box block 0 10 0 10 0 20
create_box 2 box
region Fe block 0 10 0 10 0 4
create_atoms 1 region Fe

# ----- Run Section -----

pair_coeff * * FeC.tersoff.zbl Fe C

mass 1 55.845
mass 2 12.011

group Fe type 1
group C type 2
region fix block INF INF INF INF INF 1
group fix region fix
group Femove subtract Fe fix
```

```

timestep 0.002
region dep block 0.5 28 0.5 28 25 56 units box
fix dep C deposit 2000 2 100 4564456 region dep attempt 100 near 3.0 vz -1.0 -0.1
units box
compute myT C temp
compute_modify myT dynamic/dof yes
variable Cd atom "type==2"
group Cd dynamic all var Cd
variable nC equal count(Cd)
velocity Femove create 300 372748 rot yes dist gaussian loop local
thermo 100
thermo_style custom step temp v_nC
thermo_modify temp myT flush yes lost ignore

fix blnc all balance 1000 1.05 shift xyz 10 1.05
dump mydump all custom 1000 dump.lammpstrj id type mass x y z

fix fxnvt1 C nve
fix fxnvt2 Femove nvt temp 300 300 0.2

run 200000
write_data equ.data
write_restart restart.equ

```

6.7 初始密度很小使用 npt 弛豫如何防止出现 PPPM 错误 (NaOH 溶液例子)

本例发布 OPLSAA 力场下的 NaOH 溶液的 in 和 data 文件, 获取请加微信

lmp_zhushou

只需要把一个长 run 打断成多个短 run 就可以避免出现 PPPM 错误, 以下是 NaOH 溶液例子弛豫的例子

```
# ----- Init Section -----
```

```

units real
atom_style full
bond_style harmonic
angle_style harmonic
dihedral_style opls
improper_style harmonic
pair_style lj/cut/coul/long 11.0 11.0
pair_modify mix geometric
special_bonds lj/coul 0.0 0.0 0.5
kspace_style ppm 0.0001

```

```
# ----- Atom Definition Section -----

read_data "system.data"

# ----- Settings Section -----

pair_coeff 1 1 0.1553 3.166
pair_coeff 2 2 0.0 2.058
pair_coeff 3 3 0.0005 4.07
pair_coeff 4 4 0.25 3.2
pair_coeff 5 5 0.0 0.0
bond_coeff 1 600.0 1.0
bond_coeff 2 553.0 0.945
angle_coeff 1 75.0 109.47
set type 3 charge 1.0
set type 4 charge -1.3
set type 5 charge 0.3

# ----- Run Section -----

timestep 1.0

velocity      all create 500 372748 rot yes dist gaussian loop local
thermo        1000
thermo_style  custom step temp density
thermo_modify flush yes

dump mydump all custom 10000 dump.lammpstrj id mol type mass x y z

fix fxnpt all npt temp 500 500 100.0 iso 100 100 1000

run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
```

[illegible]

[illegible]

[illegible]

```
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000
run 5000

write_data steady1.data
write_restart steady1.restart

unfix fxnpt
fix fxnpt all npt temp 300 300 100.0 iso 1 1 1000

run 100000

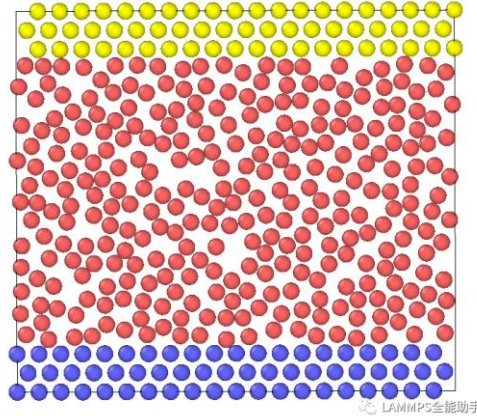
write_data steady.data
write_restart steady.restart
```

6.8 纳米流动剪切应力计算方法对比

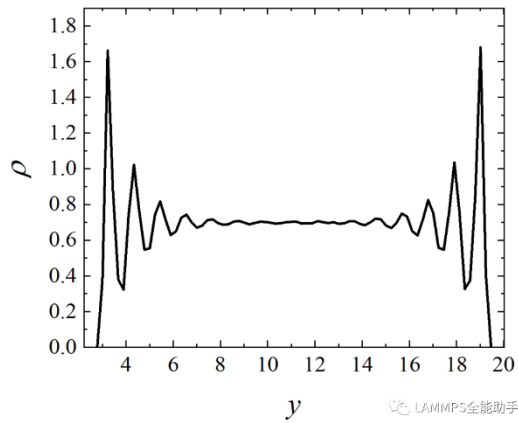
前面的例子讲到 LAMMPS 中原生的计算局部应力的方法只适应于体相系统，而不能在界面上进行计算。另一种计算方法则可以在界面上进行计算。两种方法分别是 Harashima 方法和 Irving-Kirkwood 方法，以下记为 H 方法和 IK 方法。下面通过以下简单的纳米尺度下的剪切驱动流来证明 H 方法并不能用来计算界面附近的局部应力。而现在 IK 方法在去除宏观速度方面还有待修正。通过修改源代码又可以发一篇小文章。将两种方法应用于以氩为基础流体的模拟中可以进行纳米流体的研究，得到局部粘度，热导率等信息加上理论分析，使用这个理论上的模型可以发一篇不错的论文。这对于没有太多计算资源而着急发论文的同学是再合适不过了。

计算的 in 文件加微信 lmp_zhushou 获取。

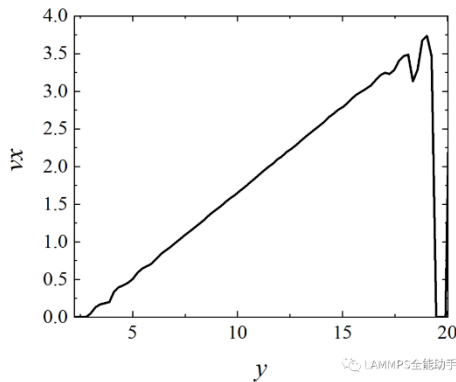
模拟的模型如下：



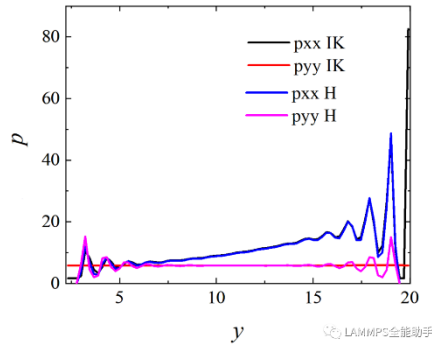
首先来看密度分布。密度分布出现了周期性振荡, 这是纳米流动的典型特征。就是在界面上的这个信息导致 H 方法在界面处的局部应力计算失败。



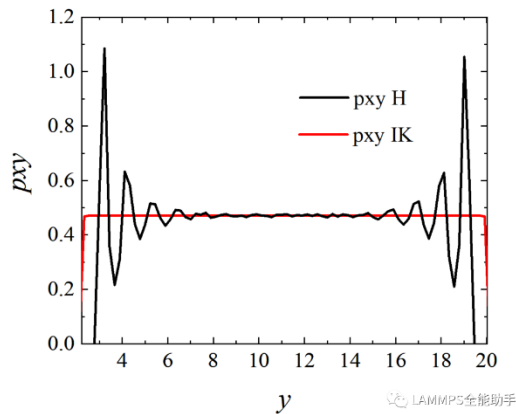
速度分布也出现了振荡, 这个是什么问题呢, 我还在纠结。



正应力如下。对于 p_{yy} 来说在 y 方向上没有宏观速度, 那么应力应该处处相等, 但是 H 方法在界面处出现了振荡, 而 IK 方法则没有。这就表明 H 方法的错误和 IK 方法的正确。对于 p_{xx} 由于在 x 方向上存在宏观速度, 速度对应力的贡献导致 IK 方法也出现了振荡并且与 H 方法结果相同。这个结果可以通过修正计算应力的方法来得到正确的 p_{xx} 。这个简单的修正和细致的计算可以发一篇小文章。



在剪切驱动流动根据定常流动的力学平衡, 可以确定剪切应力在不同位置出应该为定值。但是 H 方法只在体相区为定值, 而在界面附近不为定值, 再次说明 H 方法的不合理。而 IK 方法则完美的计算出了定值的剪切应力, 说明 IK 方法的正确性。



6.9 两种局部应力计算方法

什么叫局部应力? 局部应力是连续介质的概念, 是指任何一个位置处的应力, 不管这个位置是否存在原子。比如要算 z 方向上的应力分布。 z 的坐标是连续变化的, 但是原子是离散分布的。如何将离散分布的原子的速度和受力信息转换到连续的空间中是计算局部应力的关键之处。到目前为止使用 LAMMPS 有两种计算局部应力的方法。第一种是原生的 LAMMPS 计算功能, 第二种是 Nakamura 等对 LAMMPS 进行扩展而实现的计算功能。第一种可以在 LAMMPS 的所有版本中使用, 第二种仅限于 lammps-1Feb14 版本。当然你可以修改 Nakamura 的源代码使其适应 LAMMPS 的最新版本。

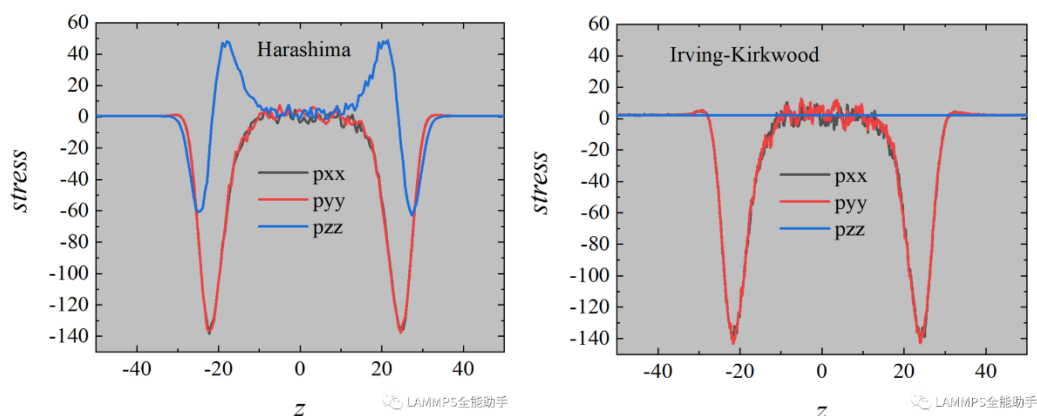
两种方法的计算细节和示例 in 文件可以加微信 [lmp_zhushou](#) 获取。

第一种是利用 `compute stress/atom` 和 `fix ave/chunk` 命令实现计算。通过 `fix ave/chunk` 命令统计平均出单原子应力 (单位是 `stress*vol`), 然后用单原子应力乘

以 bin 中的时间平均的原子个数再除以 bin 的体积就是这个 bin 位置处的应力。第二种则是直接利用 Nakamura 等开发的 `compute stress/spatial` 命令进行计算然后用 `fix ave/time` 进行平均并输出。

两种方法的区别是从离散原子信息向连续空间转换的方式不同。第一种采用 Harashima 方法, 第二种采用 Irving-Kirkwood 方法。第一种 LAMMPS 原生支持可以对任何体系进行计算, 但是统计涨落较大并且在界面处应力计算会出错。第二种方法只能计算简单液体和部分电场下的分子体系, 但是在界面和体相取都很准确且统计涨落小。

下面是两种方法计算的液体氩的气液界面上的应力分布图。有意思的是尽管两种方法在气液界面上的应力分布不同但是计算出来的表面张力确实一致的。Harashima 计算出来的是 1067.343, Irving-Kirkwood 1062.437。确实一致。所以利用 LAMMPS 中的 `compute stress/atom` 是可以直接计算界面的表面张力的。

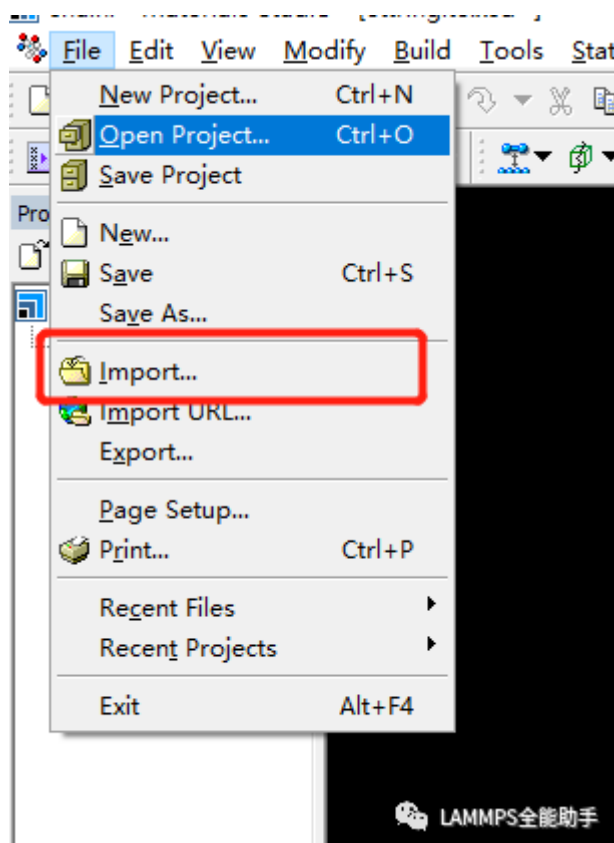


6.10 水泥体系的钙矾石拉伸的 LAMMPS 模拟

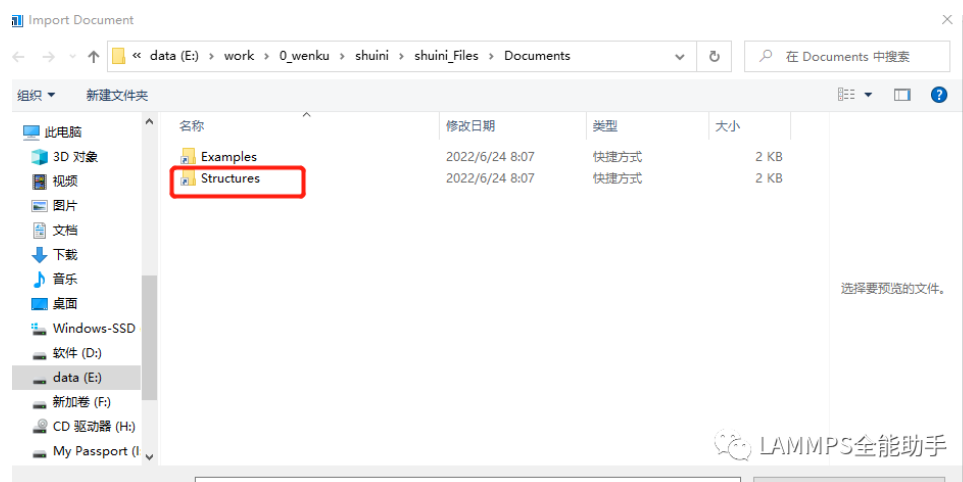
使用 ReaxFF 研究水泥基体系是其重要的应用, 可以研究其腐蚀老化等化学反应机理。本例发布钙矾石的拉伸性能模拟。in 文件和 data 文件以及力场文件加微信 [lmp_zhushou](#) 获取。文件未经过测试, 欢迎反馈问题。

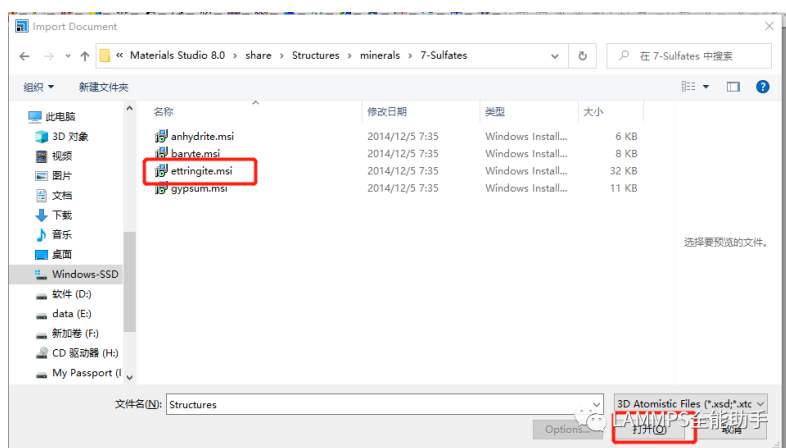
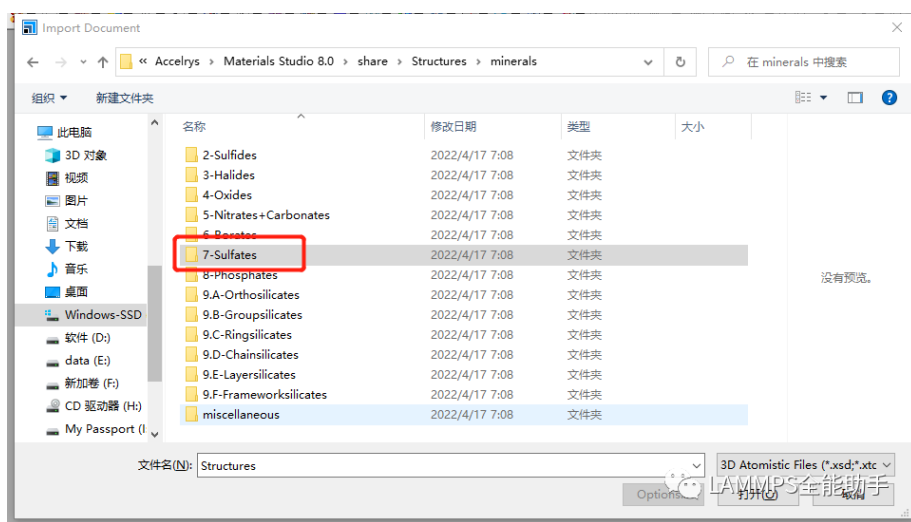
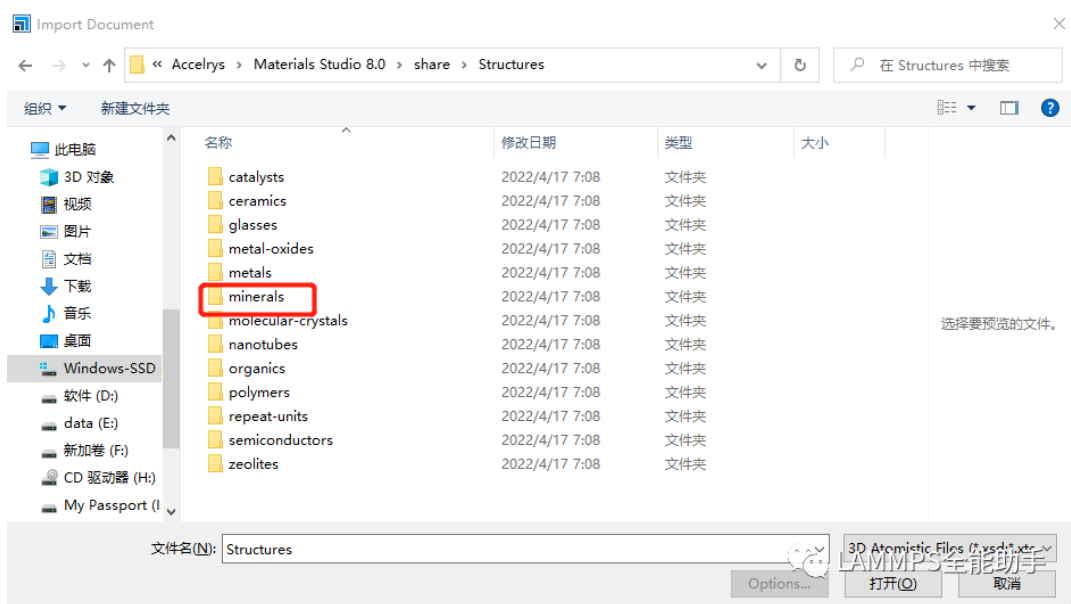
首先是建模过程

打开 MS, 点击 **import**

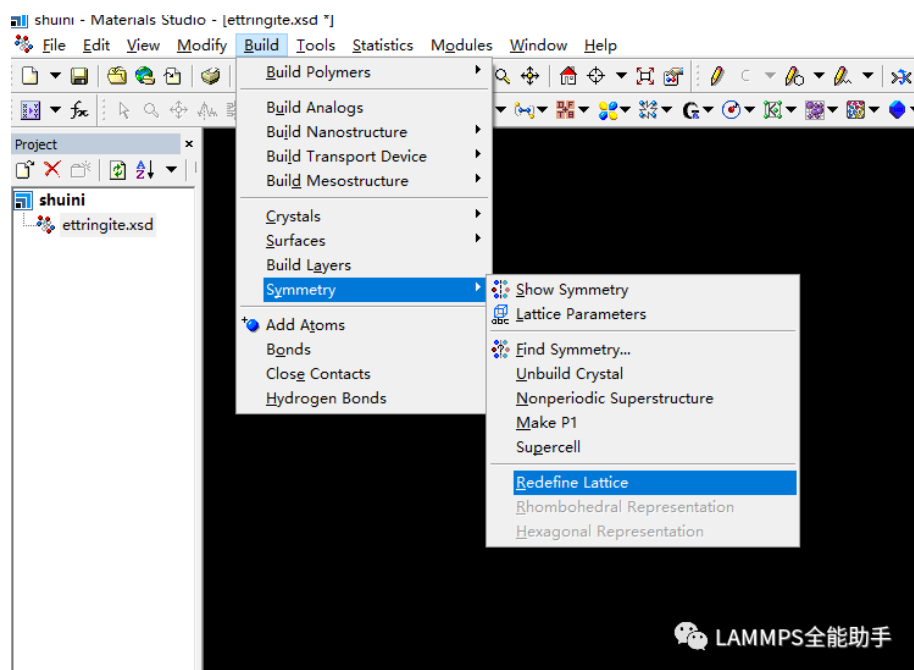


在打开的对话框中, 分别打开 Structures->minerals->7-Sulfates->ettringite

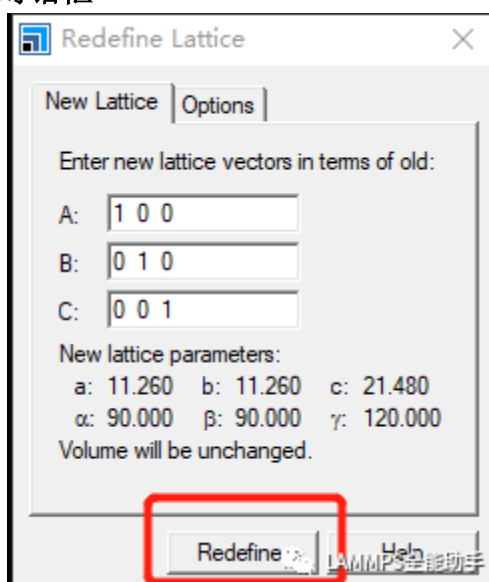




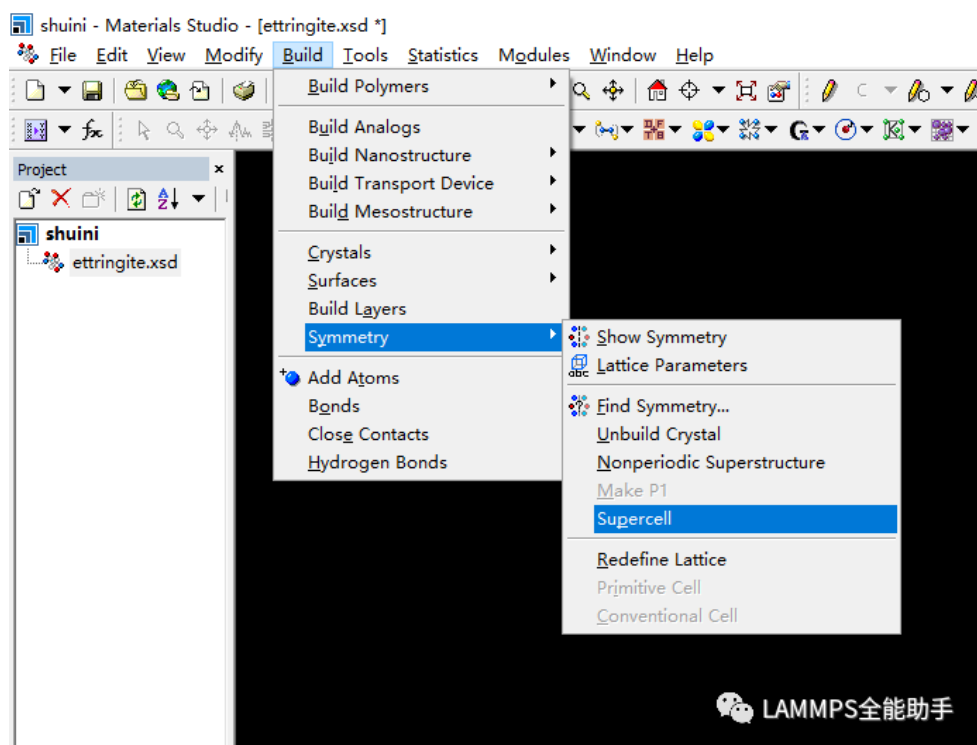
执行操作, 打开 **Redefine Lattice**



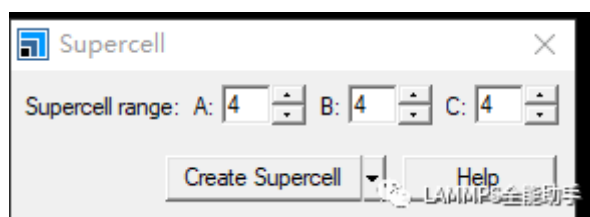
点击 **Redefine** 后关闭对话框



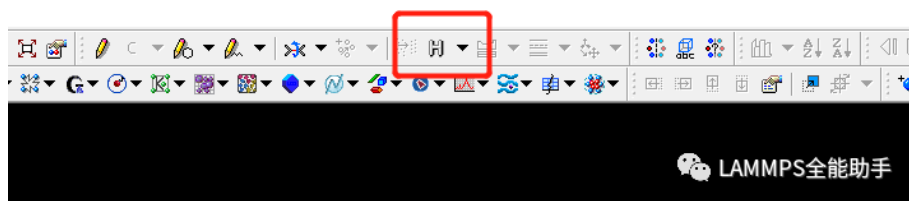
打开 **Supercell** 对话框



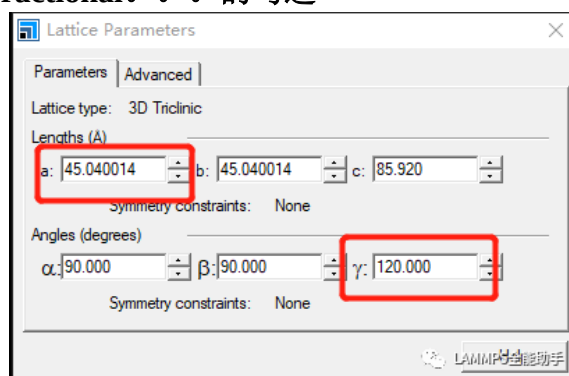
按需要修改扩胞数量, 这里设置都是 4, 然后点击 **Create Supercell**

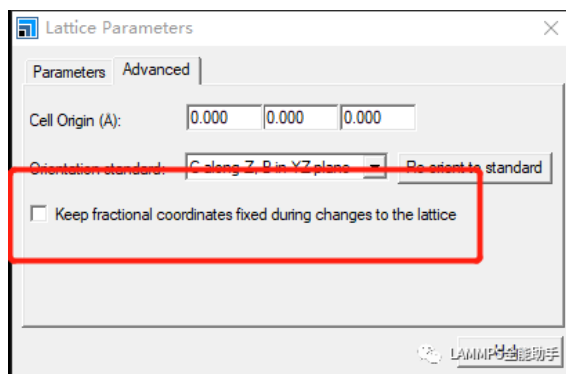


点击补氢按钮

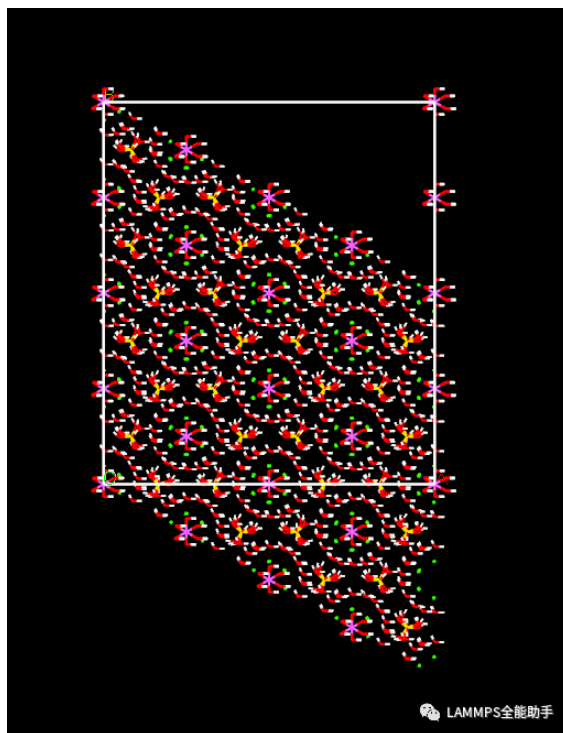


打开 Lattice Parameters, γ 修改为 90.000, a 修改为原来值乘以 $\cos 30$, 修改前要确保取消 **Keep fractional**。。的勾选

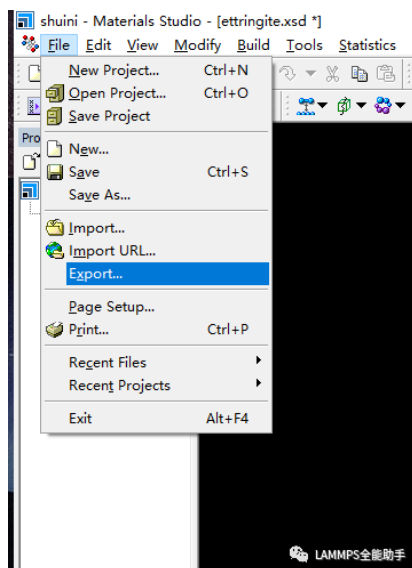




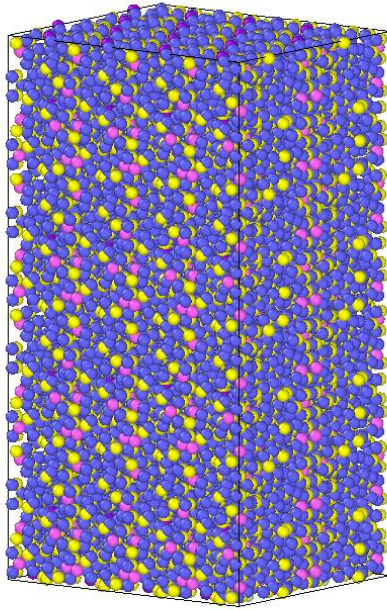
建完的模型如下图, 超出盒子的不是会由周期性边界条件显示的原因。



将模型导出为 cif 格式



用文本编辑器打开保存的 cif 文件拷贝盒子尺寸和坐标, 在 Excel 中组合为 data 文件, 记得要将坐标按照盒子三个方向上的尺寸进行缩放, 然后将内容拷出到另一个文本文件建立 data 文件。下面就是建立的模型



LAMMPS全能助手

```
in 文件内容
units      real
atom_style charge
neighbor 2.0 bin
neigh_modify every 1 delay 0 check no

read_data system.data

pair_style reaxff NULL
pair_coeff * * ff.reaxff C H O Ca Si Al S

timestep 0.1
velocity all create 300.0 4928459 rot yes dist gaussian
thermo 100
thermo_style custom step temp pe ke
thermo_modify flush yes

fix 2 all npt temp 300 300 10 iso 0 0 100
fix 3 all qeq/reaxff 1 0.0 10.0 1e-6 reaxff

fix 1 all reaxff/bonds 1000 bonds.reaxff
dump 1 all custom 1000 dump.lammpstrj id type q x y z
```

```
run      40000
```

```
unfix 2
```

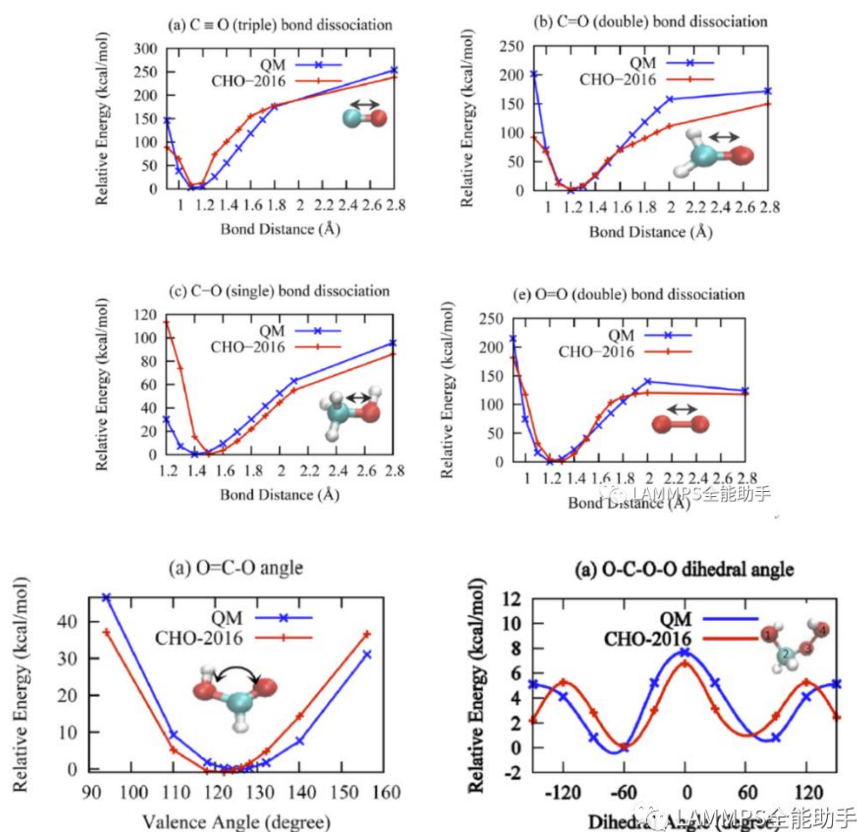
```
fix 2 all npt temp 300 300 10 y 0 0 100 z 0 0 100
```

```
fix 3 all deform 1 x erate 0.001 remap x
```

```
run 1000000
```

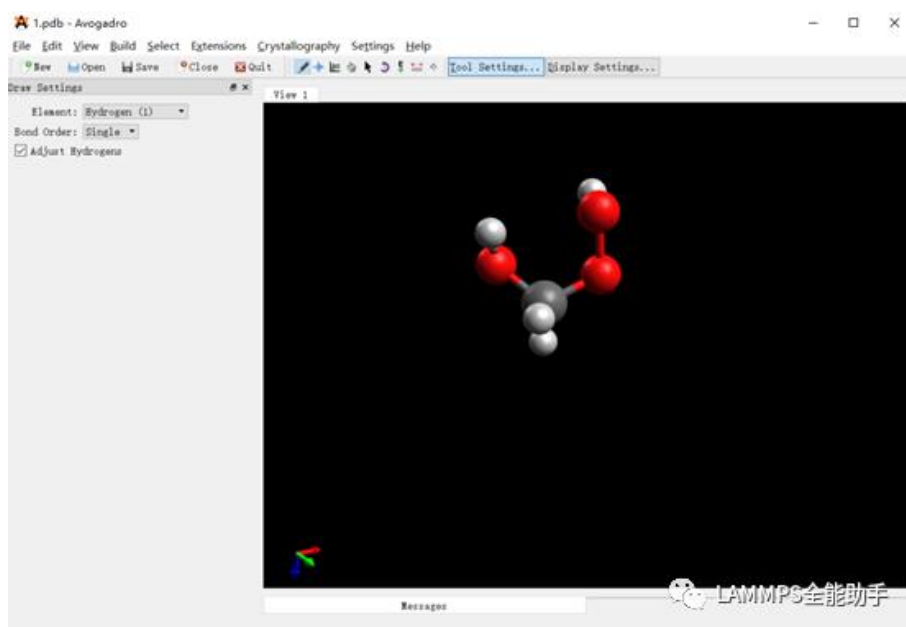
6.11 ReaxFF 力场之键结参数验证-对比 DFT 和 ReaxFF

当你找到一个力场, 要针对自己的体系进行验证, 你就需要将自己的体系中的分子进行简化, 使其在包含关键化学基团的同时越简单越好。验证的主要过程就是对比由 DFT 计算得到的能量和 ReaxFF 计算得到的能量, 分子越简单能量对比越有效。为说明这里验证的方法的可行性 (好像可行, 后面会说明), 本例复现一下论文中的结果。以下是论文 Extension of the ReaxFF Combustion Force Field toward Syngas Combustion and Initial Oxidation Kinetics 中的力场部分验证结果。

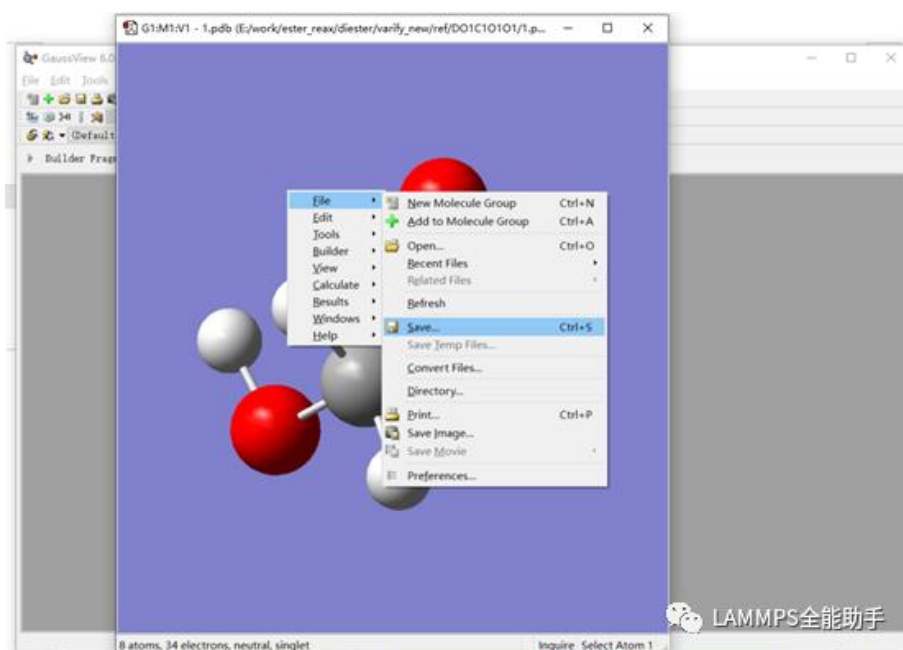


为了验证力场, 首先要获得 DFT 的计算结果, 也就是采用 DFT 方法对势能面性扫描。扫描采用高斯 16 软件进行刚性扫描。刚性扫描速度快, 并且对于小分

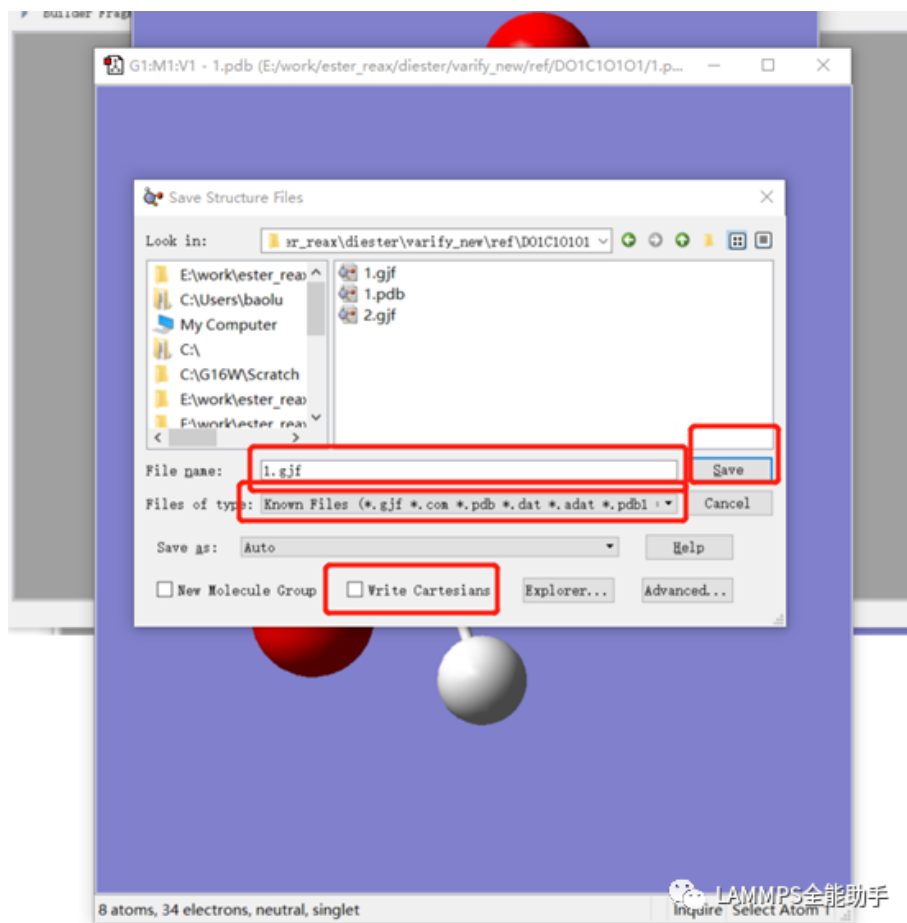
子来说柔性扫描和刚性扫描结果相差不大。下面通过上图中第六个图的结果说明高斯中如何进行刚性扫描。首先根据图中的分子在 Avogadro 软件中绘制出分子结构并保存为 pdb 文件。在绘制分子的时候尽量和图中的分子构象保持一致, 如下图所示。



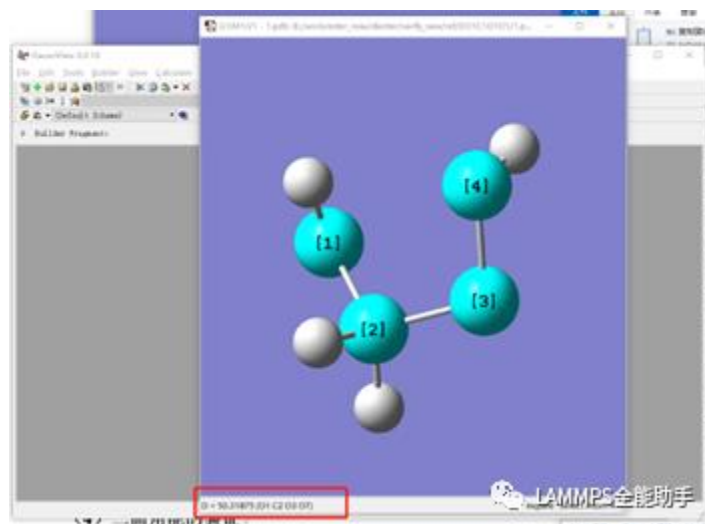
利用 GaussView 将保存的 pdb 文件打开, 并保存为 gif 格式。保存的流程是在打开的窗口中鼠标右击选择 File->Save..., 打开保存窗口。



在打开的保存窗口中 File name 中设置文件名称, Files of type 中选择 Know files, 取消 Write Cartesians 的勾选, 然后点击保存。



在刚才打开的 `pdb` 文件窗口中鼠标在原子上左击选择要扫描的二面角的四个原子。查看窗口左下角处构成该二面角的四个原子的编号。



用文本编辑器将刚才保存的 `gif` 文件打开。将第二行改为 `b3lyp/6-31g(d) scan nosymm`, 其中 `b3lyp/6-31g(d)`表示 DFT 计算要选择的基组, `scan` 表示执行扫描任务, 做刚性扫描的过程中, 如果涉及到点群的改变, 不加 `nosymm` 关键词的话有时会出问题, 而且看到的扫描轨迹可能不连续。要扫描的二面角由编号为 7 3

2 1 四个原子组成, 从文件中可以看出这个二面角的编号是 D4。因此将下面 D4 行中的二面角角度改为-90.0 24 10.0。-90.0 表示扫描初始角度为-90°, 24 表示扫描 24 个角度, 10.0 表示按 10.0° 进行角度递增。这里需要说明涉及到浮点数的一定要写为浮点数形式, 比如角度要写为-90.0, 10.0, 而不能是-90 和 10。修改完后保存文件。

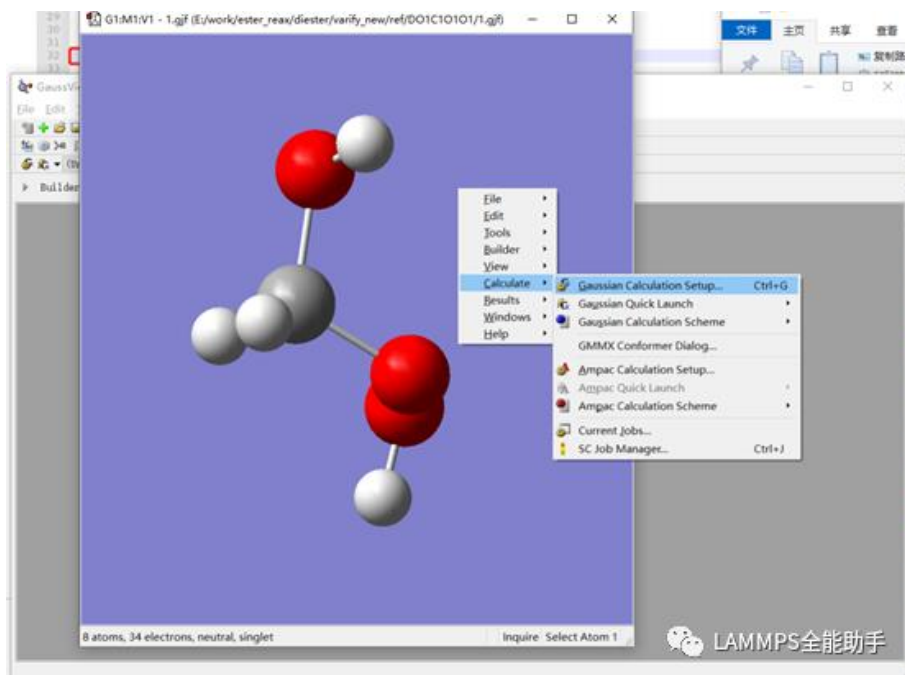
```

1 # b3lyp/6-31g(d) scan nosymm
2
3
4 Title Card Required
5
6 0 1
7 O(PDBName=O, ResName=UNL, ResNum=1)
8 C(PDBName=C, ResName=UNL, ResNum=1)
9 O(PDBName=O, ResName=UNL, ResNum=1)
10 H(PDBName=H, ResName=UNL, ResNum=1)
11 H(PDBName=H, ResName=UNL, ResNum=1)
12 H(PDBName=H, ResName=UNL, ResNum=1)
13 O(PDBName=O, ResName=UNL, ResNum=1)
14 H(PDBName=H, ResName=UNL, ResNum=1)
15
16 B1      1.41910289
17 B2      1.42404038
18 B3      0.97274354
19 B4      1.09247380
20 B5      1.09419240
21 B6      1.46186251
22 B7      0.97687512
23 A1      111.06031387
24 A2      103.95331024
25 A3      108.56857182
26 A4      110.01574060
27 A5      106.19363504
28 A6      95.67164404
29 D1      -75.58688444
30 D2      -118.91932220
31 D3      122.08190478
32 D4      -90.0 24 10.0
33 D5      -105.34367420
34
35 1 2 1.0 4 1.0
36 2 3 1.0 5 1.0 6 1.0
37 3 7 1.0
38 4
39 5
40 6
41 7 8 1.0
42 8
43
44

```

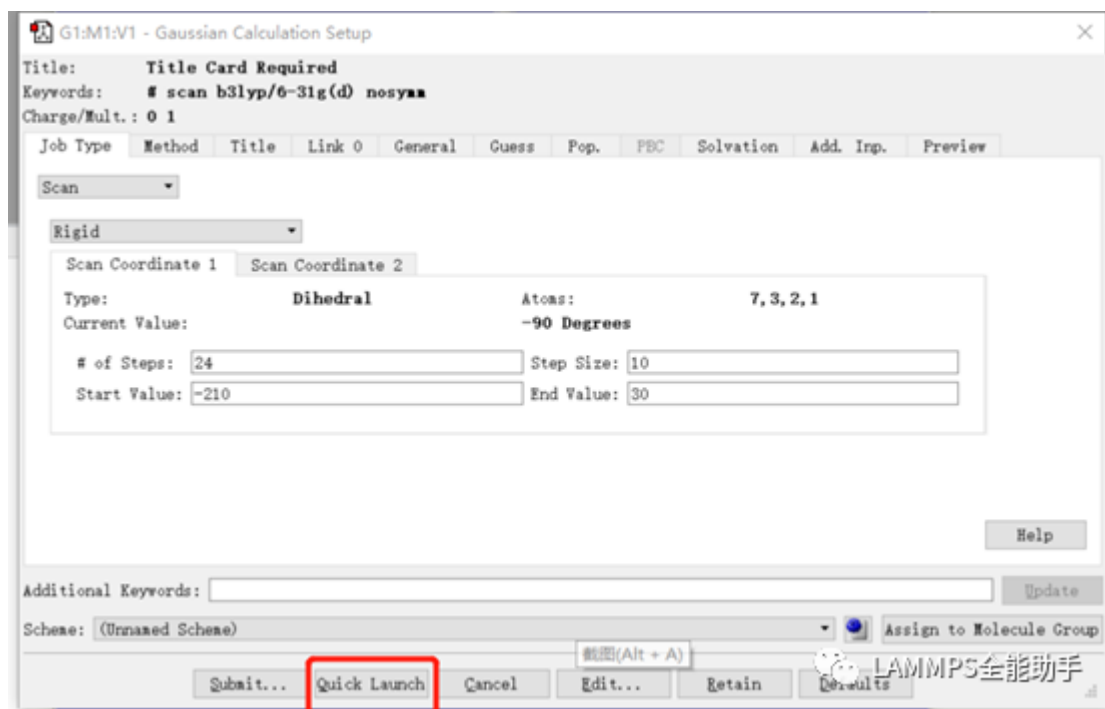
LAMMPS全能助手

用 GaussView 打开刚刚修改并保存的 gif 文件, 右击窗口空白处, 按下图进行选择, 打开计算设置窗口 (不要关闭 gif 窗口)。

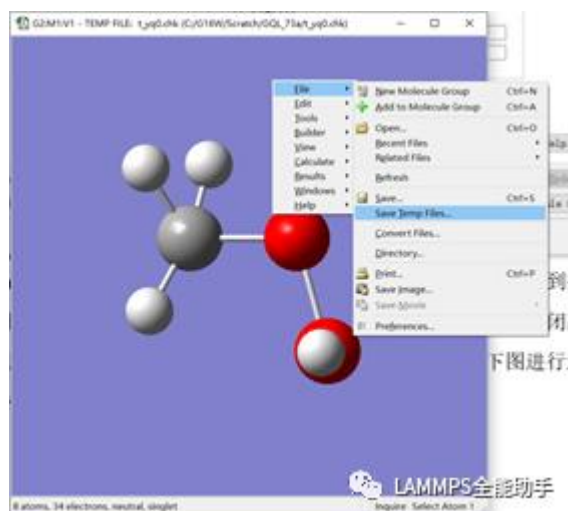


LAMMPS全能助手

在打开的计算设置窗口中点击下方 Quick Launch 启动高斯开始计算。



Quick Launch 点击后会自动弹出高斯计算界面和另一个 chk 窗口。可以看到控制台在持续输出, 由于是小分子, 计算会比较快。计算完成后会弹出是否关闭高斯的对话框, 点击是。稍等几秒钟, 在 chk 窗口空白处鼠标右击, 按下图进行选择打开保存临时文件窗口, 在打开的窗口中设置保存的文件名称, 点击保存。



会输出四个新的文件分别是 chk, fch, gif, log 文件。将 log 文件用文本编辑器打开, 拉到文件末尾可以看到扫描得到的每个角度对应的能量, 包括初始角度本例中会得到 25 行数据。将数据拷贝到绘图软件中。得到的能量的单位是 Hartree, 需要转换为 Kcal/mol 以和 lammmps 的计算结果进行对比。1 Hartree=625.7 Kcal/mol。

在对比 DFT 结果和 ReaxFF 结果时都会采用相对能量, 即将所有计算得到的能量减去能量的最小值。

scan completed.

Summary of the potential surface scan:

N	D4	SCF
1	-90.0000	-266.05860
2	-80.0000	-266.05952
3	-70.0000	-266.05979
4	-60.0000	-266.05907
5	-50.0000	-266.05718
6	-40.0000	-266.05421
7	-30.0000	-266.05061
8	-20.0000	-266.04722
9	-10.0000	-266.04504
10	0.0000	-266.04489
11	10.0000	-266.04697
12	20.0000	-266.05082
13	30.0000	-266.05544
14	40.0000	-266.05967
15	50.0000	-266.06263
16	60.0000	-266.06398
17	70.0000	-266.06383
18	80.0000	-266.06260
19	90.0000	-266.06078
20	100.0000	-266.05882
21	110.0000	-266.05708
22	120.0000	-266.05581
23	130.0000	-266.05510
24	140.0000	-266.05496
25	150.0000	-266.05523

LAMMPS全能助手

log 文件中不仅会有扫描的最终结果, 还会记录每一个角度对应的所有原子的坐标, 通过提取这些坐标就可以得到每个角度对应的 data 文件, 用以 LAMMPS 计算。提取 log 文件中的坐标并转换为一系列 data 文件的 python 脚本如下:

#设置 log 文件的路径

```
work_path = 'E:\\work\\ester_reax\\diester\\varify_new\\ref\\DO1C1O1O1\\'
```

```
input_file = open(work_path + '2.log','r')
```

#设置原子个数

```
natom = 8
```

```
line = input_file.readline()
```

```
no = 0
```

```
while line:
```

```
    tmp1 = line.split()
```

```
    if len(tmp1) == 2:
```

```
        if tmp1[0] == "Z-Matrix" and tmp1[1] == "orientation:":
```

```
            no += 1
```

```
            line = input_file.readline()
```

```
            line = input_file.readline()
```

```
            line = input_file.readline()
```

```
            line = input_file.readline()
```

```
            output_file = open(work_path + 'data.'+str(no),'w')
```

#LAMMPS 的头部内容, 可按需修改, 这里只有 CHO 三种元素

```
            output_file.write("LAMMPS data file Created\n")
```

```
            output_file.write("\n")
```

```
            output_file.write(str(natom) + " atoms\n")
```

```

output_file.write("\n")
output_file.write("3 atom types\n")
output_file.write("\n")
output_file.write("-20 20.0 xlo xhi\n")
output_file.write("-20 20.0 ylo yhi\n")
output_file.write("-20 20.0 zlo zhi\n")
output_file.write("\n")
output_file.write("Masses\n")
output_file.write("\n")
output_file.write("1 12.011\n")
output_file.write("2 1.008\n")
output_file.write("3 15.999\n")
output_file.write("\n")
output_file.write("Atoms\n")
output_file.write("\n")
for i in range(natom):

```

```

    line = input_file.readline().split()
    atom_id = line[0]
    mol_id = "1"

```

#根据原子序号分配 data 文件中的原子 type 编号, 这里只有

CHO 三种元素

```

    if line[1] == "6":
        atom_type = "1"
    elif line[1] == "1":
        atom_type = "2"
    else:
        atom_type = "3"

```

```

    atom_q = "0"

```

```

    x = line[3]

```

```

    y = line[4]

```

```

    z = line[5]

```

```

    tmp = atom_id + " " + mol_id + " " + atom_type + " " + atom_q +
    " " + x + " " + y + " " + z + "\n"

```

```

    output_file.write(tmp)

```

```

    output_file.close()

```

```

    line = input_file.readline()

```

利用得到的一系列 data 文件就可以计算 ReaxFF 下的能量, 计算的 in 文件如下:

variable a loop 25

log log.\$a

variable b index 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

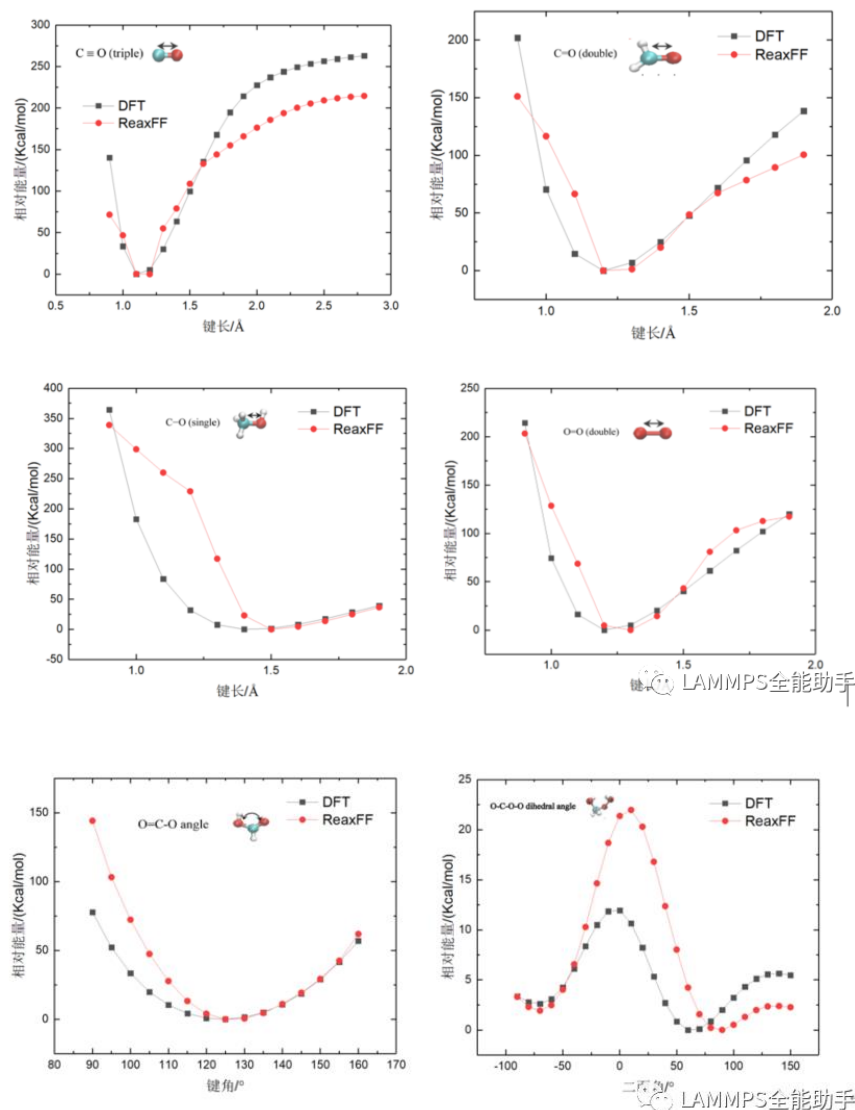
16 17 18 19 20 21 22 23 24 25

units real

atom_style full


```
neighbor 2.0
bin
neigh_modify every
1 delay 0 check no
read_data data.$b
pair_style reaxff
NULL
pair_coeff * *ffield2016.reax.cho C H O
timestep 0.25
velocity all create 300.0 4928459 rot yes dist
gaussian
thermo 100
thermo_style custom step temp pe ke etotal
thermo_modify flush yes
fix 3 all qeq/reaxff 1 0.0 10.0 1e-6 reaxff
variable mye equal etotal
run 1000
print ${mye} append info.dat screen no
clear
next b
next a
jump system.in
```

在输出的 `info.dat` 文件中就是所有 `data` 文件对应的能量，将数据拷贝到绘图软件中，并将所有数据减去这些能量中的最小值，得到相对能量。由此便可对比 DFT 和 ReaxFF 的计算结果了。以上就是力场验证的所有过程。本例中对应文献中的六个图进行对比了 DFT 和 ReaxFF 的结果，如下图。从图中可以看出得到的对比图和文献中的差不多。但是还是有差别，特别是对于键来说，当键长小于平衡键长后这里计算的能量总是比文献中的较大，对于键角来说，当键角小于平衡键角后能量也比文献中的较大，对于二面角来说当二面角在零度左右也比文献中的大。但是对于 ReaxFF 和 DFT 对比的可接受程度来说，这里的验证是合理的。因为得到的误差和文献中的基本一致。至于为什么 LAMMPS 计算的 ReaxFF 的能量会与文献中的有符合很好，也有不符合的，我也百思不得其解。知道原因的可以加我微信 `lmp_zhushou`，我们一起讨论。



6.12 ReaxFF 力场之 Qeq 电荷计算参数验证

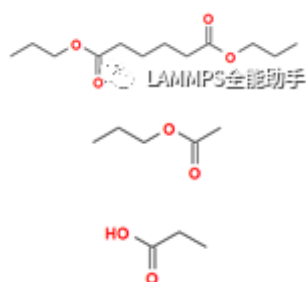
ReaxFF 已经存在很多的力场参数。很多时候能够找到包含所需元素的力场参数。但是力场参数往往是根据特定体系而开发的，是否适用于自己的体系则需要验证力场参数。验证力场参数的基本流程就是对比自己体系中的分子各种能量（如键，键角，二面角）与 DFT 计算的能量。如果吻合较好那么找到的力场参数就可以用，如果不好就要基于找到的力场参数进行进一步优化。本节介绍力场参数的验证过程。严格的验证力场需要对比各项能量和参数，包括键结、非键结和电荷平衡参数。本例介绍如何进行电荷平衡参数验证

这个可以通过计算一个完整分子和分子片段内原子的电荷，对比 DFT 和 ReaxFF 的计算结果。以合成酯为例，合成酯中只包含 CHO 三种元素。可以使用这篇文章 (Chowdhury Ashraf and Adri C.T. van Duin*)

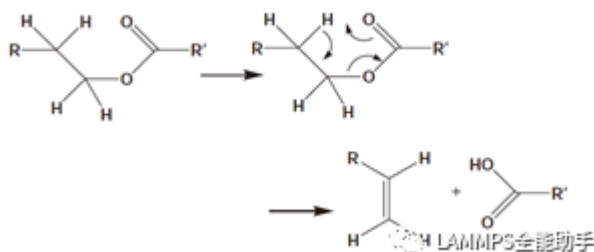
Extension of the ReaxFF Combustion Force Field toward Syngas Combustion and Initial Oxidation Kinetics J. Phys. Chem. A 121, 5, 1051-1068) 中所开发出的力场参数。把力场参数下载下来, 命名为 ffield2016.reaxcho。目前在 LAMMPS 中只用 Qeq 方法进行电荷动态平衡。该方法要求有三个参数分别是

chi eta gamma

对于 LAMMPS 来说, 可以直接采用 ReaxFF 中力场文件中的参数, 也可以自己单独写一个文件定义这些参数。通常会直接采用力场文件中的参数。这里要验证的由 lammps 的例子中给出的电荷平衡参数。对于合成酯采用以下几个分子进行验证。



为什么要选择这几种。这就需要对研究对象有充分的了解。这里要研究合成酯的热解问题, 那么首先合成酯的完整分子自然要考虑, 一半合成酯也可以考虑一下。然后根据前人研究合成酯会发生以下热解过程, 那么热解过程中的反应产物也要考虑。而不含 O 的自由基和分子就不用考虑了, 因为待验证的力场在开发时已经充分验证了。



这里采用 LAMMPS 计算 ReaxFF 所得到的电荷并与采用量子化学的电荷进行对比。要计算首先要有输入文件的坐标。可以采用 Avogadro 进行坐标生成。这里没有采用 MS 是因为 MS 太大了, 这么个小事不需要这么大的软件。使用 Avogadro 分别生成上述三个分子的 pdb 文件和 xyz 文件。在 Avogadro 中画完

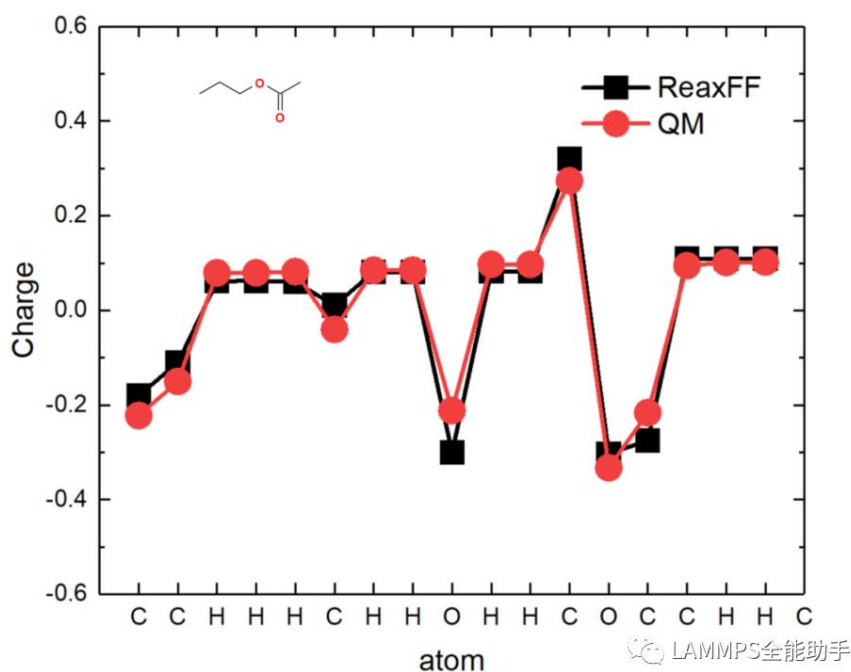
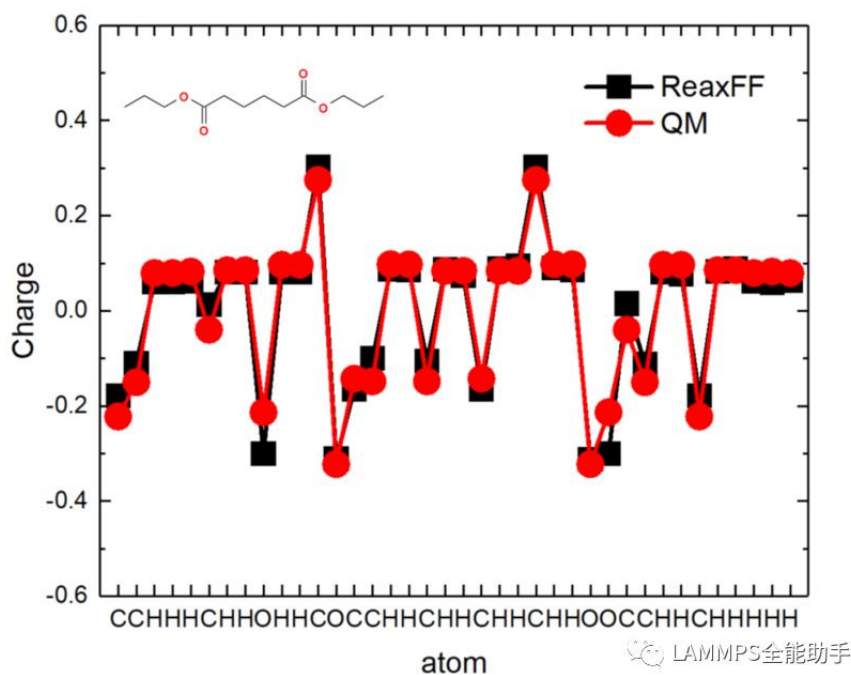
分子之后一定要做几下结构优化只有再导出文件。Qeq 方法只需要分子几何构型和原子的实验性质就可以预测分子内部的每个原子的电荷。先将导出的 pdb 文件中转换为 data 文件（一个分子就够了），然后在 lammps 中计算每个原子的部分电荷。计算 in 文件如下

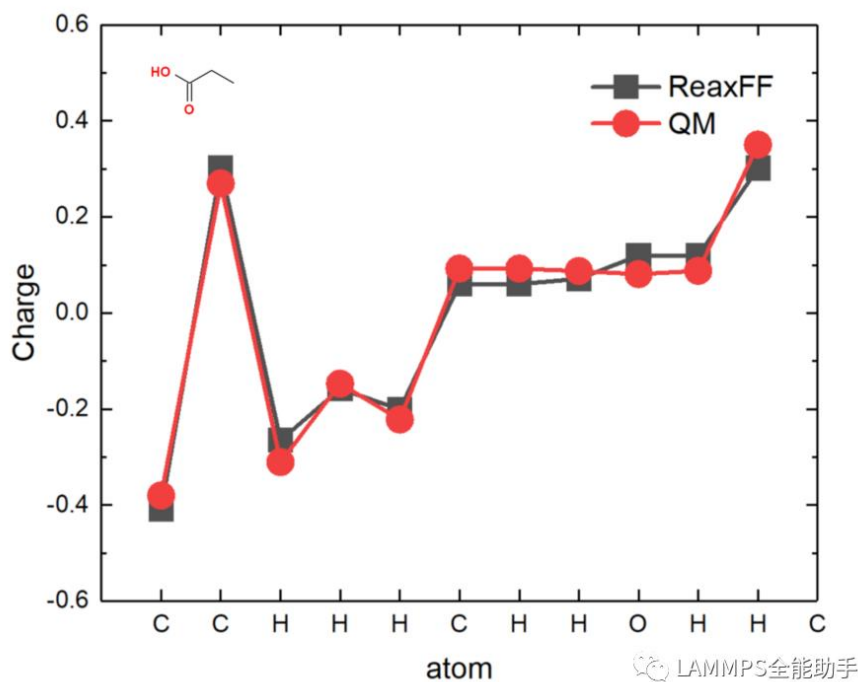
```
units real
atom_style full
neighbor 2.0
bin
neigh_modify every
1 delay 0 check no
read_data half.data
pair_style reaxff
lmp_control
pair_coeff * *ffield2016.reax.cho C H O
timestep 0.25
velocity all create 300.0 4928459 rot yes dist
gaussian
thermo 100
thermo_style custom step temp pe ke
thermo_modify flush yes
fix          1 all nve
fix 2 all temp/berendsen 300.0 300.0 50.0
fix          3 all qeq/reaxff 1 0.0 10.0 1e-6
param.qeq
run 100000
dump 1 all custom 100 dump.lammpstrj id mol type q x y
z
dump_modify 1 sort id
run 100000
```

这样就能得到由 ReaxFF 计算得到的电荷。然后使用导出的 xyz 文件，调用 Multiwfn 软件和 Gaussian 程序使用量子化学计算得到的电荷。这里我们采用 1.2CM5 方法计算电荷。计算的采用的量子化学信息为 B3LYP-D3(BJ)/def2-SVP。由于 1.2*CM5 电荷就是在 Truhlar 等人提出的原始的 CM5 电荷基础上乘上 1.2，这相当于增大了原子电荷的数量级，等效地体现出了溶剂环境对溶质的极化作用。这里的模拟没有溶剂化作用，所以需要对得到的电荷除以 1.2 得到 CM5 电荷。关于如何联用 Multiwfn 软件和 Gaussian 程序计算 CM5 电荷，参见网址[计算适用于 OPLS-AA 力场做模拟的 1.2*CM5 原子电荷的懒人脚本](#) - 思想家公社的门口：

量子化学 分子模拟 二次元 (sobereva.com)。这里要求 Gaussian 程序是 Linux 版下的。如需获取 Linux 版的 Gaussian 可加微信 baolu_yao。

以下是上述三个分子中 ReaxFF 计算得到的原子电荷和量子化学得到的原子电荷对比。从三个图中可以看到这里所采用的 Qeq 参数以很高的精度再现了量子化学计算得到的电荷, 表明采用 Qeq 参数是合理的。具体的 Qeq 参数见下表。





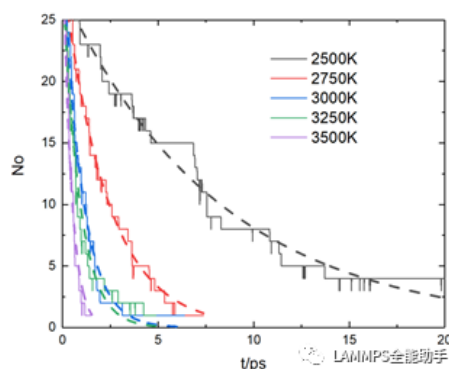
元素	χ	η	γ
C	5.8678	14.0000	0.9000
H	5.3200	14.8732	1.0206
O	8.5000	17.9978	1.0503

6.13 有机分子热分解的 ReaxFF 模拟及活化能计算

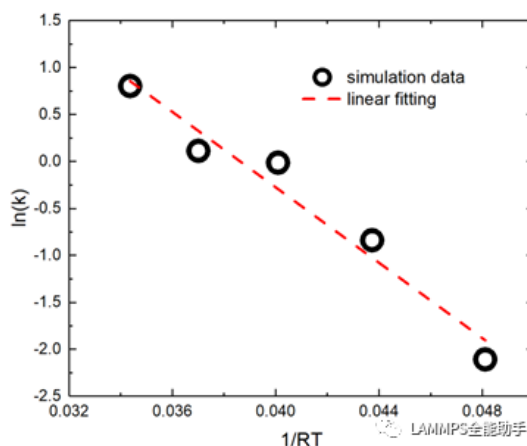
使用 ReaxFF 力场的反应分子动力学的一个重要的应用就是进行热分解模拟。关于这个主题只要能细致地考究 ReaxFF 力场参数的细节, 就可以获得准确的结果。比如, ReaxFF 的提出者 Duin 教授采用 ReaxFF 对 PDMS 的研究发表在了 JACS 上。本例发布采用 ReaxFF 计算有机分子的热分解并说明如何计算活化能。

反应力场下的模拟步长为 0.1fs, 共弛豫 10000 个分子动力学步。由于热分解反应是一个稀有事件在 1000K 温度和 10000 个时间步内不会发生分解。此时通过将系统温度瞬间提高以加速热分解的发生, 并监测残余完整分子个数随时间的变化。平衡后的初始阶段, 分子都很完整, 热分解并未发生。将系统温度提高至 2750K, 少量分子开始发生分解。随着模拟时间的推移发生热分解的原子越来越多。通过 fix 4 all reaxff/species 输出的文件信息获得原始分子随时间的变化, 提取脚本见公众号文章[小工具之 ReaxFF 产物个数随时间的变化统计](#)。如

果要观察分子式如何分解, 即键是如何断裂和生成的, 可使用公众号中的脚本: [小工具之 ReaxFF 成键断键可视化](#)。本例中合成酯的热分解温度一般在 570K 左右, 设置这么高的温度在于加速分解反应的发生, 以便在有限的模拟时间内完成全部分子热分解的模拟。因此这里假设在不同温度下 (即使是很高的温度) 分子的热分解机理和过程是一样的, 同时这也是反应力场模拟分解反应的常见策略。模拟过程中通过设置不同的温度就可以获得合成酯分子的残余完整个数随时间的变化。残余分子个数与模拟时间成指数关系, 因此采用公式对 $N=N_0e^{-kt}$ 曲线进行拟合。拟合参数就是在当前温度下的热分解速率。



根据 Arrhenius 公式 $k=Ae^{(-E_a/RT)}$ 对不同温度下的热分解速率进行拟合就可以得到反应的活化能和前置速率因子。拟合中取 $R=8.314\text{J/mol/K}$ 。下图为不同温度下根据 Arrhenius 公式的拟合结果。拟合得到的前置速率因子为 $2332.5 \times 10^{12}\text{s}^{-1}$, 活化能为 $200.7 \pm 23.9\text{ KJ/mol}$ 。实验活化能为 $204 \pm 5\text{ KJ/mol}$ 。由此表面第一高温加速反应的策略是合理的, 第二力场是很准确的。所以再次证明 ReaxFF 只要有可靠的力场就能很准确的模拟化学反应。



in 文件如下。data 文件加微信 [lmp_zhushou](#) 获取

```
#循环多个温度
variable a loop 10
log log.$a
variable T index 2750 3000 3250 3500 3750 4000 4250 4500 4750 5000
units      real
atom_style full #采用 full 方便后面追踪分子
neighbor 2.0 bin
neigh_modify every 1 delay 0 check no

read_data reax.data #data 文件只需包含一条链就行
replicate 3 3 3 #通过复制得到想要的分子个数

pair_style reaxff NULL #这里建议使用 NULL，当然也可以使用 lmp_control 文件进行细节设定
pair_coeff * *ffield2016.reax.cho H C O

timestep 0.1
velocity all create 1000.0 4928459 rot yes dist gaussian
thermo 100
thermo_style custom step temp pe ke
thermo_modify flush yes

fix 1 all nve
fix 2 all temp/berendsen 1000.0 1000.0 50.0 #使用别的温度控制方法也行
fix 3 all qeq/reaxff 1 0.0 10.0 1e-6 reax/c #使用力场中的电荷平衡参数

fix 4 all reaxff/species 1 1 100 species_${T}.out element H C O
fix 1 all reaxff/bonds 1000 bonds_${T}.reaxff #与 dump 输出频率一致以使用公众
号中的工具
dump 1 all custom 1000 dump_${T}.lammppstrj id mol type mass x y z

run 40000

unfix 2
fix 2 all temp/berendsen ${T} ${T} 50.0 #不用升温直接将温度设定为目标值开始
分解

run 200000

clear
next T
next a
jump system.in
```

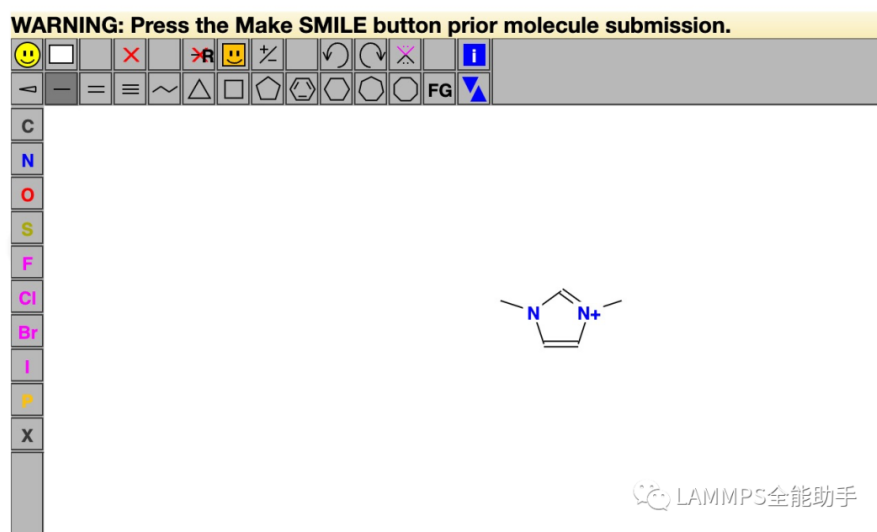

6.14 离子液体建模教程

本例发布如何结合 Ligpargen 和 moltemplate 建立离子液体的 in 和 data 文件。这里以如下的离子液体为例



其中 R=CH₃。

首先在 <http://zarbi.chem.yale.edu/ligpargen/moleculeDraw.html> 这个网址的分子结构绘制面板中画出阳离子的结构，如下图



然后点击 make smiles



然后按下图选择

Step 1: Input structure

Step 2: Options

Molecule Optimization Iterations

Select charge model:

☐ 1.14*CM1A-LBCC (Neutral molecules)

☒ 1.14*CM1A¹ (Neutral or Charged molecules)

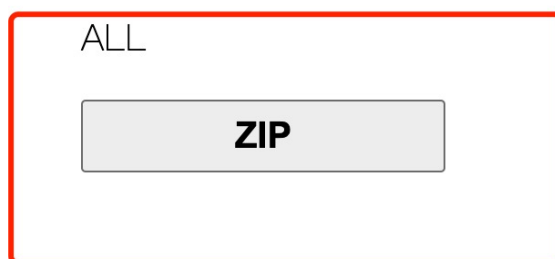
Molecule charge

¹ For charged molecules, CM1A charges are NOT scaled by a factor 1.14

LAMMPS全能助手

然后点击 submit molecule, 等待网站执行完成

然后点击 ZIP 下载文件



_OR

Q

OP

QPARM

LAMMPS全能助手

将下载文件解压缩后在解压出的文件找到后缀名为.lmp 的文件, 就是我们要的一个分子的 data 文件。

采用相同的方法建立阴离子的一个分子的 data 文件。注意建立阴离子需要将分子电荷选择为-1

将两个.lmp 文件分别重命名为 yanglizi.lmp 和 yinlizi.lmp，然后使用 moltemplate 的这个脚本 **ltemplify.py** 将 data 文件转换.lt 文件

在装完 moltemplate 软件后这个脚本可以直接使用，在终端中进入 yanglizi.lmp 和 yinlizi.lmp 这两个文件所在的目录，然后分别执行以下两条命令

```
ltemplify.py -name yanglizi yanglizi.lmp > yanglizi.lt
```

```
ltemplify.py -name yinlizi yinlizi.lmp > yinlizi.lt
```

由此就可得到阳离子和阴离子的.lt 文件

然后就可以写 system.lt 文件对阴阳离子进行整列复制得到所需的 data 文件

```
import "yanglizi.lt"
```

```
import "yinlizi.lt"
```

```
write_once("Data Boundary") {
```

```
    -50    550   xlo xhi
```

```
    -50    550   ylo yhi
```

```
    -50    550   zlo zhi
```

```
}
```

#移动间隔要根据阴阳离子的尺寸进行设定，如果不想麻烦就可以设定一个很大的值比如 50

```
YAN = new yanglizi [10].move(0, 0, 50)
```

```
        [10].move(0, 50, 0)
```

```
        [10].move(50, 0, 0)
```

```
YIN = new yinlizi [10].move(0, 0, 50)
```

```
        [10].move(0, 50, 0)
```

```
        [10].move(50, 0, 0)
```

#阴离子移动一下防止和阳离子重叠

```
YIN[*][*][*].move(50,50,50)
```

然后执行 `moltemplate.sh -atomstyle "full" system.lt`, 就可生成 `data` 文件了。in 文件需要自己写, 我们需要生成的 `system.data` 和 `system.in.settings` 文件。以下是弛豫示例的 in 文件。后续的分析可以在 in 文件中添加内容得到。

```
# ----- Init Section -----
```

```
boundary p p p
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes
atom_style full
units real
bond_style harmonic
angle_style harmonic
dihedral_style opls
improper_style cvff
pair_style lj/cut/coul/long 8.5 8.5
pair_modify mix geometric
special_bonds lj/coul 0.0 0.0 0.5
kspace_style pppm 0.0001
```

```
# ----- Atom Definition Section -----
```

```
read_data "system.data"
include "system.in.settings"
```

```
# ----- Run Section -----
```

```
timestep 1.0
```

```
velocity      all create 500 372748 rot yes dist gaussian loop local
thermo        1000
thermo_style  custom step temp
thermo_modify flush yes
```

```
dump mydump all custom 10000 dump.lammpstrj id mol type mass x y z
```

```
fix fxnpt all npt temp 500 500 100.0 iso 100 100 1000
```

```
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
run 10000  
unfix fxnpt  
fix fxnpt all npt temp 400 400 100.0 iso 1 1 1000  
run 500000  
write_data equil.data  
write_restart restart1.equ
```

计算命令是

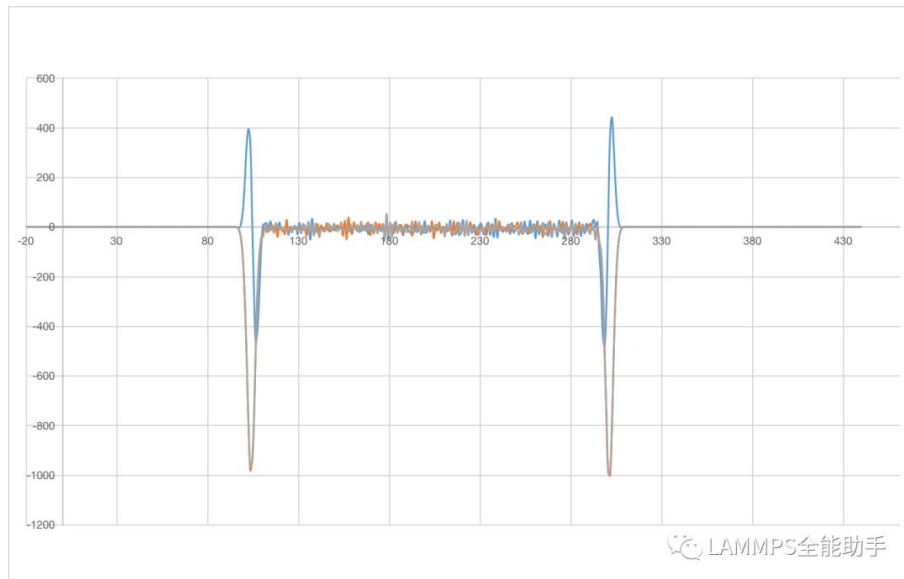
```
compute myp all stress/atom NULL
```

```
compute cc1 all chunk/atom bin/1d x lower 1.0 units box
```

```
fix 1 all ave/chunk 1 1000000 1000000 cc1 c_myp[1] c_myp[2] c_myp[3] norm sample  
file p.profile
```

计算完成后打开输出的 p.profile 文件，文件中第一列是 bin 的编号，第二列是 bin 的位置，第三列是 bin 中的平均原子数，第四列到第六列是 c_myp[1], c_myp[2] , c_myp[3]。用第四列乘以第三列再除以 bin 的体积取相反数就是 pxx，用第五列乘以第三列除以 bin 的体积取相反数就是 pyy，用第六列乘以第三列除以 bin 的体积取相反数就是 pzz。将得到的 pyy 和 pzz 取平均就得到了平行于液面的压强 pt，而 pxx 就是垂直于液面的压强 pn。将 pn 列减去 pt 列，并将得到的结果乘以 bin 的厚度，此处是 1 埃。将最终得到的结果列求和除以 2 就是表面张力值。本例计算出的表面张力值是 54mN/m，文献值是 52mN/m，实验值是 72mN/m。由此可见本例的计算结果是可靠的。

pxx，pyy 和 pzz 沿垂直液面方向的分布曲线如下图



6.16 纳米气泡电润湿形态变化

表面纳米气泡是现在的一个研究热点。本例发布纳米气泡的电润湿模拟。电润湿通过在近壁区施加一个力来等效电润湿。该方法来自论文 Atomistic simulations of electrowetting in carbon nanotubes, Nano Letters。可以看到当电润湿施加后纳米气泡的形态发生了很大的变化。由于气体原子个数是固定的，当纳米气泡接触

角减小的同时，气泡体积也大幅减小。这是符合拉普拉斯定律的。通过这个设置可以研究电润湿对纳米气泡寿命的影响，以及纳米气泡内外扩散平衡的动力学过程。本例不需要 data 文件，建模在 in 文件中完成。

variable a loop 11

log log.\$a

variable F index 0.00 0.05 0.10 0.15 0.20 0.25 0.30 0.35 0.40 0.45 0.50

dimension 3

boundary p p p

units real

atom_style atomic

neighbor 2.0 bin

neigh_modify every 1 delay 0 check yes

pair_style lj/cut 17

region box block 0 400 0 40 0 400 units box

create_box 3 box

lattice fcc 4.0

region solid block 0 400 0 40 0 20 units box

create_atoms 1 region solid

lattice fcc 6.5

region liquid block 0 400 0 40 23 300 units box

create_atoms 2 region liquid

group solid type 1

group liquid type 2

region del block 150 250 INF INF 20.1 60 units box

delete_atoms region del

lattice fcc 12

region gas block 100 300 INF INF 20.1 60 units box

create_atoms 3 region gas

group liquid type 2

group gas type 3

group fluid union liquid gas

mass 1 20.0

mass 2 20.0

mass 3 20.0

#in e_ss, e denotes epsilon; in s_ss, the first s denotes sigma, the second and third s denotes solid

#l denotes liquid; g denotes gas

variable e_ss equal 0.715077

variable e_ll equal 0.715077

variable e_gg equal 0.238359

variable s_ss equal 3.4

variable s_ll equal 3.4

variable s_gg equal 5.0

variable e_sl equal $0.1 * \{e_{ll}\}$

variable e_sg equal 0.476718

variable e_lg equal $0.25 * \{e_{ll}\}$


```
variable s_sl equal 0.5*($s_ss+$s_ll)
variable s_sg equal 0.5*($s_ss+$s_gg)
variable s_lg equal 1.62*$s_ll

pair_coeff 1 1 ${e_ss} ${s_ss}
pair_coeff 1 2 ${e_sl} ${s_sl}
pair_coeff 1 3 ${e_sg} ${s_sg}

pair_coeff 2 2 ${e_ll} ${s_ll}
pair_coeff 2 3 ${e_lg} ${s_lg}

pair_coeff 3 3 ${e_gg} ${s_gg}

fix 1 solid setforce 0 0 0
minimize 0.0 1.0e-8 1000 100000

timestep 2.0
thermo 1000
compute myT fluid temp
thermo_style custom step temp
thermo_modify temp myT flush yes

velocity fluid create 300 5645354 temp myT loop local dist gaussian rot yes
dump mydump all atom 50000 atom_$F.lammpstrj

fix fxnvt fluid nvt temp 300.0 300.0 200.0

run 500000

region add block INF INF INF INF 20 23.5 units box
```

```
fix add liquid addforce 0.0 0.0 -$F region add
```

```
run 500000
```

```
reset_timestep 0
```

```
compute cc2 liquid chunk/atom bin/2d x 0.0 3.0 z 0.0 3.0
```

```
fix ave liquid ave/chunk 1 1000000 1000000 cc2 density/mass norm sample file  
rho_-$F.profile
```

```
run 1000000
```

```
clear
```

```
next F
```

```
next a
```

```
jump system.in
```

6.17 耗散粒子动力学

耗散粒子动力学是专为流体而开发的介观粗粒化模拟方法。通过将全原子尺度下的多个原子粗粒化为一个粒子，从而降低体系的自由度个数，提升计算效率。该方法中被抹除掉的自由度由一对满足涨落-耗散定理的耗散力和随机力进行描述。这也是耗散粒子动力学的名字的由来。耗散粒子动力学经过多年发展已经开发出适用于体相流体的耗散粒子动力学、适用于自由界面模拟的多体耗散粒子动力学，适用于传热模拟的能量耗散粒子动力学等。本专题主要介绍原始的耗散粒子动力学。（有关其他耗散粒子动力学的东西，如虚拟边界滑移控制方法和代码，悬浮液非牛顿特性模拟，电磁流变液模拟可添加上述微信进行免费交流）

在 LAMMPS 中主要通过 pair dpd 进行基础或原始耗散粒子动力学模拟。pair dpd 会包含三部分力：保守力，耗散力和随机力。其中耗散力和随机力组合起来起到了温度控制器的作用。因此在耗散粒子动力学中系统一定要采用 fix nve。耗散粒子动力学最简单的例子就是模拟通道流，即两无限大平行平板间的层流现象。

在耗散粒子动力学中比较重要的是边界条件的设置。为了模拟固液之间滑移现象，即通过参数控制实现不同程度的滑移：从无滑移到无限滑移，研究者提出了很多方法，如固定粒子法，虚拟耗散边界法等。本节通过具体的 in 文件讲解来介绍。

（in 文件未经测试如果发现问题欢迎反馈）

#####基本模拟规则设置#####

units lj #耗散粒子动力学中统一采用无量纲化单位

dimension 3

boundary p p p

neighbor 0.3 bin

neigh_modify delay 0 every 1 chech yes

atom_style atomic #如果没有键直接采用这个最简单的原子类型

pair_style dpd 1.0 2.5 546564 #温度一般设定为 1

comm_modify vel yes #需要这条命令来使得 ghost 原子存储速度信息

#####定义几何模型#####

region box block -20 20 -20 20 -20 20 units box

create_box 2 box

region upwall block -20 20 -20 20 15 20 units box

lattice fcc 12 #为防止流体原子从壁面中逃逸出去，壁面的密度要设置很大

create_atoms 1 upwall

region downwall block -20 20 -20 20 -20 -15 units box

lattice fcc 12 #为防止流体原子从壁面中逃逸出去，壁面的密度要设置很大

create_atoms 1 downwall

region fluid block -20 20 -20 20 -15 15 units box

lattice fcc 3 #流体的密度一般设置为 3，也可以设置为其他值，这和你杜丽华程度有关

create_atoms 2 fluid

#####定义物理模型#####

pair_style 1 1 3.0 1.0 #保守力和耗散系数的大小可以用来控制流体的可压性和粘度

pair_style 1 2 3.0 1.0 #壁面原子和流体原子之间的保守力可以用来防止流体逃逸出壁面，如果发现有原子深入壁面了，这个值就可以扩大。壁面原子和流体原子之间的耗散系数可以用来控制滑移程度，如果是 0 就是无限滑移，如果很大就是无滑移

#####定义分组信息#####

group wall type 1

group fluid type 2

#####弛豫部分#####

timestep 0.01 #由于时粗粒化，保守力是有限值，所以时间步可以大一点

thermo 1000

thermo_style custom step temp

thermo_modify flush yes

velocity fluidcreate 1.0 3265456 rot yes distgaussian loop local

dump 1 all custom 10000 dump.lammpstrj id type x yz

fix fxnve fluid nve #只需要把壁面不包含在任何系综中，壁面就会保持不动

run 1000000

write_restart restart.equ

#####开始 Poiseuille 流动模拟#####

fix 1 fluid addforce 0.01 0.0 0.0 #驱动力不能太大，否则温度为控制不住。由于耗散粒子动力学的温度控制是动量守恒的所以不需要排除质心速度进行温度控制

run 1000000

#####统计速度分布和密度分布#####

compute layers all chunk/atom bin/1d z lower 0.01 units box

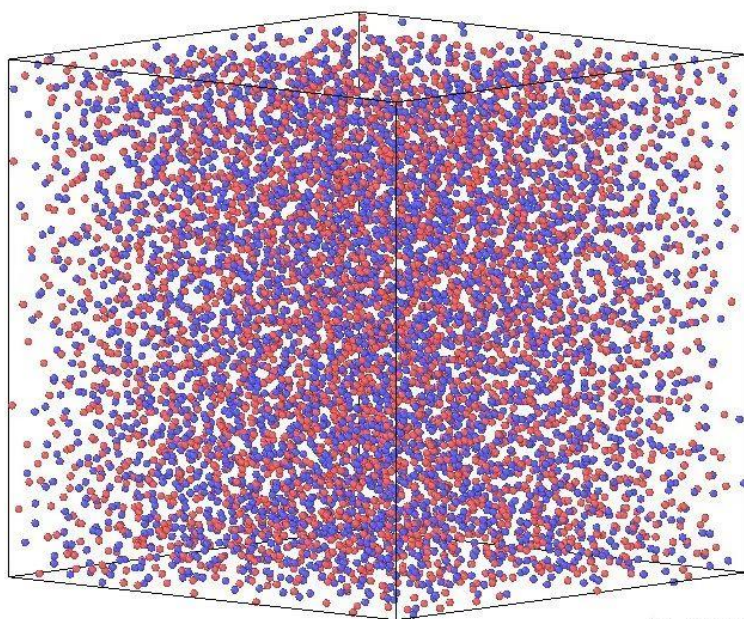
fix aveall ave/chunk 1 1000000 3000000 layers density/number vx file vx_rho.profile

run 1000000

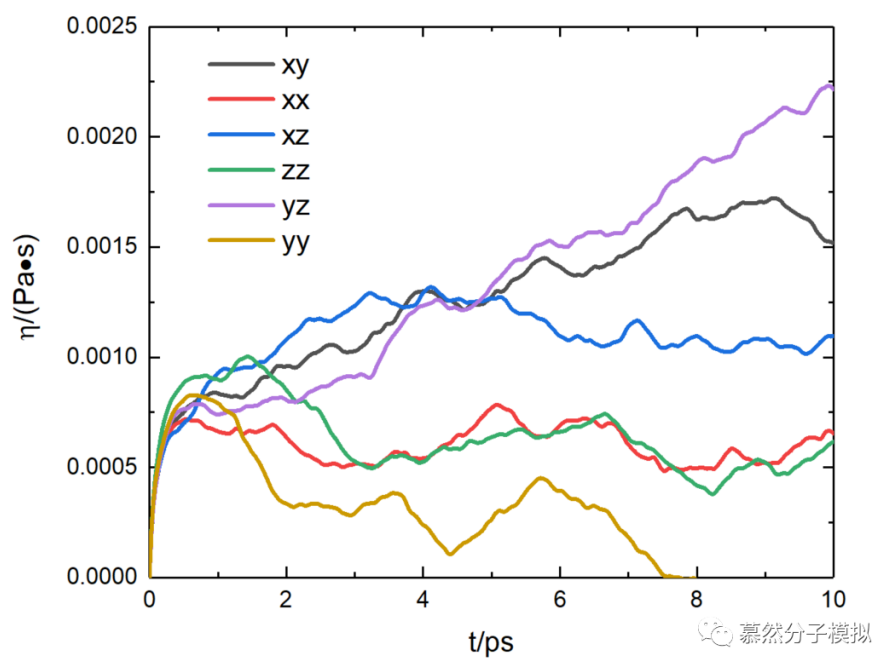
6.18 氯化钠 NaCl 熔盐粘度和扩散系数模拟

本例发布氯化钠 NaCl 熔盐粘度和扩散系数模拟。力场参数取自文献 Solid-solid phase equilibria in the NaCl-KCl system。粘度通过 Green-Kubo 关系计算，还可以

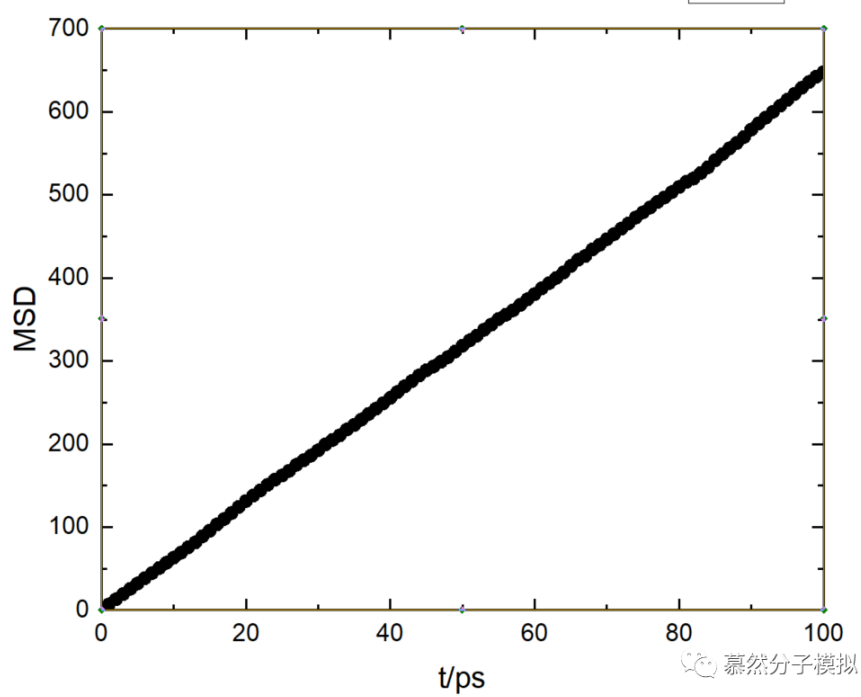
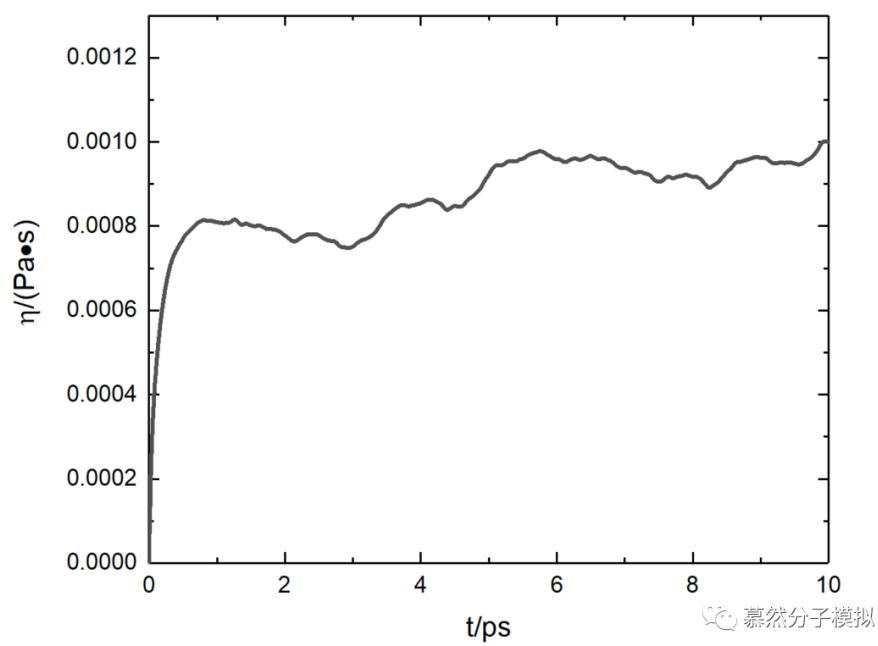
通过其他方法获得更为准确的粘度。扩散系数通过均方位移计算。在本例 in 文件的基础上还可以计算热导率, RDF 等其他参数。

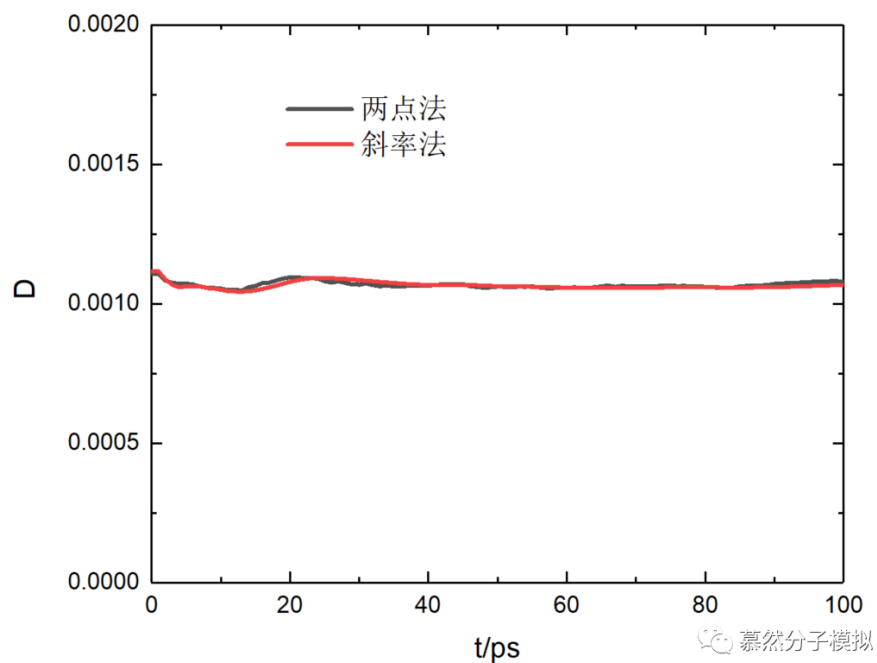


慕然分子模拟



慕然分子模拟





in 文件

units real

dimension 3

boundary p p p

neighbor 3.0 bin

neigh_modify every 1 delay 0 check yes

atom_style charge

pair_style born/coul/long 10.0 8.0

kspace_style pppm 10e-4

region box block -40 40 -40 40 -40 40 units box

create_box 2 box

lattice sc 4

region part1 block -40 40 -40 40 -40 -4.0 units box

create_atoms 1 region part1

```
lattice sc 4
region part2 block -40 40 -40 40 0.0 40 units box
create_atoms 2 region part2
#Na = 1 Cl = 2

pair_coeff 1 1 6.0772 0.3167 2.340 24.164 11.5076
pair_coeff 1 2 4.8617 0.3167 2.755 161.1 199.93
pair_coeff 2 2 3.6462 0.3267 3.170 1729.64 3473.65

set type 1 charge 1.0
set type 2 charge -1.0

mass 1 22.9898
mass 2 35.45

dump mydump all atom 1000 atom.lammpstrj

minimize 0.0 1.0e-8 1000 100000

timestep 1.0
thermo 100
thermo_style custom step temp density
thermo_modify flush yes
velocity all create 1200 5645354 loop local dist gaussian

fix fxnpt all npt temp 1200 1200 100 iso 100 100 1000

run 100000
```


unfix fxnpt

fix fxnpt all npt temp 1200 1200 100 iso 1.0 1.0 1000

run 100000

unfix fxnpt

variable dt equal 1.0

variable kB equal 1.3806504e-23

variable g_mol2kg equal 1.0e-3/6.02214e23

variable A2m equal 1.0e-10

variable fs2s equal 1.0e-15

variable atm2pa equal 1.01325e5

variable convert_eta equal

$\${atm2pa}*\${atm2pa}*\${A2m}*\${A2m}*\${A2m}*\${fs2s}$

variable kCal2J equal 4186.0/6.02214e23

variable convert_kappa equal $\${kCal2J}/\${fs2s}/\${A2m}$

variable convert_rho equal 1000

reset_timestep 0

variable p equal 2000 # correlation length

variable s equal 5 # sample interval

variable d equal $\$p*\s # dump interval

thermo \$d

variable pxy equal pxy

variable pxx equal pxx-press

variable pxz equal pxz

variable pzz equal pzz-press

variable pyz equal pyz

```
variable      ppy equal ppy-press
fix           SS all ave/correlate $s $p $d v_pxy v_pxx v_pxz v_pzz v_pyz
v_ppy type auto file profile.gk ave running
```

```
variable      scale equal 1.0/1200/${kB}*vol*$s*dt
variable      diagfac equal 2-2/3
variable      vxy equal trap(f_SS[3])*${scale}*${convert_eta}
variable      vxx equal trap(f_SS[4])*${scale}/${diagfac}*${convert_eta}
variable      vxz equal trap(f_SS[5])*${scale}*${convert_eta}
variable      vzz equal trap(f_SS[6])*${scale}/${diagfac}*${convert_eta}
variable      vyz equal trap(f_SS[7])*${scale}*${convert_eta}
variable      vyy equal trap(f_SS[8])*${scale}/${diagfac}*${convert_eta}
```

```
fix fxnvt all nvt temp 1200 1200 100.0
```

```
thermo_style  custom step temp press pxy pxz pyz v_vxy v_vxx v_vxz v_vzz
v_vyz v_vyy
thermo_modify flush yes
```

```
run          100000
```

```
unfix SS
```

```
reset_timestep 0
```

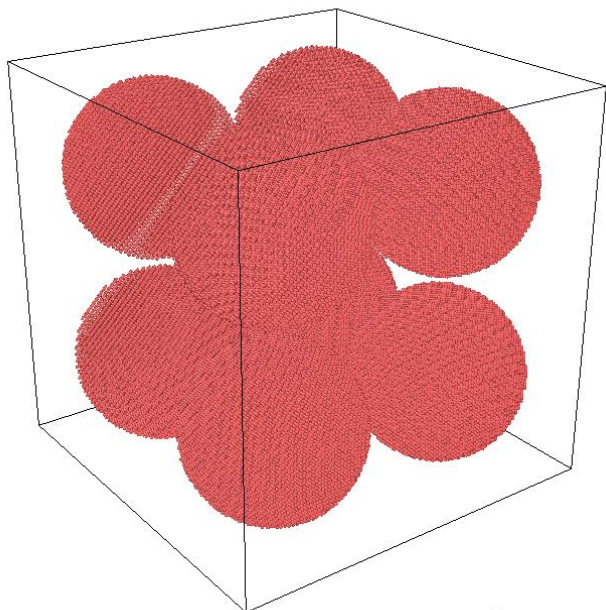
```
compute      msd all msd com yes
variable      twopoint equal c_msd[4]/6/(step*dt+1.0e-6)
fix           9 all vector 10 c_msd[4]
variable      fitslope equal slope(f_9)/6/(10*dt)
```

```
thermo       1000
```

```
thermo_style    custom step temp c_msd[4] v_twopoint v_fitslope
thermo_modify flush yes
```

```
run 100000
```

6.19 铝粉锻造 LAMMPS 模拟



嘉然分子模拟

```
# ----- Init Section -----

dimension      3
boundary       f f f
units          metal
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes
atom_style     atomic
pair_style     eam

# ----- Atom Definition Section -----

lattice        fcc 3.9860
region         box block -111.608 111.608 -111.608 111.608 -111.608 111.608 units
box
create_box     1 box
create_atoms   1 box

# ----- Settings Section -----
```

```

pair_coeff 1 1 Al_jnp.eam

# ----- group define -----

region s1 sphere -55.804 -55.804 55.804 49.825 units box
region s2 sphere 55.804 -55.804 55.804 49.825 units box
region s3 sphere 55.804 55.804 55.804 49.825 units box
region s4 sphere -55.804 55.804 55.804 49.825 units box
region s5 sphere -55.804 -55.804 -55.804 49.825 units box
region s6 sphere 55.804 -55.804 -55.804 49.825 units box
region s7 sphere 55.804 55.804 -55.804 49.825 units box
region s8 sphere -55.804 55.804 -55.804 49.825 units box
region s union 8 s1 s2 s3 s4 s5 s6 s7 s8
group s region s
group del subtract all s
delete_atoms group del
write_data 1.data

# ----- Run Section -----

timestep 0.002
thermo 1000
thermo_style custom step temp
thermo_modify flush yes

velocity      all create 580 372748 rot yes dist gaussian loop local
dump mydump all custom 10000 dump.lammpstrj id type x y z

fix      fxnvt all nvt temp 580.0 580.0 0.1

fix xwall all wall/lj93 xlo EDGE 0.001 3.405 4.0 xhi EDGE 0.001 3.405 4.0
fix ywall all wall/lj93 ylo EDGE 0.001 3.405 4.0 yhi EDGE 0.001 3.405 4.0
fix zwall all wall/lj93 zlo EDGE 0.001 3.405 4.0 zhi EDGE 0.001 3.405 4.0

run      10000

unfix xwall
unfix ywall
unfix zwall

variable v equal 0.005
variable p_xlo equal xlo+${v}*elapsed*0.002
variable p_xhi equal xhi-${v}*elapsed*0.002
variable p_ylo equal ylo+${v}*elapsed*0.002

```

```
variable p_yhi equal yhi-${v}*elapsed*0.002
variable p_zlo equal zlo+${v}*elapsed*0.002
variable p_zhi equal zhi-${v}*elapsed*0.002

fix xwall all wall/lj93 xlo v_p_xlo 0.001 3.405 4.0 xhi v_p_xhi 0.001 3.405 4.0 units
box
fix ywall all wall/lj93 ylo v_p_ylo 0.001 3.405 4.0 yhi v_p_yhi 0.001 3.405 4.0 units
box
fix zwall all wall/lj93 zlo v_p_zlo 0.001 3.405 4.0 zhi v_p_zhi 0.001 3.405 4.0 units
box

run 5000000

write_restart 1.restart
```

7. 出错解决思路

如果两次 LAMMPS 运行在不同的机器或不同数量的处理器上没有产生完全相同的结果, 这通常不是错误。理论上, 应该在任何数量的处理器和任何机器上得到相同的结果。在实践中, 数值四舍五入会导致分子动力学相空间轨迹在几百个或几千个时间步长内的微小差异和最终发散。但是, 两次运行的统计特性(例如平均能量或温度)应该仍然相同。

如果 `velocity` 命令用于设置初始原子速度, 则当模拟在不同数量的处理器或不同机器上运行时, 可以为特定原子分配不同的速度。如果发生这种情况, 两个模拟的相空间轨迹将迅速发散。有关避免此问题的详细信息和选项, 请参阅速度命令中对 `loop` 选项的讨论。

类似地, `create_atoms` 命令生成原子晶格。对于同一个物理系统, 根据处理器数量的不同, 原子 ID 对原子的排序和编号可能会有所不同。

一些命令使用随机数生成器, 可以设置为在每个处理器上生成不同的随机数流, 因此在不同数量的处理器上运行时会产生不同的效果。一个常用的示例是用于温度控制的 `fix langevin` 命令。

LAMMPS 模拟通常有两个阶段, 设置和运行。大多数 LAMMPS 错误是在设置时检测到的; 其他像键拉伸太远可能直到 `run` 中间才会发生。

LAMMPS 会尝试标记错误并打印错误消息, 以便解决问题。对于大多数错误, 它还将打印它正在处理的最后一个输入脚本命令。当然, LAMMPS 无法找出物理或数值错误, 例如选择太大的时间步长、指定错误的力场系数或有原子靠

的太近甚至重叠！不正确的几何模型和物理模型会导致一系列错误，比如 `lost atoms`、`bond/angle/dihedral/improper missing`、`simulation unstable`、`cannot compute ppm`，`too many bin` 等。这些错误出现你只能修改优化你的几何模型或者检查力场参数。

如果在输入脚本中收到有关 `invalid command` 的错误消息，可以通过查看 `log.lammps` 文件查看它来确定导致问题的命令。如果您收到诸如“`Invalid ... style`”之类的错误，这意味着输入了错误的 `style` 名称，或者该命令是未编译到可执行文件中的可选包的一部分。

对于给定的命令，LAMMPS 需要以指定顺序的某些参数。如果你搞砸了，LAMMPS 通常会标记错误，但它也可能只是读取一个虚假的参数并分配一个有效的值，但不是你想要的。例如。尝试将字符串“`abc`”读取为整数值 0。仔细阅读该命令的相关手册内容应该可以解决这些问题。在大多数情况下，LAMMPS 期望读取一个数字，无论是整数还是浮点数，它会分别对所提供的输入实际上是整数还是浮点数进行严格测试，（例如，当需要一个整数，但提供了一个浮点数 1.0）：
`ERROR: Expected integer parameter instead of '1.0' in input script or data file`

一些命令允许使用变量引用代替数字常量，以便可以计算值并且可以在运行过程中更改。这通常使用参数的语法 `v_name` 来完成，其中 `name` 是变量的名称。另一方面，在读取输入和解析命令之前，使用语法 `${name}` 会执行直接替换。

仅当相应命令的文档明确说明时才允许使用变量引用（即 `v_name`）。否则，您将收到此类错误消息：

`ERROR: Expected floating point parameter instead of 'v_name' in input script or data file`

通常，LAMMPS 会在遇到致命错误时将消息打印到屏幕和日志文件并退出。有时它会在屏幕和日志文件上打印一个警告并继续；您可以决定警告是否重要。在运行过程中生成的 `WARNING` 消息仅打印到屏幕上，而不是打印到日志文件中，以避免混淆热力学输出。如果 LAMMPS 崩溃或挂起而没有首先输出错误消息，则可能是错误（请参阅本节）或以下情况之一：

LAMMPS 在处理器允许分配的可用内存中运行。最合理的 MD 运行受计算限制，而不是内存限制，因此这不应该成为大多数平台的瓶颈。代码中几乎所有

的大内存分配都是通过 C 风格的 `malloc` 完成的，如果内存不足，则会生成错误消息。较小的内存块是通过 C++“`new`”语句分配的。如果你不走运，你可能会在发出这些小请求之一时耗尽内存，在这种情况下，代码将崩溃或挂起（并行），因为 LAMMPS 不会捕获这些错误。

非法数字会导致 LAMMPS 运行缓慢或崩溃。这通常是由于模拟计算的物理和数字无效。如果在 LAMMPS 输出中看到剧烈的热力学值或 NaN 值，则说明的模拟有问题。如果怀疑这种情况正在发生，最好通过 `thermo` 频繁地输出热力学信息（例如每个时间步长），这样就可以监控正在发生的事情。可视化原子运动也是确保模型按预期运行的好主意。

同时，LAMMPS 可能挂起的一种方式是由于不同的 MPI 实现如何处理消息的缓冲。如果代码挂起而没有错误消息，则可能是您需要指定一个或两个 MPI 设置（通常通过环境变量）以启用缓冲或增加可以缓冲的消息的大小。

当出现 **error** 错误的时候查看手册中 **error** 描述和解决办法可以解决掉绝大多数错误。Error 章的网址是：https://docs.lammps.org/Errors_messages.html

8. 专题一：纳米流动

8.1 引言

在该专题中将介绍纳米流动的基本模拟过程，其中涉及到的知识点包括：用 `fix addforce` 实现泊肃叶流动模拟，如何设置固定层、温控层和自由层，`fix ave/chunk` 计算速度分布、温度分布和密度分布，如何根据速度分布和密度分布计算流体的粘度分布。

我对纳米流动还是很熟悉的，有想在这个方向上发表论文的可以联系我（微信：**baolu_yao**）

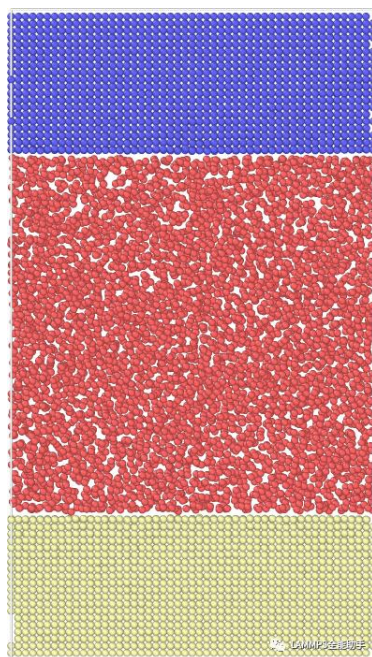
在 1990 开始分子动力学模拟被大量用来纳米流动条件下的模拟研究。研究者主要关注两个方面的性质：边界滑移和热阻。采用的流动形式也主要有两种：库艾特流动和泊肃叶流动。其流动构型都是两无限大平行平板之间的定常层流，如图 1 所示。库艾特流动是上下两个平板以相同的速度做相对平行运动以剪切流体，因此又称为剪切驱动流；泊肃叶流动是两个平板静止不动，对流体施加一个固定的压差来驱动流体，因此又称为压差驱动流。为什么大家都采用这两种流动呢？主要原因是（1）这两种流动都是层流，流动简单可从 N-S 方程求出理论解，

方便进行模拟结果和理论进行对比；(2) 分子动力学模拟的长度尺度有限都是在纳米尺度，模拟出的流动雷诺数很小，流动是层流。(3) 对于研究边界滑移和热阻，这两种流动已经足够了。为什么可以采用分子动力学研究边界滑移和热阻可以取得令人信服的结果？因为分子动力学可以在原子尺度上解析流动的速度分布，这对特征尺度在纳米级的滑移和热阻特性研究可以保证足够的精度。再加上以简单液体为研究对象，从力场和模拟设置都可以保证足够的准确度。

近年来纳米流动受到了更多的关注。主要原因是研究者在纳米尺度上发现了新的流动现象，特别是在能量转换，离子输运和纳米固液阻力方面。著名的学者 Lyderic Bocquet 是纳米流动的主要研究者，其在 *nature* 等期刊上这几年已发表多篇重要的论文。其发表在 *nature materials* 上的综述更是对纳米流动做了重要的总结和展望，大家可以看看。论文信息是 Nanofluidics coming of age, <https://www.nature.com/articles/s41563-020-0625-8>。

分子动力学对于研究纳米流动是特别合适的，不管是时间尺度还是空间尺度都可以落在分子动力学模拟的范围内。紧跟 Lyderic Bocquet 的实验研究结果，从模拟的角度去研究更多的问题，可以在短时间内发表一些论文。特别的，采用 QMMM 方法甚至可以去研究其实验中涉及量子力学的部分。

纳米流动的建模相对简单。在这个领域内研究的介质大多是水。使用 *moltemplate* 采用润湿专题中的建模方法可以方便的获得两层固体平板之间一层水的模型。这也是纳米流动最常用的模型。在实际研究中还可以在水中添加离子并研究电渗流的流动问题。下面采用简单液体对纳米流动的设置做一下介绍。



本例采用简单液体研究了泊肃叶流动条件下，粘性加热后固液之间的传热。液体和固体都通过 lj/cut 进行描述。流动达到定常后流体的温度分布和速度分布不在变化，但是会在固液界面处出现速度滑移和温度跳跃。本例可直接用来研究这些现象。液体在固体表面流动时的无滑移边界条件在流体力学发展之初就是一个假设，随着观测手段的进步，研究发现无滑移边界不一定成立。但是，同时发现即使存在滑移，滑移长度的大小也在纳米量级对宏观流动的影响可以忽略。然而，纳米流动的出现使得滑移边界条件再次进入人们的视野。

分子动力学在研究滑移的时候液体的流动剪切率相较于实验的剪切率特别高，因此会产生明显的粘性加热效应，所以在模拟时采用温度控制的方法研究等温流动时的滑移现象。这种方法的合理性在于温度控制不影响流动同时流动的特征尺度很小，流动始终处于层流阶段，并且中心区的流动符合理论解。因此，大量论文采用这种方法来研究流动滑移并取得了丰硕的成果。之所以设置较高的剪切率，是因为分子动力学模拟的时间尺度在 ns 级，为了能在短时间内统计平均获得光滑的曲线，需要流动速度很高，才能将流体的宏观速度从热运动中提取出来。尽管剪切率特别高，但是流动仍符合线性响应和理论解。

在实际应用场景或实验中，当纳米流动器件作为散热器时，粘性加热效应不明显，同时热从外部向液体内部传导。那么怎么利用分子动力学同时研究滑移和热传递现象呢？就是利用大剪切率下的粘性加热效应，即不对液体施加温度控制，只对壁面施加温度控制，这样热就由液体内部向壁面传导。此时获得的固液之间

的热传导特性足以反映由壁面向液体的热传导特性。这样就耦合了热传导和滑移。

研究因素

润湿性（壁面材质（晶格取向，热导率，晶格大小）），规则或不规则沟槽（超疏水表面），表面电荷，石墨烯修饰，界面剪切率，界面温度梯度，系统规模（包括通道高度，壁面切向面积），通道形式（双排，单排）

观测量

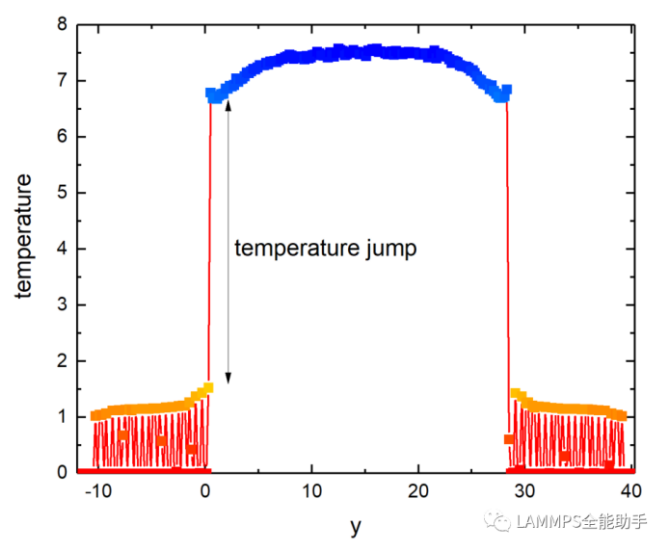
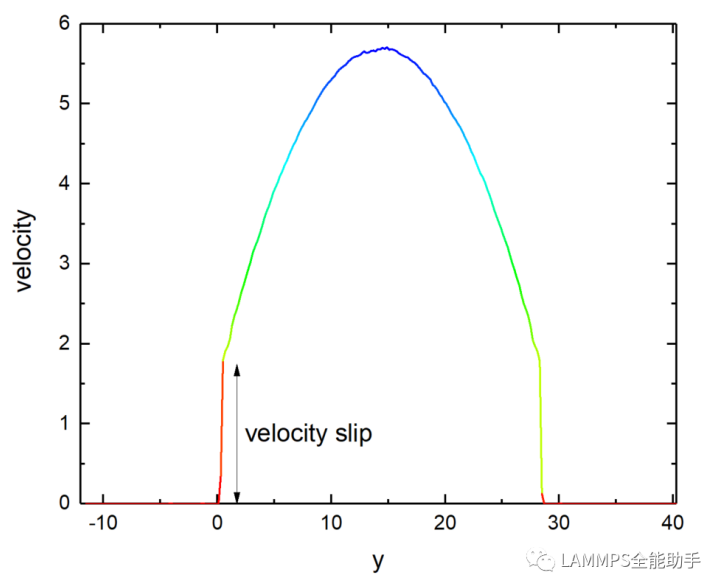
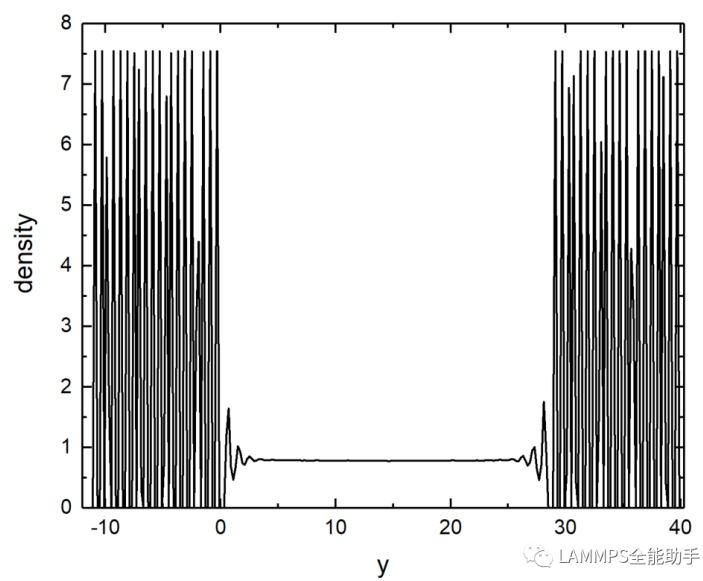
滑移长度，热阻长度，结构因子，密度法向一维分布，密度切向二维分布，固液距离或固液界面厚度

研究目标

设计高效纳米流动散热器件。纳米流动散热器件因为表面积体积比很高因此会具有很高的散热效果。纳米流动的研究结果可直接用在微流动散热器件的设计中（系统规模研究可证实）。查阅文献找到纳米流动或微流动散热器件设计的可能性。找到可行的技术方案或实际用在研究背景中。如果可行，CPU 等芯片的散热是个很好的应用场景。要是能做个实验就完美了。这么多的研究内容够你发一堆论文了。

通过改变固液相互作用强度和驱动力的大小研究不同润湿性表面和驱动强度下的界面滑移和温度跳跃，获得滑移长度和热阻长度随润湿性和驱动力的变化规律。

在 LAMMPS 中泊肃叶的流动通过对所有流体原子施加一个固定的力来实现模拟。模拟的过程分为三个阶段。第一阶段不对流体施加驱动力，让系统自然弛豫达到热力学平衡。第二阶段，对流体原子施加驱动力并跑一段时间让流动达到定常。第三阶段，通过统计平均获得流体的速度，温度和密度分布。本例中的模拟细节在 in 文件中进行详细注释。下面三个图是密度，速度和温度分布。



8.2 in 文件讲解

以下是 in 文件

```
##### Global settings #####

dimension      3
units          lj
boundary       p f p
atom_style     atomic
neighbor       0.3 bin
neigh_modify    delay 0 every 1 check yes

##### Create geometry #####

read_data      system.data

##### Define liquid groups #####

group liquid type 1
group upwall type 2
group downwall type 3
group wall union upwall downwall

##### Define wall three layers region and group #####
#利用 region 命令分别建立固定层, 温控层和自由层的原子组
region up_fix block INF INF 39.6 INF INF INF units box
region down_fix block INF INF INF -10.8 INF INF units box
region up_thermo block INF INF 37.8 39.6 INF INF units box
region down_thermo block INF INF -10.8 -9.1 INF INF units box
group up_fix region up_fix
group down_fix region down_fix
group up_thermo region up_thermo
group down_thermo region down_thermo
group up_free subtract upwall up_fix up_thermo
group down_free subtract downwall down_fix down_thermo

group move subtract all up_fix down_fix

##### Define LJ potentials #####

#liquid=1 upwall=2 downwall=3

pair_style     lj/cut 2.5
```

```

pair_coeff 1 1 1 1 2.5      #pair between liquid and liquid
pair_coeff 1 2 0.8 1 2.5    #pair between liquid and upwall
pair_coeff 1 3 0.8 1 2.5    #pair between liquid and downwall
pair_coeff 2 2 50 0.7342 2.5 #pair between upwall and downwall 0.521 2.5
pair_coeff 2 3 0 0 0         #pair between upwall and downwall
pair_coeff 3 3 50 0.7342 2.5 #pair between downwall and downwall

##### Min #####
#通过 setforce 命令使得在能量最小化时固体的结构保持不变
fix 1 upwall setforce 0 0 0
fix 2 downwall setforce 0 0 0
minimize 1.0e-4 1.0e-6 100 1000
unfix 1
unfix 2

##### Run settings #####

compute      liquid_c_t move temp #系统默认的温度计算命令是计算所有原子的温度, 我
们期望的是监测流体的温度
#创建初始速度, 通过修改种子数可以创建不同的初始速度, 从而实现重复独立测量
velocity      move create 1.0 482748 dist gaussian rot yes temp liquid_c_t units box
timestep      0.002
thermo        100
thermo_style   custom step temp
thermo_modify temp liquid_c_t
dump myDump all atom 100 dump.lammpstrj
#对流体, 温控层, 自由层同时施加温度控制和系综使系统达到热力学平衡
fix          liquid_nvt liquid nvt temp 1.0 1.0 0.2
fix          upwall_nvt1 up_thermo nvt temp 1.0 1.0 0.2
fix          upwall_nvt2 up_free nvt temp 1.0 1.0 0.2
fix          downwall_nvt1 down_thermo nvt temp 1.0 1.0 0.2
fix          downwall_nvt2 down_free nvt temp 1.0 1.0 0.2
#第一阶段 run
run 1000000

unfix liquid_nvt
unfix upwall_nvt2
unfix downwall_nvt2
#取消自由层和流体上的温度控制, 将其设置为 nve 系综
fix          liquid_nve liquid nve
fix          upwall_nve up_free nve
fix          downwall_nve down_free nve

fix add_F liquid addforce 0.1 0 0

```

#第二阶段 run 使流动达到定常

```
run      1000000
```

```
variable Ty atom (vz*vz)*mass
```

```
compute mype all pe/atom
```

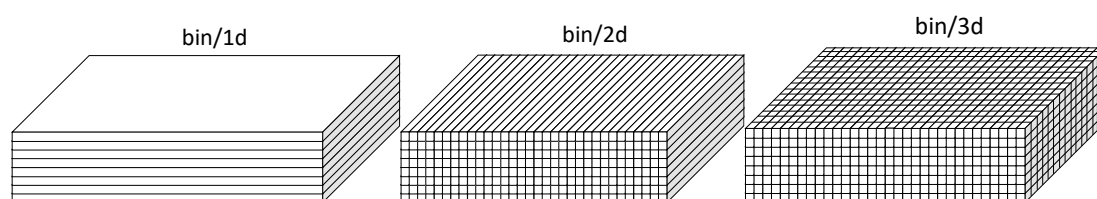
```
compute cc2 all chunk/atom bin/1d y 0.0 0.2 units box
```

```
fix ave all ave/chunk 1 1000000 3000000 cc2 density/number vx v_Ty c_mype norm sample file  
v_T.profile
```

#第三阶段 run, 统计平均输出想要的量

```
run 1000000
```

上面讲了怎么设置 in 文件来输出密度, 速度和温度分布。这里面需要讲一下 `compute chunk/atom` 和 `fix ave/chunk` 命令。`compute chunk/atom` 可以把具有部分特征的原子归结为一个 `chunk`, 然后供后面带 `chunk` 的 `compute` 和 `fix style` 进行计算。如本例中将模型在 `y` 方向上分为不同的层, 将属于同一层的原子的速度和温度和密度进行时间平均。`compute chunk/atom` 中的 `bin/1d`, `bin/2d`, `bin/3d` 可以将模拟盒子分解为一维, 二维和三维网格, 如下图:



`bin/1d` 是用一个个薄片层将整个模拟盒子进行划分, `bin/2d` 是用一个个细条将整个模拟盒子进行划分, `bin/3d` 是用一个个小块将整个模拟盒子进行划分。每一个方向上在划分的时候都会设定起点和这个方向上间隔的大小。如上面的命令例子中 `y 0.0 0.2` 的意思就是在 `y` 方向上以 `0.0` 为起点 `0.2` 为间隔将模拟盒子进行划分。`bin/2d` 设置方式的例子是 `x 0.0 0.2 y 0.0 0.2`, 类似 `bin/2d` 设置方式的例子是 `x 0.0 0.2 y 0.0 0.2 z 0.0 0.2`。起点和间隔大小都是自己定义的。LAMMPS 在划分网格时有时会额外多出一两个 `bin` 以能完全盖住整个盒子。

`fix ave all ave/chunk 1 1000000 3000000 cc2 density/number vx v_Ty c_mype norm sample file v_T.profile`。当你需要统计某个分布的时候就可以采用这个的命令。这个命令平均的是 `per-atom` 量, `global` 和 `local` 量不能用这个命令进行时间平均计算。该命令中 `ave/chunk` 后面的三个数分别称为 `Nevery` `Nrepeat` `Nfreq`。根据我的经验这个命令设置完后, 紧接着就会 `run`, 假设 `run` 了 `num_run` 步, 那么比较好的做法是 `num_run=Nevery*Nrepeat`, `Nfreq=num_run+之前所有 run 的步数`。`Nfreq`

的含义是当时间步数是 Nfreq 的整数倍的时候这个命令就会执行平均，Nrepeat 的含义是去多少步的数据进行平均，Nevery 的含义是从这个命令执行平均的时间步往前数，每多少个步取一次数据，直到取了 Nrepeat 个步数的数据。

下面我们看看 fix ave/chunk 输出了什么？井号开头的都是注释。数字部分第一行是输出信息分别是在哪一个输出的结果，总共有多少个 bin，所有 bin 中总共有多少个原子。接下来每一列别是 bin 的编号，bin 的坐标，bin 中平均有多少原子，取值 1，取值 2。。。这里的取值与 fix ave/chunk 中的设置一致（见下面的命令）。有了输出结果就可以后处理画图了。

```
compute cc2 all chunk/atom bin/1d y 0.0 0.2 units box
```

```
fix ave all ave/chunk 1 100000 210000 cc2 density/number vx v_Ty c_mype norm
sample file v_T.profile
```

v_T.profile 的内容

```
# Chunk-averaged data for fix ave and group vx
# Timestep Number-of-chunks Total-count
# Chunk Coord1 Ncount density/number vx v_Ty c_mype
210000 260 19202
 1 -11.5 0 0 0 0 0
 2 -11.3 0 0 0 0 0
 3 -11.1 0 0 0 0 0
 4 -10.9 350 7.54503 0 0 -275.14
 5 -10.7 0 0 0 0 0
 6 -10.5 0.00011 2.37129e-06 -6.29937e-05 5.4045e-05 -0.0420899
 7 -10.3 350 7.54502 1.12873e-05 1.02235 -392.174
 8 -10.1 0 0 0 0 0
 9 -9.9 268.517 5.78848 0.000169106 1.04341 -411.428
10 -9.7 81.4828 1.75654 -0.000584359 1.04091 -411.533
11 -9.5 0 0 0 0 0
12 -9.3 350 7.54503 1.16908e-05 1.07065 -415.646
13 -9.1 0 0 0 0 0
14 -8.9 0 0 0 0 0
.....
```

8.3 后处理

计算滑移长度和热阻长度：

随着剪切率的增加，液体内部的粘性加热效应变得越来越明显，同时液体的温度分布不再是均匀分布。对于纳米通道中液体的速度分布和温度分布在通道中心区的部分仍满足连续介质力学的理论预测。对于包含滑移和热阻边界条件的定

常不可压泊肃叶流动的理论解为(坐标原点在下壁面最靠近流体的那一层固体原子处)。

$$v_x(y) = \frac{\rho F_x H^2}{2\mu} \left[\frac{1}{4} - \left(\frac{y}{H} - \frac{1}{2} \right)^2 \right] + v_s$$

$$T(y) = \frac{\rho^2 F_x^2 H^4}{12\lambda\mu} \left[\frac{1}{16} - \left(\frac{y}{H} - \frac{1}{2} \right)^4 \right] + T_I + T_W$$

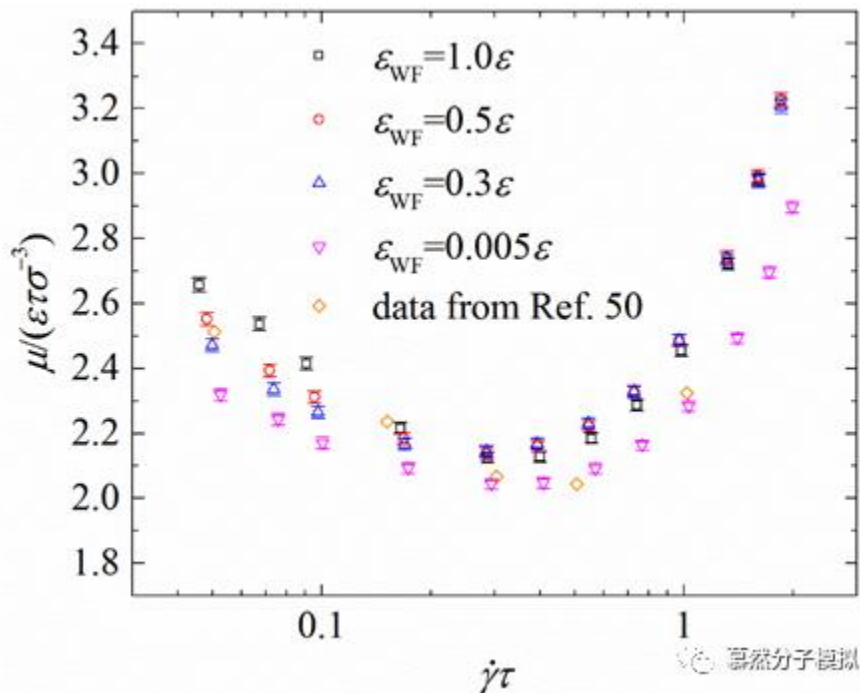
以上各两个公式会根据垂直壁面方式上的零点位置不同而略有差别。这里的零点为下壁面最靠近流体的壁面原子的位置。滑移长度采用纳维尔模型计算 $L_s = v_s / \gamma$, 其中 v_s 和 γ 分别为液体在固液界面处的滑移速度和剪切率。上下固液界面的位置分别定义为上壁面处最底层原子的位置和下壁面处最顶层原子的位置。参数 v_s 和 γ 通过对通道中心区部分(3.1~25.7, 这部分是经验取值, 取速度明显符合理论解部分)的速度剖面进行抛物线拟合并外延至固液界面处得到。采用 Kapitza 热阻长度的概念来定量表征固液界面处的热阻大小。热阻长度采用如下公式计算 $L_K = T_J / T_{\dot{}}$, 其中 T_J 和 $T_{\dot{}}$ 分别为固液界面处的温度跳跃和温度梯度。 T_J 和 $T_{\dot{}}$ 通过对通道中心部分的温度分布进行四次多项式拟合并外延至固液界面处得到。此外, $T_J = T_{F_Wall} - T_{W_Wall}$, 其中, T_{F_Wall} 是液体外延至固液界面处的温度, T_{W_Wall} 为上壁面处最底层和下壁面处最顶层的温度(这是这两个值的一种计算方法, 还可以通过直线拟合固体的温度分布外延至固液界面处得到)。

粘度分布计算

液体温度随剪切率显著升高会影响液体的粘性。并且, 由于非均匀的温度分布致使液体的粘性可能还与距离壁面的位置有关。在泊肃叶流动中, 沿垂直壁面方向也即 y 方向上, 不同位置处的液体的粘性可采用以下公式进行计算(计算原理就是体积力与表面力平衡):

$$\mu(y_1) = \mu(y_2) = \frac{F_x \Delta y A \sum_{i=1}^n \rho_i}{(\dot{\gamma}_1 + \dot{\gamma}_2) \Delta y}$$

其中 δy 为平行于壁面的薄层厚度, 取值为 0.2, A 为模拟区域在 xz 平面上的面积。 $\gamma_{\dot{1}}$ 和 $\gamma_{\dot{2}}$ 分别为液体在 y_1 和 y_2 处的剪切率。 y_1 和 y_2 关于通道中心平面位置对称。对于给定的驱动力和固液相互作用强度, 液体的温度分布在通道中心区近似为直线, 这表明在通道中心区液体的粘性可以假设为常数。因此, 计算粘性时仅需考虑温度分布非线性非常显著的区域即可。这样 y_1 和 y_2 取对称变化就可确定粘度分布。也可以计算平均粘度。事实表明尽管温度变化较明显但是粘度变化并不大。不同固液相互作用强度下液体的粘性随界面剪切率（驱动力）的变化关系, 如下图



由此不同条件下滑移长度和热阻长度就可以计算出来, 进行分析。在拓展方面可以将流体换成不同的对象比如水, 高分子。对固体壁面进行处理, 比如进行沟槽化, 添加表面电荷等。带有表面电荷下的热阻和滑移之间的关系还没有人系统的研究过, 可以做一做。利用公众号（慕然分子模拟）中的 [in-plane 静态结构因子程序](#) 对结果进行分析, 可以发一篇二区。此外把壁面进行软物质修饰, 水凝胶修饰可以做出很好的文。虽然这个内容做得人很多了, 但是还是有很多东西可以做。

9. 专题二：纳米流体

纳米流体和纳米流动不同。纳米流体是指在液体中添加纳米颗粒的流体介质, 纳米流动是指流动特征尺度在纳米量级的流动现象。在通常的纳米流体中, 纳米

颗粒的直径一般在数百个纳米。这样的量级对于分子动力学来说是很吃力的，因为计算量太大了。所以，我对分子动力学直接模拟纳米流体的热物性参数，甚至流动现象并不看好。但是根据公众号和群友的反馈，做这方面的人还挺多的。然后，你以 nanofluid 为关键词搜索引用 LAMMPS 发表的论文，会发现并没有多少顶刊论文。但是针对纳米流体我们还是可以去研究很多基础传热方面的问题。比如不同金属与液体之间热阻或热导效率，可以考虑金属颗粒的组成和形状，如直径，曲率方向等。下面分为两个部分进行介绍纳米流体的基本研究过程。

9.1 基本传热性质模拟

基本传热性质采用球套球的方式进行模拟。水分子采用粗粒化多体势模型，金属也采用多体势。为什么采用粗粒化模型呢？因为不考虑氢键水分子朝向，这些东西，只关注热物性，粗粒化已经足够描述了。粗粒化带来的优点就是计算速度快。采用全原子模型静电作用的计算就很耗时了。目前水分子的粗粒化模型有几个，我们采用最简单的那个也就是 mW 模型。在这个模型中一个水分子被当做是一个点。关于粗粒化水分子模型采用的是 2019 年在 NC 上的一篇文章：Machine learning coarse grained models for water。这里我们采用文章新开发出的 ML-mW 势函数。这个势函数是 Stillinger-Weber 形式的多体势，文章采用遗传算法对势函数参数进行了重新拟合。势函数文件可在论文的补充信息中下载，也可以联系我（微信：baolu_yao）发给你。

由于水和金属都采用多体势，所以你的 LAMMPS 在编译的时候一定要包含 manybody 包。水和金属的几何模型都可以设置为晶体，因此 LAMMPS 的 create_atoms 命令就可完成建模。下面我们通过 in 文件的编写来讲解建模和模拟过程。

```
#####基本模拟规则设置#####
units metal
dimension 3
boundary p p p
neighbor 3.0 bin
neigh_modify delay 0 every 1 check yes
atom_style atomic
pair_style hybrid sw eam lj/cut 10
#####定义几何模型#####
#使用 LAMMPS 内部命令建立模型需要三个命令联用：region 命令，create_box
命令和 create_atoms 命令
```

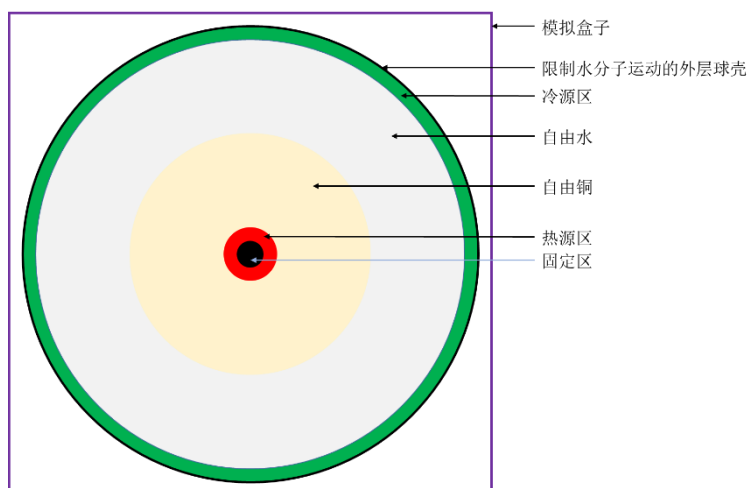
```

region box block -90 90 -90 90 -90 90 units box
create_box 2 box
region sphere_cu 0.0 0.0 0.0 45 units box side in
lattice fcc 3.615
create_atoms 1 sphere_cu
region sphere_wat_in 0.0 0.0 0.0 45 units box side out
region sphere_wat_out 0.0 0.0 0.0 85 units box side in
region sphere_wat intersect 2 sphere_wat_out sphere_wat_in
lattice fcc 5.10873
create_atoms 2 sphere_wat
#####定义物理模型#####
pair_style * * sw NULL WT
pair_style 1 1 eam Cu_u6.eam
pair_style 1 2 lj/cut 0.006 3.0 #水和铜的参数, 可以通过一些实验参数进行调节
#####定义分组信息#####
group cu type 1
group wat type 2
region cu_fix 0.0 0.0 0.0 5 units box side in
group cu_fix region cu_fix
group cu_move subtract cu cu_fix
group move union wat cu_move
#####定义热源区和冷源区#####
region cu_hot_in 0.0 0.0 0.0 5 units box side out
region cu_hot_out 0.0 0.0 0.0 10 units box side in
region cu_hot intersect 2 cu_hot_in cu_hot_out
region wat_cold_in 0.0 0.0 0.0 80 units box side out
region wat_cold_out 0.0 0.0 0.0 85 units box side in
region wat_cold intersect 2 wat_cold_in wat_cold_out
#####弛豫部分#####
timestep 0.002
thermo 1000
compute myT move temp
thermo_style custom step temp etotal press
thermo_modify temp myT flush yes
velocity move create 300 3265456 rot yes dist gaussian loop local
dump 1 all custom 10000 dump.lammpstrj id type x y z
fix fxnvt move nvt temp 300 300 0.2
fix wall wat wall/region wat_cold_out lj93 0.0001 3.0 3.1
run 1000000
write_restart restart.equ
fix fxnve move nve
fix fxhot cu_move ehex 1 100 region cu_hot
fix fxcold wat ehex 1 -100 region wat_cold
run 1000000

```

```
write_restart restart.steady
compute cc1 all chunk/atom bin/sphere 0.0 0.0 0.0 0.0 85 85 units box
fix 1 all ave/chunk 1 1000000 3000000 cc1 density/mass temp norm sample file
rho_T.profile
run 1000000
write_restart restart.stat
```

建好的模型示意图如下图所示。这是一个二维示意图，实际上圆是球。



模拟结果如下

直接模拟纳米流体的热导率和粘度

尽管我不建议这样做，但是这也算是一个模拟内容。以这样的方式进行模拟，最关键的就是要将模型设置为各项同性的，这就要求你的模型要能足够大，包含足够多的纳米颗粒。由于大多数纳米流体的都是水基的，所以我们还是可以采用上面的模型进行模拟，只是建模有少许差别。后续模拟也有差别。

9.2 Green-Kubo 法计算热导率和粘度

Green-Kubo 方法的计算热导率的原理是利用系统热流的时间自相关函数进行计算，其公式为

$$\kappa = \frac{V}{k_B T^2} \int_0^\infty \langle J_x(0) J_x(t) \rangle dt = \frac{V}{3k_B T^2} \int_0^\infty \langle \mathbf{J}(0) \cdot \mathbf{J}(t) \rangle dt$$

由于体系是各向同性的所以 xyz 三个方向上的热流自相关函数计算出来的热导率应该是相同的，所以可以计算出来三个方向上的热导率系数然后取平均。在固液共存相中这里没有考虑焓的影响，所以计算结果有问题。因此内容可选择性借鉴。在 LAMMPS 中利用 `compute heat/flux` 来计算系统的热流。这个 `compute` 会输出 6 个全局矢量。该矢量的前三个分量分别是全局热流，除以系统体积就为热

流了。以 x 方向为例, $J_x = c_flux[1]/vol$ 。把这个代入上面公式中的

$$\kappa_x = \frac{1}{k_B T^2 V} \int_0^\infty \langle c_flux[1](0) c_flux[1](t) \rangle dt$$

$\langle c_flux[1](0) c_flux[1](t) \rangle$ 使用命令 `fix JJ all ave/correlate $s $p $d c_flux[1] c_flux[2] c_flux[3] type auto file profile.heatflux ave running` 进行计算。在实际计算中积分不可能取无穷大。但是上面的积分当积分上限足够大时积分值基本保持不变, 所以可以用有限代替无限。fix JJ all ave/correlate 命令中 \$p 就是积分上限值。这个值的设定需要保证积分值已经趋于不变了, 可以查看这个命令输出的文件内容, 对相关列进行积分看积分值是否已趋于不变了。公式中 $dt = \$s * \text{时间步长}$ 。\$s 也是个经验值一般会选为 5 或 10。有了 $\langle c_flux[1](0) c_flux[1](t) \rangle$ 随时间的变化对其进行积分代入公式就能计算出热导率系数了。

粘度的计算与热导类似, 使用的是应力的自相关。其公式为

$$\begin{aligned} \eta_{xy} &= \frac{V}{k_B T} \int_0^\infty \langle p_{xy}(0) p_{xy}(t) \rangle dt \\ \eta_{xz} &= \frac{V}{k_B T} \int_0^\infty \langle p_{xz}(0) p_{xz}(t) \rangle dt \\ \eta_{yz} &= \frac{V}{k_B T} \int_0^\infty \langle p_{yz}(0) p_{yz}(t) \rangle dt \end{aligned}$$

$\langle p_{xy}(0) p_{xy}(t) \rangle$, $\langle p_{xz}(0) p_{xz}(t) \rangle$, $\langle p_{yz}(0) p_{yz}(t) \rangle$ 使用命令 `fix SS all ave/correlate $s $p $d v_pxy v_pxz v_pyz type auto file profile.gk ave running` 进行计算。设置方法与热导类似。 $\langle p_{xy}(0) p_{xy}(t) \rangle$, $\langle p_{xz}(0) p_{xz}(t) \rangle$, $\langle p_{yz}(0) p_{yz}(t) \rangle$

建模分为两个部分第一部分可以借鉴上一节的过程:

#####基本模拟规则设置#####

units metal

dimension 3

boundary p p p

neighbor 3.0 bin

neigh_modify delay 0 every 1 check yes

atom_style atomic

pair_style hybrid sw eam lj/cut 10

#####定义几何模型#####

#使用 LAMMPS 内部命令建立模型需要三个命令联用: region 命令, create_box 命令和 create_atoms 命令

region box block -90 90 -90 90 -90 90 units box

create_box 2 box

region sphere_cu 0.0 0.0 0.0 45 units box side in

lattice fcc 3.615

create_atoms 1 sphere_cu

```
region wat subtract 2 box sphere_cu
lattice fcc 5.10873
create_atoms 2 wat
#####定义物理模型#####
pair_style * * sw NULL WT
pair_style 1 1 eam Cu_u6.eam
pair_style 1 2 lj/cut 0.006 3.0 #水和铜的参数，可以通过一些实验参数进行调节
write_data pre.data
run 1
```

通过控制水区域和铜区域的比例就可以调控纳米颗粒的体积分数。写出 data

文件后重新读入 data 文件开始模拟。in 文件如下：

```
#####基本模拟规则设置#####
units metal
dimension 3
boundary p p p
neighbor 3.0 bin
neigh_modify delay 0 every 1 check yes
atom_style atomic
pair_style hybrid sw eam lj/cut 10
#####定义几何模型#####
read_data pre.data
replicate 5 5 5 #利用复制命令建立更大的模型，具体模型多大需要经验。具体可以跑一下算例
#####定义物理模型#####
pair_style * * sw NULL WT
pair_style 1 1 eam Cu_u6.eam
pair_style 1 2 lj/cut 0.006 3.0 #水和铜的参数，可以通过一些实验参数进行调节
#####定义分组信息#####
group cu type 1
group wat type 2
#####弛豫部分#####
timestep 0.002
thermo 1000
thermo_style custom step temp etotal press
thermo_modify flush yes
velocity all create 300 3265456 rot yes dist gaussian loop local
dump 1 all custom 10000 dump.lammpstrj id type x y z
fix fxnvt move nvt temp 300 300 0.2
run 1000000
write_restart restart.equ
#####粘度和热导率计算之 Green-Kubo 法#####
variable p equal 20000 # correlation length
variable s equal 5 # sample interval
```

```

variable      d equal $p*$s    # dump interval
variable      t equal 300
variable      kB equal 1.3806504e-23
variable      A2m equal 1.0e-10
variable      ps2s equal 1.0e-12
variable      bar2pa equal 100000
variable      eV2J 1.60218E-19
variable                               convert_eta          equal
${bar2pa}*$${bar2pa}*$${A2m}*$${A2m}*$${A2m}*$${ps2s}
variable      convert_kappa equal ${eV2J}*$${eV2J}/${ps2s}/${A2m}
reset_timestep 0
compute       myKE all ke/atom
compute       myPE all pe/atom
compute       myStress all stress/atom NULL virial
compute       flux all heat/flux myKE myPE myStress
variable      Jx equal c_flux[1]/vol
variable      Jy equal c_flux[2]/vol
variable      Jz equal c_flux[3]/vol
unfix fxnvt
fix           1 all nve
fix           JJ all ave/correlate $s $p $d c_flux[1] c_flux[2] c_flux[3] type auto
file profile.heatflux ave running
variable      scale equal $s*dt/$t/$kB)/vol
variable      k11 equal trap(f_JJ[3])*${scale}*$${convert_kappa}
variable      k22 equal trap(f_JJ[4])*${scale}*$${convert_kappa}
variable      k33 equal trap(f_JJ[5])*${scale}*$${convert_kappa}

variable      pxy equal pxy
variable      pxz equal pxz
variable      pyz equal pyz
fix           SS all ave/correlate $s $p $d v_pxy v_pxz v_pyz type auto file
profile.gk ave running
variable      scale equal $s*dt/$t/$kB)*vol
variable      diagfac equal 2-2/3
variable      vxy equal trap(f_SS[3])*${scale}*$${convert_eta}
variable      vxz equal trap(f_SS[5])*${scale}*$${convert_eta}
variable      vyz equal trap(f_SS[7])*${scale}*$${convert_eta}

thermo        $d
thermo_style  custom step temp press pe ke v_k11 v_k22 v_k33 v_vxy v_vxz
v_vyz
thermo_modify flush yes
run           1000000
计算结果分析

```

9.3 RNEMDS 方法计算热导率和粘度

建模方法和 Green-Kubo 方法一样，只是计算方法不一样。由于热导率和粘度计算过程不一样需要分开计算，但两个计算可以共享一个弛豫后的 restart 文件。RNEMDS 方法的原理是将模拟盒子划分为偶数个薄层，然后人为的交换最中间的一层与最两边两层之间的速度，从而形成由中间流向两边的热流或动量流。通过交换速度的累积过程就能知道多少时间交换了多少能量或动量，除以交换累积时间和薄层面积再除以温度或速度梯度就能得到热导或粘度。

热导率计算

```
#####基本模拟规则设置#####
units metal
dimension 3
boundary p p p
neighbor 3.0 bin
neigh_modify delay 0 every 1 check yes
atom_style atomic
pair_style hybrid sw eam lj/cut 10
#####定义几何模型#####
read_data pre.data
replicate 5 5 5 #利用复制命令建立更大的模型，具体模型多大需要经验。具体可以跑一下算例
#####定义物理模型#####
pair_style * * sw NULL WT
pair_style 1 1 eam Cu_u6.eam
pair_style 1 2 lj/cut 0.006 3.0 #水和铜的参数，可以通过一些实验参数进行调节
#####定义分组信息#####
group cu type 1
group wat type 2
#####弛豫部分#####
timestep 0.002
thermo 1000
thermo_style custom step temp etotal press
thermo_modify flush yes
velocity all create 300 3265456 rot yes dist gaussian loop local
dump 1 all custom 10000 dump.lammpstrj id type x y z
fix fxnvt move nvt temp 300 300 0.2
run 1000000
write_restart restart.equ
#####开始交换速度产生定常温度分布#####
reset_timestep 0
```



```
unfix fxnvt
fix fxnve all nve
fix      MP_T wat thermal/conductivity 10 z 200 #通过控制交换频率和每次交换的
原子个数就能产生不同大小的热流以及温度分布
run      1000000
#重置 fix thermal/conductivity
reset_timestep 0
unfix MP_T
fix      MP_T wat thermal/conductivity 10 z 200
compute          layers_kappa all chunk/atom bin/1d z lower 0.005 units reduced
fix      ave_T all ave/chunk 1 1000000 1000000 layers_kappa temp file T.profile
thermo_style custom step temp ke f_MP_T
run      1000000
```

粘度计算

```
#####基本模拟规则设置#####
units metal
dimension 3
boundary p p p
neighbor 3.0 bin
neigh_modify delay 0 every 1 check yes
atom_style atomic
pair_style hybrid sw eam lj/cut 10
#####定义几何模型#####
read_data pre.data
replicate 5 5 5 #利用复制命令建立更大的模型，具体模型多大需要经验。具体可
以跑一下算例
#####定义物理模型#####
pair_style * * sw NULL WT
pair_style 1 1 eam Cu_u6.eam
pair_style 1 2 lj/cut 0.006 3.0 #水和铜的参数，可以通过一些实验参数进行调节
#####定义分组信息#####
group cu type 1
group wat type 2
#####弛豫部分#####
timestep 0.002
thermo 1000
thermo_style custom step temp etotal press
thermo_modify flush yes
velocity all create 300 3265456 rot yes dist gaussian loop local
dump 1 all custom 10000 dump.lammpstrj id type x y z
fix fxnvt move nvt temp 300 300 0.2
run 1000000
write_restart restart.equ
#####开始交换速度产生定常速度分布#####
```

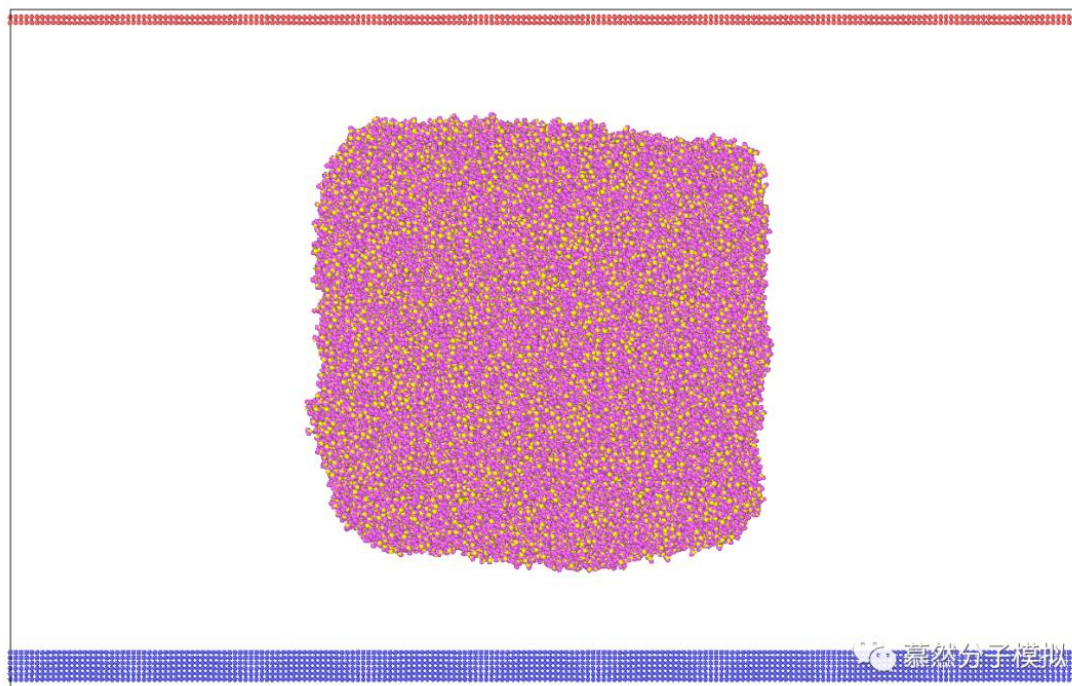
```

reset_timestep 0
unfix fxnvt
fix fxnve all nve
fix MP_V wat viscosity 10 x z 200 #通过控制交换频率和每次交换的原子个数就能
产生不同大小的动量流以及速度分布
run 1000000
#重置 fix viscosity
reset_timestep 0
unfix MP_V
fix MP_V wat viscosity 10 x z 200
compute layers_eta all chunk/atom bin/1d z lower 0.005 units reduced
fix ave_T all ave/chunk 1 1000000 1000000 layers_eta vx file vx.profile
thermo_style custom step ke temp f_MP_V
run 1000000

```

10. 专题三：润湿

工业和生活生产中很多情况都涉及到液滴在固体表面上润湿和铺展。在润湿基础理论方面，分子动力学特别适合用来研究液滴在光滑固体表面上的润湿。从基本的铺展规律到原子尺度机理都可以得到可靠的结果。大量论文已经采用分子动力学方法研究润湿过程，其中不乏顶刊论文。实践表明纳米尺度的研究对理想情况下的润湿已经足够了。本文开始介绍润湿的模拟。使用的工具就是 LAMMPS。首先是如何建立润湿模型。本例讲解如何建立如下图所示的润湿模型。该模型研究圆柱形水滴在光滑固体表面的润湿。为什么是水滴，因为水最常见。为什么是圆柱形。因为圆柱形水滴可以捕捉到润湿的关键过程，同时在相同直径下用到的原子数更少。在理论上，圆柱形水滴也更容易处理。液滴尺寸的选择一般会设置初始直径大于 10nm 就可以了。固体表面需要保证液滴在铺展时可以充分铺展而不会受到周期性边界的影响。下面介绍如何使用 `moltemplate` 建立上图的模型。模型采用 TIP4P/2005 水分子力场，壁面为刚性。z 方向为非周期性边界条件。x 和 y 方向为周期性边界条件。更复杂的模型也可以建立。



10.1 moltemplate 建模

LAMMPS 的使用重点前处理任务之一就是建立模型。实现 LAMMPS 的通用建模方法是很困难的。往往需要组合几个工具才能实现建模目的。LAMMPS 官网给出了多个前后处理的工具 (<https://lammps.sandia.gov/prepost.html>), 阅读工具介绍找到自己需要的工具。在这些工具中 moltemplate 是一款专门为 LAMMPS 建模开发的软件, 与 LAMMPS 是无缝衔接的。使用 moltemplate 建模首先查看安装包中的例子, 如果能找到相似的例子, 恭喜你你的任务 moltemplate 可以实现, 如果找不到相似的, 那么就用 moltemplate 就很难了。目前网上的教程不能令人满意。



首先是 moltemplate 安装。这个软件安装很简单, 但是需要有 Linux 环境。如果是 Windows 10 系统, 可以安装 ubuntu 子系统获得 Linux 环境。首先确保你的系统中有 python, 2.7 和 3.0 版本或更新的版本。执行
which python 或 which python3
如果输出路径则 python 存在, 如果不存在就安装一个 python。然后官网下载 moltemplate (<http://www.moltemplate.org/download.html>) 使用 git 命令或下载源代

码压缩包都行。如果是下载的压缩包, 就解压缩到合适位置。假设 moltemplate 在路径/home/user 下。那么就把以下两个路径加入到.bashrc 文件末尾

```
export PATH=/home/user/moltemplate/moltemplate:$PATH
```

```
export PATH=/home/user/moltemplate/moltemplate/scripts:$PATH
```

然后执行

```
source ~/.bashrc
```

```
which moltemplate.sh
```

若输出路径则安装完成。上述安装假设你的 shell 是 bash。

分子动力学的模型往往是由大量重复组分构成, 如水体系就有大量水分子构成, 金属块有大量晶胞构成等等。moltemplate 建模的基本原理就是先手动设定一个重复单元的信息, 然后通过 x, y, z 三个方向上阵列复制重复单元得到想要的模型。通过阵列复制 moltemplate 内部自动处理 atom id, type 以及坐标这些信息。moltemplate 没有 GUI 界面和 LAMMPS 一样通过编写文本文件然后读入文件内容执行。这些文件被称为 lt 文件。一般你的模型有几部分组成就会写几个 lt 文件加一个最终控制执行的文件。比如本文即将用到的例子就由上壁面, 水和下壁面构成, 因此会写 4 个 lt 文件: upwall.lt, downwall.lt, water.lt 和 system.lt。前三个分别设定上壁面, 下壁面和水重复单元信息, system.lt 控制阵列重复的设置。

我们先来看看 water.lt 学习 lt 文件的基本内容。

```
wat {

  write("Data Atoms") {
    $atom:O $mol:. @atom:O -1.1128 0.000000 0.00000 0.000000
    $atom:H1 $mol:. @atom:H 0.5564 0.7569503 0.00000 0.5858823
    $atom:H2 $mol:. @atom:H 0.5564 -0.7569503 0.00000 0.5858823
  }

  write_once("Data Masses") {
    @atom:O 15.9994
    @atom:H 1.008
  }

  write("Data Bonds") {
    $bond:OH1 @bond:OH $atom:O $atom:H1
    $bond:OH2 @bond:OH $atom:O $atom:H2
  }
}
```

```

}

write("Data Angles") {
  $angle:HOH @angle:HOH $atom:H1 $atom:O $atom:H2
}

write_once("In Settings") {
  bond_coeff    @bond:OH 600.0    1.0
  angle_coeff    @angle:HOH 75.0    109.47
  pair_coeff     @atom:O @atom:O lj/cut/tip4p/long 0.1553  3.166
  pair_coeff     @atom:H @atom:H lj/cut/tip4p/long 0.0      0.0
}

write_once("In Init") {
  dimension      3
  units metal
  boundary p p f
  neighbor 2.0 bin
  neigh_modify every 1 delay 0 check yes
  atom_style     full
  pair_style     hybrid lj/cut/tip4p/long 3 4 1 1 0.1546 8.5 8.5 eam
  bond_style     harmonic
  angle_style    harmonic
  kspace_style   pppm/tip4p 0.0001
  kspace_modify slab 3.0
}
}

```

首先在 `water.lt` 中给重复单元定义一个名称（这个名称是自定义的就像任何编程语言中的变量定义一个道理）。比如上面 `wat` 然后写一对花括号构成如下结构。花括号中的内容就是 `wat` 这个重复单位的设定，`wat` 名称会在 `system.lt` 中调用。

```

wat {
  .....
}

```

花括号中有多个 `write(){}` 命令和 `write_once(){}` 命令组成，这两个命令实现向输出文件中写相应的内容。`moltemplate` 每次执行会生成三个目标文本文件：`system.in.init`, `system.in.settings` 和 `system.data`。`write(){}` 命令和 `write_once(){}` 命令圆括号中的内容指出花括号中的内容向哪个文件进行写入。`write(){}` 命令表示内容对多次写入，`write_once(){}` 命令表示内容只写一次。`write("Data Atoms") {}`

向 data 文件中 Atoms 部分写内容, write_once("Data Masses") {} 向 data 文件中 Masses 部分写内容, write("Data Bonds") {} 向 data 文件中 Bonds 部分写内容, write("Data Angles") {} 向 data 文件中 Angles 部分写内容。本例中的体系是水加上金属壁面因此可以知道 data 文件的主体内容就包括这三个部分。write_once("In Settings") {} 向 Settings 文件写内容, write_once("In Init") {} 向 Init 文件写内容。

来看 write("Data Atoms"){} 中的内容。这里面每行的内容与你设定的 atom_style 对应, 比如本例中设定的 atom_style 是 full, 那么这里面的内容以及 data 文件 Atoms 部分每一行就包括

atom-ID mol-ID atom-type charge x y z

这些内容在 moltemplate 中对应为

```
$atom:O $mol:. @atom:O -1.1128 0.0000000 0.00000 0.000000
```

我们看到这里有两个特殊字符\$和@,\$表示 id,@表示 type,如\$atom:,\$mol:,\$bond:,\$angle:分别表示 atom-ID,mol-ID,bond-ID 和 angle-ID;@atom:,@bond:,@angle:分别表示 atom type, bond type, angle type。这些都是 moltemplate 内部的关键词(包括冒号),冒号后面是自定义的原子,键和键角的名称,或原子,键和键角类型的名称。这些名称在定义和引用时要一致。在一个重复单元中每个原子都有自己的名称,但是可能会共用 atom_type,以及共用一个 mol-ID。键和键角也类似。

在一个水分子中有三个原子,两个键,一个键角所以 write("Data Atoms") {} 中有三行内容代表三个原子的信息,分别表示一个氧原子和两个氢原子。三个原子分别表示为\$atom:O,\$atom:H1,\$atom:H2,这三个原子属于分子\$mol:.,其 atom_style 分别为@atom:O 和@atom:H。

```
$atom:O $mol:. @atom:O -1.1128 0.0000000 0.00000 0.000000
```

```
$atom:H1 $mol:. @atom:H 0.5564 0.7569503 0.00000 0.5858823
```

```
$atom:H2 $mol:. @atom:H 0.5564 -0.7569503 0.00000 0.5858823
```

write("Data Bonds") {} 有两行信息

```
$bond:OH1 @bond:OH $atom:O $atom:H1
```

```
$bond:OH2 @bond:OH $atom:O $atom:H2
```

\$bond:OH1 键的类型是@bond:OH,由\$atom:O 和\$atom:H1 构成,\$bond:OH2 键

的类型是@bond:OH, 由\$atom:O 和\$atom:H2 构成。

write("Data Angles") {} 有一行信息

\$angle:HOH @angle:HOH \$atom:H1 \$atom:O \$atom:H2

\$angle:HOH 键角的类型是@angle:HOH, 由\$atom:H1, \$atom:O 和\$atom:H2 构成。

write_once("Data Masses") {} 有两行内容因为有两个 type 的原子。这样就把一个水分子定义完成了。

write_once("In Settings") {} 和 write_once("In Init") {} 你写什么内容后面文件中就出现什么内容。通常这部分我们会在自己的 in 文件中修改, 这里写什么不是很关键, 我们要的是 data 文件的内容。

类似的 upwall.lt 和 downwall.lt 中的内容也类似

upwall {

#AtomID MolID AtomType Charge X Y Z

write("Data Atoms") {

\$atom:UC	\$mol:.	@atom:U	0.0	0.000	0.000	0.000
\$atom:UX	\$mol:.	@atom:U	0.0	0.000	2.040	2.040
\$atom:UY	\$mol:.	@atom:U	0.0	2.040	0.000	2.040
\$atom:UZ	\$mol:.	@atom:U	0.0	2.040	2.040	0.000

}

write_once("In Settings") {

pair_coeff @atom:U @atom:U 1.0 1.0

}

write_once("Data Masses") {

@atom:U 1.0

}

}

downwall {

#AtomID MolID AtomType Charge X Y Z

write("Data Atoms") {

\$atom:DC	\$mol:.	@atom:D	0.0	0.000	0.000	0.000
\$atom:DX	\$mol:.	@atom:D	0.0	0.000	2.040	-2.040
\$atom:DY	\$mol:.	@atom:D	0.0	2.040	0.000	-2.040
\$atom:DZ	\$mol:.	@atom:D	0.0	2.040	2.040	-0.000

}

```

write_once("In Settings") {
  pair_coeff @atom:D @atom:D 1.0 1.0
}

write_once("Data Masses") {
  @atom:D 1.0
}
}

```

定义好个重复单元的内容后剩下的就交给 moltemplate 了。system.lt 中的内容如下

```

import "upwall.lt"
import "downwall.lt"
import "water.lt"

write_once("Data Boundary") {
  0      408      xlo xhi
  0      32.64    ylo yhi
  -15    245      zlo zhi
}

UPWALL = new upwall [1].move(0.00, 0.00, 4.08)
               [8].move(0.00, 4.08, 0.0)
               [100].move(4.08, 0.0, 0.0)
UPWALL[*][*][*].move(0.0,0.0,240.0)

DOWNWALL = new downwall [3].move(0.00, 0.00, -4.08)
               [7].move(0.00, 4.08, 0.0)
               [100].move(4.08, 0.0, 0.0)

wats = new wat[50].move(0.00, 0.00, 3.0)
               [10].move(0.00, 3.0, 0.0)
               [50].move(3.0, 0.0, 0.0)

wats[*][*][*].move(129,0.0,45)

```

首先把建立好的 water.lt, upwall.lt 和 downwall.lt 使用 import 命令导入。然后用 write_once 命令写出盒子边界。然后使用 new 关键词和.move 命令阵列复制重复单元得到我们需要的模型。比如

```

WAT = new wat [30].move(0, 0, -3.0)

```



```
[19].move(0, 3.0, 0)
```

```
[19].move(3.0, 0, 0)
```

这块的效果就是生成一个水块名字叫 WAT 使用的是 wat 重复单元, 分别以 3.0 为间隔在 z 负方向 (-3.0) 阵列复制 30 次, x 和 y 正方向阵列复制 19 次, 这样就生成了一个长方体形状的水块。UPWALL 和 DOWNWALL 也类似。需要指出的是 .move 命令当加了 new 关键词就会新生成分子, 不加就会单纯的移动已经生成的部分。

在写好所有文件后执行下面命令生成 data 文件

```
moltemplate.sh -atomstyle full system.lt
```

然后在 ovito 中观察模型是否合理, 不合理再进行修改。

上面教程只是 moltemplate 的最基本使用, 更复杂的功能可参考 moltemplate 的例子。但是已经够生成大部分模型了。

10.2 in 文件讲解

上一节中的水分子模型采用 TIP4P/2005 模型。在弛豫完成后通过将水滴向下移动一段距离使其与下壁面接触从而自发润湿。在移动水滴时要调试移动距离以确保水滴下端距离下壁面足够近, 但又不会引发原子重叠, 一般距离 3 埃左右为宜。

in 文件内容为

```
#####基本模拟规则#####
```

```
dimension      3
boundary       p p f
units          real
atom_style     full
pair_style lj/cut/tip4p/long 1 2 1 1 0.1546 8.5
bond_style     harmonic
angle_style    harmonic
kspace_style   pppm/tip4p 0.0001
kspace_modify slab 3.0
```

```
#####定义几何模型#####
```

```
read_data "system.data"
```

```
#####定义物理模型#####
```

```
#type 1=upwall type 2=downwall type 3=O type 4=H
```

```
pair_coeff 1 1 0.1852 3.1589
pair_coeff 2 2 0.0 2.058
pair_coeff 3 3 0.1852 3.1589
pair_coeff 4 4 0.1852 3.1589
```

```
bond_coeff 1 600.0 0.9572
angle_coeff 1 75.0 104.52
```

```
#####定义分组信息#####
```

```
group water type 3 4
group upwall type 1
group downwall type 2
```

```
#####弛豫过程#####
```

```
fix 1 upwall setforce 0 0 0
fix 2 downwall setforce 0 0 0
minimize 1.0e-5 1.0e-7 1000 10000
```

```
unfix 1
unfix 2
```

```
compute myT water temp
timestep 1.0
thermo 1000
thermo_style custom step temp
thermo_modify temp myT flush yes
fix fShakeSPCE water shake 0.0001 10 0 b 1 a 1
velocity water create 300 372748 temp myT rot yes dist gaussian
dump mydump all custom 10000 dump.lammpstrj id type x y z
fix fxnvt water nvt temp 300.0 300.0 100.0
run 1000000
```

```
#####润湿过程#####
```

```
displace_atoms water move 0 0 -14 units box
```

```
run 1000000
```

10.3 MS+packmol 建模

本节内容来自网友马化云克, 感谢网友的贡献

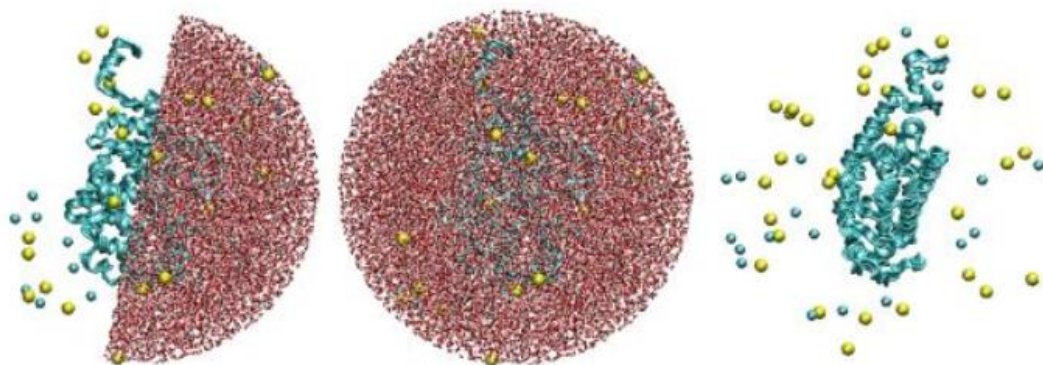
10.3.1 所需软件

目前市面上充斥着各种建模、模拟、后处理和绘图软件, 我们不可能面面俱

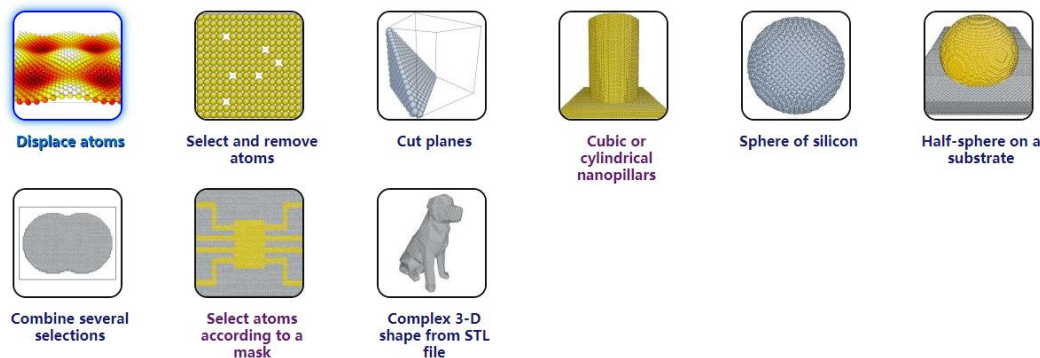
到，每个软件都会。所以只挑选了几个上手简单，又可以完成相对比较复杂的功能的软件。

建模工具包括 Packmol，Material Studio 和 AtomsK。

Packmol(<http://leandro.iqm.unicamp.br/m3g/packmol/home.shtml>)使用方法非常简单，只需少量脚本就可以建立比较复杂的模型，如下图所示。我们将使用 Packmol 建立所需形状的各类液滴，比如水滴、离子液滴、溶液液滴等。



AtomsK(<https://atomsk.univ-lille.fr/index.php>)用于构建具有复杂结构的基底。类似于 Packmol，AtomsK 同样具有容易上手，脚本简单，功能强大的特点，特别对于各种晶体模型（无论 2D 或者 3D）的建立很方便。在润湿模拟，基底通常具有很多微观结构，使用 AtomsK 无疑可以达到事半功倍的效果。



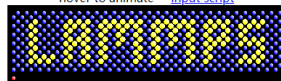
Material Studio 用于一些简单的建模、模型组装以及加力场，也可以用于可视化出图。

模拟主要靠 LAMMPS 全家桶，包括 lammmps 主程序以及一些附件工具，如 msi2lmp 等。

LAMMPS Molecular Dynamics Simulator

lamp: a device that generates light, heat, or therapeutic radiation; something that illumines the mind or soul -- www.dictionary.com

hover to animate -- [input script](#)



[physical analog](#)

new The LAMMPS Homepage has moved to <https://www.lammps.org>. Please update your links.

new [2021 Virtual LAMMPS Workshop and Symposium](#), Aug 10-13, hosted by Temple University.

Big Picture	Code	Documentation	Results	Related Software	Context	User Support
Features	Download	Manual	Publications	Pre/Post processing	Authors	Mail list
Non-features	GitHub	Programmer guide	Pictures	External packages & tools	History	MatSci forum
Packages	SourceForge	Tutorials	Movies	Pizza.py Toolkit	Funding	Slack & IRC channel
FAQ	Latest features & bug fixes	MD to LAMMPS glossary	Benchmarks	Visualization	Open source	Workshops
Wish list	Report bugs & request features	Commands	Citing LAMMPS	Other MD codes	Contribute to LAMMPS	Books about MD

可视化和后处理工具主要包括 OVITO 和 VMD, 我们还需要会一些简单的脚本和小程序。

10.3.2 使用 Packmol 建立液滴模型

使用 Packmol 建立液滴模型需要以下几个文件: Packmol 执行程序、分子模板文件和 Packmol 脚本文件。

Packmol 源代码和执行文件可以在官网获取, 我们也在 QQ 群(670911765)里上传了, 需要的同学在群文件中自行下载。

<http://www.ime.unicamp.br/~martinez/packmol/userguide.shtml>

Packmol 语法非常简单, 堪称一看就会, 一做就对, 就没有专门讲解的必要了, 如果同学们有需求可以专门写一期。

现在准备分子模板, 分子模板可以认为是我们需要建立模型的最小单位, 比如水滴的分子模板就是水分子。如果液滴中含有不同分子、或者离子, 则分子模板也需要分开准备。下面以纯水液滴和氯化钠溶液液滴为例。

水分子模板

HEADER water

COMPND

SOURCE

HETATM 1 H HOH 1 9.626 6.787 12.673

HETATM 2 H HOH 1 9.626 8.420 12.673

HETATM 3 O HOH 1 10.203 7.604 12.673

CONECT 1 3

CONECT 2 3

CONECT 3 1 2

END

水分子需要给出原子序号、类型和相对位置(HETATM 开头的行)，以及连接情况(CONECT 开头的行)，其余行应该每个分子模板都是一样的。

氯离子模板

REMARK Materials Studio PDB file

REMARK Created: Sun Apr 08 01:52:21 中国标准时间 2018

ATOM 1 Cl MOL 2 -0.457 1.541 18.284 1.00 0.00 Cl1-

TER 2

钠离子模板

REMARK Materials Studio PDB file

REMARK Created: Sun Apr 08 01:52:05 中国标准时间 2018

ATOM 1 Na MOL 2 0.653 1.541 18.284 1.00 0.00 Na1+

TER 2

氯离子模板和钠离子模板仅需给出离子的位置就行(ATOM 开头的行)，可以随意修改。细心的同学可以看出水分子模板和氯离子、钠离子的模板格式有些不同，那么笔者是怎么知道该怎么写呢？事实上，我也不知道，其实也不用关心，因为这个模板是从 Materials Studio 里面导出出来的。只需在 Materials Studio 建立一个水分子、氯离子或者钠离子在导出 pdb 格式文件即可，需要注意的事这个时候并不需要建立 box。所以读者可以自行建立 pdb 文件，也可以采用本文给出的模板，将模板内容复制进入文档，并分别命名为 H2O.pdb、Cl.pdb 和 Na.pdb 即可。

Packmol 脚本文件

下面开始准备 Packmol 脚本文件，本文以球形液滴为例。Packmol 脚本文件的逻辑非常简单，即：读入分子模板==>把指定数量的分子放入指定区域==>写出模型文件。以下是 Packmol 脚本文件示例：

##表示注释

tolerance 2.0 #tolerance 表示距离约束容差，可以理解为原子间最小距离不低于该值，对于水分子和氯离子钠离子来说 2Å 是个不错的值

filetype pdb #filetype 表示原子类型，这里选择 pdb 格式

output Droplet.pdb #output 表示输出文件，droplet.pdb 为输出文件名

1000 个水分子置于半径为 20 球内部

```
# 球心位于原点
structure H2O.pdb #以 H2O.pdb 为模板建立 structure
    number 1000      #水分子数量为 1000
    inside sphere 0.0 0.0 0.0 20.0 #inside sphere 表示在球内部，0.0 0.0 0.0 为球心坐标，20.0 为半径
end structure      #停止使用该模板建模
```

10 个氯离子和钠离子置于半径为 20 球内部

```
# 球心位于原点
structure Cl.pdb
    number 10
    inside sphere 0.0 0.0 0.0 20.0
end structure
```

```
structure Na.pdb
    number 10
    inside sphere 0.0 0.0 0.0 20.0
end structure
```

10.3.3 使用 packmol 和 MS 建模

将该脚本命名为 Droplet.inp,将 Packmol 执行程序、分子模板文件和 Packmol 脚本文件放进同一个文件夹，如下图：

名称	修改日期	类型	大小
Cl.pdb	2018/4/8 星期日 ...	Materials Studio...	1 KB
Droplet.inp	2021/8/25 星期...	Materials Studio...	1 KB
H2O.pdb	2006/3/22 星期...	Materials Studio...	1 KB
Na.pdb	2018/4/8 星期日 ...	Materials Studio...	1 KB
packmol.exe	2016/7/5 星期二 ...	应用程序	2,521 KB

打开命令窗口，使用以下命令即可完成建模。

Packmol<Droplet.inp

```
C:\Users\Administrator\Desktop\Packmol>packmol<Droplet.inp
```

```

管理员: C:\windows\system32\cmd.exe

Success!
Final objective function value: .11093E-03
Maximum violation of target distance: 0.000000
Maximum violation of the constraints: .11093E-03

-----

Please cite this work if Packmol was useful:

L. Martinez, R. Andrade, E. G. Birgin, J. M. Martinez,
PACKMOL: A package for building initial configurations for
molecular dynamics simulations.
Journal of Computational Chemistry, 30:2157-2164,2009.

=====

Running time: 0.7580000 seconds.
    
```

文件夹中已经输出模型文件 Droplet.pdb

名称	修改日期	类型	大小
Cl.pdb	2018/4/8 星期日 ...	Materials Studio...	1 KB
Droplet.inp	2021/8/25 星期...	Materials Studio...	1 KB
Droplet.pdb	2021/8/25 星期...	Materials Studio...	243 KB
H2O.pdb	2006/3/22 星期...	Materials Studio...	1 KB
Na.pdb	2018/4/8 星期日 ...	Materials Studio...	1 KB
packmol.exe	2016/7/5 星期二 ...	应用程序	2,521 KB

用 Materials Studio 打开即可。

10.3.4 使用 MS 建立基底+基底液滴合并

本期是润湿系列教程的第三期, 主要讲解使用 MS 建立基底, 并与上期使用 Packmol 建立的液滴合并, 导出之后即可进行下一步操作, 即模拟。

MS 建立基底

在实际的模拟, 基底常常会有不同的结构, 使用 atomsk 可以轻易得到, 我们在这里不讲解关于 atomsk 的操作。假设基底具有为原子级光滑的表面, 首先需要得到基底的晶胞坐标, 也可以直接下载晶胞的结构文件, 然后在 MS 中进行扩胞, 得到需要的尺寸即可, 在此我们采用后一种方法。

假设润湿过程为: 纳米水液滴在铜基底上润湿, 所以我们先得到铜的 cif 文件, 可从以下网址获取:

<http://rruff.geo.arizona.edu/AMS/amcsd.php>

在 Mineral 一栏中输入: copper, 然后点击 search 即可

American Mineralogist Crystal Structure Database

This site is an interface to a crystal structure database that includes every structure published in the American Mineralogist, The Canadian Mineralogist, European Journal of Mineralogy and Physics and Chemistry of Minerals, as well as selected datasets from other journals. The database is maintained under the care of the Mineralogical Society of America and the Mineralogical Association of Canada, and financed by the National Science Foundation.

Mineral

Author

Chemistry Search

Cell Parameters and Symmetry

Diffraction Search

General Search

Logic interface ☐ AND ☐ OR

Viewing (About [File Formats](#)) ☐ amc long form ☐ amc short form ☐ cif

Download ☐ amc ☒ cif ☐ diffraction data

得到结果如下图所示, 有 14 个搜索结果, 都大同小异, 我们选择第一个结果, 点击下载 CIF data 即可。

American Mineralogist Crystal Structure Database

14 matching records for this search.

☐ Copper

Wyckoff R W G

Crystal Structures 1 (1963) 7-83

Second edition. Interscience Publishers, New York, New York

Cubic closest packed, ccp, structure

_database_code_amcscd 0011145

3.61496 3.61496 3.61496 90 90 90 Fm3m

atom x y z

Cu 0 0 0

[Download AMC data \(View Text File\)](#)

[Download CIF data \(View Text File\)](#)

[Download diffraction data \(View Text File\)](#)

[View JMOI 3-D Structure \(permalink\)](#)

☐ Copper

Otte H M

Journal of Applied Physics 32 (1961) 1536-1546

Lattice parameter determinations with an X-ray spectrogoniometer by the Debye-Scherrer method and the effect of specimen condition

Locality: synthetic

_database_code_amcscd 0012903

3.6150 3.6150 3.6150 90 90 90 Fm3m

atom x y z

Cu 0 0 0

[Download AMC data \(View Text File\)](#)

[Download CIF data \(View Text File\)](#)

[Download diffraction data \(View Text File\)](#)

[View JMOI 3-D Structure \(permalink\)](#)

☐ Copper

Suh I-K, Ohta H, Waseda Y

Journal of Materials Science 23 (1988) 757-760

High-temperature thermal expansion of six metallic elements measured by dilatation method and X-ray diffraction

Locality: synthetic

Sample: at T = 293 K

_database_code_amcscd 0013087

3.613 3.613 3.613 90 90 90 Fm3m

atom x y z

Cu 0 0 0

[Download AMC data \(View Text File\)](#)

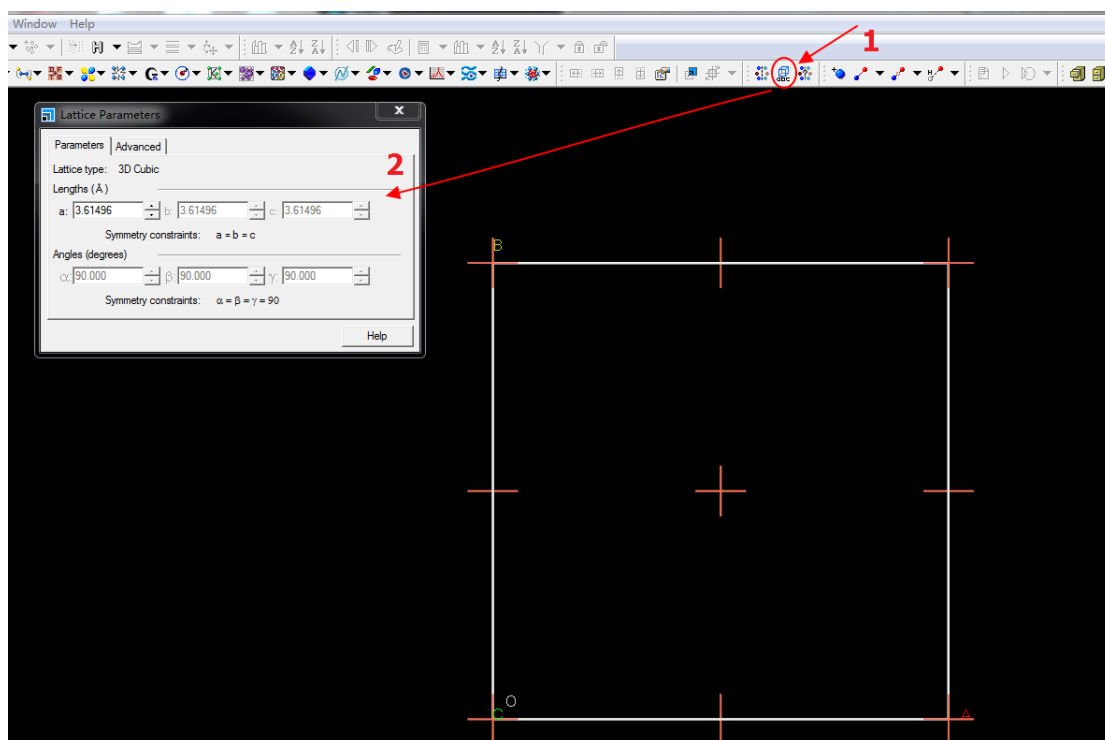
[Download CIF data \(View Text File\)](#)

[Download diffraction data \(View Text File\)](#)

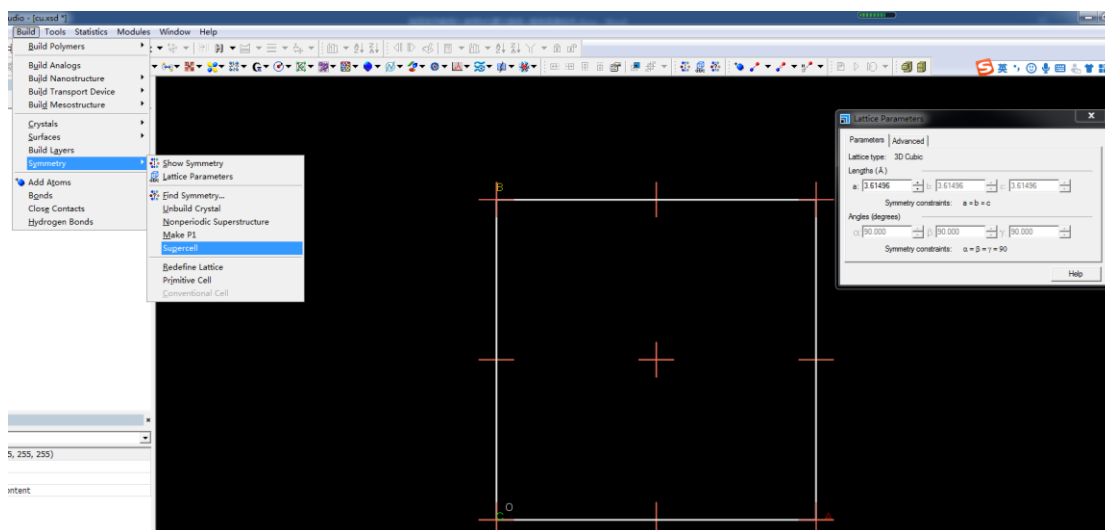
[View JMOI 3-D Structure \(permalink\)](#)

使用 MS 打开下载的 cif 文件, 点击如图所示的小立方体, 就可知晶胞的参数, 三个长度数据和三个角度数据, 此案例中 $a=b=c$, 三个角度均为 90 度。

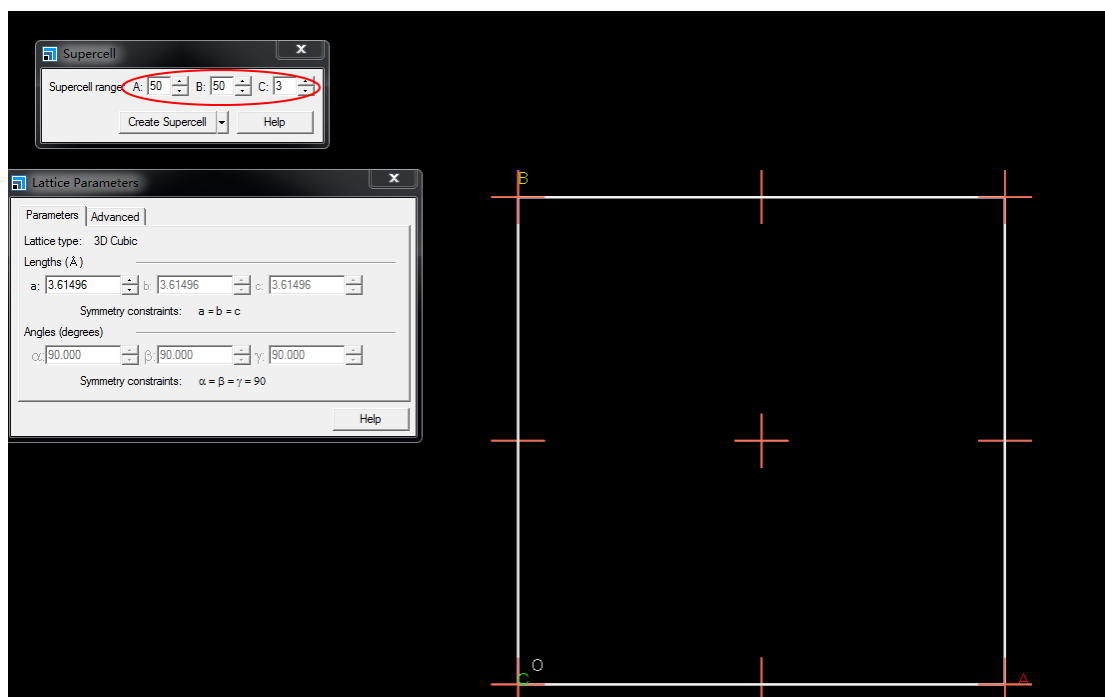
164



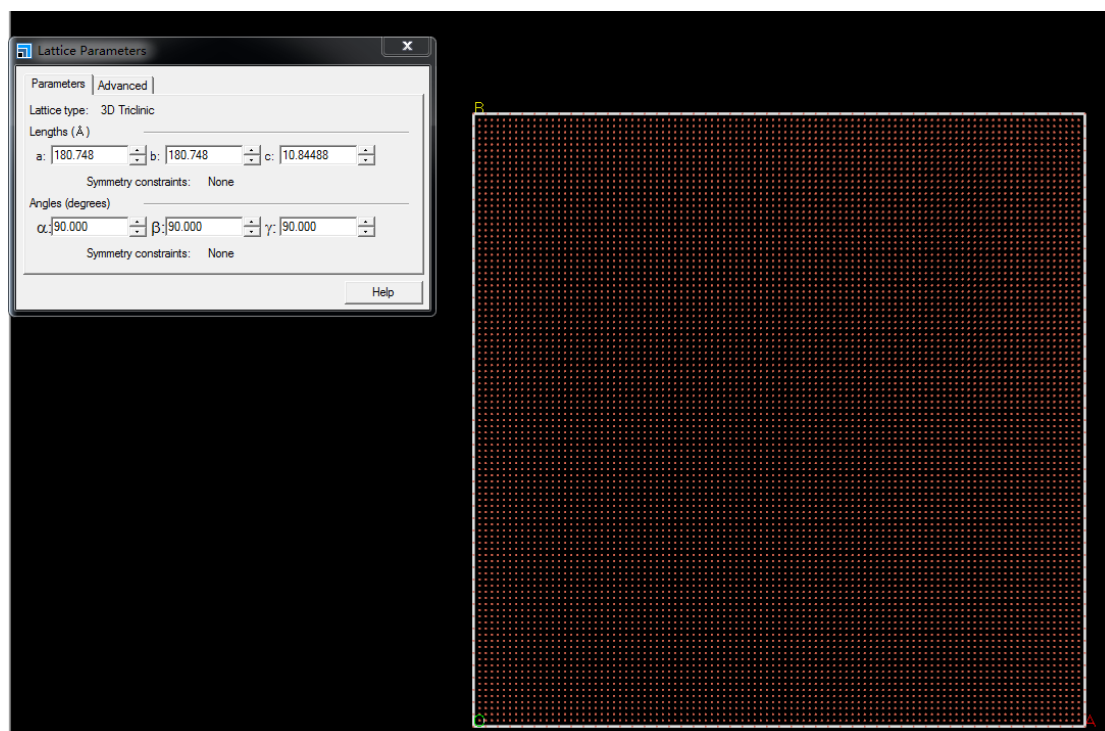
我们假设润湿模拟中基底的 a、b 方向长度为 50*晶胞棱长，c 方向为 3*晶胞棱长，所以需要对该模型进行扩胞，即建立 supercell。路径为：Build>Symmetry>supercell。



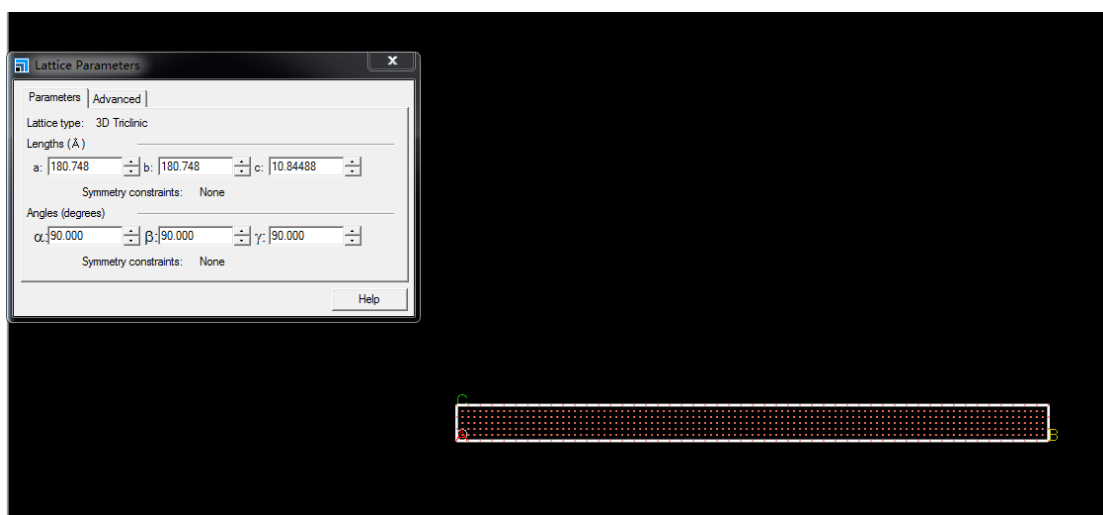
点击之后如下图所示，输入所需的倍数即可，这里为 50*50*3。



得到模型，如下：



换个显示角度：



建立液滴模型

建立球状液滴, 建立方法在教程二中已有说明, 为了使液滴和基底可以顺利合并, 并使模拟顺利进行, 需要注意以下 2 点:

(1) packmol 建立的液滴模型需要在 MS 中加个 box, 即建立成“晶体”, 才可以与基底合并, 且“晶体”的 a 和 b 需要和基底的保持一致, 即 $a=b=180.748$ 。

(2) 液滴的密度应用在模拟温度下的密度保持一致, 300K 下水的密度约为 1 g/cm^3 。

假设液滴的半径的 2.5 nm, 根据密度可以算出该液滴中水分子有个。液滴在 a 和 b 方向上在基底中央, c 方向上在基底表面上方 5 Å, 则可知液滴球心位置为: 90.374 90.374 35.84488。

Packmol 的输入文件如下:

##表示注释

tolerance 2.0 #tolerance 表示距离约束容差, 可以理解为原子间最小距离不低于该值, 对于水分子和氯离子钠离子来说 2Å 是个不错的值

filetype pdb #filetype 表示原子类型, 这里选择 pdb 格式

output Droplet_R20.pdb #output 表示输出文件, droplet.pdb 为输出文件名

1000 个水分子置于半径为 20 球内部

球心位于原点

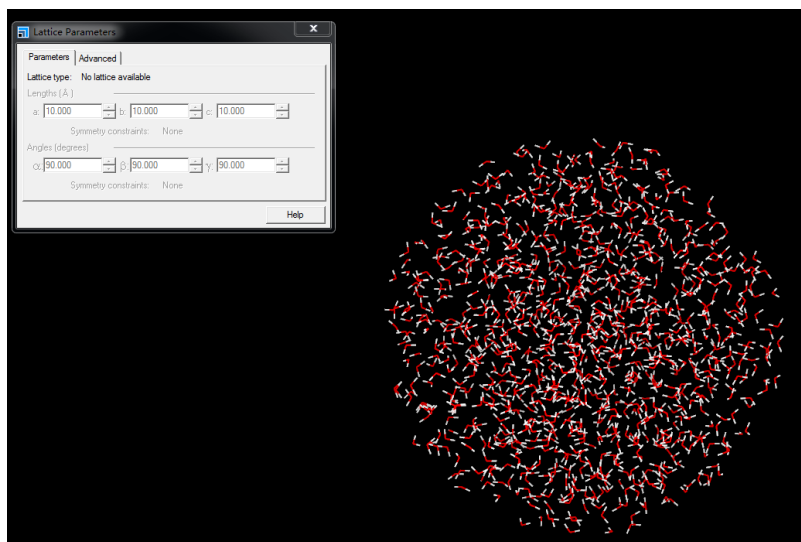
structure H2O.pdb #以 H2O.pdb 为模板建立 structure

number 1000 #水分子数量为 1000

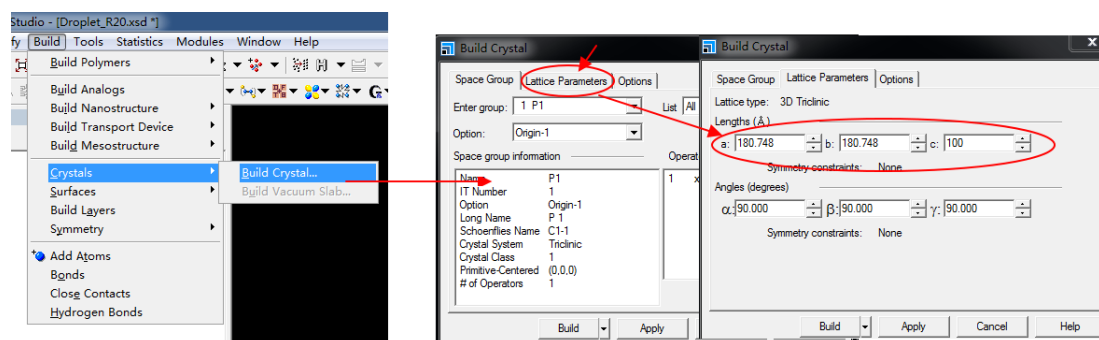
inside sphere 90.374 90.374 35.84488 20.0 #inside sphere 表示在球内部, 90.374 90.374 35.84488 35.84488 为球心坐标, 20.0 为半径

end structure #停止使用该模板建模

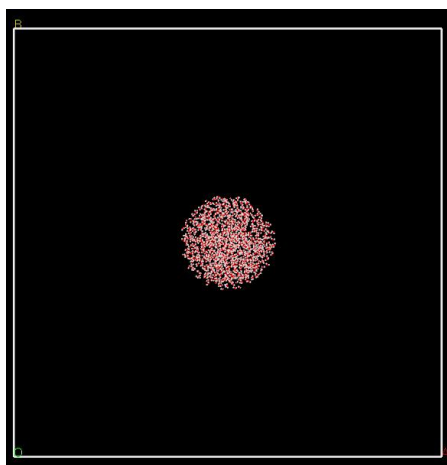
运行文件并把得到的文件用 MS 打开, 如下图所示:



建立“晶体”, 路径为: Build>crystal>build crystal 并设 a=b=180.748, c=100 即可。

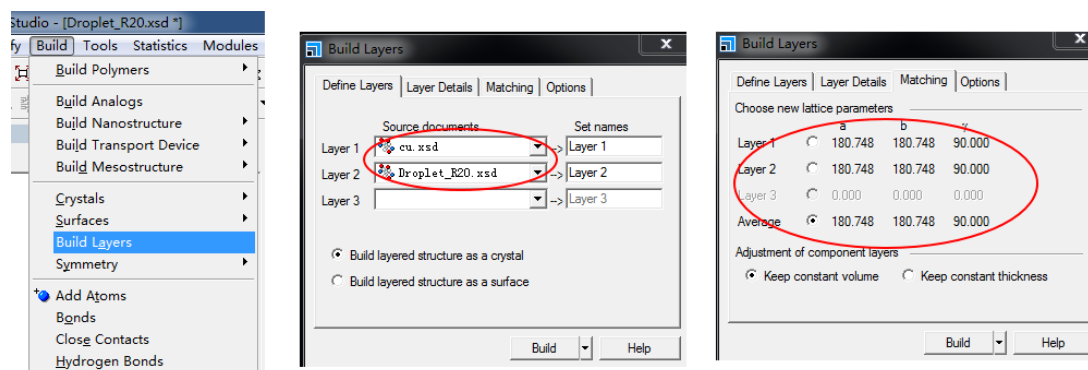


得到模型如下:

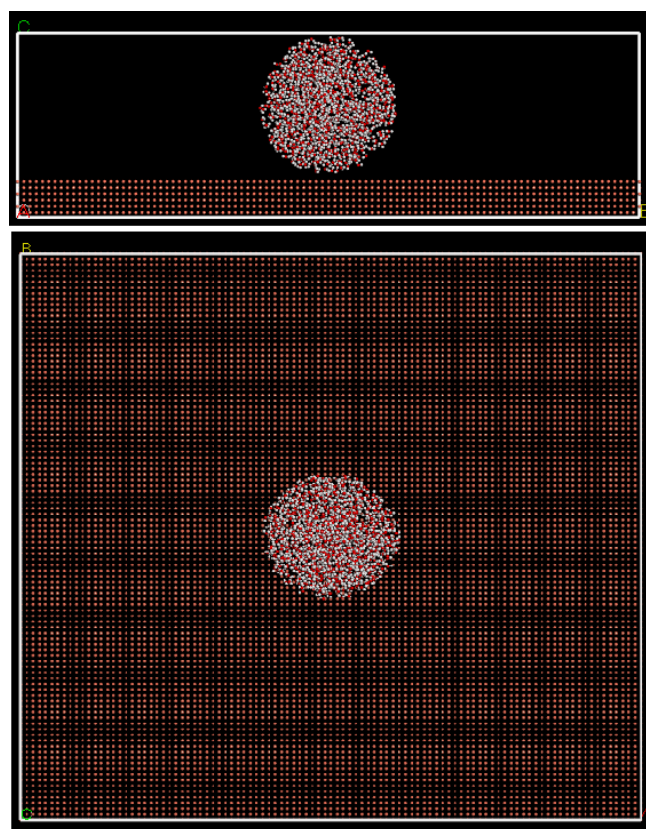


合并基底和液滴

合并液滴和基底, 路径为 Build>Build layers, 层 1 为基底, 层 2 为液滴, 注意查看两个模型的 a 和 b 是否相等, 不等的话最好进行调整, 此处模型没有问题。



合并后模型如下, 该模型经过简单的设置就可以导出进行模拟了!

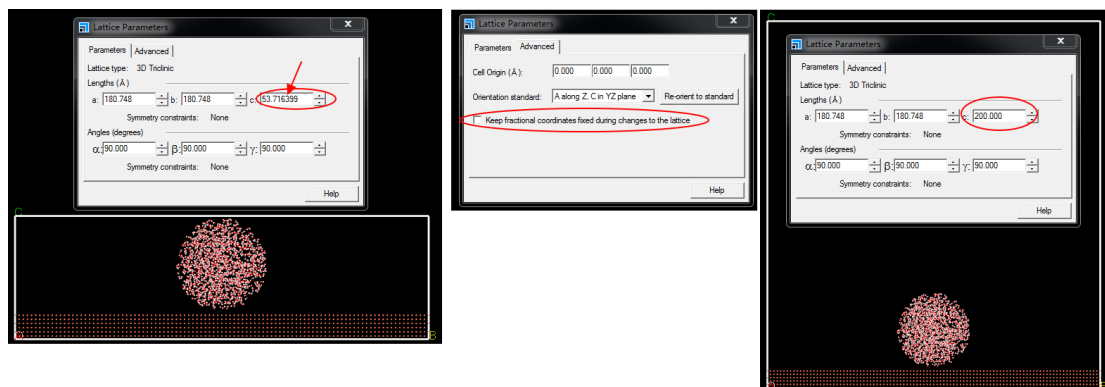


10.3.5 LAMMPS 的 data 文件导出与 in 文件编写

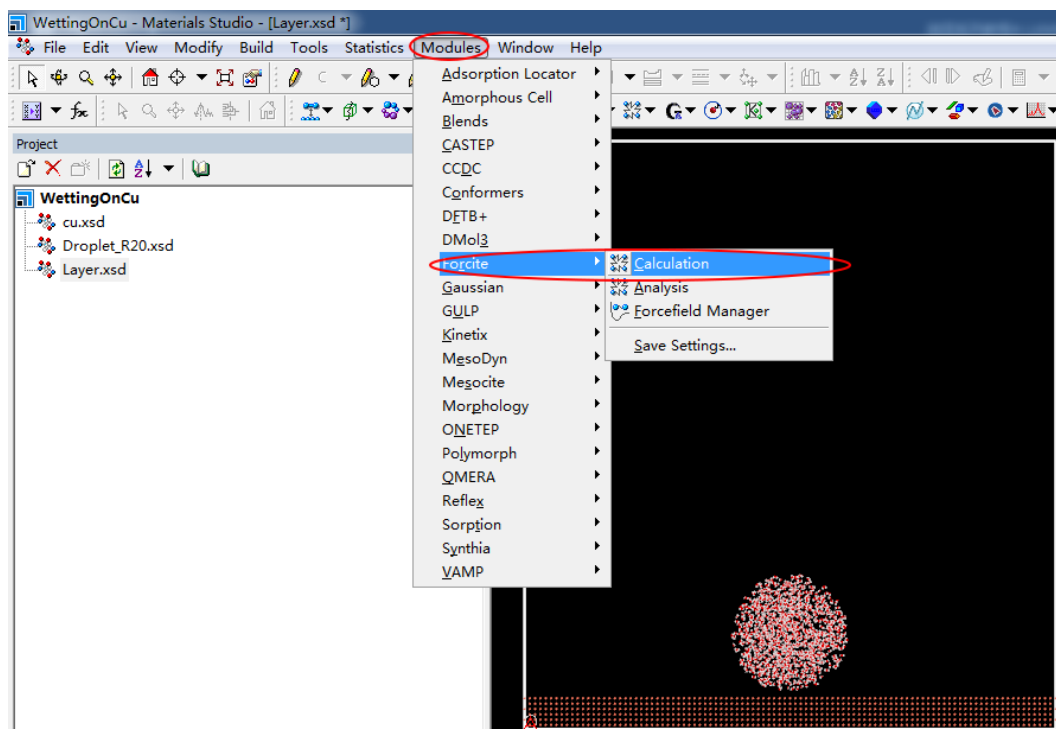
从 MS 中导出 LAMMPS 的 data 文件

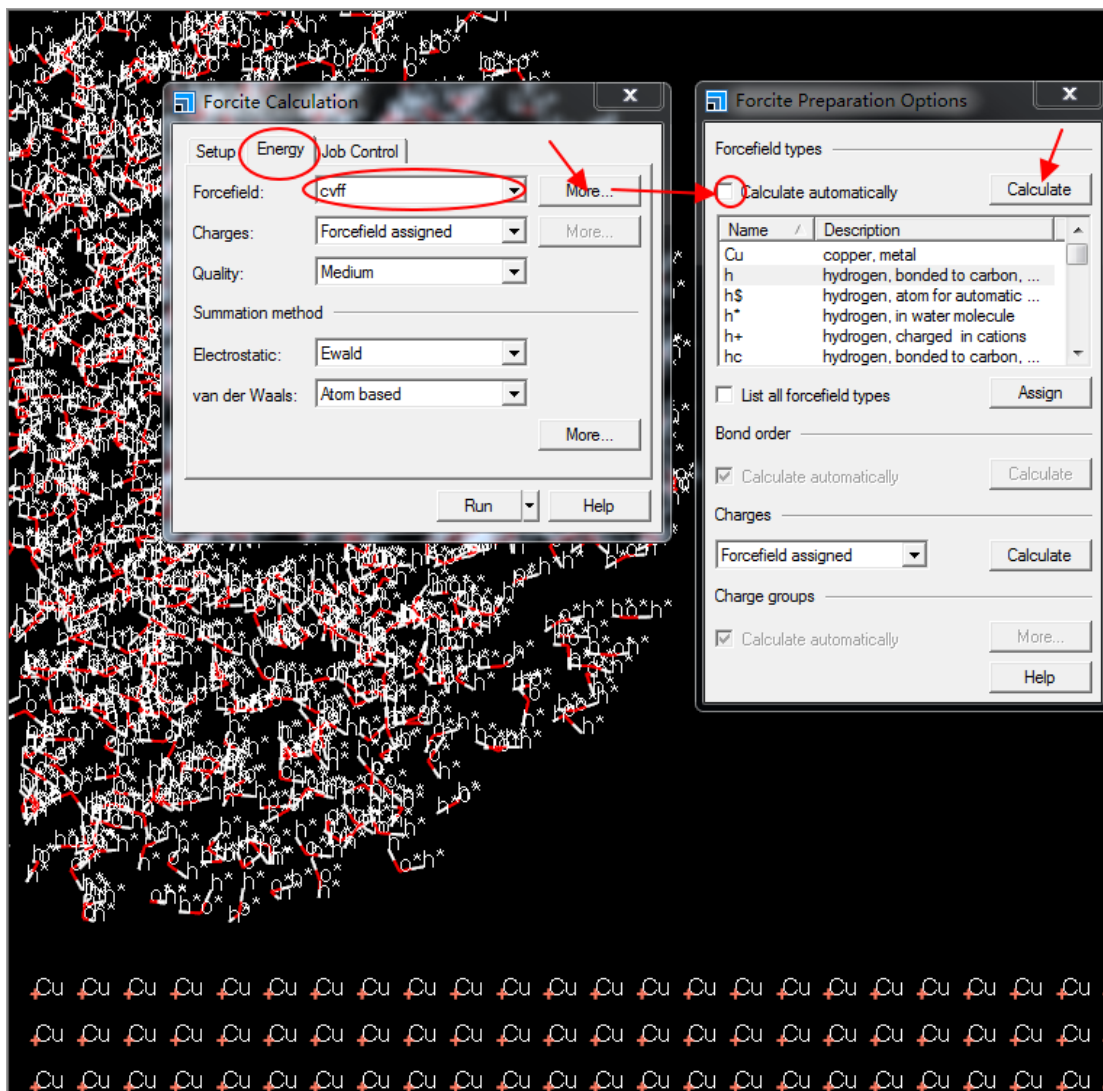
在导出模型之前先做两件事, 第一是把模型 box 在 Z 方向加大一点, 可以避免 Z 方向上由于周期性而导致非物理的作用和现象发生, 也就是说避免基底在液滴上方对水分子有作用, 第二是给模型赋力场。

原来的模型 Z 方向 box 长 53.72Å, 现在扩大为 200 Å, 如下图所示, 注意中间的图中不要勾选。

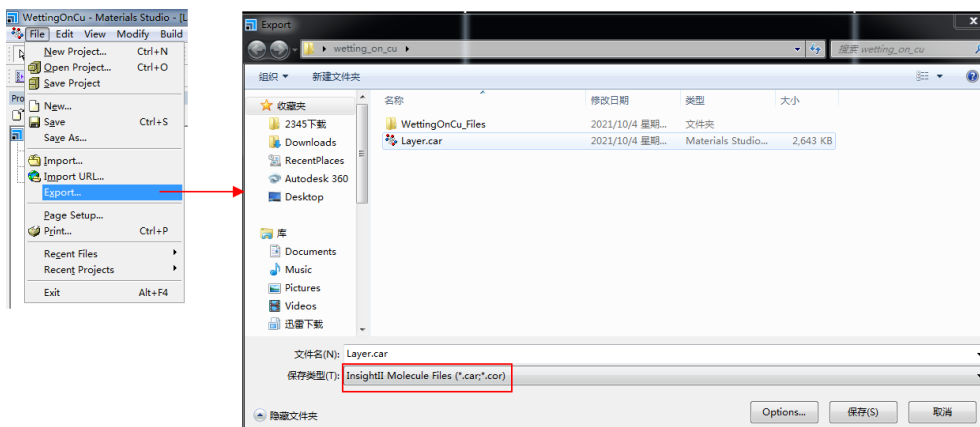


接下来是给模型赋力场, 这是后面使用 lammps 自带的程序将模型转为 data 文件的关键一步, 否则程序无法识别原子。注意按照下面图片中的步骤来, 改勾选的地方勾选, 不该勾选的地方不勾。



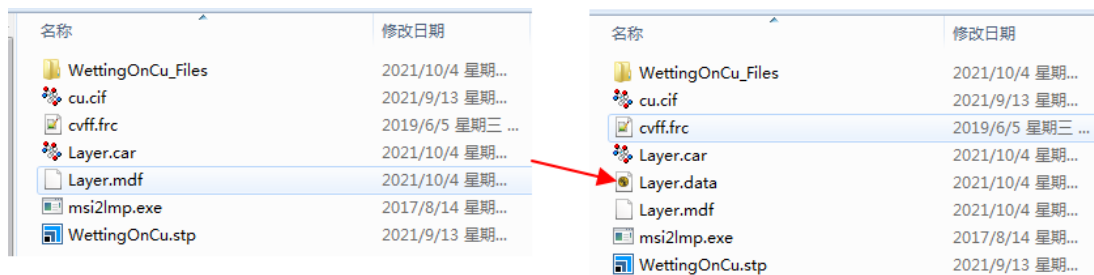


赋完力场之后就可以导出转成 data 文件了, 导出格式为 car, 然后会在文件夹中得到两个文件 Layer.car 和 Layer.mdf。

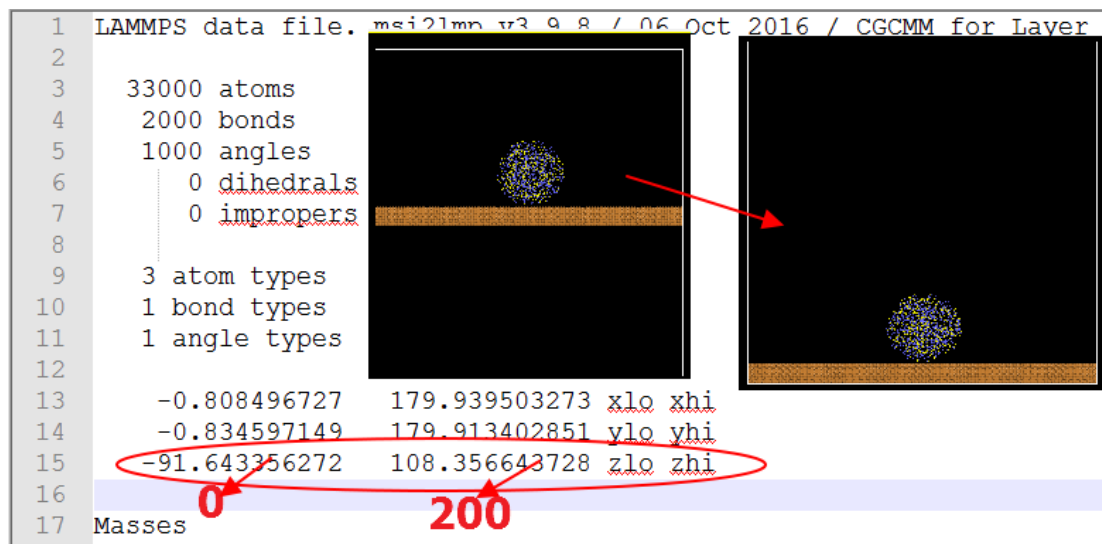


把 lammps 中自带的 msi2lmp.exe 和 cvff.frc 复制到存放 Layer.car 和 Layer.mdf 的文件夹, 打开 cmd, 输入命令即可转化为 data 文件, 如下图所示:


```
msi2lmp.exe Layer -class 1 -f \Users\Administrator\Desktop\wetting_on_cu\cvff -
i>info.data
```



注意使用 cvff 力场文件的绝对路径, 否则会报错。用 OVITO 打开生成的 data 文件, 如下图所示, 生成的 data 文件 box 上下边界与 MS 中不一致, 可以在 data 文件中直接修改即可。



接下来是修改势函数, 直接从力场文件中导出的参数有时候会有错漏或者需要人为进行修改以适应研究目的。这里假设基底原子和水分子之间的相互作用由 12-6 LJ 势函数描述, 水分子之间的相互作用由 12-6 LJ 势函数和库仑力来描述。并且润湿时基底固定不动, 改变基底原子和水分子之间的势函数参数来调节基底的润湿性。因此基底原子与基底原子之间的相互作用参数可以随意设置, 水分子这里假设使用 spce 模型, 具体参数可从论文中获得。势函数参数可以写在 data 文件中, 也可以写在 in 文件中。当势函数函数参数写在 data 文件里面时, 在 in 文件中指定所需的原子间的相互作用的混合模式即可, 在 in 文件中则可以直接指定不同原子间的相互作用, 因此本文中的势函数参数均在 in 文件中。

编写 in 文件

编写 in 文件需要注意为模拟完成后求接触角提供便利, 目前论文中求接触角主

要有两种方式, 第一是直接在轨迹文件中拟合液滴, 第二是求得气液界面, 然后用球或者圆拟合气液界面, 得到接触角。无疑第二种方法是更适合的, 求取气液界面需要获得水分子密度的分布, 在 `lammps` 中, 可以进行对区域进行圆柱形划分, 也可以进行长方体划分。本例中采用的液滴是球形的, 使用圆柱形划分会更省时间, 但圆柱形划分要确定圆心的位置, 如果圆心位置稍有差别, 结果可能会截然不同, 因此本文推荐使用更稳健的长方体划分。本文将 `data` 文件命名为 `wetting.data`, `data` 文件中有原子质量信息以及拓扑信息, 如下图所示, 势函数参数在 `in` 文件中。

```

1 LAMMPS data file. msi2lmp v3.9.8 / 06 Oct 2016 / CGCMM for Layer
2
3 33000 atoms
4 2000 bonds
5 1000 angles
6 0 dihedrals
7 0 impropers
8
9 3 atom types
10 1 bond types
11 1 angle types
12
13 -0.808496727 179.939503273 xlo xhi
14 -0.834597149 179.913402851 ylo yhi
15 0 200 zlo zhi
16
17 Masses
18
19 1 63.546000 # Cu
20 2 1.007970 # h*
21 3 15.999400 # o*
22
23 Atoms # full
24
25 1 1 1 0.000000 0.000000000 0.000000000 10.437399976 0 0 0 # Cu
26 2 1 1 0.000000 0.000000000 1.807480000 1.399999976 0 0 0 # Cu
27 3 1 1 0.000000 1.807480000 0.000000000 1.399999976 0 0 0 # Cu
28 4 1 1 0.000000 1.807480000 1.807480000 10.437399976 0 0 0 # Cu
29 5 1 1 0.000000 3.614960000 0.000000000 10.437399976 0 0 0 # Cu
30 6 1 1 0.000000 3.614960000 1.807480000 1.399999976 0 0 0 # Cu
31 7 1 1 0.000000 5.422440000 0.000000000 1.399999976 0 0 0 # Cu
32 8 1 1 0.000000 5.422440000 1.807480000 10.437399976 0 0 0 # Cu
33 9 1 1 0.000000 7.229920000 0.000000000 10.437399976 0 0 0 # Cu
34 10 1 1 0.000000 7.229920000 1.807480000 1.399999976 0 0 0 # Cu
35 11 1 1 0.000000 9.037400000 0.000000000 1.399999976 0 0 0 # Cu
36 12 1 1 0.000000 9.037400000 1.807480000 10.437399976 0 0 0 # Cu
37 13 1 1 0.000000 10.844880000 0.000000000 10.437399976 0 0 0 # Cu
38 14 1 1 0.000000 10.844880000 1.807480000 1.399999976 0 0 0 # Cu
39 15 1 1 0.000000 12.652360000 0.000000000 1.399999976 0 0 0 # Cu

```

`in` 文件入所示:

```

# ----- Init Section -----
dimension          3
units              real
boundary           p p f
atom_style         full
bond_style         harmonic
angle_style        harmonic
neighbor           2.0 bin
neigh_modify       every 1 delay 0 check yes
pair_style          lj/cut/coul/long 10 12

```

```

kspace_style          pppm 1e-4
kspace_modify         slab 3.0
timestep              1.0
# ----- Atom Definition Section -----
read_data             system.data
# ----- group define -----
group                 water type 2 3
group                  Cu type 1
# ----- Settings Section -----
pair_coeff             1 1 0.0      0.0
pair_coeff             1 2 0.449    2.752
pair_coeff             1 3 0.0      0.0
pair_coeff             2 2 0.1553    3.166
pair_coeff             2 3 0.0      0.0
pair_coeff             3 3 0.0      0.0
bond_coeff             1 540.6336    1.0
angle_coeff            1 50          109.47
# ----- Run Section -----
velocity              Cu set 0 0 0
fix                   fix_Cu Cu setforce 0 0 0
#能量优化
dump                  1 all atom 100 mini.xyz
minimize              1e-5 1e-7 1000 10000
#undump               1
#初始温度
velocity              water create 298 98763 rot yes dist gaussian units box
fix                   fix_water water shake 0.0001 10 0 b 1 a 1
#计算温度
compute               newTemp water temp
#控温
fix                   1 water nvt temp 298 298 100
fix_modify            1 temp newTemp
#计算 Z 方向密度分布
compute               z water chunk/atom bin/1d z 3.61 1 units box
fix                   2 water ave/chunk 100 150 15000 z density/mass norm sample file
density_Z.profile
dump                  2 all custom 5000 dump.lammpstrj id mol type x y z
run                   30000

```

data 文件生成和 in 文件编写的思路大体如上所示, 遇到问题可加 QQ 群 670911765 讨论。值得注意的是, 纳米尺度下的润湿, 线张力的作用不可忽视, 会使平衡接触角偏离宏观观测, 因此建议本文建议如果润湿不涉及研究线张力的影响时, 优

先采用二维液滴,即圆柱状液滴。本文推荐两篇参考文献,一篇是中文学位论文,里面给出了润湿的 in 文件,可以作为参考,一篇是采用二维液滴研究基底与液滴相互作用对润湿性的影响,可为研究润湿的同学提供一些理论指导,推荐论文如下:

Effect of Solid–Liquid Interactions on Substrate Wettability and Dynamic Spreading of Nanodroplets: A Molecular Dynamics Study

<https://sci-hub.se/10.1021/acs.jpcc.0c07919>

固着纳米水滴润湿行为的分子动力学模拟

https://xueshu.baidu.com/usercenter/paper/show?paperid=652d7a7b1261e10f631ccaa7ca776d69&site=xueshu_se

使用下面 MATLAB 代码可以方便测量接触角

```
clear
rho=imread('C:\Users\baolu\Desktop\3.png');
imshow(rho)
set(gcf,'outerposition',get(0,'screensize'));
hold on
[p1,p2] = ginput(3);
x=[p1(1),p2(1)];
y=[p1(2),p2(2)];
linex=[x(1),y(1)];
liney=[x(2),y(2)];
plot(linex,liney,'r-','Linewidth',3);
hold on
z=[p1(3),p2(3)];
A=[x(1)-y(1),x(2)-y(2);z(1)-y(1),z(2)-y(2)];
B=[x(1)^2-y(1)^2+x(2)^2-y(2)^2;z(1)^2-y(1)^2+z(2)^2-y(2)^2];

ab=A\B;
a=ab(1)/2;
b=ab(2)/2;
circleCenter = [a,b];

c2 = (x(1)-a)^2+(x(2)-b)^2;
radius = sqrt(c2);
theta = (0:pi/360:2*pi)';
Result = zeros(size(theta,1),4);
for i = 1: size(theta,1)
    Result(i,1) = i;
    Result(i,2) = theta(i);
```

```

    Result(i,3) = circleCenter(1) + radius*cos(theta(i));
    Result(i,4) = circleCenter(2) + radius*sin(theta(i));
end
plot(Result(:,3),Result(:,4),'r-','Linewidth',3);
hold on
ca_left=90-asin((circleCenter(2)-x(2))/radius)/3.1415926*180;
ca_right=90-asin((circleCenter(2)-y(2))/radius)/3.1415926*180;

```

11. 专题四：拉伸、压缩、剪切、弯曲、扭转、疲劳

11.1 引言

这部分模拟我不是很熟悉，但是可以写出一些基本的 in 文件。这部分大难点应该是建立各种位错和孪晶（我不会）和机械变形导致的结构变化分析（我也不会）。所以这个专题重点讲解怎么进行相关机械变形的设置。主要以一根纳米铜柱为例进行模拟。在进行模拟时我推荐使用 `fix move` 方法而不是 `fix deform`。因为 `fix deform` 是人为根据盒子的变形将原子位置进行重排。这对于变形不大的情况下是合适的，但是通常变形模拟需要实现材料断裂等现象，这个时候断裂出现的时候原子不会反映应力导致的剧烈变化。相反 `fix move` 是材料内部变化的真实反映。还有不建议使用分子动力学进行体相材料的拉伸等模拟。因为边界条件不好控制。如进行拉伸时，材料在垂直拉伸方向应该会出现收缩。体相模拟需要借助周期性边界条件，在周期性边界条件下实现盒子缩放需要使用 `fix npt` 但是 `npt` 又会干扰材料正常的力学响应。所以比较适合的是对纳米尺度的材料进行力学观测。边界条件使用 `fix nvt` 就行。

11.2 拉伸

通过一个方柱形铜纳米线进行拉伸模拟，具体 in 文件如下：

```

dimension      3
boundary       s s s
units          metal
atom_style     atomic
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes
pair_style     eam

region box block 0 119.295 0 36.15 0 36.15 units box
create_box 1 box

lattice fcc 3.6150
region Cu block 0 119.295 0 36.15 0 36.15 units box

```

```
create_atoms 1 region Cu

pair_coeff * * Cu_u3.eam

region left block INF 0.1 INF INF INF INF units box
group left region left
region right block 119.1 INF INF INF INF INF units box
group right region right

group cu subtract all left right

timestep 0.002
thermo 100
thermo_style custom step temp pxx pyy pzz
thermo_modify flush yes

velocity cu create 300 5645354 loop local dist gaussian rot yes
dump mydump1 all atom 10000 atom1.lammpstrj

fix fxnvt cu nvt temp 300.0 300.0 0.2

run 100000

fix 1 right move linear 0.01 0.0 0.0
```

```
run 1500000
```

11.3 压缩

```
dimension      3
boundary       s s s
units          metal
atom_style     atomic
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes
pair_style     eam

region box block -10 129.295 -10 46.15 -10 46.15 units box
create_box 1 box

lattice fcc 3.6150
region Cu block 0 119.295 0 36.15 0 36.15 units box
create_atoms 1 region Cu

pair_coeff * * Cu_u3.eam
```

```
region left block INF 0.1 INF INF INF INF units box
group left region left
region right block 119.1 INF INF INF INF INF units box
group right region right

group cu subtract all left right

timestep 0.002
thermo 100
thermo_style custom step temp pxx pyy pzz
thermo_modify flush yes

velocity cu create 300 5645354 loop local dist gaussian rot yes
dump mydump1 all atom 10000 atom1.lammpstrj

fix fxnvt cu nvt temp 300.0 300.0 0.2

run 100000

fix 1 right move linear -0.01 0.0 0.0

run 500000
```

11.4 剪切

```
dimension      3
boundary       p p p
units          metal
atom_style     atomic
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes
pair_style     eam

region box block 0 119.295 0 36.15 0 119.295 units box
create_box 1 box

lattice fcc 3.6150
region Cu block 0 119.295 0 36.15 0 119.295 units box
create_atoms 1 region Cu

pair_coeff * * Cu_u3.eam

region left block INF 0.1 INF INF INF INF units box
group left region left
region right block 117.1 INF INF INF INF INF units box
```

group right region right

group cu subtract all left right

timestep 0.002

thermo 100

thermo_style custom step temp pxx pyy pzz

thermo_modify flush yes

velocity cu create 300 5645354 loop local dist gaussian rot yes

dump mydump1 all atom 10000 atom1.lammpstrj

fix fxnvt cu nvt temp 300.0 300.0 0.2

run 100000

fix 1 right move linear 0.0 0.0 0.01

run 500000

11.5 弯曲

dimension 3

boundary s s s

units metal

atom_style atomic

neighbor 3.0 bin

neigh_modify every 1 delay 0 check yes

pair_style eam

region box block 0 119.295 0 36.15 0 36.15 units box

create_box 1 box

lattice fcc 3.6150

region Cu block 0 119.295 0 36.15 0 36.15 units box

create_atoms 1 region Cu

pair_coeff * * Cu_u3.eam

region left block INF 0.1 INF INF INF INF units box

group left region left

region right block 119.1 INF INF INF INF INF units box

group right region right

region mid block 59.5 59.7 INF INF INF INF units box

group mid region mid

```
group cu subtract all left right mid

timestep 0.002
thermo 100
thermo_style custom step temp pxx pyy pzz
thermo_modify flush yes

velocity cu create 300 5645354 loop local dist gaussian rot yes
dump mydump1 all atom 10000 atom1.lammpstrj

fix fxnvt cu nvt temp 300.0 300.0 0.2

run 100000

fix 1 mid move linear 0.0 0.0 -0.01
```

```
run 500000
```

11.6 扭转

```
dimension      3
boundary       m m m
units          metal
atom_style     atomic
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes
pair_style     eam

region box block -1 120.295 -1 37.15 -1 37.15 units box
create_box 1 box

lattice fcc 3.6150
region Cu block 0 119.295 0 36.15 0 36.15 units box
create_atoms 1 region Cu

pair_coeff * * Cu_u3.eam

region left block INF 0.1 INF INF INF INF units box
group left region left
region right block 119.1 INF INF INF INF INF units box
group right region right

group cu subtract all left right

timestep 0.002
```


thermo 10000

```
compute cc all chunk/atom bin/1d x lower 10 units box
compute myChunk all torque/chunk cc
variable tx equal c_myChunk[1][1]
variable ty equal c_myChunk[2][1]
variable tz equal c_myChunk[3][1]
fix 1 all ave/time 1 10000 10000 v_tx v_ty v_tz
```

```
thermo_style custom step temp f_1[1] f_1[2] f_1[3]
thermo_modify flush yes
```

```
velocity cu create 300 5645354 loop local dist gaussian rot yes
dump mydump1 all atom 10000 atom1.lammpstrj
```

```
fix fxnvt cu nvt temp 300.0 300.0 0.2
```

```
fix 2 right move rotate 119.295 17.17125 17.17125 1 0 0 10000 units box
```

```
run 1000000
```

11.7 疲劳

听过要加缺陷才能出现疲劳损伤，可以在下面的 in 文件中添加一个缺陷试试。
具体的方法要看论文

```
dimension      3
boundary       s s s
units          metal
atom_style     atomic
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes
pair_style     eam
```

```
region box block 0 119.295 0 36.15 -54.225 54.225 units box
create_box 2 box
```

```
lattice fcc 3.6150
region Cu block 0 119.295 0 36.15 -18.075 18.075 units box
create_atoms 1 region Cu
```

```
region cu_up block 0 36.15 0 36.15 18.075 54.225 units box
create_atoms 2 region cu_up
region cu_down block 0 36.15 0 36.15 -54.225 -18.075 units box
create_atoms 2 region cu_down
pair_coeff * * Cu_u3.eam
```

```
region left block INF 0.1 INF INF INF INF units box
group left region left
region right block 119.1 INF INF INF INF INF units box
group right region right
group up region cu_up
group down region cu_down

group cu subtract all left right up down

timestep 0.002
thermo 100
thermo_style custom step temp pxx pyy pzz
thermo_modify flush yes

velocity cu create 300 5645354 loop local dist gaussian rot yes
dump mydump1 all atom 10000 atom1.lammpstrj

fix fxnvt cu nvt temp 300.0 300.0 0.2

run 100000

fix 1 right move wiggle 0.0 0.0 20.0 1000 units box
#为了出现疲劳可能要跑很长时间
run 1000000
```

12. 专题五：传热

13. 专题六：回转半径和扩散

13.1 建模

采用 OPLS-UA 模型使用 moltemplate 建立十六烷的模型。It 文件如下

ch2group.lt 内容如下

```
import "oplsaa.lt"
import "loplsaa.lt"
# Then define "CH2":
CH2 inherits OPLSAA {

    # atom-id mol-id atom-type charge  x    y    z
```

```

write("Data Atoms") {
  $atom:c $mol:... @atom:13  0.0  0.000000      0.000000      0.000000
}
} # CH2
CH2.move(0,0.4431163,0)
ch3group.lt 内容如下
import "oplsaa.lt"
import "loplsaa.lt"

# Then define "CH3":

CH3 inherits OPLSAA {

  # atom-id  mol-id atom-type  charge  x          y          z

  write("Data Atoms") {
    $atom:c $mol:... @atom:10    0.0  0.000000      0.000000      0.000000
  }

} # CH3
CH3.move(0,0.4431163,0)
alkane16.lt 内容如下
import "ch2group.lt"  # load the definition of the "CH2" object
import "ch3group.lt"  # load the definition of the "CH3" object

Alkane16 inherits OPLSAA {

  create_var {$mol}

  monomers = new CH2 [16].move(1.54,0,0)

```

```

delete monomers[0]
delete monomers[15]

monomers[0] = new CH3
monomers[15] = new CH3

monomers[15].move(23.1,0,0)

write('Data Bond List') {
    $bond:b1  $atom:monomers[0]/c $atom:monomers[1]/c
    $bond:b2  $atom:monomers[1]/c $atom:monomers[2]/c
    $bond:b3  $atom:monomers[2]/c $atom:monomers[3]/c
    $bond:b4  $atom:monomers[3]/c $atom:monomers[4]/c
    $bond:b5  $atom:monomers[4]/c $atom:monomers[5]/c
    $bond:b6  $atom:monomers[5]/c $atom:monomers[6]/c
    $bond:b7  $atom:monomers[6]/c $atom:monomers[7]/c
    $bond:b8  $atom:monomers[7]/c $atom:monomers[8]/c
    $bond:b9   $atom:monomers[8]/c $atom:monomers[9]/c
    $bond:b10  $atom:monomers[9]/c $atom:monomers[10]/c
    $bond:b11  $atom:monomers[10]/c $atom:monomers[11]/c
    $bond:b12  $atom:monomers[11]/c $atom:monomers[12]/c
    $bond:b13  $atom:monomers[12]/c $atom:monomers[13]/c
    $bond:b14  $atom:monomers[13]/c $atom:monomers[14]/c
    $bond:b15  $atom:monomers[14]/c $atom:monomers[15]/c
}
}

system.lt 内容如下
import "alkane16.lt"  # Defines the "Alkane50" molecule

```

Specify the size of the world the polymer lives in:

```
write_once("Data Boundary") {
    0.0 120.0 xlo xhi
    0.0 120.0 ylo yhi
    0.0 120.0 zlo zhi
}
```

```
polymers = new Alkane16 [24].move(0, 0, 5.0)
                [24].move(0, 5.0, 0)
                [4].move(30.0, 0, 0)
```

在生成 data 文件后要使用 cleanup_moltemplate.sh 命令简化一下 data 文件

13.2 模拟

使用 LAMMPS 模拟了十六烷在 npt 系综下的扩散系数和回转半径随时间的变化。十六烷采用 OPLSAA 联合原子力场。十六烷的扩散系数通过均方位移得到。在计算高分子的扩散系数是有两种方法，第一种直接计算所有原子的均方位移并对于所有原子取平均即 g_1，第二种计算每个分子质心的均方位移并对所有分子取平均，即 g_3。本算例对比了两种计算方法，从下图中可以看出两种计算方法在初始阶段差别很大，在足够长时间后才会重合，因此计算高分子的扩散系数或均方位移需要计算足够长时间才行。回转半径是计算每个分子的回转半径然后对所有分子取平均。算例中还输出了用于计算分子端对端距离的 dump 文件，通过编程可方便计算分子的端对端距离随时间的变化。端对端距离和回转半径都是描述分子所占空间大小的方法。

以下是 in 文件内容

```
# ----- Init Section -----

units real
boundary p p p
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes
atom_style full
```

```
bond_style harmonic
angle_style harmonic
dihedral_style opls
pair_style lj/cut 10.0
pair_modify mix geometric
special_bonds lj/coul 0.0 0.0 0.5

# ----- Atom Definition Section -----

read_data "system.data"

# ----- Settings Section -----

pair_coeff 1 1 0.175 3.905
pair_coeff 2 2 0.118 3.905
bond_coeff 1 260.0 1.526
bond_coeff 2 260.0 1.526
angle_coeff 1 63.0 112.4
angle_coeff 2 63.0 112.4
dihedral_coeff 1 -3.4 1.25 3.1 0.0
dihedral_coeff 2 -3.4 1.25 3.1 0.0

# ----- Run Section -----

minimize 1.0e-5 1.0e-7 1000 10000
timestep 1.0
velocity      all create 400 372748 rot yes loop local dist gaussian
dump 1 all custom 10000 dump.lammpstrj id mol type x y z
fix          fxnpt all npt temp 400.0 400.0 100.0 iso 1.0 1.0 1000.0
```

```
group start id 15:36863:16
dump start start custom 200 start id mol xu yu zu
dump_modify start sort id
```

```
group end id 16:36864:16
dump end end custom 200 end id mol xu yu zu
dump_modify end sort id
```

```
compute cc1 all chunk/atom molecule
compute gy all gyration/chunk cc1
variable gy equal ave(c_gy)
```

```
thermo 200
thermo_style custom step v_gy
thermo_modify flush yes
```

```
run 500000
```

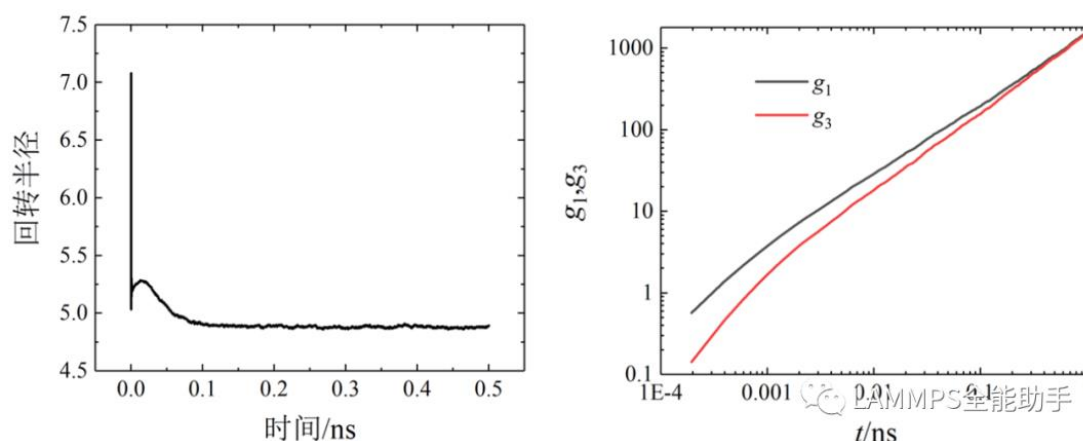
```
compute g1 all msd
```

```
compute g3 all msd/chunk cc1
variable g3 equal ave(c_g3[4])
```

```
thermo 200
thermo_style custom step c_g1[4] v_g3 v_gy
thermo_modify flush yes
```

```
run 1000000
```

13.3 结果



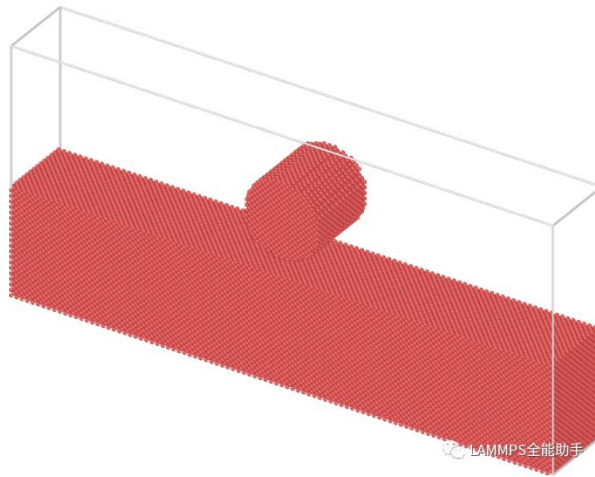
14. 专题八：摩擦和磨损

14.1 引言

在纳米尺度下研究摩擦和磨损价值有限, 因为工程中的粗糙度一般都是微米级的, 这对于分子动力学来说尺度太大了。但是对于一些特定的场景, 比如粗糙度特别小, 摩擦界面很光滑, 或者控制粗糙度在纳米尺度。分子动力学的模拟还是很有价值的。在这方面分子动力学更大的价值在于理解一般摩擦过程中的机理或者发现新的减摩方法。分子动力学模拟另一个重要的应用就是研究边界润滑过程中固体润滑机理或者润滑油及其添加剂摩擦膜的生成, 这需要用到反应力场。另一方面使用全原子模拟分子动力学模拟或者耗散粒子动力学模拟可以研究滑动轴承流体润滑过程中的油膜厚度问题。这些都是研究摩擦磨损的好方向。

14.2 一个简单的例子

本算例研究了铁球在铁基底上的摩擦的分子动力学。由于是完整的晶格结构, 所以可以直接在 LAMMPS 中建模。使用 `region` 和 `create_atoms` 命令可以创建处很复杂的模型。通过控制铁球压入深度和移动速度, 可以研究不同载荷和摩擦速度下的摩擦系数和磨损过程。其中统计输出了不同时间步的摩擦力和载荷, 对其进行时间平均可得到实际的摩擦力和载荷, 由此得到摩擦系数。通过下面 `in` 文件学习如何统计输出载荷和摩擦力。



in 文件如下

```
units metal
dimension 3
boundary p p p
neighbor 3 bin
neigh_modify every 1 delay 0 check yes
atom_style atomic
pair_style eam/fs
```

```
region box block 0 285.5324 0 39.974536 0 128.48958 units box
create_box 1 box
```

```
lattice fcc 2.855324
region part1 block 0 285.5324 0 39.974536 0 55.678818 units box
create_atoms 1 region part1
```

```
lattice fcc 2.855324
region part2 cylinder y 142.7662 84.987268 19.987268 0 39.974536 units box
create_atoms 1 region part2
```

```
pair_coeff * * Fe_mm.eam.fs Fe
```

```
region qiu block INF INF INF INF 60 INF units box
group qiu region qiu
```

```
region nomove block INF INF INF INF INF 5 units box
group nomove region nomove
```

```
region sub block INF INF INF INF 5 60 units box
group sub region sub
```

```
compute myT sub temp
```

```
timestep 0.001
thermo 100
thermo_modify temp myT flush yes
velocity sub create 300 5645354 temp myT loop local dist gaussian

fix fxnvt sub nvt temp 300 300 0.1

dump mydump all atom 2000 atom.lammpstrj

run 200000

fix move1 qiu move linear 0 0 -0.1 units box

run 94000

unfix move1
fix move2 qiu move linear 0.1 0 0 units box

compute fx qiu reduce sum fx
compute fz qiu reduce sum fz

thermo_style custom step temp c_fx c_fz
thermo_modify temp myT

run 1000000
```

14.3 二维粗糙度铜块对磨模拟

带有粗糙度的摩擦需要一些很小新的设置。这里举例说明，如何施加额定的法向载荷和切向载荷，局部控温和载荷监控。具体方法在 in 文件的注释中。这里要说明的是，磨损过程中温度会升高很快，这是正常的，因为剧烈的摩擦产生大量的热不能及时传递出去，采用较慢的摩擦速度会降低温度升高这一现象。这里采用了铜块。单质铜是比较软的，所以摩擦过程中会出现焊接现象。改用一硬一软的材料应该就会出现实际中的磨损现象。粗糙度建模分为两步首先在 LAMMPS 中或者其他软件建立一个金属块，然后通过 python 代码切除粗糙度。data 文件中的模型是一个长方体的金属材料。粗糙度在 z 方向上生成。如果要生成上下两块则可以分别生成然后在 Excel 中将两块组合起来。

该方法的原理是使用 Weierstrass-Mandelbrot (W-M)函数生成坐标，然后基于生成的坐标删除多余的原子。

$$R(x) = G^{(D-1)} \sum_{n=n_1}^{\infty} \frac{\cos(2\pi\gamma^n x)}{\gamma^{(2-D)n}}; \quad 1 < D < 2; \quad \gamma > 1$$

```
import math
import matplotlib.pyplot as plt
#写出原始 data 文件的路径和修改后 data 的路径
data = open("E:\\work\\0_wenku\\磨损\\1.data", 'r')
data_modify = open("E:\\work\\0_wenku\\磨损\\3.data", 'w')

line = data.readline()
line = data.readline()
line = data.readline().split()
natom = int(line[0])
line = data.readline()
line = data.readline()
line = data.readline().split()
xlo = float(line[0])
xhi = float(line[1])
line = data.readline().split()
ylo = float(line[0])
yhi = float(line[1])
line = data.readline().split()
zlo = float(line[0])
zhi = float(line[1])
line = data.readline()
line = data.readline()
line = data.readline()
line = data.readline()
line = data.readline()
line = data.readline()
line = data.readline()
line = data.readline()
line = data.readline()

data_modify.write("LAMMPS data of ice slab with rough\n")
data_modify.write("\n")
data_modify.write(str(natom) + " atoms\n")
data_modify.write("\n")
data_modify.write("1 atom types\n")
data_modify.write("\n")
data_modify.write(str(xlo) + " " + str(xhi) + " xlo xhi\n")
data_modify.write(str(ylo) + " " + str(yhi) + " ylo yhi\n")
data_modify.write(str(zlo) + " " + str(zhi) + " zlo zhi\n")
data_modify.write("\n")
data_modify.write("Atoms\n")
```

```

data_modify.write("\n")
#调整 Gx, Dx 和 rx 可获得不同形状的粗糙度
Gx=500.0
Dx=1.555
rx=1.45
x=0
jx=1
z=0
nx = []
ny = []
ix = 0.1
inter=18.075 #金属块 x 方向长度除以 20
num = 24; #num*inter 要超过金属块在 x 方向上的长度
while ix < num: #
    x = ix*inter
    z = 0.0
    jx=1
    ix += 1
    while jx < 100:
        z_temp = (rx**(Dx*jx-2*jx))*(math.cos(2*(math.pi)*(rx**jx)*x))
        z += z_temp
        jx += 1
    z = (Gx**(Dx-1))*z+50 #50 是粗糙度的 z 方向上的偏移量，调整以得到合适
    的坐标
    nx.append(x)
    ny.append(z)
ny = list(reversed(ny))
#将粗糙度画出来看是否合理
plt.plot(nx,ny)
plt.xlim(0,361.5)
plt.ylim(0,361.5)
plt.show()
natom_modify = 0
done = 0
while not done:
    line = data.readline()
    if line != '\n':
        pos = line.split()
        x = float(pos[2])
        i = int((x)//inter)
        z0 = ((ny[i+1] - ny[i])/(nx[i+1] - nx[i]))*(x - nx[i]) + ny[i]
        if float(pos[4]) >= z0:
            data_modify.write(line)
            natom_modify += 1

```

```

else:
    done = 1

data.close()
data_modify.close()
#最后要用打印出的原子数替换修改后的 data 文件的原子数
print(str(natom_modify))

本例中的 data 文件就是采用上面代码生成的，以下是模拟的 in 文件：
dimension      3
boundary        p p s
units           metal
atom_style      atomic

pair_style      eam

read_data system.data

pair_coeff * * Cu_u3.eam
#上下两块都会分为三层，分别是固定层，温控层和自由层，通过 region 命令将
三层分别定义为不同组
region upfix block INF INF INF INF 377 INF units box
group upfix region upfix
region downfix block INF INF INF INF INF 1.9 units box
group downfix region downfix

region upt block INF INF INF INF 357 377 units box
group upt region upt
region downt block INF INF INF INF INF 1.9 21.9 units box
group downt region downt
#为了后面统计受力取消下面一块金属固定层之间的相互作用
neighbor 3.0 bin
neigh_modify every 1 delay 0 check yes exclude group downfix downfix

group up type 2
group down type 1

group upfree subtract up upfix upt
group downfree subtract down downfix downt

group t union upt downt
group free union upfree downfree

timestep 0.002

```

```
thermo 100
#分别监控温控层和自由层的温度
compute myTfree free temp
compute myTt t temp
#下面一块金属的固定层的受力都来自上方，因此计算该固定层的切向力和法向
力就可以得到切向摩擦力和载荷的大小
compute fx downfix reduce sum fx
compute fz downfix reduce sum fz
#监控输出温度和载荷
thermo_style custom step c_myTfree c_myTt c_fx c_fz
thermo_modify flush yes

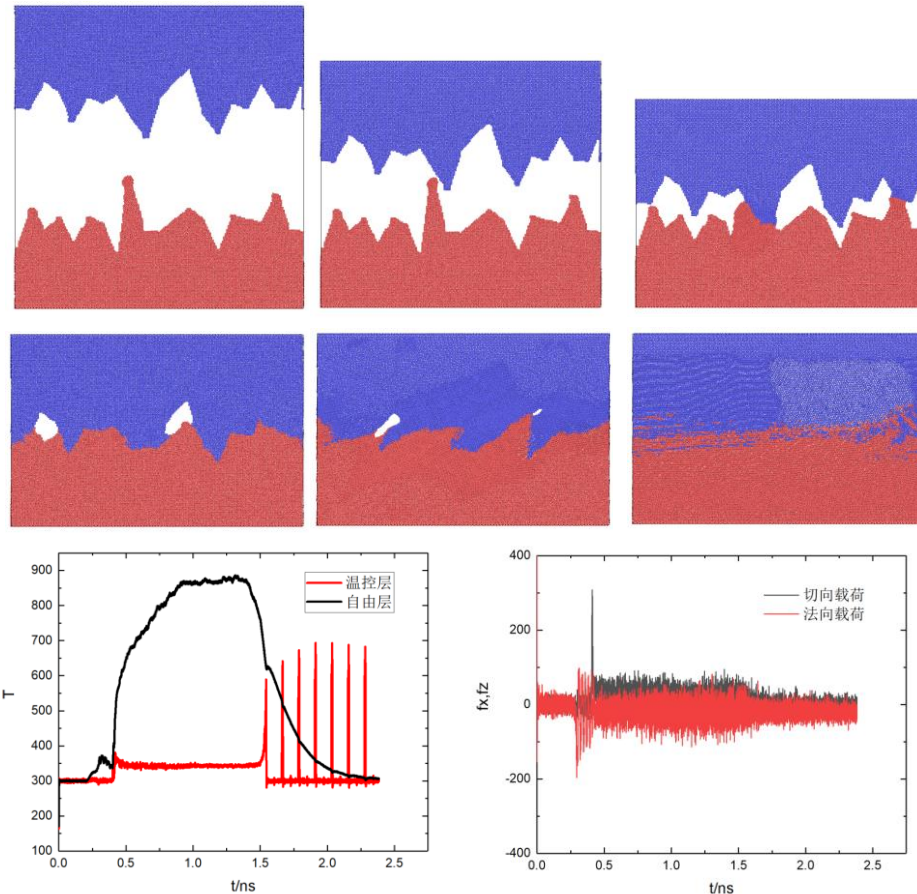
velocity t create 300 5645354 temp myTt loop local dist gaussian rot yes
velocity free create 300 565454 temp myTfree loop local dist gaussian rot yes

dump mydump1 all atom 1000 atom1.lammpstrj
#对自由层和温控层进行弛豫
fix fxnvt1 t nvt temp 300.0 300.0 0.2
fix fxnvt2 free nvt temp 300.0 300.0 0.2

run 100000
#取消自由层上的温度控制
unfix fxnvt2
fix fxnve free nve
#结合使用一下三个 fix 施加法向载荷使两块金属接触
fix 1 upfix setforce 0.0 0.0 NULL
fix 2 upfix aveforce 0.0 0.0 -0.01
fix 3 upfix nve

run 100000
#施加切向载荷开始剪切
unfix 1
unfix 2
fix 1 upfix setforce NULL 0.0 NULL
fix 2 upfix aveforce 0.1 0.0 -0.01

run 1000000
结果如下：
```



温度变化的基本规律是正确的, 但是这些尖峰是不对的。这可能是因为温控层也在移动, 没有减去宏观速度就进温控的原因。载荷的变化是符合预期的, 刚开始法向和切向载荷都为零。然后上下两块开始接触固体块出现弹性振荡, 两个载荷也出现振荡。当开始剪切的时候切向载荷会迅速升高, 因为粗糙度之间开始磨损。待粗糙度磨平后, 载荷开始下降并维持在基本不变的范围内。可以看到, 由于原子数没有足够大, 载荷会出现明显的涨落。这是我们应该对载荷取平均来获得平均载荷用以计算摩擦系数。为了改进温度的输出, 采用一下改进的 in 文件 (未经测试, 欢迎加微信 baolu_yao, 和我讨论)

```
dimension      3
boundary       p p s
units          metal
atom_style     atomic

pair_style      eam

read_data      system.data

pair_coeff      ** Cu_u3.eam
```

#上下两块都会分为三层, 分别是固定层, 温控层和自由层, 通过 region 命令将三层分别定义为不同组

region upfix block INF INF INF INF 377 INF units box

group upfix region upfix

region downfix block INF INF INF INF INF 1.9 units box

group downfix region downfix

region upt block INF INF INF INF 357 377 units box

group upt region upt

region downt block INF INF INF INF 1.9 21.9 units box

group downt region downt

#为了后面统计受力取消下面一块金属固定层之间的相互作用

neighbor 3.0 bin

neigh_modify every 1 delay 0 check yes exclude group downfix downfix

group up type 2

group down type 1

group upfree subtract up upfix upt

group downfree subtract down downfix downt

group t union upt downt

group free union upfree downfree

timestep 0.002

thermo 100

#分别监控温控层和自由层的温度

compute myTupfree upfree temp/partial 0 1 0

compute myTdownfree downfree temp/partial 0 1 0

compute myTupt upt temp/partial 0 1 0

compute myTdownt downt temp/partial 0 1 0

#下面一块金属的固定层的受力都来自上方, 因此计算该固定层的切向力和法向力就可以得到切向摩擦力和载荷的大小

compute fx downfix reduce sum fx

compute fz downfix reduce sum fz

#监控输出温度和载荷

thermo_style custom step c_myTupfree c_myTdownfree c_myTupt c_myTdownt c_fx c_fz

thermo_modify flush yes

velocity upt create 300 5643454 temp myTupt loop local dist gaussian rot yes

velocity downt create 300 564654354 temp myTdownt loop local dist gaussian rot yes

velocity upfree create 300 567854 temp myTupfree loop local dist gaussian rot yes

velocity downfree create 300 590454 temp myTdownfree loop local dist gaussian rot

yes

```
dump mydump1 all atom 1000 atom1.lammpstrj
```

#对自由层和温控层进行弛豫

```
fix fxnvt1 t nvt temp 300.0 300.0 0.2
```

```
fix fxnvt2 free nvt temp 300.0 300.0 0.2
```

```
run 100000
```

#取消自由层上的温度控制

```
unfix fxnvt1
```

```
unfix fxnvt2
```

```
fix fxnvt1 upt nvt temp 300.0 300.0 0.2
```

```
fix_modify fxnvt1 temp myTupt
```

```
fix fxnvt2 downt nvt temp 300.0 300.0 0.2
```

```
fix_modify fxnvt2 temp myTdownt
```

```
fix fxnve free nve
```

#结合使用一下三个 fix 施加法向载荷使两块金属接触

```
fix 1 upfix setforce 0.0 0.0 NULL
```

```
fix 2 upfix aveforce 0.0 0.0 -0.01
```

```
fix 3 upfix nve
```

```
run 100000
```

#施加切向载荷开始剪切

```
unfix 1
```

```
unfix 2
```

```
fix 1 upfix setforce NULL 0.0 NULL
```

```
fix 2 upfix aveforce 0.1 0.0 -0.01
```

```
run 1000000
```

15. 专题九：反应分子动力学 ReaxFF

15.1 ReaxFF 力场公式介绍

在常用的分子动力学中，化学键是不能被打断和形成，因此无法模拟化学反应。为克服这一缺陷，研究者提出了反应分子动力学模拟方法。在反应分子动力学中最重要的就是采用反应力场。在经典力场中如 OPLSAA，AMBER 等，同一种元素的原子根据所处化学环境的不同会具有不同的类型。与经典力场不同，在反应力场中不存在原子类型，同一个分子内的原子没有用固定的化学键直接相连，而是通过计算任两个原子之间的键级来确定当前时刻的连接性。在反应分子动力

学模拟过程中, 随着键级的变化, 化学键会不断的断裂和生成, 而原子之间的连接列表即成键性也在不断的更新。因此在反应力场中键级是最重要的量, 在键级的基础上将原子之间的相互作用表达为键级的函数, 由此计算出经典力场中的键, 键角, 二面角, 共轭, 库伦, 范德华等相互作用。对于反应力场, 系统中的总能量表示为:

$$E_{system} = E_{bond} + E_{lp} + E_{over} + E_{under} + E_{val} + E_{pen} + E_{coa} \\ + E_{C2} + E_{triple} + E_{tors} + E_{conj} + E_{H-bond} + E_{vdWaals} + E_{Coulomb}$$

键能: 在键级的定义中一个基本假设就是键级和原子之间的距离成特定的数学关系, 通过原子之间的距离可以计算出键级的大小。在反应力场中根据原子之间的距离的不同将化学键分为单键, 双键和三键。一个原子的键级来自三种键的加和贡献:

$$BO'_{ij} = BO^{\sigma}_{ij} + BO^{\pi}_{ij} + BO^{\pi\pi}_{ij} \\ = \exp \left[p_{bo1} \cdot \left(\frac{r_{ij}}{r_o^{\sigma}} \right)^{p_{bo2}} \right] + \exp \left[p_{bo3} \cdot \left(\frac{r_{ij}}{r_o^{\pi}} \right)^{p_{bo4}} \right] + \exp \left[p_{bo5} \cdot \left(\frac{r_{ij}}{r_o^{\pi\pi}} \right)^{p_{bo6}} \right]$$

直接计算的键级是未经校正过的键级, 需要对键级进行校正。在此之前首先要定义过配位数

$$\Delta'_i = -Val_i + \sum_{j=1}^{neighbours(i)} BO'_{ij}$$

上式中的过配位数也是未经校正的, 式中 Val_i 是原子 i 的总键级, 例如 C 原子的总键级是 4, 氢原子的总键级是 1。在未经校正的键级和过配位数的基础上, 反应力场通过下式对过配位数进行, 为使得校正不至于太硬, 引入了第二过配位数, 即

$$\Delta'^{boc}_i = -Val_i^{boc} + \sum_{j=1}^{neighbours(i)} BO'_{ij} \\ BO^{\sigma}_{ij} = BO'^{\sigma}_{ij} \cdot f_1(\Delta'_i, \Delta'_j) \cdot f_4(\Delta'_i, BO'_{ij}) \cdot f_5(\Delta'_j, BO'_{ij}) \\ BO^{\pi}_{ij} = BO'^{\pi}_{ij} \cdot f_1(\Delta'_i, \Delta'_j) \cdot f_1(\Delta'_i, \Delta'_j) \cdot f_4(\Delta'_i, BO'_{ij}) \cdot f_5(\Delta'_j, BO'_{ij}) \\ BO^{\pi\pi}_{ij} = BO'^{\pi\pi}_{ij} \cdot f_1(\Delta'_i, \Delta'_j) \cdot f_1(\Delta'_i, \Delta'_j) \cdot f_4(\Delta'_i, BO'_{ij}) \cdot f_5(\Delta'_j, BO'_{ij}) \\ BO_{ij} = BO^{\sigma}_{ij} + BO^{\pi}_{ij} + BO^{\pi\pi}_{ij} \\ f_1(\Delta'_i, \Delta'_j) = \frac{1}{2} \cdot \left(\frac{Val_i + f_2(\Delta'_i, \Delta'_j)}{Val_i + f_2(\Delta'_i, \Delta'_j) + f_3(\Delta'_i, \Delta'_j)} + \frac{Val_j + f_2(\Delta'_i, \Delta'_j)}{Val_j + f_2(\Delta'_i, \Delta'_j) + f_3(\Delta'_i, \Delta'_j)} \right) \\ f_2(\Delta'_i, \Delta'_j) = \exp(-p_{boc1} \cdot \Delta'_i) + \exp(-p_{boc1} \cdot \Delta'_j)$$

$$f_3(\Delta'_i, \Delta'_j) = \ln \left\{ \frac{1}{2} \cdot \left[\exp(-p_{boc2} \cdot \Delta'_i) + \exp(-p_{boc2} \cdot \Delta'_j) \right] \right\}$$

$$f_4(\Delta'_i, BO'_{ij}) = \frac{1}{1 + \exp(-p_{boc3} \cdot (p_{boc4} \cdot BO'_{ij} \cdot BO'_{ij} - \Delta'_i{}^{boc}) + p_{boc5})}$$

$$f_5(\Delta'_j, BO'_{ij}) = \frac{1}{1 + \exp(-p_{boc3} \cdot (p_{boc4} \cdot BO'_{ij} \cdot BO'_{ij} - \Delta'_j{}^{boc}) + p_{boc5})}$$

经过校正就可以得到新的键级, 同时新的过配位数由下式得到

$$\Delta_i = -Val_i + \sum_{j=1}^{neighbours(i)} BO_{ij}$$

由此任一个键的键能由下式得到:

$$E_{bond} = -D_e^\sigma \cdot BO_{ij}^\sigma \cdot \exp[p_{be1} (1 - (BO_{ij}^\sigma)^{p_{be2}})] - D_e^\pi \cdot BO_{ij}^\pi - D_e^{\pi\pi} \cdot BO_{ij}^{\pi\pi}$$

孤对电子能: 在某些体系中存在孤对电子, 而孤对电子对体系的结构和能量也有着较大的影响。反应力场中为了能够处理这样的体系引入了下述两个函数来表达原子外层孤对电子对系统能量的贡献:

$$\Delta_i^e = -Val_i^e + \sum_{j=1}^{neighbours(i)} BO_{ij}$$

$$n_{lp,i} = \text{int} \left(\frac{\Delta_i^e}{2} \right) + \exp \left[-p_{lp1} \cdot \left(2 + \Delta_i^e - 2 \cdot \text{int} \left(\frac{\Delta_i^e}{2} \right) \right)^2 \right]$$

其中 $n_{lp,i}$ 为原子外层的孤对电子数, Val_i^e 为原子的最外层电子数。在原子的键级发生变化时, 孤对电子的变化为:

$$\Delta_i^{lp} = n_{lp,opt} - n_{lp,i}$$

此时孤对电子对体系能量的贡献为:

$$E_{lp} = \frac{p_{lp2} \cdot \Delta_i^{lp}}{1 + \exp(-75 \cdot \Delta_i^{lp})}$$

过配位的能量校正项: 当原子的过配位数大于零是, 由键级得到体系能量需要进行校正, 同时考虑孤对电子的存在的校正分别为:

$$E_{over} = \frac{\sum_{j=1}^{nbond} p_{ovun1} \cdot D_e^\sigma \cdot BO_{ij}}{\Delta_i^{lpcorr} + Val_i} \cdot \Delta_i^{lpcorr} \cdot \left[\frac{1}{1 + \exp(p_{ovun2} \cdot \Delta_i^{lpcorr})} \right]$$

$$\Delta_i^{lpcorr} = \Delta_i - \frac{\Delta_i^{lp}}{1 + p_{ovun3} \cdot \exp \left(p_{ovun4} \cdot \left\{ \sum_{j=1}^{neighbours(i)} (\Delta_j - \Delta_j^{lp}) \cdot (BO_{ij}^{\pi} + BO_{ij}^{\pi\pi}) \right\} \right)}$$

过配位能量校正项: 当过配位数小于零时, 同样需要对体系的能量进行校正, 但是这种校正只有存在 π 键时才执行

$$E_{under} = -p_{ovun5} \cdot \frac{1 - \exp(p_{ovun6} \cdot \Delta_i^{lpcorr})}{1 + \exp(-p_{ovun2} \cdot \Delta_i^{lpcorr})} \cdot \frac{1}{1 + p_{ovun7} \cdot \exp \left[p_{ovun8} \cdot \left[\sum_{j=1}^{neighbours(i)} (\Delta_j - \Delta_j^{lp}) \cdot (BO_{ij}^{\pi} + BO_{ij}^{\pi\pi}) \right] \right]}$$

键角能量项: 与键能量项类似, 在反应力场中键角的能量也表示为键级的函数, 键角能量的计算通过下式来完成, 其中平衡键角取决于 π 键键级总和的函数 SBO, 通过条件判断, 可以硅键角中心原子的杂化状态进行判断:

$$E_{val} = f_7(BO_{ij}) \cdot f_7(BO_{jk}) \cdot f_8(\Delta_j) \cdot \{ p_{val1} - p_{val1} \cdot \exp[-p_{val2} \cdot (\Theta_0(BO) - \Theta_{ijk})^2] \}$$

$$f_7(BO_{ij}) = 1 - \exp(-p_{val3} \cdot BO_{ij}^{p_{val4}})$$

$$f_8(\Delta_j) = p_{val5} - (p_{val5} - 1) \cdot \frac{2 + \exp(p_{val6} \cdot \Delta_j^{angle})}{1 + \exp(p_{val6} \cdot \Delta_j^{angle}) + \exp(-p_{val7} \cdot \Delta_j^{angle})}$$

$$SBO = \sum_{n=1}^{neighbors(j)} (BO_{jn}^{\pi} + BO_{jn}^{\pi\pi}) + \left[1 - \prod_{n=1}^{neighbors(j)} \exp(-BO_{jn}^8) \right] \cdot (-\Delta_j^{angle} - p_{val8} \cdot n_{p,j})$$

$$\Delta_i^{angle} = -Val_i^{angle} + \sum_{j=1}^{neighbours(i)} BO_{jn}$$

$$\begin{aligned} SBO2 &= 0 \quad \text{if } SBO \leq 0 \\ SBO2 &= SBO^{p_{val9}} \quad \text{if } 0 < SBO < 1 \\ SBO2 &= 2 - (2 - SBO)^{p_{val9}} \quad \text{if } 1 < SBO < 2 \\ SBO2 &= 2 \quad \text{if } SBO > 2 \end{aligned}$$

$$\Theta_0(BO) = \pi - \Theta_{0,0} \cdot \{1 - \exp[-p_{val10} \cdot (2 - SBO2)]\}$$

为处理键角中心原子两边各连两个双键的情况, 反应力场中加入了键角能量惩罚项:

$$E_{pen} = p_{pen1} \cdot f_9(\Delta_j) \cdot \exp[-p_{pen2} \cdot (BO_{ij} - 2)^2] \cdot \exp[-p_{pen2} \cdot (BO_{jk} - 2)^2]$$

$$f_9(\Delta_j) = \frac{2 + \exp(-p_{pen3} \cdot \Delta_j)}{1 + \exp(-p_{pen3} \cdot \Delta_j) + \exp(p_{pen4} \cdot \Delta_j)}$$

在一般的共轭体系中, 反应力场的对四体共轭项也能合理的处理, 但是在处理特定基团的共轭时会出现问题, 为此引入三体共轭项:

$$E_{coa} = p_{coa1} \cdot \frac{1}{1 + \exp(p_{coa2} \cdot \Delta_j^{val})} \cdot \exp\left[-p_{coa3} \cdot (-BO_{ij} + \sum_{n=1}^{neighbours(i)} BO_{in})^2\right] \cdot \exp\left[-p_{coa3} \cdot (-BO_{jk} + \sum_{n=1}^{neighbours(i)} BO_{kn})^2\right] \cdot \exp[-p_{coa4} \cdot (BO_{ij} - 1.5)^2] \cdot \exp[-p_{coa4} \cdot (BO_{jk} - 1.5)^2]$$

四体作用项: 与键角类似, 在处理二面角时需要确定四个相连原子的键级数, 因此在反应力场中四体作用项描述为:

$$E_{tors} = f_{10}(BO_{ij}, BO_{jk}, BO_{kl}) \cdot \sin(\Theta_{ijk}) \cdot \sin(\Theta_{jkl}) \cdot \left[\frac{1}{2} \cdot V_1 \cdot (1 + \cos \omega_{ijkl}) + \frac{1}{2} \cdot V_2 \cdot \exp\{p_{tor1} \cdot (BO_{jk}^\pi - 1 + f_{11}(\Delta_j, \Delta_k)^2)\} \cdot (1 + \cos \omega_{ijkl}) + \frac{1}{2} \cdot V_3 \cdot (1 + \cos 3\omega_{ijkl})\right]$$

$$f_{10}(BO_{ij}, BO_{jk}, BO_{kl}) = [1 - \exp(-p_{tor2} \cdot BO_{ij})] \cdot [1 - \exp(-p_{tor2} \cdot BO_{jk})] \cdot [1 - \exp(-p_{tor2} \cdot BO_{kl})]$$

$$f_{11}(\Delta_j, \Delta_k) = \frac{2 + \exp[-p_{tor3} \cdot (\Delta_j^{angle} + \Delta_k^{angle})]}{1 + \exp[-p_{tor3} \cdot (\Delta_j^{angle} + \Delta_k^{angle})] + \exp[p_{tor4} \cdot (\Delta_j^{angle} + \Delta_k^{angle})]}$$

对于四体作用中的共轭情况, 采用四体共轭项进行描述:

$$E_{conj} = f_{12}(BO_{ij}, BO_{jk}, BO_{kl}) \cdot p_{cot1} \cdot [1 + (\cos \omega_{ijkl}^2 - 1) \cdot \sin \Theta_{ijk} \cdot \sin \Theta_{jkl}]$$

$$f_{12}(BO_{ij}, BO_{jk}, BO_{kl}) = \exp\left[-p_{cot2} \cdot (BO_{ij} - 1\frac{1}{2})^2\right] \cdot \exp\left[-p_{cot2} \cdot (BO_{jk} - 1\frac{1}{2})^2\right] \cdot \exp\left[-p_{cot2} \cdot (BO_{kl} - 1\frac{1}{2})^2\right]$$

氢键作用项:

$$E_{Hbond} = p_{hb1} \cdot [1 - \exp(p_{hb2} \cdot BO_{XH})] \cdot \exp\left[p_{hb3} \cdot \left(\frac{r_{hb}^o}{r_{HZ}} + \frac{r_{HZ}}{r_{hb}^o} - 2\right)\right] \cdot \sin\left(\frac{\Theta_{XHZ}}{2}\right)^8$$

C2 修正项: 在反应力场中对于两个 C 原子分子当存在三键时, 这个三键的能量要远远高于 CC 双键, 但是, 实际上三键会很大程度上被末端的自由基削弱, 是的这个三键的能量低于双键。为对这种情况进行修正, 反应力场引入了新的能量修正项:

$$E_{C2} = k_{c2} \cdot (BO_{ij} - \Delta_i - 0.04 \cdot \Delta_i^4 - 3)^2 \quad \text{if } BO_{ij} - \Delta_i - 0.04 \cdot \Delta_i^4 > 3$$

$$E_{C2} = 0 \quad \text{if } BO_{ij} - \Delta_i - 0.04 \cdot \Delta_i^4 \leq 3$$

三键修正项: 为对一氧化碳分子的碳氧三键进行修正引入:

$$E_{trip} = p_{trip1} \cdot \exp[-p_{trip2} \cdot (BO_{ij} - 2.5)^2] \cdot \frac{\exp[-p_{trip4} \cdot (\sum_{k=1}^{neighbours(i)} BO_{ik} - BO_{ij})] + \exp[-p_{trip4} \cdot (\sum_{k=1}^{neighbours(i)} BO_{jk} - BO_{ij})]}{1 + 25 \cdot \exp[p_{trip3} \cdot (\Delta_i + \Delta_j)]}$$

范德华相互作用项: 除了电子云重叠而出现的键结能量外, 体系中原子还存在着非键结相互作用, 其中一项就是范德华相互作用。这部分能量的计算与经典分子动力学相似, 所不同的是当原子跨越截断半径的时候能量会出现不连续现象, 为避免这种现象的出现特引入 Taper 函数:

$$Tap = Tap_7 \cdot r_{ij}^7 + Tap_6 \cdot r_{ij}^6 + Tap_5 \cdot r_{ij}^5 + Tap_4 \cdot r_{ij}^4 + Tap_3 \cdot r_{ij}^3 + Tap_2 \cdot r_{ij}^2 + Tap_1 \cdot r_{ij} + Tap_0$$

这个七阶的多项式能够保证非键结相互作用具有一阶, 二阶和三阶导数连续性同时可保证在截断半径位置出的相互作用能量连续趋于零, 各项的数学表达式如下:

$$\begin{aligned} Tap_7 &= 20/R_{cut}^7 \\ Tap_6 &= -70/R_{cut}^6 \\ Tap_5 &= 84/R_{cut}^5 \\ Tap_4 &= -35/R_{cut}^4 \\ Tap_3 &= 0 \\ Tap_2 &= 0 \\ Tap_1 &= 0 \\ Tap_0 &= 1 \end{aligned}$$

在范德华相互作用中采用 Morse 函数形式, 同时为避免键结的两个原子与相间的两个原子之间出现过高的排斥作用引入近程修正项:

$$E_{vdWals} = Tap \cdot D_{ij} \cdot \left\{ \exp \left[\alpha_{ij} \cdot \left(1 - \frac{f_{13}(r_{ij})}{r_{vdW}} \right) \right] - 2 \cdot \exp \left[\frac{1}{2} \cdot \alpha_{ij} \cdot \left(1 - \frac{f_{13}(r_{ij})}{r_{vdW}} \right) \right] \right\}$$

$$f_{13}(r_{ij}) = \left[r_{ij}^{p_{vdW1}} + \left(\frac{1}{\gamma_w} \right)^{p_{vdW1}} \right]^{\frac{1}{p_{vdW1}}}$$

库伦相互作用: 在计算库伦相互作用是采用 QEq 方法, 即通过原子间的距离变化计算瞬时的原子电荷, 以满足反应力场的化学键断裂和生成的要求:

$$E_{coulomb} = Tap \cdot C \cdot \frac{q_i \cdot q_j}{[r_{ij}^3 + (1/\gamma_{ij})^3]^{1/3}}$$

15.2 ReaxFF 力场拟合软件介绍

GARFfield 是由加州理工学院 Andres Jaramillo-Botero 所开发的针对 ReaxFF 力场的拟合软件, 全称是 Genetic Algorithm based Reactive Force field optimizer method。它是基于遗传算法, 爬山法和共轭梯度最小化法开发的多目标参数优化软件, 可以实现 Pareto-optimal 参数优化。目前 GARFfield 支持 ReaxFF 力场, eFF-ECP 力场, Morse 力场, COMB 力场, Tersoff 力场和 EAM 合金力场的开发。其开发流程和商业的 SCM 软件类似, 包括输入文件等。

14.2.1 安装

GARFfield 下载地址加微信 baolu_yao

NOTE: GARFfield is currently available for non-profit academic and research activities. Its use for commercial purposes requires a commercial license that can be requested via email to ajaramil@caltech.edu.

If you want to try GARFfield, please read and accept the terms of the LICENSE.

本教程为该软件在 Linux 下的安装过程。安装需要确保你系统中已安装有 gcc, g++和 gfortran

首先下载软件, 将软件放在合适的目录下, 比如就放在了如下目录中

/home/bly/software

输入解压缩命令

tar -xvzf GARFfield.tar.gz

然后下载 mpich3.4.3 和 FFTW

下载链接分别为

<https://www.mpich.org/static/downloads/3.4.3/mpich-3.4.3.tar.gz>

<http://www.fftw.org/fftw-3.3.10.tar.gz>

下载好后放在和 GARFfield 一样的目录下, 如

/home/bly/software

分别输入解压缩命令解压 mpich 和 fftw 的安装包

```
tar -xvzf mpich-3.4.3.tar.gz
```

```
tar -xvzf fftw-3.3.10.tar.gz
```

进入 mpich 解压缩出的目录，依次执行以下命令

```
./configure --prefix=/home/bly/software/mpich --enable-shared --disable-fortran
```

```
make
```

```
make install
```

进入 fftw 解压缩出的目录，依次执行以下命令

```
./configure --prefix=/home/bly/software/fftw --enable-shared
```

```
make
```

```
make install
```

mpich 和 fftw 安装完成后打开.bashrc，在末尾添加如下两行

```
export PATH=/home/bly/software/mpich/bin:$PATH
```

```
export
```

```
LD_LIBRARY_PATH=/home/bly/software/mpich/lib:$LD_LIBRARY_PATH
```

保存文件，然后执行

```
source ~/.bashrc
```

进入 GARFfield 下的这个目录

```
/home/bly/software/GarFfield/lib/lammps/lib/reax
```

执行以下命令

```
make -f Makefile.gfortran
```

打开如下目录中的 Makefile.ion 文件

```
/home/bly/software/GarFfield/lib/lammps/src/MAKE
```

在打开的 Makefile.ion 文件，在文件头部位置处修改 MPICH 和 FFTW 两个变量的值为

```
MPICH = /home/bly/software/mpich
```

```
FFTW = /home/bly/software/fftw
```

保存文件，进入 lammps 的 src 目录

```
/home/bly/software/GarFfield/lib/lammps/src
```


执行以下命令

```
make ion
```

```
make makeshlib
```

```
make -f Makefile.shlib ion
```

进入以下目录

```
/home/bly/software/GarFfield/lib/optlist
```

执行

```
make
```

进入以下目录

```
/home/bly/software/GarFfield/lib/pgapack/source
```

打开该目录下的 Makefile 文件，在该文件的头部修改 MPI_HOME 变量的值为

```
MPI_HOME = /home/bly/software/mpich
```

保存文件，执行以下命令

```
make
```

进入以下目录

```
/home/bly/software/GarFfield/src
```

打开该目录下的 Makefile 文件修改变量 MPI_HOME，LAMMPS，FFTW 的值为

```
MPI_HOME = /home/bly/software/mpich
```

```
LAMMPS = /home/bly/software/GarFfield/lib/lammps
```

```
FFTW = /home/bly/software/fftw
```

删掉该 Makefile 文件中 LAMMPS_LIB 变量的-ljpeg，保存文件

执行以下命令

```
make
```

编译完成后会在以下目录中找到 garffield_ion 文件

```
/home/bly/software/GarFfield/bin
```

打开.bashrc 文件在文件末尾添加以下内容

```
export PATH=/home/bly/software/GarFfield/bin:$PATH
```

```
export
```

```
LD_LIBRARY_PATH=/home/bly/software/GarFfield/lib/lammps/src:$LD_LIBRAR
```

```
Y_PATH
```

export

LD_LIBRARY_PATH=/home/bly/software/GarFfield/lib/optlist:\$LD_LIBRARY_PATH

export

LD_LIBRARY_PATH=/home/bly/software/GarFfield/lib/pgapack/lib/ion:\$LD_LIBRARY_PATH

执行

source ~/.bashrc

运行

mpirun -np 2 garffield_ion

输入以下内容则软件安装成功

GARFfield force field parameter optimization framework, version 1.0

Author: Andres Jaramillo-Botero (ajaramil@caltech.edu, Caltech (c) 2010-2014)

Usage:

garffield_ion <geo> <ffield> <trainset> <params> [<restraints>] [GA options] [other options] ... or

mpirun -np NumProcs garffield_ion <geo> <ffield> <trainset> <params> [<restraints>] [GA options] [other options]

GA options are:

-i ['range' or 'percent'] {parameter interval}

-g ['GRGA' or 'SSGA'] {replacement strategy: generational or steady-state}

-m [float between 0-1] {mutation rate}

-o [float between 0-1] {crossover rate}

-p [int] {population size}

-s ['maxiter' or 'nochange'] {convergence criteria}

-t [int] {stop after int iterations}

-z [int] {set SSGA population replacement percent}

-x {perform mutation and crossover}

Other options are:

- c [int] {Hillclimb every int iterations}
- C [int] {mincap array size in reax/c allocations}
- d ['full_debug', 'lammers_debug'] debug mode
- e ['MPE' or 'SIWE' or 'RMSE' or 'NRMSE' or 'MSE'] {error function}
- f {check forces on unit cells, instead of performing minimization}
- F ['reax']
- G [int] {switch to Conjugate Gradient after int iterations}
- I [int] {Number of molecular mechanics energy minimization steps}
- l <filename> {write log to filename, instead of console}
- M ['fire' or 'sd'] {switch structure minimization algorithm - default is 'cg'}
- q <filename> {Use pQeq for ReaxFF charges}
- r [int] {print report every int iterations}
- R [int] {Restart every int iterations}

- S [float] {safezone factor in reax/c memory allocation}
- u ['real'] {Force units to be real, i.e. lengths in Angstroms, energies in kcal/mol and forces in kcal/mol-Angstrom}
- v {Use low-gradient van der Waals for reax/c}
- w <filename> {write best force field file at same print report report_frequency}
- W {randomize section weights in training file}

Abort(1) on node 0 (rank 0 in comm 0): application called
MPI_Abort(MPI_COMM_WORLD, 1) - process 0

14.2.2 基本使用

GARFfield 的使用采用以下命令

```
mpirun -n p garffield <GeoFile> <ForceFieldFile> <TrainSetFile> <ParamsFile>
[Options]
```

garffield 是编译好的可执行文件, 可能是其他名字, 具体查看自己编译好的文件。

使用 GARFfield 需要准备几个输入文件。

第一个是几何文件 **GeoFile**。下面是 GeoFile 的一个示例截图。

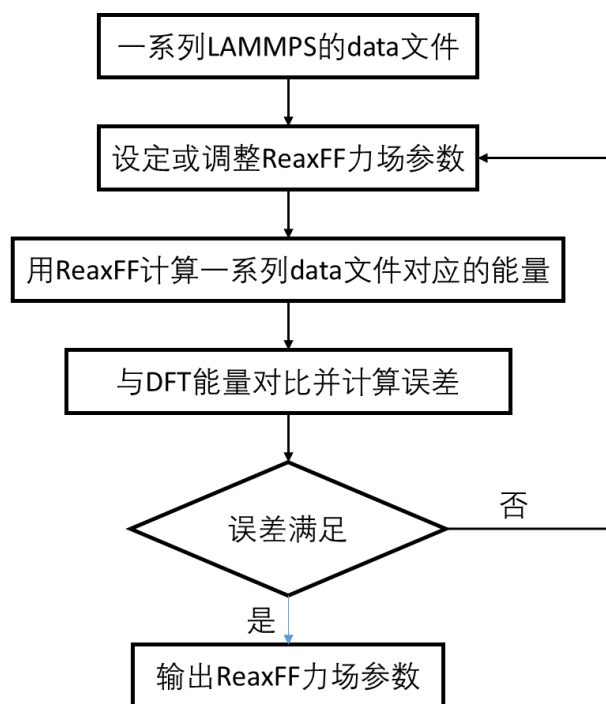
```
BIOGRF 200
DESCRP C12_414
REMARK .bgf-file generated by xtob-script
#      At1 At2  R12      Force1 Force2  dR12/dIter(MD) Start (MD) End (MD)
BOND RESTRAINT 1 2 4.14 7500.00 7500.00 0.0000000 0 0
HETATM 1 C1 0.00000 0.00000 2.07009 C1 0 0 0.00000
HETATM 2 C1 0.00000 0.00000 -2.07009 C1 0 0 0.00000
END

BIOGRF 200
DESCRP C12_404
REMARK .bgf-file generated by xtob-script
#      At1 At2  R12      Force1 Force2  dR12/dIter(MD) Start (MD) End (MD)
BOND RESTRAINT 1 2 4.04 7500.00 7500.00 0.0000000 0 0
HETATM 1 C1 0.00000 0.00000 2.02009 C1 0 0 0.00000
HETATM 2 C1 0.00000 0.00000 -2.02009 C1 0 0 0.00000
END

BIOGRF 200
DESCRP C12_394
REMARK .bgf-file generated by xtob-script
#      At1 At2  R12      Force1 Force2  dR12/dIter(MD) Start (MD) End (MD)
BOND RESTRAINT 1 2 3.94 7500.00 7500.00 0.0000000 0 0
HETATM 1 C1 0.00000 0.00000 1.97008 C1 0 0 0.00000
HETATM 2 C1 0.00000 0.00000 -1.97008 C1 0 0 0.00000
END

BIOGRF 200
DESCRP C12_384
REMARK .bgf-file generated by xtob-script
#      At1 At2  R12      Force1 Force2  dR12/dIter(MD) Start (MD) End (MD)
BOND RESTRAINT 1 2 3.84 7500.00 7500.00 0.0000000 0 0
HETATM 1 C1 0.00000 0.00000 1.92008 C1 0 0 0.00000
HETATM 2 C1 0.00000 0.00000 -1.92008 C1 0 0 0.00000
END
```

反应力场的开发过程需要对比设定分子中某一项相互作用的能量的 ReaxFF 计算值和 DFT 的计算值, 由此计算误差函数并调整力场参数, 使得两个能量尽可能的接近。某一项相互作用的能量是一个包含两列数据的列表, 第一列是参数变化, 第二列是能量。比如要对比键能, 第一列就是不同键长值, 第二列就是分子的能量; 再比如二面角能, 第一列就是不同二面角值, 第二列就是分子的能量。为了在 ReaxFF 下计算不同力场参数下的分子能量, 我们需要向软件提供不同键长, 键角, 二面角等值时的分子中的原子坐标。而 GeoFile 文件就是提供这一作用。在软件中会将 GeoFile 文件转换为 data 文件, 从而利用 LAMMPS 计算能量。反应力场的开发流程简单的说就是对比 DFT 和 ReaxFF 计算的能量。如果不满足误差要求就调整力场参数, 然后计算新的力场参数下的 ReaxFF 能量并与 DFT 进行对比, 如果还不满足误差要求, 就重复这一过程直到有 ReaxFF 计算的能量与 DFT 计算的能量满足误差要求。开发流程如下图所示



ReaxFF 力场开发的基本流程

对于 GARFfield 软件如果你直接提供了一系列 data 文件那么 GeoFile 可以不提供。

GARFfield 需要提供的第二个文件就是力场文件。为了让 GARFfield 识别你在开发 ReaxFF 力场, 你的力场文件的第一行要以“Reactive”这个词开头。首先你需要准备一个满足最后要输出的力场文件的格式, 包括所有的参数。这些参数必须具有初始值。也就是说必须提供一个完整的力场文件, 而力场文件中的参数要尽可能的接近最后要输出的参数。通常开发反应力场并不是从零开始, 而是在前人的力场基础上进行开发。所以首先找一个和目标力场尽可能接近的已有力场文件。调整力场文件中元素个数和参数值使其格式与目标力场一致。由此力场文件就准备好了。在准备力场文件的时候, 可以将已有的力场文件进行删减增改。也可以把几个不同的力场文件进行组合。要尽可能准确的找到目标力场中的所有参数。如果实在找不到了就用化学性质相似的元素或者键结形式相似的参数进行替代。

第三个需要提供的文件就是训练集文件 TrainSetFile。训练集文件可以包含很多选项。最常见的选项就是能量项。通常 ReaxFF 力场开发是逐步进行的。一次训练会包含一个能量选项或其他选项。支持 ReaxFF 的选项包括

```

CHARGE <weight>
#Key weight Atom Charge (a,f,i)
ENDCHARGE
STRESS <weight>
#Key weight xx[yy zz xy xz yz] (a,f,a)
ENDSTRESS
PRESSURE <weight>
#Key weight press|xx|yy|zz|xy|xz|yz Value (a,f,a,f)
ENDPRESS
STSTRAIN <weight>
#Key weight xx Value [yy Value zz Value xy Value xz Value yz Value] (a,f,a,f[a,f])
ENDSTSTRAIN
ATOM_FORCE
#Key weight Atom1 fx, fy, fz (a,f,i,f,f,f)
ENDATOM_FORCE
CELL PARAMETERS <weight>
#Key weight a[b,c] Value (a,f,a,f)
ENDCELL PARAMETERS
ENDHEATFO
GEOMETRY <weight>
#Key weight Atom1 Atom2 Atom3 Atom4 Value (a,f,i,i,(i,i),f)
ENDGEOMETRY
ENERGY <weight>
#weight op1 Key1 / n1 op2 Key2 / n2 op3 Key3 / n3 op4 Key4 / n4 ...
opn Keyn / nn Value (f,(a,a,i),f)
ENDENERGY
    
```

其中, Key 对应结构文件名。该文件名在 GeoFile 中设定, 或者是 LAMMPS 的 data.Key 文件。weight 是权重, 其可以用在每个部分或每一条输入中, 权重用来计算该条选项在总的误差中的加权和。op 是数学符号+或-。Value 是从量子力学或实验中得到的参考值。当你不显示指定一个部分的权重时, 它会默认为 1, 或者利用-W 选项来是的 GARFfield 随机产生。所有涉及到的单位都和 LAMMPS 中的 real 单位系统一致。括号中的字母表示每个参数的数据类型, i=integer, f=real, a=ASCII, 这些字母不出现在实际的输入中。() 表示可能重复的输入。

CHARGE 部分用来优化 ReaxFF 中原子电荷平衡参数, 包括电负性, 硬度, 和半径 (electronegativity, hardness 和 radius)。

STRESS 部分相对于一个没有应力的周期性元胞用来优化力场参数

PRESSURE 部分相对于一个周期性元胞压强标量或者张量分量来优化力场参数

STSTRAIN 部分相对于一系列应力张量分量来优化力场参数, 也就是对角分量 (xx, yy, zz) 和非对角分量 (xy, xz, yz)。同一个权重会被用于所有的张量分量。

ATOM_FORCE 部分使用原子受力来优化力场参数, 通常这会提高力场的精度和可迁移性, 即加入了能量梯度。

CELL_PARAMETERS 部分相对于周期性元胞晶格参数来优化力场参数

GEOMETRY 部分相对于几何特征如键长, 键角, 二面角来优化力场参数

ENERGY 部分相对于能量差来优化力场参数。

第四个要提供的文件是参数文件 **ParamsFile**。参数文件确定哪些力场参数会被 GARFfield 优化。一行代表一个参数, 可以有多行来设定多个参数。每一行的格式为

Section Entry Parameter 0.0 Low High (i,i,i,f,f,f)

当在命令中使用 -i range 选项时, 每一个参数都会有一个优化范围, 即介于 Low 和 High 之间。当我们提供的初始力场文件中参数是可接受的, 那么 GARFfield 也提供另一种格式来优化参数, 即提供参数的平均值和 ± 百分比浮动来探索力场参数。此时每一行的格式变为

Section Entry Parameter 2.0 Mean %Offset (i,i,i,f,f,f)

其中 Mean 的值直接来自力场文件, %Offset 来自参数文件。此时每一行中的第四个值要小于等于 2.0。

第五个可选提供的是约束文件 **RestrainFile**。在开发反应力场时, 计入反应物, 产物和过渡态是有用的。由于 GARFfield 通过所选择的力场引擎来进行几何优化, 因此必须采用相应的约束来使得结构不会偏离想要的反应路径。约束可以是两个原子之间的距离, 三个原子时间的角度以及四个原子之间的二面角。在 RestrainFile 中定义这些约束的格式是

RESTRAINT Key Keyword args

其中, Key 为在 GeoFile 文件中的结构文件名, Keyword 可以是 BOND, ANGLE, TORSION。BOND args 为 particle1 particle2 Kstart Kstop r0, particle1, particle2 为键中的两个原子编号, Kstart, Kstop 为起始和结束约束参数 (能量单位), r0 是平衡键长 (距离单位)。ANGLE args = particle1 particle2 particle3 Kstart Kstop

θ_0 , particle1, particle2, particle3 为键角中的三个原子编号, particle2 是那个中间的原子, Kstart, Kstop 为起始和结束约束参数 (能量单位), θ_0 是平衡键角 (度), TORSION args = particle1 particle2 particle3 particle4 Kstart Kstop Φ_0 , particle, particle2, particle3, particle4 为二面角中的原子编号, Kstart, Kstop 为起始和结束约束参数 (能量单位), Φ_0 是平衡二面角 (度)

GARFfield 会自动检测并将 RestraineFile 用作命令行中的第五个参数。当使用 bgf 格式的 GeoFile 是, 可以直接通过 RESTRAIN 关键词直接描述约束, 这样就不用提供一个单独的 RestraineFile 了。

GARFfield 的输出信息:

当 GARFfield 开始的时候, 它会向终端输出以下信息: 默认设置以及对默认设置的修改; 优化开始前对 ForceFieldFile, GeoFile, TrainSetFile 和 ParamsFile 的读取信息; 每个迭代的总误差。除此之外, GARFfield 还会输出以下文件:

- **Force Field parameters:** ffield.best 包含每次 run 结束之后最优的力场参数, 其余初始力场文件的格式一样。ForceFieldFile.original 对应初始力场文件, 每个 GA 迭代中间力场文件 ffield.new。如果使用命令选项 -w, 那么, 还写入了一个 ffield.restart 文件, 该文件对应于在最后一次 GA 报告迭代之前评估的最佳参数 (重启的频率通过 -r 选项控制)
- **误差信息:** trainset.err.best 文件写入了从 TrainSetFile 和 ffield.best 对比获得的误差值。train-set.err.initial 文件中写入了从 ForceFieldFile.original 文件和 TrainSetFile 对比获得的错误值。这些文件遵循原始 TrainSetFile 的格式, 除了额外的几列, 即力场值、每个条目的计算加权误差和总累积加权误差。
- **调试信息:** GARFfield 会写入两级调试信息, 这些信息会输出到终端中或者 logfile 中 (如果使用了 -l 选项), 调试的写出受命令行中 -d 选项的控制。
- **重启:** 在 GARFfield 中提供了两个重启选项。第一个是使用命令行中 -R 选项来从最佳累积力场 (即 mutated variants of the seed, with mutation probability at 0.5) 产生新的 population reseeded 进行周期性的重启。第二个是 -w BestForceField 选项来周期性的写出最佳的力场, 该力场可以作为输入文件来重启优化。第二个选项在需要多个优化过程时会很有用。第一个选项在初始力场比较合理时推荐使用, 此时可以设置重启频率为一个较大的值。

在并行过程中还会输出一些中间文件, 这些文件在运行结束后会被删除。

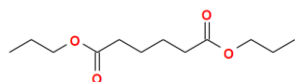
15.3 ReaxFF 力场验证

ReaxFF 已经存在很多元素的力场参数。很多时候能够找到包含所需元素的力场参数。但是力场参数往往是根据特定体系而开发的, 是否适用于自己的体系则需要验证力场参数。验证力场参数的基本流程就是对比自己体系中的分子各种能量(如键, 键角, 二面角)与 DFT 计算的能量。如果吻合较好那么找到的力场参数就可以用, 如果不好就要基于找到的力场参数进行进一步优化。本节介绍力场参数的验证过程。严格的验证力场需要对比各项能量和参数, 包括键结、非键结和电荷平衡参数。

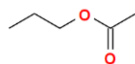
(1) 电荷平衡参数验证

这个可以通过计算一个完整分子和分子片段内原子的电荷, 对比 DFT 和 ReaxFF 的计算结果。以合成酯为例, 合成酯中只包含 CHO 三种元素。可以使用这篇文章 (Chowdhury Ashraf and Adri C.T. van Duin* Extension of the ReaxFF Combustion Force Field toward Syngas Combustion and Initial Oxidation Kinetics J. Phys. Chem. A 121, 5, 1051-1068) 中所开发出的力场参数。把力场参数下载下来, 命名为 ffield2016.reax.cho。目前在 LAMMPS 中只用 Qeq 方法进行电荷动态平衡。该方法要求有三个参数分别是
chi eta gamma

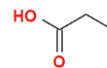
对于 LAMMPS 来说, 可以直接采用 ReaxFF 中力场文件中的参数, 也可以自己单独写一个文件定义这些参数。通常会直接采用力场文件中的参数。这里要验证的就是力场文件中的这三个参数。对于合成酯采用以下几个分子进行验证。



分子 1

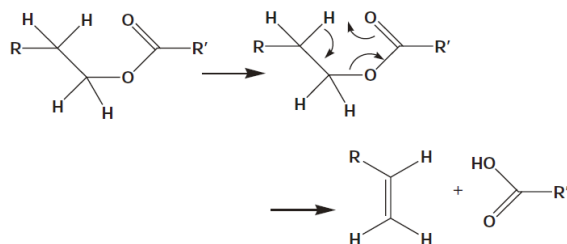


分子 2



分子 3

为什么要选择这几种。这就需要对研究对象有充分的了解。这里要研究合成酯的热解问题, 那么首先合成酯的完整分子自然要考虑, 一半合成酯也可以考虑一下。然后根据前人研究合成酯会发生以下热解过程, 那么热解过程中的反应产物也要考虑。而不含 O 的自由基和分子就不用考虑了, 因为待验证的力场在开发时已经充分验证了。



这里采用 LAMMPS 计算 ReaxFF 所得到的电荷并与采用量子化学的电荷进行对比。要计算首先要有输入文件的坐标。可以采用 Avogadro 进行坐标生成。这里没有采用 MS 是因为 MS 太大了, 这么个小事不需要这么大的软件。而且 MS 收费, 使用收费软件不符合讲义特点。使用 Avogadro 分别生成上述三个分子的 pdb 文件和 xyz 文件。在 Avogadro 中画完分子之后一定要做几下结构优化只有再导出文件。

Qeq 方法只需要分子几何构型和原子的实验性质就可以预测分子内部的每个原子的电荷。先将导出的 pdb 文件中转换为 data 文件 (一个分子就够了), 然后在 lammps 中计算每个原子的部分电荷。计算 in 文件如下

```
units          real
atom_style     full
neighbor       2.0 bin
neigh_modify   every 1 delay 0 check no
read_data      half.data
pair_style     reaxff lmp_control
pair_coeff     * *ffield2016.reax.cho C H O
timestep       0.25
velocity       all create 300.0 4928459 rot yes dist gaussian
thermo         100
thermo_style   custom step temp pe ke
thermo_modify  flush yes
fix            1 all nve
fix            2 all temp/berendsen 300.0 300.0 50.0
fix            3 all qeq/reaxff 1 0.0 10.0 1e-6 param.qeq
run            100000
dump           1 all custom 100 dump.lammpstrj id mol type q x y z
dump_modify    1 sort id
run            100000
```

计算完成后将 dump 中的电荷取时间平均, 对应的 python 脚本为

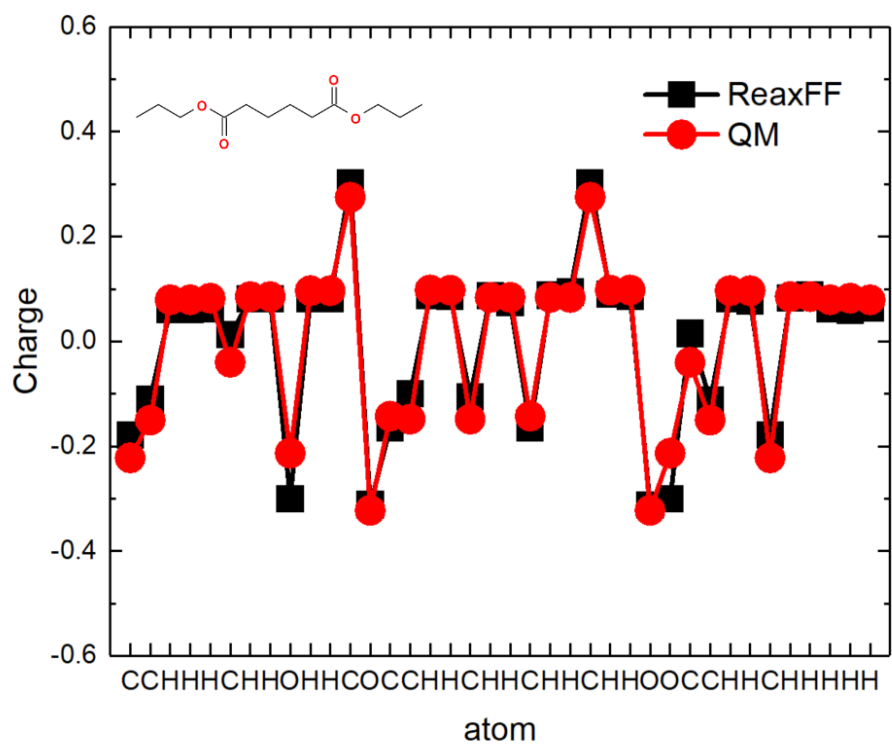
```
input_file = open('E:\\work\\ester_reax\\diester\\力场验证\\third\\dump.lammpstrj','r')
output_file = open('E:\\work\\ester_reax\\diester\\力场验证\\third\\aveq','w')
natom = 11
```

```
aveq = [[0]*2 for _ in range(natom)]
line = input_file.readline()
tmp = ""
no = 0
while line:
    if line == "ITEM: TIMESTEP\n":
        line = input_file.readline()
        line = input_file.readline()
        line = input_file.readline()
        line = input_file.readline()
        line = input_file.readline()
        line = input_file.readline()
        line = input_file.readline()
        line = input_file.readline()
        for i in range(natom):
            line = input_file.readline().split()
            aveq[i][0] += float(line[3])
        no += 1
    line = input_file.readline()

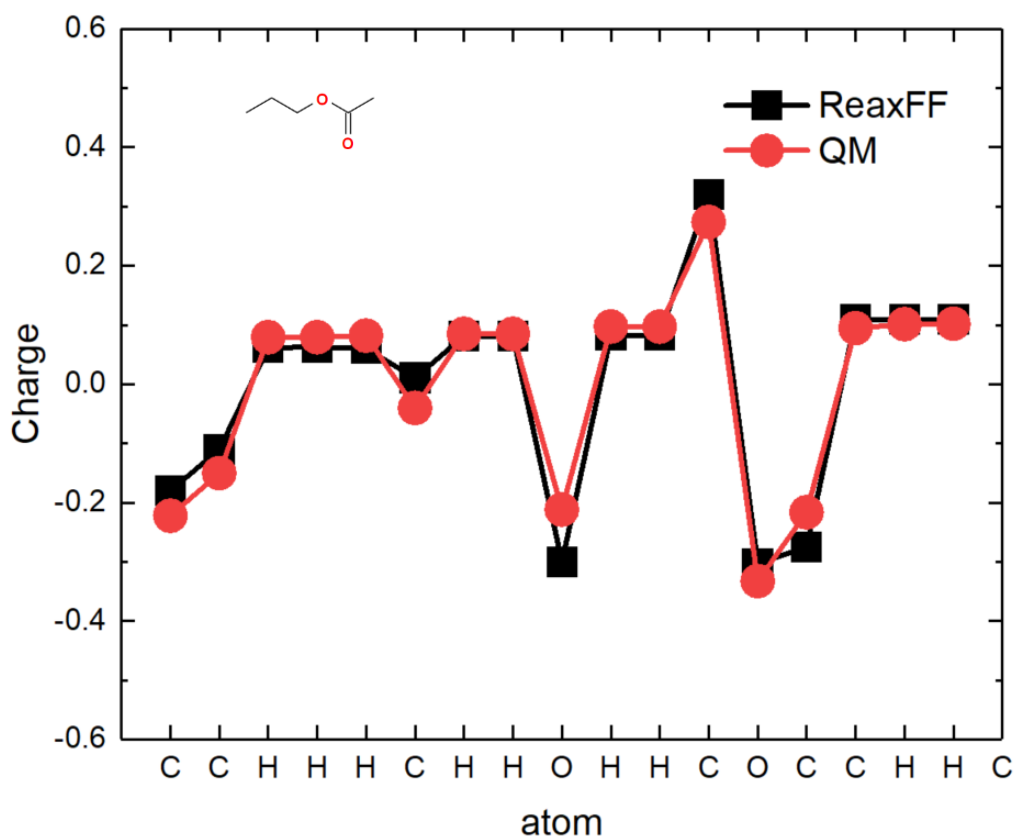
for i in range(natom):
    aveq[i][0] = aveq[i][0]/no
    output_file.write(str(aveq[i][0]) + "\n")
output_file.close()
```

这样就能得到有 ReaxFF 计算得到的电荷。然后使用导出的 xyz 文件, 调用 Multiwfn 软件和 Gaussian 程序使用量子化学计算得到的电荷。这里我们采用 1.2CM5 方法计算电荷。计算的采用的量子化学信息为 B3LYP-D3(BJ)/def2-SVP。由于 1.2*CM5 电荷就是在 Truhlar 等人提出的原始的 CM5 电荷基础上乘上 1.2, 这相当于增大了原子电荷的数量级, 等效地体现出了溶剂环境对溶质的极化作用。这里的模拟没有溶剂化作用, 所以需要对得到的电荷除以 1.2 得到 CM5 电荷。关于如何联用 Multiwfn 软件和 Gaussian 程序计算 CM5 电荷, 参见网址[计算适用于 OPLS-AA 力场做模拟的 1.2*CM5 原子电荷的懒人脚本 - 思想家公社的门口: 量子化学·分子模拟·二次元 \(sobereva.com\)](#)。这里要求 Gaussian 程序是 Linux 版下的。如需获取 Linux 版的 Gaussian 可加微信 baolu_yao。

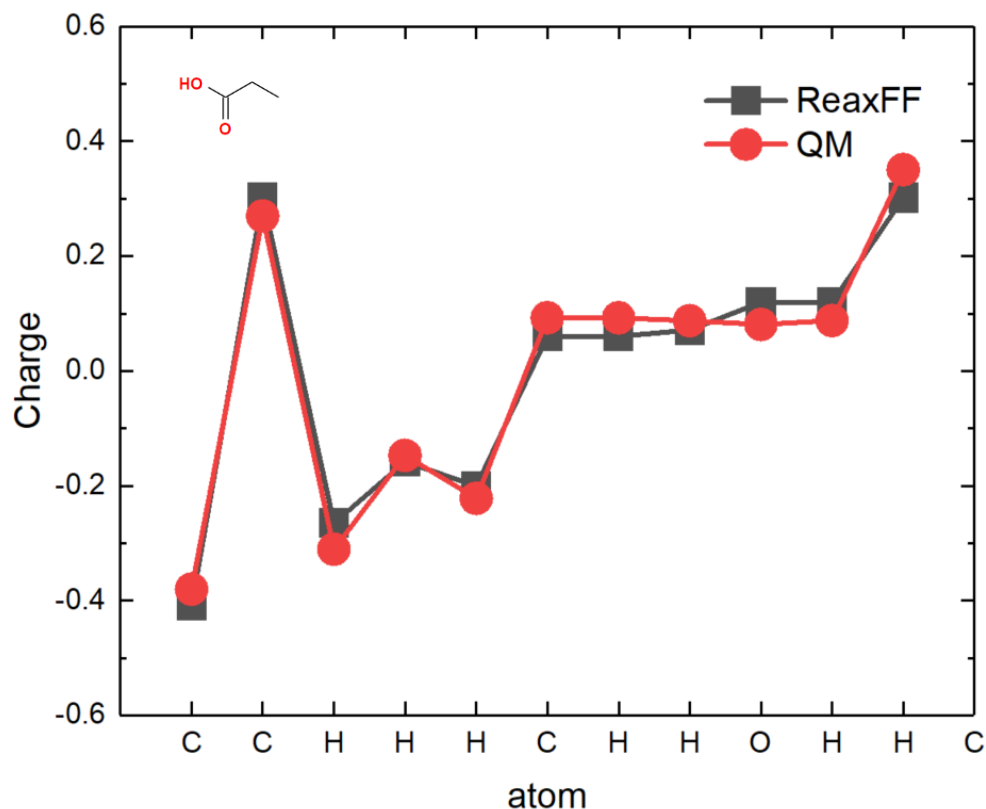
以下是上述三个分子中 ReaxFF 计算得到的原子电荷和量子化学得到的原子电荷对比。从三个图中可以看到这里所采用的 Qeq 参数以很高的精度再现了量子化学计算得到的电荷, 表明采用 Qeq 参数是合理的。具体的 Qeq 参数见下表。



分子 1 中 ReaxFF 和量子化学计算得到的原子电荷对比



分子 2 中 ReaxFF 和量子化学计算得到的原子电荷对比



分子 3 中 ReaxFF 和量子化学计算得到的原子电荷对比

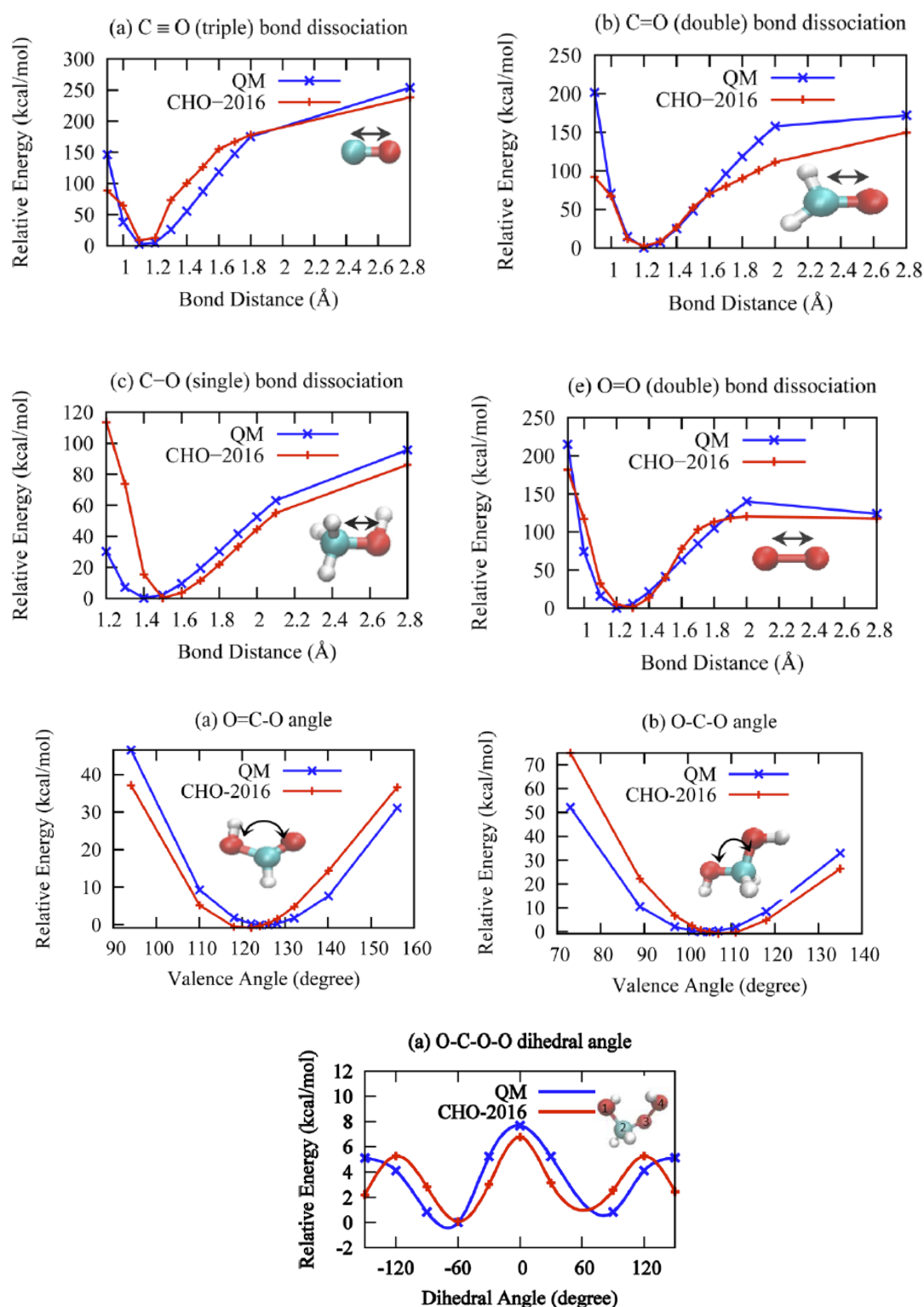
ReaxFF 模拟中采用的 Qeq 参数

元素	χ	η	γ
C	5.8678	14.0000	0.9000
H	5.3200	14.8732	1.0206
O	8.5000	17.9978	1.0503

(2) 键结能量验证方法

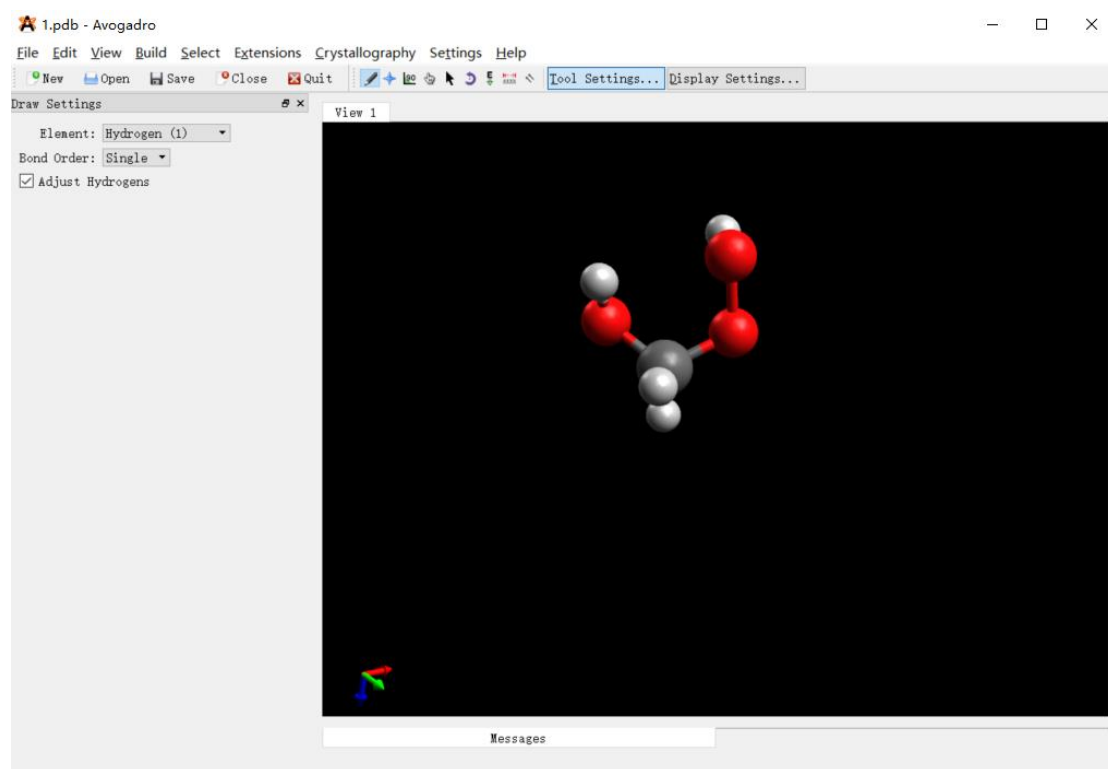
要验证的力场是针对 CHO 体系开发的。这里计划模拟合成酯，那么就要计算合成酯关键键的键能。前人研究表明合成酯在热解时会从 CO 单键处发生断键，所以需要重点验证这个键能。更一般的来说，当你找到一个力场，要针对自己的体系进行验证，你就需要将自己的体系中的分子进行简化，使其在包含关键化学基团的同时越简单越好。比如为了验证 CO 单键的能量这里选取了这个分子进行验证。首先验证关键的 CO 单键。验证的主要过程就是对比由 DFT 计算得到的能量和 ReaxFF 计算得到的能量，分子越简单能量对比越有效。为说明这里验证的方法的可行性（好像可行，后面会说明），首先复现一下论文中的结果。以下

是论文 Extension of the ReaxFF Combustion Force Field toward Syngas Combustion and Initial Oxidation Kinetics 中的力场部分验证结果。

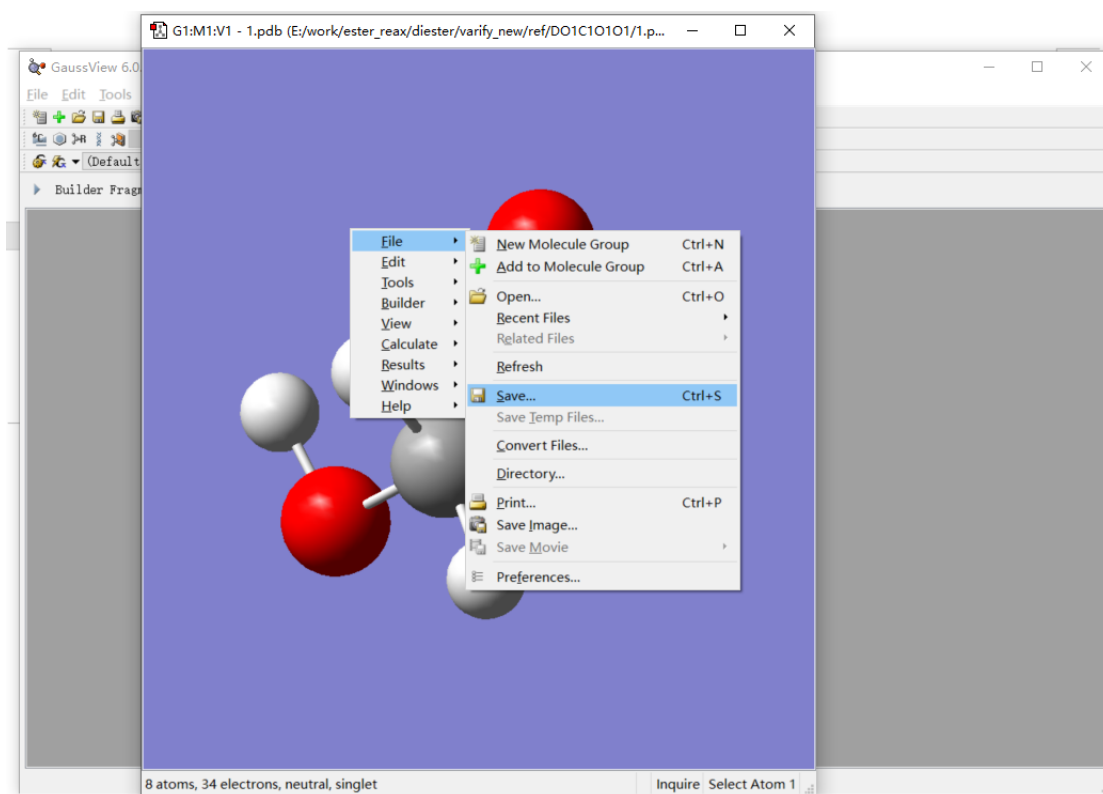


为了验证力场, 首先要获得 DFT 的计算结果, 也就是采用 DFT 方法对势能面性扫描。扫描采用高斯 16 软件进行刚性扫描。刚性扫描速度快, 并且对于小

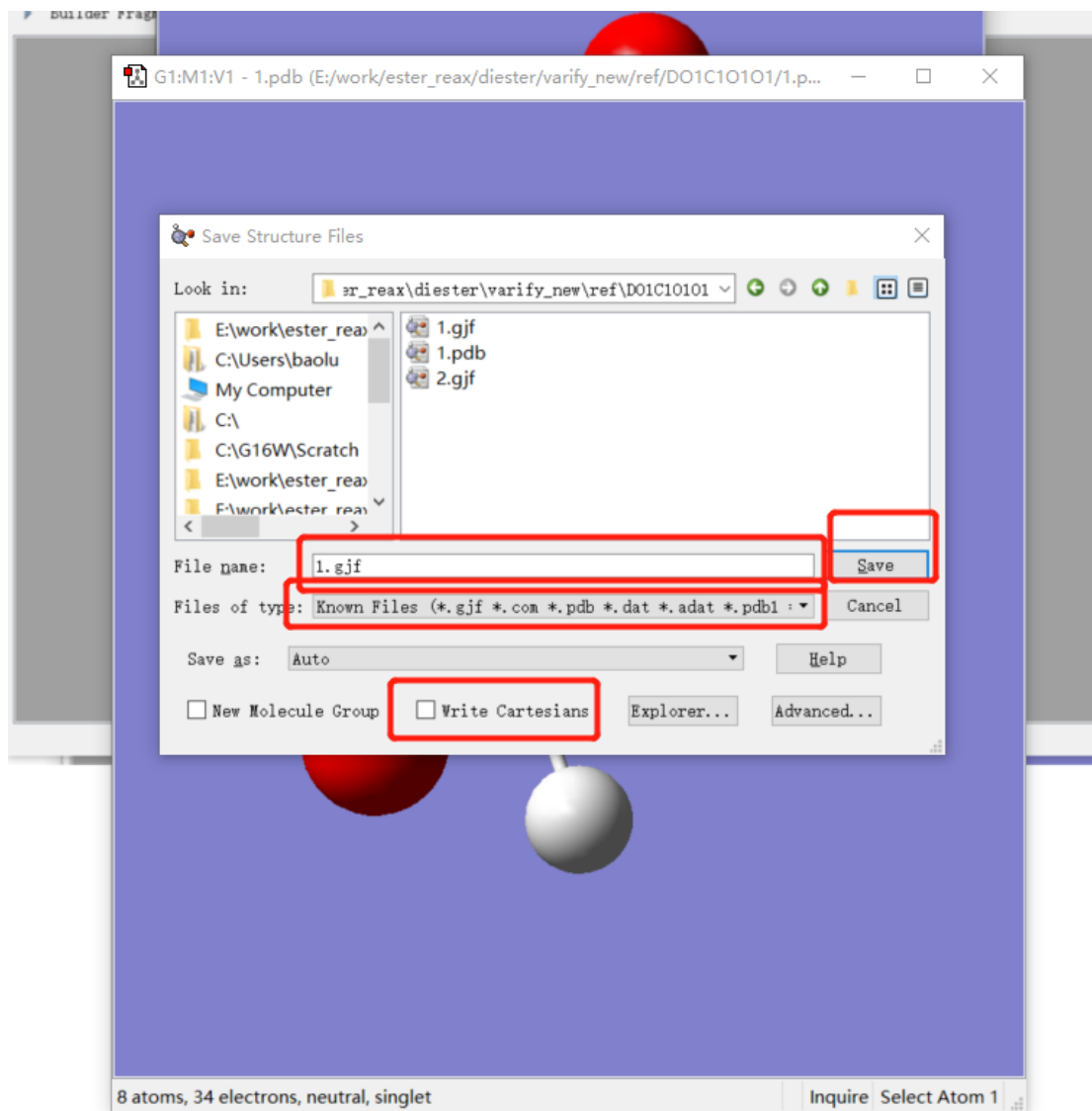
分子来说柔性扫描和刚性扫描结果相差不大。下面通过上图中第六个图的结果说明高斯中如何进行刚性扫描。首先根据图中的分子在 Avogadro 软件中绘制出分子结构并保存为 pdb 文件。在绘制分子的时候尽量和图中的分子构象保持一致，如下图。



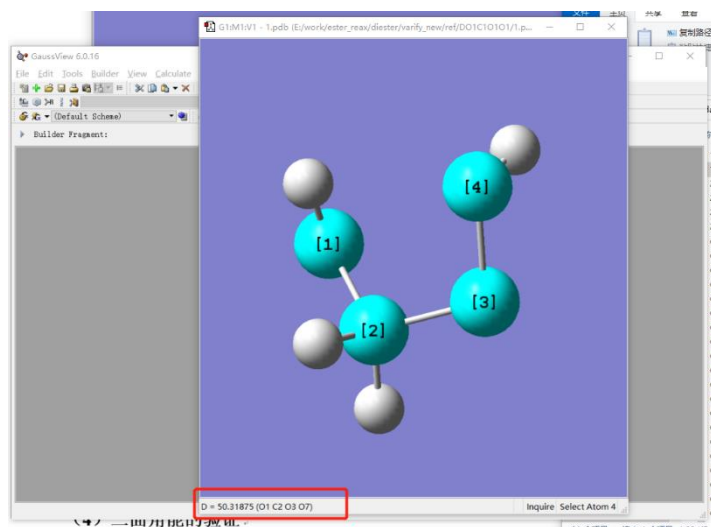
利用 GaussView 将保存的 pdb 文件打开，并保存为 gif 格式。保存的流程是在打开的窗口中鼠标右击选择 File->Save...，打开保存窗口。



在打开的保存窗口中 File name 中设置文件名称, Files of type 中选择 Know files, 取消 Write Cartesians 的勾选, 然后点击保存。



在刚才打开的 `pdb` 文件窗口中鼠标在原子上左击选择要扫描的二面角的四个原子。查看窗口左下角处构成该二面角的四个原子的编号。



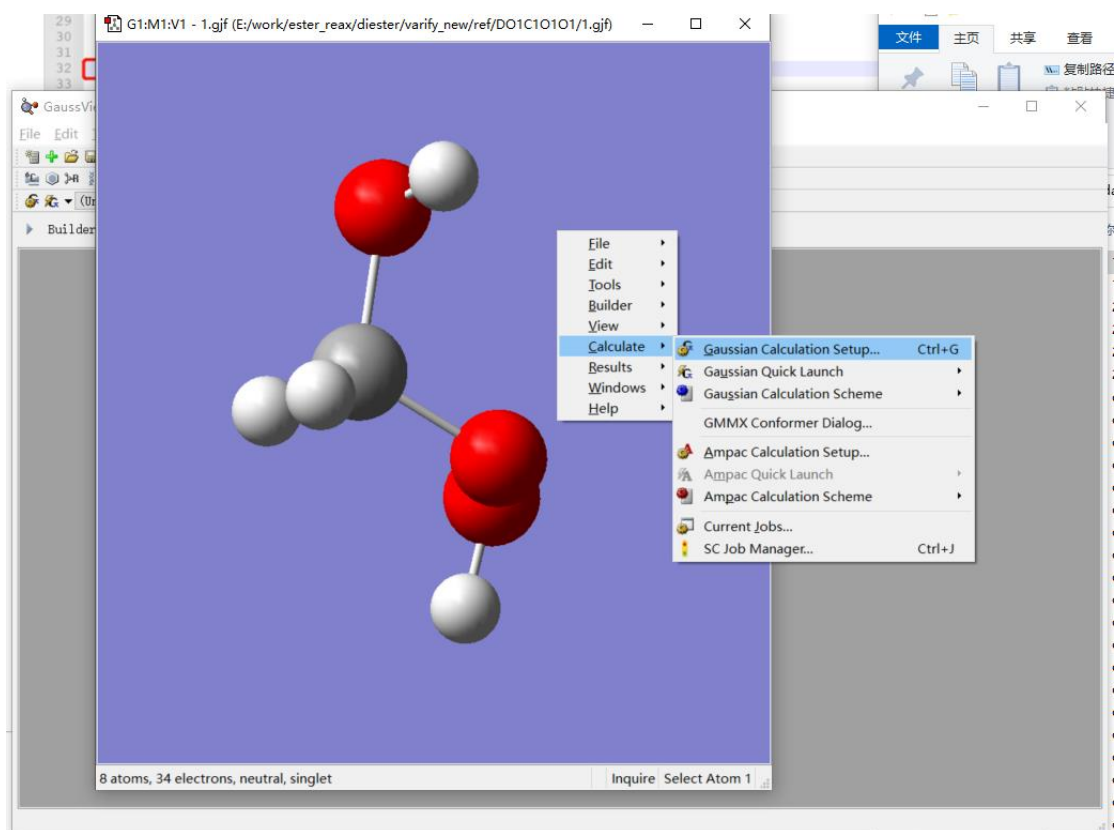
用文本编辑器将刚才保存的 gif 文件打开。将第二行改为 b3lyp/6-31g(d) scan nosymm, 其中 b3lyp/6-31g(d)表示 DFT 计算要选择的基组, scan 表示执行扫描任务, 做刚性扫描的过程中, 如果涉及到点群的改变, 不加 nosymm 关键词的话有时会出问题, 而且看到的扫描轨迹可能不连续。要扫描的二面角由编号为 7 3 2 1 四个原子组成, 从文件中可以看出这个二面角的编号是 D4。因此将下面 D4 行中的二面角角度改为-90.0 24 10.0。-90.0 表示扫描初始角度为-90°, 24 表示扫描 24 个角度, 10.0 表示按 10.0°进行角度递增。这里需要说明涉及到浮点数的一定要写为浮点数形式, 比如角度要写为-90.0, 10.0, 而不能是-90 和 10。修改完后保存文件。

```

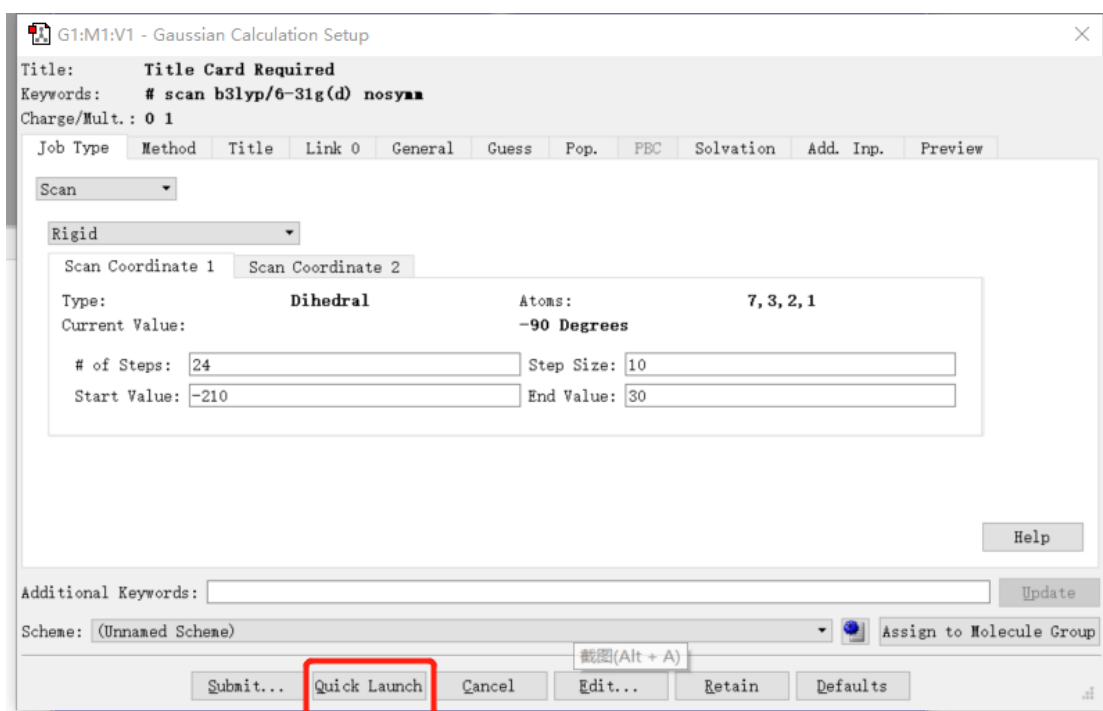
1  chk=F:\work\ester_reax\diester\varify_new\ref\DO1C10101\1.chk
2  # b3lyp/6-31g(d) scan nosymm
3
4  Title Card Required
5
6  0 1
7  O(PDBName=O, ResName=UNL, ResNum=1)
8  C(PDBName=C, ResName=UNL, ResNum=1)
9  O(PDBName=O, ResName=UNL, ResNum=1)
10 H(PDBName=H, ResName=UNL, ResNum=1)
11 H(PDBName=H, ResName=UNL, ResNum=1)
12 H(PDBName=H, ResName=UNL, ResNum=1)
13 O(PDBName=O, ResName=UNL, ResNum=1)
14 H(PDBName=H, ResName=UNL, ResNum=1)
15
16 B1      1.41910289
17 B2      1.42404038
18 B3      0.97274354
19 B4      1.09247380
20 B5      1.09419240
21 B6      1.46186251
22 B7      0.97687512
23 A1     111.06031387
24 A2     103.95331024
25 A3     108.56857182
26 A4     110.01574060
27 A5     106.19363504
28 A6      95.67164404
29 D1     -75.58688444
30 D2     -118.91932220
31 D3      122.08390478
32 D4     -90.0 24 10.0
33 D5     -105.34367420
34
35 1 2 1.0 4 1.0
36 2 3 1.0 5 1.0 6 1.0
37 3 7 1.0
38 4
39 5
40 6
41 7 8 1.0
42 8
43
44

```

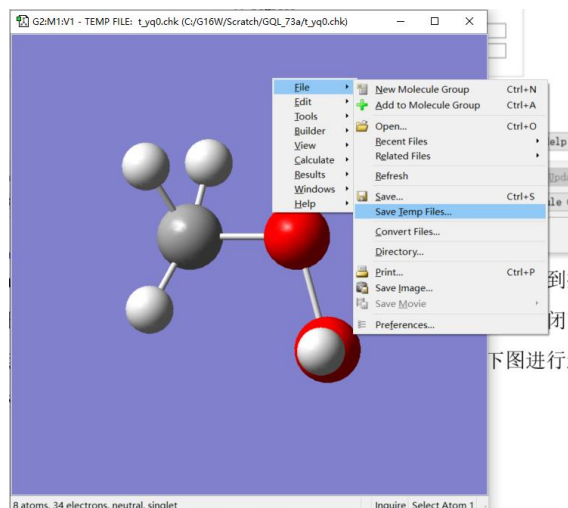
用 GaussView 打开刚刚修改并保存的 gif 文件, 右击窗口空白处, 按下图进行选择, 打开计算设置窗口 (不要关闭 gif 窗口)。



在打开的计算设置窗口中点击下方 Quick Launch 启动高斯开始计算。



Quick Launch 点击后会弹出高斯计算界面和另一个 chk 窗口。可以看到控制台在持续输出，由于是小分子，计算会比较快。计算完成后会弹出是否关闭高斯的对话框，点击是。稍等几秒钟，在 chk 窗口空白处鼠标右击，按下图进行选择打开保存临时文件窗口，在打开的窗口中设置保存的文件名称，点击保存。



会输出四个新的文件分别是 chk, fch, gif, log 文件。将 log 文件用文本编辑器打开, 拉到文件末尾可以看到扫描得到的每个角度对应的能量, 包括初始角度本例中会得到 25 行数据。将数据拷贝到绘图软件中。得到的能量的单位是 Hartree, 需要转换为 Kcal/mol 以和 lammps 的计算结果进行对比。1 Hartree=625.7 Kcal/mol。在对比 DFT 结果和 ReaxFF 结果时都会采用相对能量, 即将所有计算得到的能量减去能量的最小值。

scan completed.

Summary of the potential surface scan:

N	D4	SCF
1	-90.0000	-266.05860
2	-80.0000	-266.05952
3	-70.0000	-266.05979
4	-60.0000	-266.05907
5	-50.0000	-266.05718
6	-40.0000	-266.05421
7	-30.0000	-266.05061
8	-20.0000	-266.04722
9	-10.0000	-266.04504
10	0.0000	-266.04489
11	10.0000	-266.04697
12	20.0000	-266.05082
13	30.0000	-266.05544
14	40.0000	-266.05967
15	50.0000	-266.06263
16	60.0000	-266.06398
17	70.0000	-266.06383
18	80.0000	-266.06260
19	90.0000	-266.06078
20	100.0000	-266.05882
21	110.0000	-266.05708
22	120.0000	-266.05581
23	130.0000	-266.05510
24	140.0000	-266.05496
25	150.0000	-266.05523

log 文件中不仅会有扫描的最终结果, 还会记录每一个角度对应的所有原子的坐标, 通过提取这些坐标就可以得到每个角度对应的 data 文件, 用以 LAMMPS 计算。提取 log 文件中的坐标并转换为一系列 data 文件的 python 脚本如下:

```
work_path = 'E:\\work\\ester_reax\\diester\\varify_new\\ref\\DO1C1O1O1\\' #设置 log 文件的目录
```

```

input_file = open(work_path + '2.log','r')

natom = 8 #设置原子个数
line = input_file.readline()
no = 0
while line:
    tmp1 = line.split()
    if len(tmp1) == 2:
        if tmp1[0] == "Z-Matrix" and tmp1[1] == "orientation:":
            no += 1
            line = input_file.readline()
            line = input_file.readline()
            line = input_file.readline()
            line = input_file.readline()
            #data 文件的头部，可按需修改，这里元素只有 CHO 三种
            output_file = open(work_path + 'data.'+str(no),'w')
            output_file.write("LAMMPS data file Created\n")
            output_file.write("\n")
            output_file.write(str(natom) + " atoms\n")
            output_file.write("\n")
            output_file.write("3 atom types\n")
            output_file.write("\n")
            output_file.write("-20 20.0 xlo xhi\n")
            output_file.write("-20 20.0 ylo yhi\n")
            output_file.write("-20 20.0 zlo zhi\n")
            output_file.write("\n")
            output_file.write("Masses\n")
            output_file.write("\n")
            output_file.write("1 12.011\n")
            output_file.write("2 1.008\n")
            output_file.write("3 15.999\n")
            output_file.write("\n")
            output_file.write("Atoms\n")
            output_file.write("\n")
            for i in range(natom):
                line = input_file.readline().split()
                atom_id = line[0]
                mol_id = "1"
                #根据原子序号赋予 LAMMPS 中原子的 type 编号，可按需修改，这里只有 CHO 三种元素
                if line[1] == "6":
                    atom_type = "1"
                elif line[1] == "1":
                    atom_type = "2"
                else:

```

```

        atom_type = "3"
        atom_q = "0"
        x = line[3]
        y = line[4]
        z = line[5]
        tmp = atom_id + " " + mol_id + " " + atom_type + " " + atom_q + " " + x + " " + y + " " + z + "\n"
        output_file.write(tmp)
    output_file.close()
    line = input_file.readline()

```

利用得到的一系列 data 文件就可以计算 ReaxFF 下的能量, 计算的 in 文件如下:

```

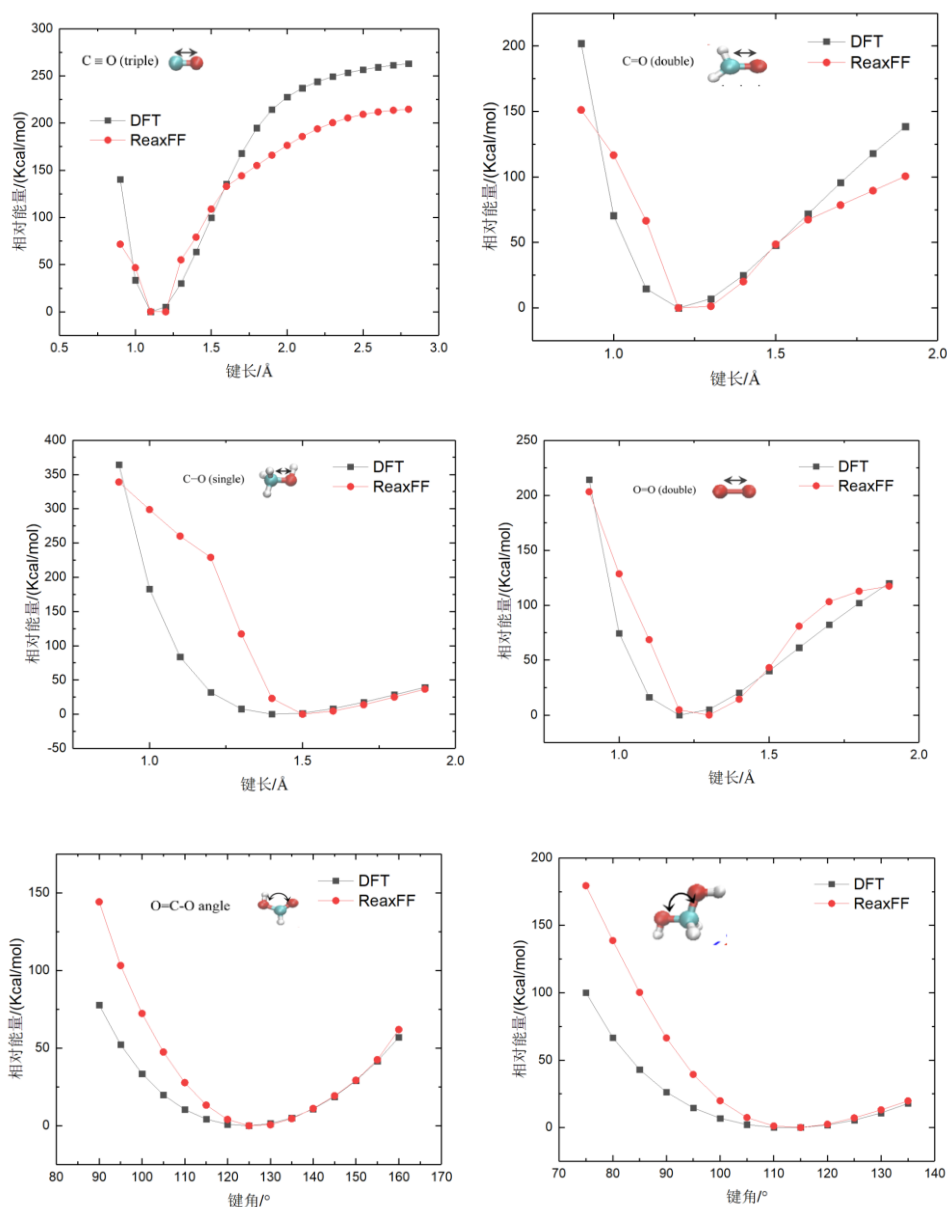
variable a loop 25
log log.$a
variable b index 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
units          real
atom_style     full
neighbor       2.0 bin
neigh_modify   every 1 delay 0 check no
read_data     data.$b
pair_style     reaxff NULL
pair_coeff * *ffield2016.reax.cho C H O
timestep 0.25
velocity all create 300.0 4928459 rot yes dist gaussian
thermo 100
thermo_style custom step temp pe ke etotal
thermo_modify flush yes
fix 3 all qeq/reaxff 1 0.0 10.0 1e-6 reaxff
variable mye equal etotal
run 1000
print ${mye} append info.dat screen no
clear
next b
next a
jump system.in

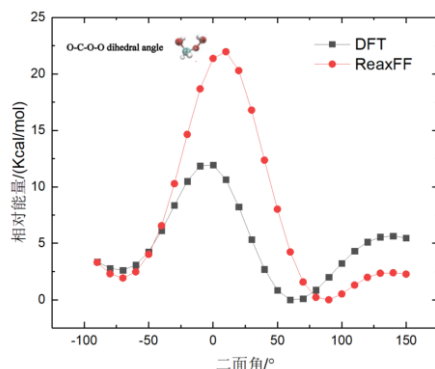
```

在输出的 info.dat 文件中就是所有 data 文件对应的能量, 将数据拷贝到绘图软件中, 并将所有数据减去这些能量中的最小值, 得到相对能量。

由此便可对比 DFT 和 ReaxFF 的计算结果了。以上就是力场验证的所有过程。本例中对应文献中的六个图进行对比了 DFT 和 ReaxFF 的结果, 如下图。从图中可以看出得到的对比图和文献中的差不多。但是还是有差别, 特别是对于键来说, 当键长小于平衡键长后这里计算的能量总是比文献中的较大, 对于键角来说, 当

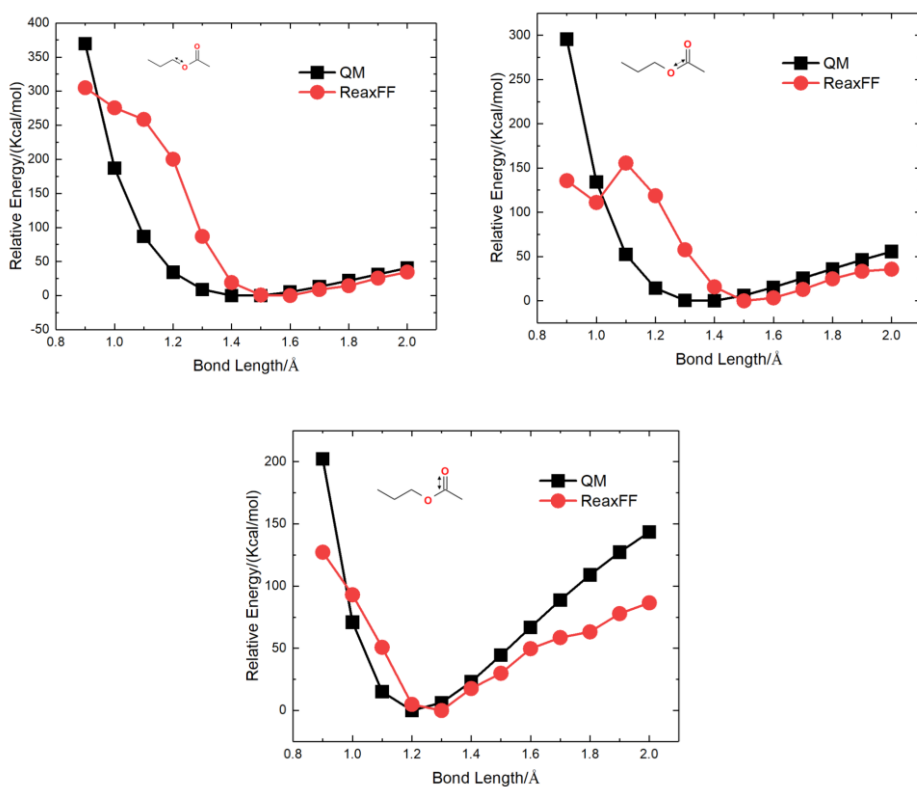
键角小于平衡键角后能量也比文献中的较大, 对于二面角来说当二面角在零度左右也比文献中的大。但是对于 ReaxFF 和 DFT 对比的可接受程度来说, 这里的验证是合理的。因为得到的误差和文献中的基本一致。至于为什么 LAMMPS 计算的 ReaxFF 的能量会与文献中的有符合很好的也有不符合的, 我也百思不得其解。知道原因的可以加我微信 baolu_yao, 我们一起讨论。



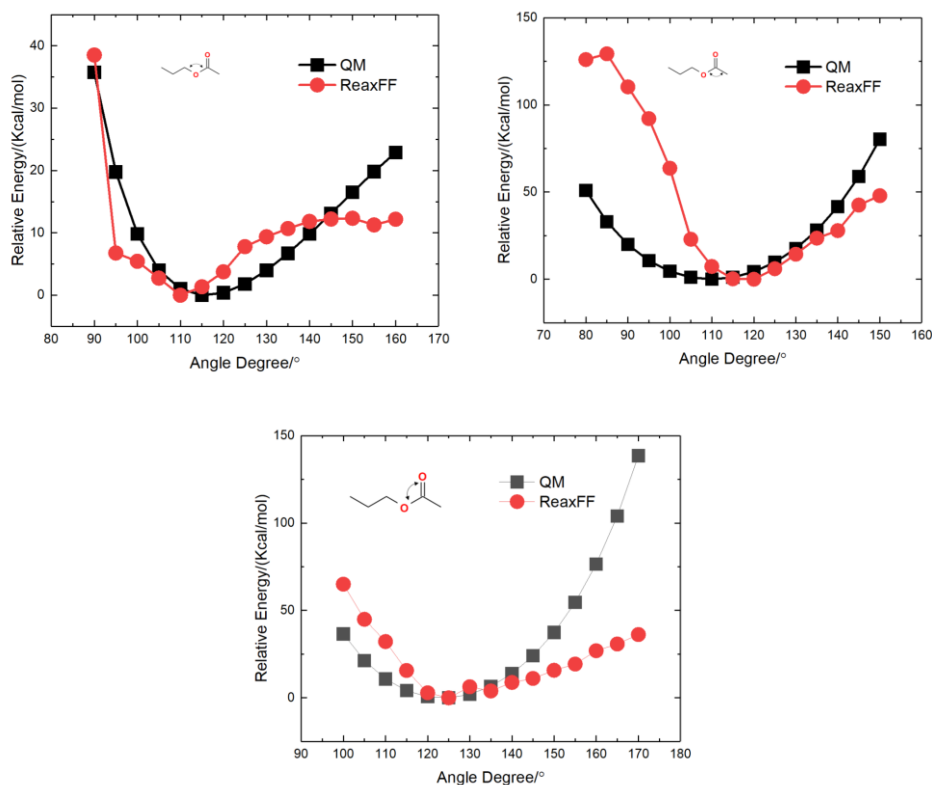


以下是针对合成酯分子的能量验证分别验证了酯基内的各类键，键角和二面角，不含酯基的部分不需要验证，力场本身已经在开发时验证过了。下面的结果表明包含酯基的 ReaxFF 计算的能量与 DFT 能量对比与原始力场相似，表明力场可以进行合成酯的模拟。

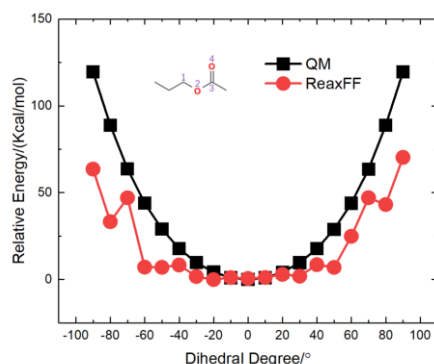
(3) 键能的验证



(4) 键角能的验证



(5) 二面角能的验证



15.4 ReaxFF 力场开发

16. 专题十: 耗散粒子动力学

16.1 简单流动

耗散粒子动力学是专为流体而开发的介观粗粒化模拟方法。通过将全原子尺度下的多个原子粗粒化为一个粒子, 从而降低体系的自由度个数, 提升计算效率。该方法中被抹除掉的自由度由一对满足涨落-耗散定理的耗散力和随机力进行描述。这也是耗散粒子动力学的名字的由来。耗散粒子动力学经过多年发展已经开

发出适用于体相流体的耗散粒子动力学、适用于自由界面模拟的多体耗散粒子动力学，适用于传热模拟的能量耗散粒子动力学等。本专题主要介绍原始的耗散粒子动力学。

在 LAMMPS 中主要通过 pair dpd 进行基础或原始耗散粒子动力学模拟。pair dpd 会包含三部分力：保守力，耗散力和随机力。其中耗散力和随机力组合起来起到了温度控制器的作用。因此在耗散粒子动力学中系统一定要采用 fix nve。耗散粒子动力学最简单的例子就是模拟通道流，即两无限大平行平板间的层流现象。在耗散粒子动力学中比较重要的是边界条件的设置。为了模拟固液之间滑移现象，即通过参数控制实现不同程度的滑移：从无滑移到无限滑移，研究者提出了很多方法，如固定粒子法，虚拟耗散边界法等。本节通过具体的 in 文件讲解来介绍。

```
#####基本模拟规则设置#####
units lj #耗散粒子动力学中统一采用无量纲化单位
dimension 3
boundary p p p
neighbor 0.3 bin
neigh_modify delay 0 every 1 check yes
atom_style atomic #如果没有键直接采用这个最简单的原子类型
pair_style dpd 1.0 2.5 546564 #温度一般设定为 1
comm_modify vel yes #需要这条命令来使得 ghost 原子存储速度信息
#####定义几何模型#####
region box block -20 20 -20 20 -20 20 units box
create_box 2 box
region upwall block -20 20 -20 20 15 20 units box
lattice fcc 12 #为防止流体原子从壁面中逃逸出去，壁面的密度要设置很大
create_atoms 1 upwall
region downwall block -20 20 -20 20 -20 -15 units box
lattice fcc 12 #为防止流体原子从壁面中逃逸出去，壁面的密度要设置很大
create_atoms 1 downwall
region fluid block -20 20 -20 20 -15 15 units box
lattice fcc 3 #流体的密度一般设置为 3，也可以设置为其他值，这和你杜丽华程度有关
create_atoms 2 fluid
#####定义物理模型#####
pair_style 1 1 3.0 1.0 #保守力和耗散系数的大小可以用来控制流体的可压性和粘度
pair_style 1 2 3.0 1.0 #壁面原子和流体原子之间的保守力可以用来防止流体逃逸出壁面，如果发现有原子深入壁面了，这个值就可以扩大。壁面原子和流体原子之间的耗散系数可以用来控制滑移程度，如果是 0 就是无限滑移，如果很大就是无滑移
```

```
#####定义分组信息#####
group wall type 1
group fluid type 2
#####弛豫部分#####
timestep 0.01 #由于时粗粒化, 保守力是有限值, 所以时间步可以大一点
thermo 1000
thermo_style custom step temp
thermo_modify flush yes
velocity fluidcreate 1.0 3265456 rot yes dist gaussian loop local
dump 1 all custom 10000 dump.lammpstrj id type x y z
fix fxnve fluid nve #只需要把壁面不包含在任何系综中, 壁面就会保持不动
run 1000000
write_restart restart.equ
#####开始 Poiseuille 流动模拟#####
fix 1 fluid addforce 0.01 0.0 0.0 #驱动力不能太大, 否则温度为控制不住。由于耗
散粒子动力学的温度控制是动量守恒的所以不需要排除质心速度进行温度控制
run 1000000
#####统计速度分布和密度分布#####
compute layers all chunk/atom bin/1d z lower 0.01 units box
fix ave all ave/chunk 1 1000000 3000000 layers density/number vx file
vx_rho.profile
run 1000000
```

计算得到的结果就可以使用讲义纳米流动部分进行粘度计算和其他分析。

16.2 滑移边界条件控制方法

耗散粒子动力学在复杂流动方面具有广泛的应用。但是在流体在固体表面流动时, 如何控制滑移边界条件是耗散粒子动力学的一个问题。本例通过修改 LAMMPS 源代码中的 fix wall 和 fix wall/lj126 实现了文献 Tunable-slip boundaries for coarse-grained simulations of fluid flow 中的滑移边界条件控制方法。采用的 lammps 版本是 29sep2021.通过设置参数可实现从无滑移到无限滑移的连续控制。实现的边界条件是一个通道流, 上下壁面只能是 y 方向的两个盒子边界, 流动方向只能是 x 方向。修改后的源代码可添加微信 baolu_yao 获取。该方法主要通过在近壁区引入一个耗散力来控制近壁区的速度, 具体方法是: 当粒子进入近壁区后就会受到一个额外的力, 这个力包含两部分一个耗散力一个随机力。为了防止粒子跑出边界需要施加一个保守力, 这里保守力采用 LJ/126 模型。通过修改源代码可以方便采用其他形式的保守力。耗散力用来控制速度, 随机力和耗散力组合

起来保证近壁区的温度不变。其中 γ_{WT} 为控制强度, 调节该值的大小就可实现滑移量的控制。

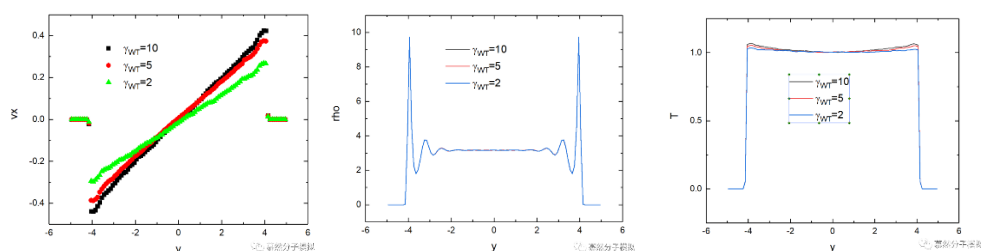
修改后的代码测试效果如下。图中结果表明代码修改成功。但是有温度分布可以看出, 这里设置的剪切率太大了。

$$\mathbf{F}_i^{WT} = \mathbf{F}_i^D + \mathbf{F}_i^R$$

$$\mathbf{F}_i^D = -\gamma_{WT} \omega(y_i/y_c)(\mathbf{v}_i - \mathbf{v}_{WT})$$

$$\omega\left(\frac{y_i}{y_c}\right) = \begin{cases} 1 - \frac{y_i}{y_c}, & y_i < y_c \\ 0, & y_i \geq y_c \end{cases}$$

$$\mathbf{F}_i^R = \sqrt{2\gamma_{WT}k_B T \omega\left(\frac{y_i}{y_c}\right)} \alpha(\Delta t)^{-1/2}$$



下面是测试所用到的 in 文件

```
#####
#####
#####这是一个通道流, 上下壁面只能是 y 方向的两个盒子边界, 流动方向只能是
x
units          lj
boundary p f p
neighbor 1.0 bin
neigh_modify   every 1 delay 0 check yes
atom_style     atomic
pair_style     dpd 1.0 1.0 343587
comm_modify mode single vel yes

log 01.log
variable vx equal 1.0
```

```

variable vx_up equal ${vx}/2
variable vx_down equal -${vx}/2

# ----- Atom Definition Section -----

region box block -5 5 -5 5 -5 5 units box
create_box 1 box
lattice fcc 3.0
region fluid block -5 5 -4.0 4.0 -5 5 units box
create_atoms 1 region fluid

# ----- Settings Section -----

pair_coeff      1 1 25 1.0
mass 1 1.0

# ----- Run Section -----

timestep 0.01
velocity       all create 1.0 672748 rot yes dist gaussian

thermo 100
thermo_style custom step temp
thermo_modify flush yes

dump mydump all custom 10000 dump.lammpstrj id type x y z

fix mynve all nve
#fix fix_id group_id wall/lj126 face coord epsilon sigma cutoff myT myseed
mygamma myv
fix ywall_down all wall/lj126 ylo EDGE 0.001 1.0 1.1225 1.0 5632456 2.0
${vx_down} units box
fix ywall_up all wall/lj126 yhi EDGE 0.001 1.0 1.1225 1.0 687541 2.0 ${vx_up} units
box

run 1000000

reset_timestep 0

compute cc1 all chunk/atom bin/1d y lower 0.1 units box
fix 2 all ave/chunk 1 100000 100000 cc1 density/number vx temp norm sample file
rho_vx_T.profile

run 100000

```

17. 专题十一：自由能

18. 专题十二：LAMMPS 实现小分子有机半导体的沉积

本节作者 **Lethe** 同学，感谢他的贡献

18.1 研究背景

有机电子学的快速发展在电子工业中显示出了巨大的潜力，引起了学术界的极大关注。有机半导体在有机光伏器件(OPV)、有机发光二极管(OLED)、有机场效应晶体管(OFET)和有机生物电子学等领域的各种新应用，使有机半导体成为有机电子器件中最重要的功能元件。有机薄膜晶体管(OTFT)是一种用有机半导体制备的有机电子器件，广泛应用于有机传感器、存储器件和柔性电路等领域。在 OTFT 中（图 1），有机半导体分子（DNTT）需要站在基底上，这就保证了有机半导体之间的 π 键方向垂直于表面。因此载流子（图 1b 中蓝色箭头）在有机半导体薄膜中水平传输，以获得最大载流子迁移率。由于有机半导体中载流子的输运具有高度的各向异性，因此有机半导体薄膜的分子堆积结构对有机电子学至关重要，我们可以理解为：较差的薄膜质量会导致载流子传输受阻（类比为路况极差的公路上汽车始终无法高速行驶）。

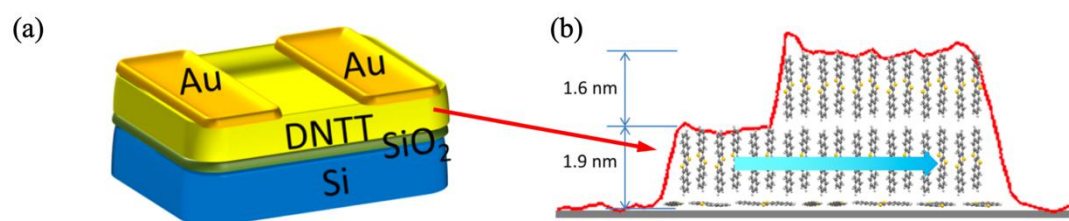


图 1 （a）有机薄膜晶体管（OTFT）示意图；（b）有序的 DNTT 层状薄膜示意图。

而实验中有有机半导体薄膜的制备通常无法达到理想的情况，如图 2 所示，沉积在纯金（bare Au）上的并五苯（pentacene）分子会呈现细长的绳岛状，而不是我们希望得到的平铺在整个基底表面的均匀层状，研究人员发现在金表面加入自组装单分子层（SAMs）后，有机半导体分子会自发的站立在基底表面，这是我们理想的分子沉积结构，因此，我们使用分子动力学模拟了有机半导体在不同基底上沉积的情况。

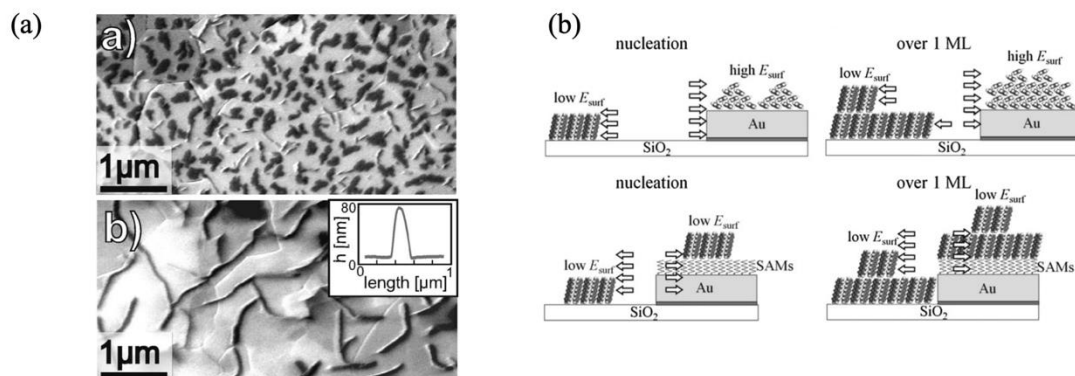


图 2 (a) 并五苯分子在纯金表面的绳岛状生长结构; (b) 并五苯在 SAMs 修饰的金表面以直立形态的层状生长。

18.2 建模细节

我们使用 Materials Studio 分别构建了两种不同表面形貌的 Au 基底, 其中粗糙基底是由光滑基底切割而来的, 其模型尺寸为 $105 \text{ \AA} \times 105 \text{ \AA}$, 同时, 对于两种 Au 表面分别覆盖了两种极性差异极大的 SAMs 分子, 分别为 SAM(-CH₃)和 SAM(-COOH)分子, 不同的末端官能团展现出明显的极性差异, 这可能对有机半导体的沉积造成一定的影响。沉积所使用的小分子有机半导体为 pentacene。模拟中所使用的模型和分子结构如图 3 所示。

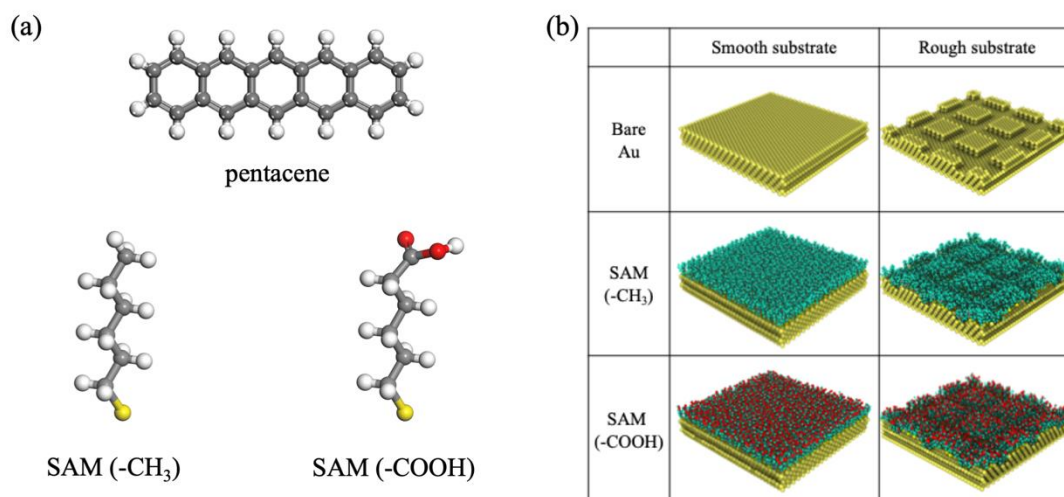


图 3 (a) 模拟中所使用的有机小分子 pentacene 和两种不同极性的 SAMs 分子; (b) 模拟中所使用的的六种不同基底结构。

18.3 模拟过程

具体的 in 文件如下:

```
#####
##                                     ##
```

```

## Pentacene in file ##
##                               ##
#####

##### Initial setup #####

dimension          3
units              real
boundary           p p f

#### Set  variable ####

variable           addatomscount   equal      count(addatoms)
variable           fix1count       equal      count(fix1)
variable           Aucount         equal      count(Au)

variable           velocity        equal      -0.005
variable           Temperature     equal      350
variable           D               equal      count(del)

##### Potentials #####

atom_style         full
bond_style         harmonic
angle_style        harmonic
dihedral_style     charmm
pair_style         hybrid morse 8.0 lj/cut/coul/cut 10.0
special_bonds      amber
timestep           1

##### region #####

neighbor           2.0 bin
neigh_modify       delay 5 every 1 check yes

read_data          Au_CH3_different_rough120.data extra/atom/types 2 extra/bond/types 2 extra/angle/types 2 extra/dihedral/types

```


3 extra/bond/per/atom 3 extra/angle/per/atom 7 extra/dihedral/per/atom 20 extra/special/per/atom 13

region fix1 block EDGE EDGE EDGE EDGE 0.0 3.0 units box
region delete block EDGE EDGE EDGE EDGE 190.0 200.0 units box

group addatoms type 6 7
group Au type 1
group fix1 region fix1
group SAM_CH3 type 2 3 4 5
#group del dynamic all region delete every 100

mass

mass 1 197 # Au
mass 2 32 # S
mass 3 12 # C3
mass 4 12 # C2
mass 5 1 # H

mass 6 12 # CP
mass 7 1 # H

LJ potential

Au_UFF

pair_coeff 1 1 morse 10.954 1.583 3.024 8
pair_coeff 1 2 morse 8.763 1.470 2.650 8
pair_coeff 1 3 lj/cut/coul/cut 0.0634 3.182
pair_coeff 1 4 lj/cut/coul/cut 0.0634 3.182
pair_coeff 1 5 lj/cut/coul/cut 0.0414 2.752
pair_coeff 1 6 lj/cut/coul/cut 0.0634 3.182
pair_coeff 1 7 lj/cut/coul/cut 0.0414 2.752

GAFF

pair_coeff 2 2 lj/cut/coul/cut 0.25 3.56

pair_coeff 2 3 lj/cut/coul/cut 0.15 3.48
 pair_coeff 2 4 lj/cut/coul/cut 0.15 3.48
 pair_coeff 2 5 lj/cut/coul/cut 0.06 3.10
 pair_coeff 2 6 lj/cut/coul/cut 0.15 3.48
 pair_coeff 2 7 lj/cut/coul/cut 0.06 3.10

pair_coeff 3 3 lj/cut/coul/cut 0.086 3.4
 pair_coeff 3 4 lj/cut/coul/cut 0.086 3.4
 pair_coeff 3 5 lj/cut/coul/cut 0.036 3.0
 pair_coeff 3 6 lj/cut/coul/cut 0.086 3.4
 pair_coeff 3 7 lj/cut/coul/cut 0.036 3.0

pair_coeff 4 4 lj/cut/coul/cut 0.086 3.4
 pair_coeff 4 5 lj/cut/coul/cut 0.036 3.0
 pair_coeff 4 6 lj/cut/coul/cut 0.086 3.4
 pair_coeff 4 7 lj/cut/coul/cut 0.036 3.0

pair_coeff 5 5 lj/cut/coul/cut 0.015 2.6
 pair_coeff 5 6 lj/cut/coul/cut 0.086 3.4
 pair_coeff 5 7 lj/cut/coul/cut 0.086 3.4

pair_coeff 6 6 lj/cut/coul/cut 0.086 3.4
 pair_coeff 6 7 lj/cut/coul/cut 0.036 3.0

pair_coeff 7 7 lj/cut/coul/cut 0.015 2.6

bond coeffer

bond_coeff 1 569.4 1.3343 # C2-C2
 bond_coeff 2 343.1 1.0879 # C2-H
 bond_coeff 3 569.4 1.3343 # C2-C3
 bond_coeff 4 281.5 1.7340 # C2-S
 bond_coeff 5 343.1 1.0879 # C2-H

 bond_coeff 6 478.4 1.387 # CP-CP
 bond_coeff 7 344.3 1.087 # CP-H

angles coeffer

angle_coeff 1 69.3 121.81 # C2-C2-C2

```

angle_coeff 2 49.9 120.43 # C2-C2-H
angle_coeff 3 38.1 116.90 # H-C2-H
angle_coeff 4 69.3 121.81 # C2-C2-C3
angle_coeff 5 49.9 120.43 # C3-C2-H
angle_coeff 6 61.2 129.37 # C2-C2-S
angle_coeff 7 43.4 115.70 # S-C2-H
angle_coeff 8 49.9 120.43 # C2-C3-H
angle_coeff 9 38.1 116.90 # H-C3-H

angle_coeff 10 35.00 120.00 # CP-CP-H
angle_coeff 11 63.00 120.00 # CP-CP-CP

```

dihedrals coeffes

```

dihedral_coeff 1 4 2 180 0 # C2-C2-C2-C2
dihedral_coeff 2 4 2 180 0 # C2-C2-C2-H
dihedral_coeff 3 4 2 180 0 # H-C2-C2-H
dihedral_coeff 4 4 2 180 0 # C2-C2-C2-C3
dihedral_coeff 5 4 2 180 0 # C3-C2-C2-H
dihedral_coeff 6 4 2 180 0 # C2-C2-C2-S
dihedral_coeff 7 4 2 180 0 # S-C2-C2-H
dihedral_coeff 8 4 2 180 0 # C2-C2-C3-H
dihedral_coeff 9 4 2 180 0 # H-C2-C3-H

```

```

dihedral_coeff 10 3.625 2 180 0 # c2-c2-c2-h
dihedral_coeff 11 3.625 2 180 0 # c2-c2-c2-c2
dihedral_coeff 12 3.625 2 180 0 # h-c2-c2-h

```

balance

```

comm_style tiled
fix fixbalance all balance 1000 1.2 rcb

```

thermo

```

minimize 1.0e-4 1.0e-6 5000 10000
thermo_style custom step temp pe etotal time

```

```

thermo                1000
dump                  1 all custom 10000 pentacene.xyz id type x y z

##### initialization velocity #####

velocity              all          create ${Temperature} 78273974 mom yes rot yes loop local dist gaussian

velocity              fix1 set 0 0 0 units box

##### Freeze boundary #####

fix                   fl fix1 setforce 0.0 0.0 0.0

fix                   r1 all wall/reflect zhi EDGE

#### NVT ####

fix                   nvt1 Au nvt temp ${Temperature} ${Temperature} $(100*dt)

fix                   nvt2 addatoms nvt temp ${Temperature} ${Temperature} $(100*dt)

fix                   nvt3 SAM_CH3 nvt temp ${Temperature} ${Temperature} $(100*dt)

run                   5000

##### compute #####

compute              Au Au temp
compute              SAM_CH3 SAM_CH3 temp
compute              add addatoms temp
compute              all all temp

compute_modify        add dynamic/dof yes
compute_modify        Au dynamic/dof yes
compute_modify        SAM_CH3 dynamic/dof yes
compute_modify        all dynamic/dof yes

```

```
fix_modify      nvt2      temp add dynamic/dof yes energy yes

thermo_style     custom step etotal pe ke epair evdwl ecoul c_Au c_SAM_CH3 c_add c_all temp
thermo           50000
thermo_modify     lost ignore lost/bond ignore
```

```
##### Deposit #####
```

```
molecule        Pentacene  Pentacene_R.txt
region           slab block EDGE EDGE EDGE EDGE 100 100
fix              deposit_pentacene_1  addatoms deposit 800 0 250000 12345 region slab id next near 3.0 mol Pentacene vz
${velocity} ${velocity} units box
```

```
dump             dump1 addatoms custom 200000 pentaceneid.dump mol id type x y z mass
dump_modify      dump1 sort id
```

```
##### delete #####
```

```
fix              d1 addatoms evaporate 10000 36 delete 49892 molecule yes
```

```
##### Run #####
```

```
run             125000000
```

```
##### nvt #####
```

```
write_data       Au_SAM_CH3_Pentacene_ML1.data
write_restart     Au_SAM_CH3_Pentacene_desposit_1st.restart
```

```
print "Program successfully finish!"
```

Pentacene 分子模板如下

```
# pentacene molecule topology structure
```

36 atoms

40 bonds

66 angles
104 dihedrals

Coords

```

1  1.41020643 0.029404622  0.016905863  #cp
2  0.772300649 -1.209895195  0.110532533 #cp
3  1.459025378 -2.424179496  0.223790084  #cp
4  0.785318385 -3.636784401  0.317488763  #cp
5  1.513151622 -4.878998804  0.453677267  #cp
6  0.864216813 -6.051146578  0.5545002  #cp
7  -0.581490243 -6.092841482  0.522616617  #cp
8  -1.295323752 -4.938965535  0.401302087  #cp
9  -0.652762553 -3.65376428 0.302942869  #cp
10 -1.34711203 -2.480678924 0.190897082  #cp
11 -0.686946957 -1.239743872 0.089557743  #cp
12 2.341190713 0.057668601 0.025100233  #h
13 2.390043057 -2.41896526 0.235617092  #h
14 2.443796059 -4.865874741 0.472134231  #h
15 1.34864434 -6.83964628 0.64520484  #h
16 -1.024936558 -6.908790959 0.584056028  #h
17 -2.225226889 -4.983017907 0.381749096  #h
18 -2.278326812 -2.50282174 0.180295114 #h
19 -1.41020643 -0.029404622 -0.016905863  #cp
20 -0.772300649 1.209895195 -0.110532533  #cp
21 -1.459025378 2.424179496 -0.223790084  #cp
22 -0.785318385 3.636784401 -0.317488763  #cp
23 -1.513151622 4.878998804 -0.453677267  #cp
24 -0.864216813 6.051146578 -0.5545002  #cp
25 0.581490243 6.092841482 -0.522616617  #cp
26 1.295323752 4.938965535 -0.401302087  #cp
27 0.652762553 3.65376428 -0.302942869  #cp
28 1.34711203 2.480678924 -0.190897082  #cp
29 0.686946957 1.239743872 -0.089557743  #cp
30 -2.341190713 -0.057668601 -0.025100233  #h
31 -2.390043057 2.41896526 -0.235617092  #h
32 -2.443796059 4.865874741 -0.472134231  #h
33 -1.34864434 6.83964628 -0.64520484 #h
34 1.024936558 6.908790959 -0.584056028  #h
35 2.225226889 4.983017907 -0.381749096  #h
36 2.278326812 2.50282174 -0.180295114  #h

```

Types

1	6
2	6
3	6
4	6
5	6
6	6
7	6
8	6
9	6
10	6
11	6
12	7
13	7
14	7
15	7
16	7
17	7
18	7
19	6
20	6
21	6
22	6
23	6
24	6
25	6
26	6
27	6
28	6
29	6
30	7
31	7
32	7
33	7
34	7
35	7
36	7

Charges

1	-0.2795
2	0.1438
3	-0.2795
4	0.1438
5	-0.2795

6 -0.1011
7 -0.1011
8 -0.2795
9 0.1438
10 -0.2795
11 0.1438
12 0.15395
13 0.15395
14 0.152675
15 0.128625
16 0.128625
17 0.152675
18 0.15395
19 -0.2795
20 0.1438
21 -0.2795
22 0.1438
23 -0.2795
24 -0.1011
25 -0.1011
26 -0.2795
27 0.1438
28 -0.2795
29 0.1438
30 0.15395
31 0.153975
32 0.152675
33 0.128625
34 0.128625
35 0.152675
36 0.153975

Bonds

1 6 1 2
2 7 1 12
3 6 1 29
4 6 2 3
5 6 2 11
6 6 3 4
7 7 3 13
8 6 4 5
9 6 4 9
10 6 5 6


```

11 7 5 14
12 6 6 7
13 7 6 15
14 6 7 8
15 7 7 16
16 6 8 9
17 7 8 17
18 6 9 10
19 6 10 11
20 7 10 18
21 6 11 19
22 6 19 20
23 7 19 30
24 6 20 21
25 6 20 29
26 6 21 22
27 7 21 31
28 6 22 23
29 6 22 27
30 6 23 24
31 7 23 32
32 6 24 25
33 7 24 33
34 6 25 26
35 7 25 34
36 6 26 27
37 7 26 35
38 6 27 28
39 6 28 29
40 7 28 36

```

Angles

```

1 10 2 1 12
2 11 2 1 29
3 10 29 1 12
4 11 1 2 3
5 11 1 2 11
6 11 3 2 11
7 11 2 3 4
8 10 2 3 13
9 10 4 3 13
10 11 3 4 5
11 11 3 4 9

```

12 11 5 4 9
13 11 4 5 6
14 10 4 5 14
15 10 6 5 14
16 11 5 6 7
17 10 5 6 15
18 10 7 6 15
19 11 6 7 8
20 10 6 7 16
21 10 8 7 16
22 11 7 8 9
23 10 7 8 17
24 10 9 8 17
25 11 4 9 8
26 11 4 9 10
27 11 8 9 10
28 11 9 10 11
29 10 9 10 18
30 10 11 10 18
31 11 2 11 10
32 11 2 11 19
33 11 10 11 19
34 10 20 19 30
35 11 20 19 11
36 10 11 19 30
37 11 19 20 21
38 11 19 20 29
39 11 21 20 29
40 11 20 21 22
41 10 20 21 31
42 10 22 21 31
43 11 21 22 23
44 11 21 22 27
45 11 23 22 27
46 11 22 23 24
47 10 22 23 32
48 10 24 23 32
49 11 23 24 25
50 10 23 24 33
51 10 25 24 33
52 11 24 25 26
53 10 24 25 34
54 10 26 25 34
55 11 25 26 27

```

56 10 25 26 35
57 10 27 26 35
58 11 22 27 26
59 11 22 27 28
60 11 26 27 28
61 11 27 28 29
62 10 27 28 36
63 10 29 28 36
64 11 20 29 28
65 11 20 29 1
66 11 28 29 1

```

Dihedrals

```

1  10 3  2  1 12
2  10 11 2  1 12
3  11 29 1  2  3
4  11 29 1  2 11
5  11 2  1 29 20
6  11 2  1 29 28
7  10 20 29 1 12
8  10 28 29 1 12
9  11 1  2  3  4
10 10 1  2  3 13
11 11 11 2  3  4
12 10 11 2  3 13
13 11 1  2 11 10
14 11 1  2 11 19
15 11 3  2 11 10
16 11 3  2 11 19
17 11 2  3  4  5
18 11 2  3  4  9
19 10 5  4  3 13
20 10 9  4  3 13
21 11 3  4  5  6
22 10 3  4  5 14
23 11 9  4  5  6
24 10 9  4  5 14
25 11 3  4  9  8
26 11 3  4  9 10
27 11 5  4  9  8
28 11 5  4  9 10
29 11 4  5  6  7
30 10 4  5  6 15

```

31	10	7	6	5	14
32	12	14	5	6	15
33	11	5	6	7	8
34	10	5	6	7	16
35	10	8	7	6	15
36	12	15	6	7	16
37	11	6	7	8	9
38	10	6	7	8	17
39	10	9	8	7	16
40	12	16	7	8	17
41	11	7	8	9	4
42	11	7	8	9	10
43	10	4	9	8	17
44	10	10	9	8	17
45	11	4	9	10	11
46	10	4	9	10	18
47	11	8	9	10	11
48	10	8	9	10	18
49	11	9	10	11	2
50	11	9	10	11	19
51	10	2	11	10	18
52	10	19	11	10	18
53	11	2	11	19	20
54	10	2	11	19	30
55	11	10	11	19	20
56	10	10	11	19	30
57	10	21	20	19	30
58	10	29	20	19	30
59	11	11	19	20	21
60	11	11	19	20	29
61	11	19	20	21	22
62	10	19	20	21	31
63	11	29	20	21	22
64	10	29	20	21	31
65	11	19	20	29	28
66	11	19	20	29	1
67	11	21	20	29	28
68	11	21	20	29	1
69	11	20	21	22	23
70	11	20	21	22	27
71	10	23	22	21	31
72	10	27	22	21	31
73	11	21	22	23	24
74	10	21	22	23	32

```

75 11 27 22 23 24
76 10 27 22 23 32
77 11 21 22 27 26
78 11 21 22 27 28
79 11 23 22 27 26
80 11 23 22 27 28
81 11 22 23 24 25
82 10 22 23 24 33
83 10 25 24 23 32
84 12 32 23 24 33
85 11 23 24 25 26
86 10 23 24 25 34
87 10 26 25 24 33
88 12 33 24 25 34
89 11 24 25 26 27
90 10 24 25 26 35
91 10 27 26 25 34
92 12 34 25 26 35
93 11 25 26 27 22
94 11 25 26 27 28
95 10 22 27 26 35
96 10 28 27 26 35
97 11 22 27 28 29
98 10 22 27 28 36
99 11 26 27 28 29
100 10 26 27 28 36
101 11 27 28 29 20
102 11 27 28 29 1
103 10 20 29 28 36
104 10 1 29 28 36

```

其中值得注意的几点分别为:

1) read_data 中需要注意的事项

由于我们的模拟中需要在模拟盒使用 `fix deposit` 命令不断插入新的 pentacene 分子, 所以在 `read_data` 命令后面需要额外注明 `extra/atom/.../types` 和 `extra/bond/.../per/atom`, 其中对于 `type` 类型的参数设置比较简单, 直接找额外插入的分子中不同键角对应的 `type` 种类即可, 比较难搞懂的是 `extra/per/atom`, 这里我自己摸索的理解是这样的: 每个原子所具有的连接键、角和二面角等的最大值, 所以我们需要查阅插入的分子文件, 如图 4 所示。在 Bonds 中, 对于 1 号原子相关的键有三个, 因此 `extra/bond/per/atom` 的值应至少大于 3; 在 Angles 中,

对于 2 号原子相关的角有 7 个, 因此 `extra/angle/per/atom` 的值应至少大于 7, 以此类推; 对于 `extra/special/per/atom`, 由于我们在原有的五种原子类型的基础上额外插入了两种原子, 因此在 `pair_coeff` 中与 6 和 7 号原子相关的一共有 13 个, 因此 `extra/special/per/atom` 应至少大于 13。

Bonds				Angles				
1	6	1	2	1	10	2	1	12
2	7	1	12	2	11	2	1	29
3	6	1	29	3	10	29	1	12
4	6	2	3	4	11	1	2	3
5	6	2	11	5	11	1	2	11
6	6	3	4	6	11	3	2	11
7	7	3	13	7	11	2	3	4
8	6	4	5	8	10	2	3	13
9	6	4	9	9	10	4	3	13
10	6	5	6	10	11	3	4	5
11	7	5	14	11	11	3	4	9
12	6	6	7	12	11	5	4	9
13	7	6	15	13	11	4	5	6
14	6	7	8	14	10	4	5	14
15	7	7	16	15	10	6	5	14
16	6	8	9	16	11	5	6	7
17	7	8	17	17	10	5	6	15
18	6	9	10	18	10	7	6	15
19	6	10	11	19	11	6	7	8
20	7	10	18	20	10	6	7	16
21	6	11	19	21	10	8	7	16
22	6	19	20	22	11	7	8	9
23	7	19	30	23	10	7	8	17
24	6	20	21	24	10	9	8	17
25	6	20	29	25	11	4	9	8

图 4 分子模板

2) 有关分子模板的制作

由于本文计算所使用的是 3Mar2020 版本, 所以对应的分子模板也是按照此版本的手册编写的, 其中为了制作沉积分子所需的单个 `pentacene` 分子模板, 只需要以下几个信息即可: (1) `atom`、`bond`、`angle` 以及 `dihedrals` (如果有 `improper` 也需添加) 的数量; (2) 分子的笛卡尔坐标 (`Coords`); (3) 所有原子的类型 (`Types`); 所有原子的电荷 (`Charges`); (4) 分子中所有的键 (`Bonds`)、角 (`Angles`)、二面角 (`Dihedrals`), 这几项均可从单分子的 `data` 文件中获取。本文所用分子模板会放在文章最后。

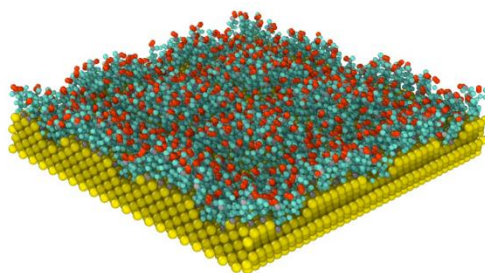
3) `fix deposit` 命令的使用

`fix deposit` 命令是做分子 (原子) 沉积最常用的命令, 其具体的使用方法在 `lammps` 官网有很详细的说明, 在这里不再赘述, 我们的研究中取了一个论文中经常用到的沉积频率: 每隔 250000 步 (即 0.25ns) 在系统真空区域插入一个 `pentacene` 分子, 一共插入了 800 个 `pentacene` 分子, 但是由于在模拟过程中会有

分子从基底脱离飞入真空层, 因此我们在模拟盒真空层顶端设置一个删除区域, 将脱离基底的 pentacene 分子迅速删除, 以保证其不会影响后续分子的沉积。值得注意的是, 由于在模拟过程中会不断有分子插入, 因此越到模拟后期系统运行速度会下降的很厉害, 对服务器的性能要求较高。

18.4 模拟结果

将模拟结果展示以下视频:



视频链接: [利用 LAMMPS 实现小分子有机半导体的沉积 哔哩哔哩 bilibili](#)

我们可以发现, 随着沉积过程的进行, 模拟初期平躺在基底表面的 pentacene 分子会逐渐聚集在一起, 形成一个晶核, 而随着沉积的进行, 越来越多的分子聚集到已经形成的晶核上, 晶核会在短时间内迅速发生转向, 从平躺状态变为直立状态, 并引导新沉积的 pentacene 分子形成有序的直立有机薄膜, 整个过程完全是在模拟系统中完全自发进行的。

18.5 结果分析

I SAM 和粗糙度对生长的影响

并五苯分子在六种不同表面形态结构的基底表面(光滑裸金、粗糙裸金、光滑 SAM(-CH₃)、光滑 SAM(-COOH)、粗糙 SAM(-CH₃)和粗糙 SAM(-COOH))的沉积过程如图 5 所示。在沉积初期, 并五苯分子以平躺形态出现并聚集在基底上。在光滑裸金和粗糙裸金表面, 并五苯分子迅速聚集, 随着沉积过程的进行呈现三维岛状生长模式。研究发现, 不同官能团的 SAM 会影响并五苯分子的初始分布, 在光滑 SAM(-CH₃)表面并五苯分子均匀分布, 而在光滑 SAM(-COOH)表面则倾向于较早的形成晶核。尽管并五苯分子仍平行于基底, 但是 SAM 的加入能使并五苯分子取向更加有序。粗糙 SAM(-CH₃)和 SAM(-COOH)表面沉积结果表明, 并五苯分子在这两种表面上均能形成直立的分子层, 而且粗糙 SAM(-COOH)表面形成了的两层直立分子层, 更有利于并五苯的生长。

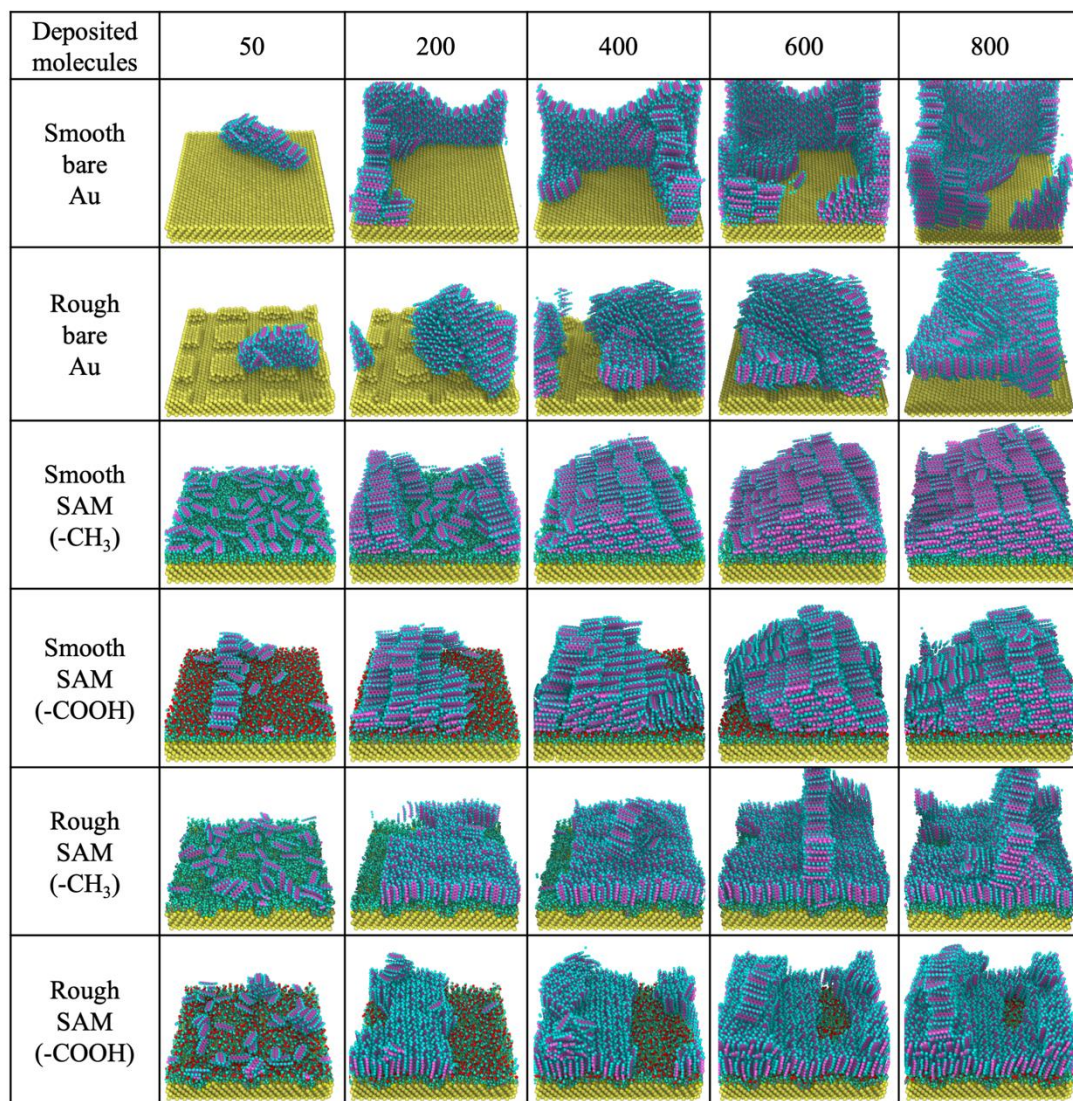


图 5 并五苯在不同基底上动力学生长过程。

研究还监测了并五苯分子在粗糙 SAM(-CH₃)和粗糙 SAM(-COOH)表面上的平均倾斜角和质心高度随沉积分子数的变化(图 6)。随着沉积分子数的增加,在两种表面上第一层分子(ML1)的平均倾斜角从 0°迅速变化到 80°,并随后保持稳定,这表明 ML1 分子发生了重新定向;而且在粗糙 SAM(-COOH)表面上并五苯形成了第二层直立分子。同时,直立晶体岛形成后,每层的质心高度保持不变;SAM(-COOH)表面上不同分子层质心高度的变化说明形成的并五苯薄膜具有阶梯形貌,这与文献中并五苯生长的实验结果一致。

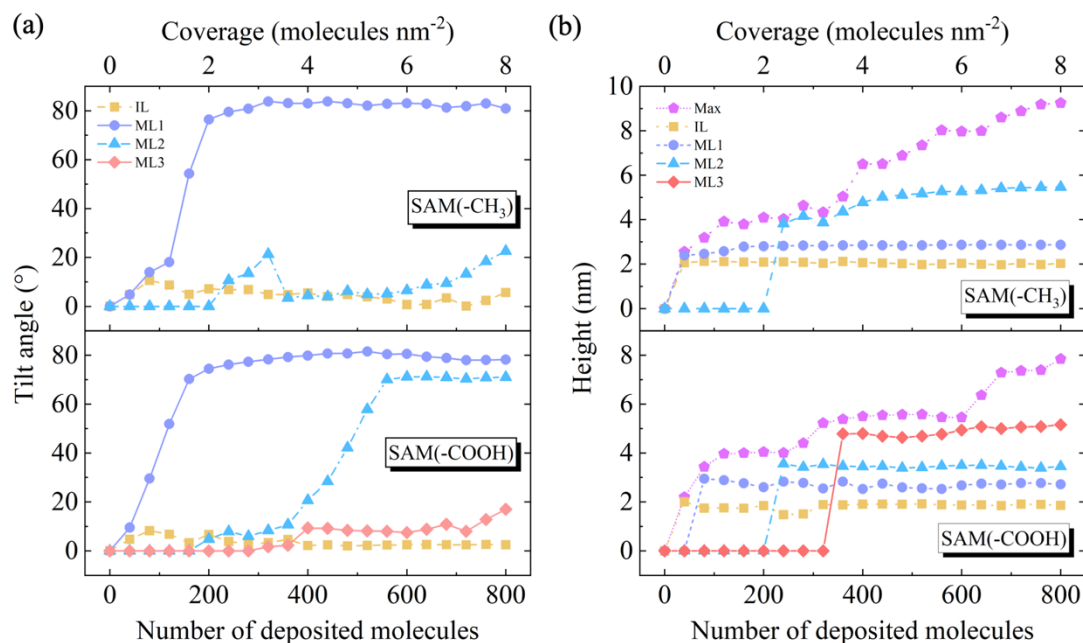


图 6 (a) 粗糙 SAM(-CH₃)和粗糙 SAM(-COOH)表面上不同分子层的平均倾斜角随沉积分子数的变化。(b) 粗糙 SAM(-CH₃)和粗糙 SAM(-COOH)表面上不同分子层的质心高度随沉积分子数的变化。

II 温度对生长的影响

另外, 基底温度也会对不同表面上的并五苯成核产生影响。较高的温度会延迟两个粗糙 SAM 表面上晶核出现和重新定向, 这种现象在粗糙 SAM(-CH₃)表面更为明显。研究计算了不同基底温度下并五苯分子间相互作用能 ($E_{\text{Pen-Pen}}$) 以及并五苯和 SAM 分子间相互作用能 ($E_{\text{Pen-SAM}}$) 的变化。如图 7 所示, 当沉积分子超过一定数量时, $E_{\text{Pen-Pen}}$ 会急剧上升, $E_{\text{Pen-SAM}}$ 则出现反弹, 说明两者间发生了能量交换; 此时并五苯晶核倾斜角也随之发生变化, 这就表明并五苯-并五苯和并五苯-SAM 分子间的能量交换驱动了晶核转向。随着基底温度的升高, 并五苯-并五苯分子和并五苯-SAM 分子间的能量交换出现延迟, 但晶核的重新定向过程进行更快。

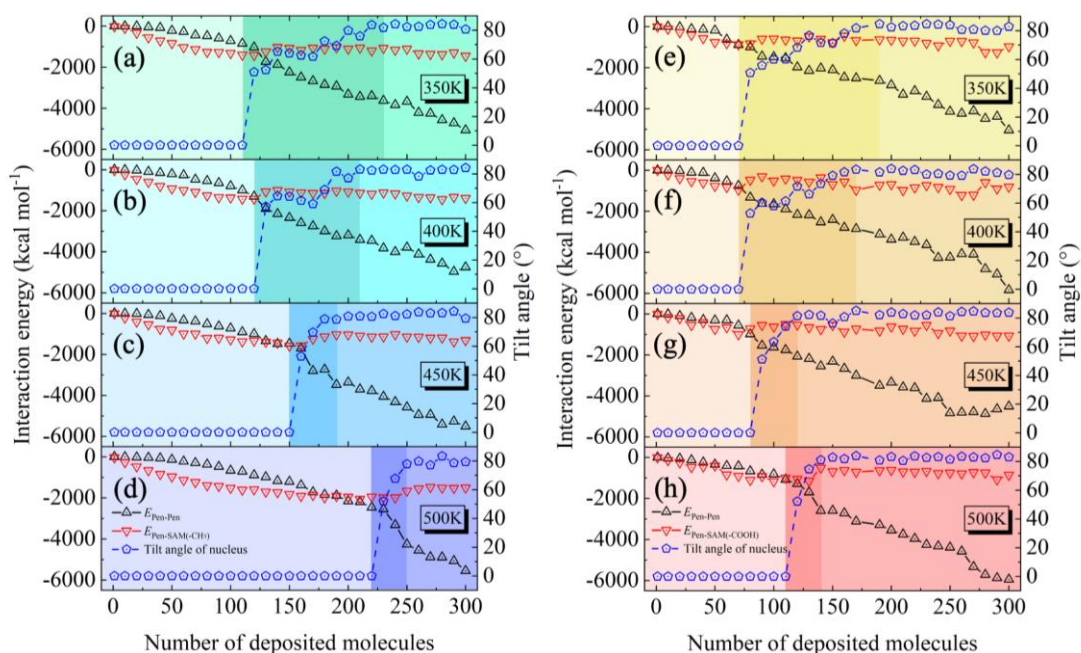


图3 在 350 到 500 K 的基底温度下, (a-d) 粗糙 SAM(-CH₃)表面和 (e-h) 粗糙 SAM(-COOH)表面上, 并五苯分子间相互作用能量 ($E_{\text{Pen-Pen}}$)、并五苯和 SAM 分子间相互作用能量 ($E_{\text{Pen-SAM}}$) 以及并五苯晶核平均倾斜角随沉积分子数的变化。

III 自由能对生长的影响

为了揭示基底粗糙度对有机半导体分子生长的影响, 研究进一步计算了薄膜表面自由能 (图 8)。结果表明, 粗糙度会引起表面自由能的变化, 从而调控了有机半导体分子沉积的动力学行为。同时, 在并五苯薄膜台阶边缘两侧存在局部自由能的最大值和最小值, 这就形成了 Ehrlich-Schwöbel 势垒。由于并五苯的 (001) 取向具有最小的能量, 因此沉积在基底上的并五苯分子更倾向于沿着能量减小的路径向台阶边缘移动以达到最稳定状态, 促使了并五苯以 (001) 取向直立依附在台阶边缘。同时, Ehrlich-Schwöbel 势垒阻碍了并五苯分子跨越台阶边缘, 沉积在不同分子层上的并五苯分子只能在当前分子层移动, 促进了并五苯阶梯形貌的形成。

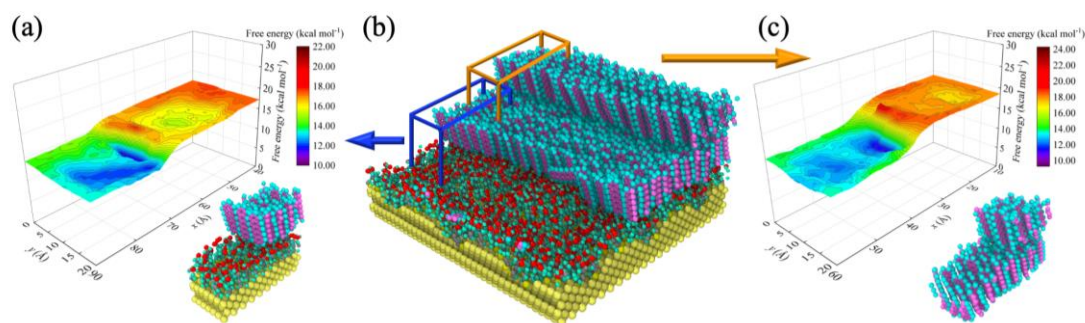


图8 粗糙 SAM(-COOH)基底、ML1 和 ML2 的表面自由能分布。

原文链接: <https://doi.org/10.1016/j.apsusc.2021.152203>

感兴趣的同学可以给我发邮件交流: 412874001@qq.com, 如果您对我的研究赞同麻烦写论文的时候引用一下☺

19. LAMMPS 源代码解析

19.1 LAMMPS 的一个时间步是如何工作的

理解 LAMMPS 代码工作方式的第一也是最基础的方法就是明白一个时间步使如何执行的。一个时间步是通过调用 `Update` 类中的积分实例的方法来实现。由于积分是一个基类, 它指向一个导出类的实例, 该导出类与 `in` 文件中 `run_style` 命令的设置相对应。这里介绍由 `Verlet` 类所执行的时间步。对于 `r-RESPA` 多级时间步方法, `Respa` 类执行了相似的过程。`Min` 类用来执行能量最小化, 因此并未执行字面意思的时间步。但是其与这里描述的过程是相似的, 即在每个能量最小化迭代中计算受力和调用 `fix`。时间积分和能量最小化的区别将在本节最后进行强调。

`Verlet` 类写在 `src/verlet.cpp` 和 `verlet.h` 文件中。它执行速度-Verlet 时间步算法。具体执行这一算法的方法是 `Verlet::run()` 方法。在介绍这个方法前, 首先要介绍该类中的几个其他方法。

- `init()` 方法在每个 `run` 之前都会调用。该方法基于用户在其他代码部分的设置, 初始化了一些内部的 `flag`
- `setup()` 和 `setup_minimal()` 方法在 `run` 前也会调用。速度-Verlet 方法要求当前受力在第一个时间步之前计算, 这些根据原子相互作用计算受力的过程与接下来描述的逻辑是相同的。一些 `fix` 采用下面描述的机制被调用。各种 `counters` 也会在 `run` 前被初始化。`setup_minimal` 是一个变体, 其有一些用来执行较少设置的 `flag`。当之前有多个 `run` 连续调用, 并且之前 `run` 的信息在后续依旧成立, 那么这个方法会被调用。比如, 在 `in` 文件中通过 `pre no` 和 `every` 的 `run` 选项设置多个重复的较短的 `run` (这些 `run` 之间有其他命令), 那么 `setup_minimal()` 方法会被使用
- `force_clear()` 方法在每个时间步之前初始化受力和其他数组, 这样受力以及扭矩等可以被累积。

下面介绍 `Verlet::run()` 方法。该方法的基本结构以伪代码的形式进行描述。在实际 `src/verlet.cpp` 的代码中这些操作会被有选择的调用。

```

loop over N timesteps:
  if timeout condition: break
  ev_set()

  fix->initial_integrate()
  fix->post_integrate()

  nflag = neighbor->decide()
  if nflag:
    fix->pre_exchange()
    domain->pbcb()
    domain->reset_box()
    comm->setup()
    neighbor->setup_bins()
    comm->exchange()
    comm->borders()
    fix->pre_neighbor()
    neighbor->build()
    fix->post_neighbor()
  else:
    comm->forward_comm()

  force_clear()
  fix->pre_force()

  pair->compute()
  bond->compute()
  angle->compute()
  dihedral->compute()
  improper->compute()
  kspace->compute()

  fix->pre_reverse()
  comm->reverse_comm()

  fix->post_force()
  fix->final_integrate()
  fix->end_of_step()

  if any output on this step:
    output->write()

# after loop
fix->post_run()

```

LAMMPS 一个时间步执行的伪代码

`ev_set()`方法（在 `Integrate` 父类中），设置两个 flag（`eflag` 和 `vflag`），这两个 flag 用来控制能量和维里的计算。每个 flag 控制 `global` 或 `per-atom` 能量和维里是否在当前时间步进行计算，这是因为能量和维里可能会被某些 `fix` 或 `variable` 或 `output` 需要。这些 flag 会被传递给计算粒子相互作用的各种方法中，这样如果能量和维里不需要的时候一些 `compute` 或 `tally` 相应的数据可以跳过额外的计算。查看 `Integrate::ev_set()`方法中对这些 flag 取值的描述。

在时间步的各个阶段，不同的 `fix` 会被调用，比如 `fix->initial_intergrate()`。在代码中，这实际上是通过 `Modify` 类来实现。`Modify` 类存储了所有的 `Fix` 对象，以及

一个列表来记录在时间步的哪个阶段调用哪个 `fix`。`Fix` 是 LAMMPS 中用来针对一个模拟来在一个时间步内实现某种操作。每一个 `fix` 都有一个或多个方法，每一个方法都会在时间步的特定阶段被调用，具体见时间步的伪代码。所有在 `in` 文件中定义的 `fix` 都有一个 `initial_integrate()` 方法，该方法在每个时间步开始时调用。如 `fix nve`，`fix nvt` 和 `fix npt` 执行时间步起始的速度-Verlet 积分操作来以半个时间步长更新速度，以整个时间步长更新位置。`post_integrate()` 方法在这些更新后会被立即调用。只有少数一些 `fix` 使用这个方法，如在 `FixWallRefect` 类将例子反射回盒子。

在 `Neighbor` 类中 `decide()` 方法决定是否需要在当前时间步重建邻居列表（重建条件可以使用 `neigh_modify` 中的 `every/delay/check` 参数进行修改）。如果不需要重建，则通过 `Comm` 类中的 `forward_comm()` 方法获取每个处理器的 `ghost` 原子的坐标。如果邻居列表需要被重建，在伪代码中 `if` 条件语句中的一些操作会首先被调用。任何定义的 `fix` 中的 `pre_exchange()` 方法先被调用。通常这会往系统中插入或从系统中删除粒子。

周期性边界条件则通过 `Domain` 类中的 `pbc()` 方法来实现，即将那些跑出模拟盒子的粒子重新调整回盒子。注意这不是每个时间步都会执行，只有在邻居列表重建时执行。这样每个处理所负责的子区域对于 `owned` 原子和 `ghost` 原子将会具有一致的原子坐标。这也就是为什么当 `dump` 不在邻居列表重建时间步输出时，`dump` 出的原子坐标会稍微处于模拟盒子外部的原因。

接下来，如果需要，盒子边界会通过 `Domain` 类中的 `reset_box()` 进行重新调整，如盒子边界缩放至当前粒子的坐标。盒子尺寸或形状的改变要求通信 `ghost` 原子（`Comm` 类）和邻居列表 `bin`（邻居类）的内部信息被更新。`Comm` 类中的 `setup()` 方法和 `Neighbor` 类中的 `setup_bins()` 方法来执行这些更新。

这时，代码已经准备来将原子从一个处理器的几何区域迁移至另一个处理器中。这个操作通过 `Comm` 类中的 `exchange()` 方法进行实现。接着，`Comm` 类中的 `borders()` 方法识别出围绕在处理器子区域周围的 `ghost` 原子并且和周围处理器通信 `ghost` 原子信息。这通过循环处理器所有 `owned` 原子来构建这些原子的列表并发送给处理器的每个邻居处理器。在接下来的时间步中，这些列表会被 `Comm::forward_comm()` 方法使用。

具有 `pre_neighbor()` 方法的 `fix` 会被接着调用。这通常会重建一些由 `fix` 所存储的数据结构，之所以这样是因为这些 `fix` 取决于每个处理器当前 `owned` 用字。

现在每个处理器都有了其当前 `owned` 原子和 `ghost` 原子的列表。LAMMPS 会通过 `Neighbor` 类中的 `build()` 方法来重建邻居列表。这通常通过将处理器 `owned` 原子和 `ghost` 原子进行 binning，并扫描每个原子的 `bin` 周围的 `bin` 模板来在受力截断半径+`skin` 距离之内构建邻居原子的 Verlet 列表。

在接下里时间步部分，在使用 `force_clear()` 置零每个原子的力矢量后，计算原子之间所有的相互作用力。如果 `newton flag` 通过 `newton` 命令被设置为 `on`，受力会被加到处理器 `owned` 原子和 `ghost` 原子上，否则只加到 `owned`（即当地）原子上。对相互作用先会被计算，这允许全局维里（在 `Pair::compute()` 方法末尾以 $O(N)$ 而不是 $O(N^2)$ ）被便宜的计算。维里通过点乘原子坐标和受力来计算。通过在点乘中包含 `owned` 和 `ghost` 原子，周期性边界条件可以被正确的考虑。分子拓扑相互作用（键，键角，二面角，离平面）被接着计算。最有通过 `Kspace` 类计算来自长程库伦相互作用的贡献。

在 `fix` 中的 `pre_exchange()` 方法被用来执行那些在反向通信（比如在受力计算和存储的 `ghost` 原子期间执行额外的数据传递或数据减少）之前的操作。

如果 `newton flag` 是 `on`，`ghost` 原子的受力会被通信并被累加回它们所对应的 `owned` 原子上。`Comm` 类中的 `reverse_comm()` 来执行这个操作。这其实就是发送 `owned` 原子的坐标到其他处理器 `ghost` 原子的反向操作。

在时间步的这个阶段，每个原子的受力都知道了。通过 `fix`（具有 `post_force()`）所施加的额外受力约束（外部力，SHAKE 等）被执行。速度-Verlet 的第二阶段通过类似 `fix nve`，`nvt`，`npt` 中的 `final_integrate()` 方法进行执行（即，更新第二个半步长的速度）

在时间步的末尾，具有 `end_of_step()` 方法的类会被调用。这通常执行后处理计算，如 `fix ave/time`，`ave/chunk`。时间步的最后操作是执行任何输出的设置，即通过 `Output` 类的 `write()` 方法。在 LAMMPS 中有三类输出：输出到屏幕或日志文件中的热力学信息，输出到 `dump` 文件中的原子快照和 `restart` 文件。查看 `thermo_style`，`dump` 和 `restart` 命令获取详细信息。

能量最小化迭代的流程控制和一个动力学时间步的是相似的。受力被计算，邻居

列表按需建立，原子迁移到新的处理器，原子坐标和受力被通信到邻居处理器。唯一不同的是调用 `Fix` 类的不同。只有有限的一些 `fix` 在能量最小化中是有用的。相关的 `Fix` 类中的方法是 `min_pre_exchange()`, `min_pre_force()` 和 `min_post_force()`。每个 `fix` 在能量最小化迭代的适当阶段被调用。如，`min_post_force()` 和动力学中的 `post_force()` 是类似的。它用来改变或这约束每个原子的受力，这会影响能量最小化的过程。

在所有迭代完成后会有一个 `cleanup` 步骤来调用 `fix` 中的 `post_run` 方法来执行只有在计算末尾才需要的操作，如释放临时存储或者创建最终输出。